Artificial Intelligence — Exam Preparation Notes

Chapters 1–8 | George F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th edition
Print-ready study edition

These notes are organised by textbook chapter and retain the detailed explanations, worked examples, definitions, comparison tables, formulas, and pseudocode from the supplied notes. Symbols and logical notation are preserved as Unicode text. The final appendix contains selected textbook figure extracts to complement figure descriptions in the notes.

Printing recommendation: A4 paper, double-sided, normal margins. Use the PDF for printing and the DOCX if you need to annotate or customise the notes.

Contents

- Chapter 1 — AI: History and Applications
- Chapter 2 — The Predicate Calculus
- Chapter 3 — Structures and Strategies for State Space Search
- Chapter 4 — Heuristic Search
- Chapter 5 — Stochastic Methods
- Chapter 6 — Control and Implementation of State Space Search
- Chapter 7 — Knowledge Representation
- Chapter 8 — Strong Method Problem Solving
- Appendix — Selected Textbook Diagram Extracts

Chapter 1: AI — History and Applications

Complete Study Notes — Based strictly on Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th ed., Chapter 1

1. WHY CHAPTER 1 IS THERE

Luger states the goal of Chapter 1 explicitly:
"Chapter 1 introduces artificial intelligence, beginning with a brief history of attempts to understand mind and intelligence in philosophy, psychology, and other areas of research. In an important sense, AI is an old science, tracing its roots back at least to Aristotle. An appreciation of this background is essential for an understanding of the issues addressed in modern research. We also present an overview of some of the important application areas in AI. Our goal in Chapter 1 is to provide both background and a motivation for the theory and applications that follow."
The chapter therefore has three purposes:
Purpose
What it provides
Historical foundation
AI's intellectual roots from Aristotle onward
Definitional grounding
An attempt to define AI, and why that is difficult
Application survey
An overview of the major AI subfields

Luger's initial definition:
"Artificial intelligence (AI) may be defined as the branch of computer science that is concerned with the automation of intelligent behavior."
Why Luger says this definition is appropriate: It emphasizes "our conviction that AI is a part of computer science and, as such, must be based on sound theoretical and applied principles of that field." These principles include:
Principle
Detail
Data structures
Used in knowledge representation
Algorithms
Needed to apply that knowledge
Languages and programming techniques
Used in their implementation

Why the definition suffers:

"However, this definition suffers from the fact that intelligence itself is not very well defined or understood. Although most of us are certain that we know intelligent behavior when we see it, it is doubtful that anyone could come close to defining intelligence in a way that would be specific enough to help in the evaluation of a supposedly intelligent computer program, while still capturing the vitality and complexity of the human mind."
The questions the definition raises (Luger lists them directly):
Question
Is intelligence a single faculty, or is it just a name for a collection of distinct and unrelated abilities?
To what extent is intelligence learned as opposed to having an a priori existence?
Exactly what does happen when learning occurs?
What is creativity?
What is intuition?
Can intelligence be inferred from observable behavior, or does it require evidence of a particular internal mechanism?
How is knowledge represented in the nerve tissue of a living being, and what lessons does this have for the design of intelligent machines?
What is self-awareness; what role does it play in intelligence?
Is it necessary to pattern an intelligent computer program after what is known about human intelligence, or is a strict "engineering" approach sufficient?
Is it even possible to achieve intelligence on a computer, or does an intelligent entity require the richness of sensation and experience that might be found only in a biological existence?
Luger's observation on these questions:

"These are unanswered questions, and all of them have helped to shape the problems and solution methodologies that constitute the core of modern AI. In fact, part of the appeal of artificial intelligence is that it offers a unique and powerful tool for exploring exactly these questions. AI offers a medium and a test-bed for theories of intelligence: such theories may be stated in the language of computer programs and consequently tested and verified through the execution of these programs on an actual computer."
Luger's working definition:
"For the time being, we will simply define it as the collection of problems and methodologies studied by artificial intelligence researchers. This definition may seem silly and meaningless, but it makes an important point: artificial intelligence, like every science, is a human endeavor, and perhaps, is best understood in that context."

3. FROM EDEN TO ENIAC: ATTITUDES TOWARD INTELLIGENCE, KNOWLEDGE, AND HUMAN ARTIFICE

3.1 The Promethean Myth

Luger opens with Aeschylus's *Prometheus Bound*:
"Prometheus speaks of the fruits of his transgression against the gods of Olympus: his purpose was not merely to steal fire for the human race but also to enlighten humanity through the gift of intelligence or nous: the rational mind. This intelligence forms the foundation for all of human technology and ultimately all human civilization."
Zeus's punishment:
"Though Prometheus's action freed humanity from the sickness of ignorance, it also earned him the wrath of Zeus. Outraged over this theft of knowledge that previously belonged only to the gods of Olympus, Zeus commanded that Prometheus be chained to a barren rock to suffer the ravages of the elements for eternity."
Luger's interpretation:
"The notion that human efforts to gain knowledge constitute a transgression against the laws of God or nature is deeply ingrained in Western thought. It is the basis of the story of Eden and appears in the work of Dante and Milton. Both Shakespeare and the ancient Greek tragedians portrayed intellectual ambition as the cause of disaster. The belief that the desire for knowledge must ultimately lead to disaster has persisted throughout history, enduring the Renaissance, the Age of Enlightenment, and even the scientific and philosophical advances of the nineteenth and twentieth centuries. Thus, we should not be surprised that artificial intelligence inspires so much controversy in both academic and popular circles."
Frankenstein:
"The legends of Prometheus, Eve, and Faustus have been retold in the language of technological society. In her introduction to Frankenstein, subtitled, interestingly enough, The Modern Prometheus, Mary Shelley writes…"
Luger quotes Shelley's introduction at length (the conversation between Byron and Shelley about galvanism and reanimation), then concludes:
"Mary Shelley shows us the extent to which scientific advances such as the work of Darwin and the discovery of electricity had convinced even nonscientists that the workings of nature were not divine secrets, but could be broken down and understood systematically. Frankenstein's monster is not the product of shamanistic incantations or unspeakable transactions with the underworld: it is assembled from separately 'manufactured' components and infused with the vital force of electricity. Although nineteenth-century science was inadequate to realize the goal of understanding and creating a fully intelligent agent, it affirmed the notion that the mysteries of life and intellect might be brought into the light of scientific analysis."

3.2 A Brief History of the Foundations for AI

Luger's logical starting point is Aristotle:
"The logical starting point for such a history is the genius of Aristotle, or as Dante in the Divine Comedy refers to him, 'the master of them that know'."
Aristotle's three contributions:
a) Matter and Form (Physics)

"For Aristotle, the most fascinating aspect of nature was change. In his Physics, he defined his 'philosophy of nature' as the 'study of things that change'. He distinguished between the matter and form of things: a sculpture is fashioned from the material bronze and has the form of a human. Change occurs when the bronze is molded to a new form. The matter/form distinction provides a philosophical basis for modern notions such as symbolic computing and data abstraction. In computing (even with numbers) we are manipulating patterns that are the forms of electromagnetic material, with the changes of form of this material representing aspects of the solution process. Abstracting the form from the medium of its representation not only allows these forms to be manipulated computationally but also provides the promise of a theory of data structures, the heart of modern computer science. It also supports the creation of an 'artificial' intelligence."
b) Epistemology and Logic
"In his Metaphysics, beginning with the words 'All men by nature desire to know', Aristotle developed a science of things that never change, including his cosmology and theology. More relevant to artificial intelligence, however, was Aristotle's epistemology or analysis of how humans 'know' their world, discussed in his Logic. Aristotle referred to logic as the 'instrument' (organon), because he felt that the study of thought itself was at the basis of all knowledge. In his Logic, he investigated whether certain propositions can be said to be 'true' because they are related to other things that are known to be 'true'. Thus if we know that 'all men are mortal' and that 'Socrates is a man', then we can conclude that 'Socrates is mortal'. This argument is an example of what Aristotle referred to as a syllogism using the deductive form modus ponens. Although the formal axiomatization of reasoning needed another two thousand years for its full flowering in the works of Gottlob Frege, Bertrand Russell, Kurt Godel, Alan Turing, Alfred Tarski, and others, its roots may be traced to Aristotle."
The Renaissance and the Copernican revolution:
"Renaissance thought, building on the Greek tradition, initiated the evolution of a different and powerful way of thinking about humanity and its relation to the natural world. Science began to replace mysticism as a means of understanding nature. Clocks and, eventually, factory schedules superseded the rhythms of nature for thousands of city dwellers. Most of the modern social and physical sciences found their origin in the notion that processes, whether natural or artificial, could be mathematically analyzed and understood. In particular, scientists and philosophers realized that thought itself, the way that knowledge was represented and manipulated in the human mind, was a difficult but essential subject for scientific study."
"Perhaps the major event in the development of the modern world view was the Copernican revolution, the replacement of the ancient Earth-centered model of the universe with the idea that the Earth and other planets are

actually in orbits around the sun. After centuries of an 'obvious' order, in which the scientific explanation of the nature of the cosmos was consistent with the teachings of religion and common sense, a drastically different and not at all obvious model was proposed to explain the motions of heavenly bodies. For perhaps the first time, our ideas about the world were seen as fundamentally distinct from that world's appearance. This split between the human mind and its surrounding reality, between ideas about things and things themselves, is essential to the modern study of the mind and its organization. This breach was widened by the writings of Galileo, whose scientific observations further contradicted the 'obvious' truths about the natural world and whose development of mathematics as a tool for describing that world emphasized the distinction between the world and our ideas about it. It is out of this breach that the modern notion of the mind evolved: introspection became a common motif in literature, philosophers began to study epistemology and mathematics, and the systematic application of the scientific method rivaled the senses as tools for understanding the world."

3.3 Francis Bacon (1620) — Novum Organum

"In 1620, Francis Bacon's Novum Organum offered a set of search techniques for this emerging scientific methodology. Based on the Aristotelian and Platonic idea that the 'form' of an entity was equivalent to the sum of its necessary and sufficient 'features', Bacon articulated an algorithm for determining the essence of an entity."

Bacon's algorithm:

"First, he made an organized collection of all instances of the entity, enumerating the features of each in a table. Then he collected a similar list of negative instances of the entity, focusing especially on near instances of the entity, that is, those that deviated from the 'form' of the entity by single features. Then Bacon attempts - this step is not totally clear - to make a systematic list of all the features essential to the entity, that is, those that are common to all positive instances of the entity and missing from the negative instances."

AI relevance:

"It is interesting to see a form of Francis Bacon's approach to concept learning reflected in modern AI algorithms for Version Space Search, Chapter 10.2. An extension of Bacon's algorithms was also part of an AI program for discovery learning, suitably called Bacon (Langley et al. 1981). This program was able to induce many physical laws from collections of data related to the phenomena. It is also interesting to note that the question of whether a general purpose algorithm was possible for producing scientific proofs awaited the challenges of the early twentieth century mathematician Hilbert (his Entscheidungsproblem) and the response of the modern genius of Alan Turing (his Turing Machine and proofs of computability and the halting problem); see Davis et al. (1976)."

3.4 Early Calculating Machines

"Although the first calculating machine, the abacus, was created by the Chinese in the twenty-sixth century BC, further mechanization of algebraic processes awaited the skills of the seventeenth century Europeans."

Inventor
Year
Machine
Luger's description
Chinese
26th c. BC
Abacus
First calculating machine
John Napier
1614
Logarithms, Napier's Bones
"created logarithms, the mathematical transformations that allowed multiplication and the use of exponents to be reduced to addition and multiplication. Napier also created his bones that were used to represent overflow values for arithmetic operations"
Wilhelm Schickard
1623
Calculating Clock
"a German mathematician and clergyman of Tubingen, who in 1623 invented a Calculating Clock for performing addition and subtraction. This machine recorded the overflow from its calculations by the chiming of a clock"
Blaise Pascal
1642
Pascaline
"Although the mechanisms of Schickard and Pascal were limited to addition and subtraction - including carries and borrows - they showed that processes that previously were thought to require human thought and skill could be fully automated"
Leibniz
1694
Leibniz Wheel
(Luger mentions Leibniz's contribution in the formal logic section)

Pascal's insight:

"As Pascal later stated in his Pensees (1670), 'The arithmetical machine produces effects which approach nearer to thought than all the actions of animals'."

3.5 René Descartes (1596–1650) — Mind–Body Dualism

Luger presents Descartes through two consequences:

"1. By attempting to separate the mind from the physical world, Descartes and related thinkers established that the structure of ideas about the world was not necessarily the same as the structure of their subject matter. This underlies the methodology of AI, along with the fields of epistemology, psychology, much of higher mathematics, and most of modern literature: mental processes have an existence of their own, obey their own laws, and can be studied in and of themselves.

Once the mind and the body are separated, philosophers found it necessary to find a way to reconnect the two, because interaction between Descartes mental, res cogitans, and physical, res extensa, is essential for human existence.

Although millions of words have been written on this mind-body problem, and numerous solutions proposed, no one has successfully explained the obvious interactions between mental states and physical actions while affirming a fundamental difference."

Note: The text provided jumps from page 32 to page 33, omitting the detailed account of Descartes' method ("Cogito ergo sum", the rejection of sensory input, the inference of God's existence). The summary above (the two consequences) is what the text contains on pages 32–33.

3.6 AI and the Rationalist and Empiricist Traditions

Rationalist tradition:

"The rationalist tradition, as seen in the previous section, had an early proponent in Plato, and was continued on through the writings of Pascal, Descartes, and Liebniz. For the rationalist, the external world is reconstructed through the clear and distinct ideas of a mathematics. A criticism of this dualistic approach is the forced disengagement of representational systems from their field of reference. The issue is whether the meaning attributed to a representation can be defined independent of its application conditions. If the world is different from our beliefs about the world, can our created concepts and symbols still have meaning?"

"Many AI programs have very much of this rationalist flavor. Early robot planners, for example, would describe their application domain or 'world' as sets of predicate calculus statements and then a 'plan' for action would be created through proving theorems about this 'world' (Fikes et al. 1972, see also Section 8.4). Newell and Simon's Physical Symbol System Hypothesis (Introduction to Part II and Chapter 16) is seen by many as the archetype of this approach in modern AI. Several critics have commented on this rationalist bias as part of the failure of AI at solving complex tasks such as understanding human languages (Searle 1980, Winograd and Flores 1986, Brooks 1991a)."

Empiricist tradition:

"Rather than affirming as 'real' the world of clear and distinct ideas, empiricists continue to remind us that 'nothing enters the mind except through the senses'. This constraint leads to further questions of how the human can possibly perceive general concepts or the pure forms of Plato's cave (Plato 1961). Aristotle was an early empiricist, emphasizing in his De Anima, the limitations of the human perceptual system. More modern empiricists, especially Hobbes, Locke, and Hume, emphasize that knowledge must be explained through an introspective but empirical psychology. They distinguish two types of mental phenomena perceptions on one hand and thought, memory, and imagination on the other. The Scots philosopher, David Hume, for example, distinguishes between impressions and ideas. Impressions are lively and vivid, reflecting the presence and existence of an external object and not subject to voluntary control, the qualia of Dennett (2005). Ideas on the other hand, are less vivid and detailed and more subject to the subject's voluntary control."

Associationism:

"Given this distinction between impressions and ideas, how can knowledge arise? For Hobbes, Locke, and Hume the fundamental explanatory mechanism is association. Particular perceptual properties are associated through repeated experience. This repeated association creates a disposition in the mind to associate the corresponding ideas, a precursor of the behaviorist approach of the twentieth century. A fundamental property of this account is presented with Hume's skepticism. Hume's purely descriptive account of the origins of ideas cannot, he claims, support belief in causality. Even the use of logic and induction cannot be rationally supported in this radical empiricist epistemology."

Hume on miracles and Bayes:

"In An Inquiry Concerning Human Understanding (1748), Hume's skepticism extended to the analysis of miracles. Although Hume didn't address the nature of miracles directly, he did question the testimony-based belief in the miraculous. This skepticism, of course, was seen as a direct threat by believers in the bible as well as many other purveyors of religious traditions. The Reverend Thomas Bayes was both a mathematician and a minister. One of his papers, called Essay towards Solving a Problem in the Doctrine of Chances (1763) addressed Hume's questions mathematically. Bayes' theorem demonstrates formally how, through learning the correlations of the effects of actions, we can determine the probability of their causes."

AI manifestations of empiricism:

"The associational account of knowledge plays a significant role in the development of AI representational structures and programs, for example, in memory organization with semantic networks and MOPS and work in natural language understanding (see Sections 7.0, 7.1, and Chapter 15). Associational accounts have important influences of machine learning, especially with connectionist networks (see Section 10.6, 10.7, and Chapter 11). Associationism also plays an important role in cognitive psychology including the schemas of Bartlett and Piaget as well as the entire thrust of the behaviorist tradition (Luger 1994). Finally, with AI tools for stochastic analysis, including the Bayesian belief network (BBN) and its current extensions to first-order Turing-complete systems for stochastic modeling, associational theories have found a sound mathematical basis and mature expressive power. Bayesian tools are important for research including diagnostics, machine learning, and natural language understanding (see Chapters 5 and 13)."

Kant's synthesis:

"Immanuel Kant, a German philosopher trained in the rationalist tradition, was strongly influenced by the writing of Hume. As a result, he began the modern synthesis of these two traditions. Knowledge for Kant contains two collaborating energies, an a priori component coming from the subject's reason along with an a posteriori component coming from active experience. Experience is meaningful only through the contribution of the subject. Without an active organizing form proposed by the subject, the world would be nothing more than passing transitory sensations. Finally, at the level of judgement, Kant claims, passing images or representations are bound together by the active subject and taken as the diverse appearances of an identity, of an 'object'. Kant's realism began the modern enterprise of psychologists such as Bartlett, Brunner, and

Piaget. Kant's work influences the modern AI enterprise of machine learning (Section IV) as well as the continuing development of a constructivist epistemology (see Chapter 16)."

3.7 The Development of Formal Logic

"Once thinking had come to be regarded as a form of computation, its formalization and eventual mechanization were obvious next steps."

Leibniz:

"Gottfried Wilhelm von Leibniz, with his Calculus Philosophicus, introduced the first system of formal logic as well as proposed a machine for automating its tasks (Leibniz 1887). Furthermore, the steps and stages of this mechanical solution can be represented as movement through the states of a tree or graph."

Euler:

"Leonhard Euler, in the eighteenth century, with his analysis of the 'connectedness' of the bridges joining the riverbanks and islands of the city of Königsberg (see the introduction to Chapter 3), introduced the study of representations that can abstractly capture the structure of relationships in the world as well as the discrete steps within a computation about these relationships (Euler 1735)."

"The formalization of graph theory also afforded the possibility of state space search, a major conceptual tool of artificial intelligence. We can use graphs to model the deeper structure of a problem. The nodes of a state space graph represent possible stages of a problem solution; the arcs of the graph represent inferences, moves in a game, or other steps in a problem solution. Solving the problem is a process of searching the state space graph for a path to a solution (Introduction to II and Chapter 3). By describing the entire space of problem solutions, state space graphs provide a powerful tool for measuring the structure and complexity of problems and analyzing the efficiency, correctness, and generality of solution strategies."

Babbage:

"As one of the originators of the science of operations research, as well as the designer of the first programmable mechanical computing machines, Charles Babbage, a nineteenth century mathematician, may also be considered an early practitioner of artificial intelligence (Morrison and Morrison 1961). Babbage's difference engine was a special-purpose machine for computing the values of certain polynomial functions and was the forerunner of his analytical engine. The analytical engine, designed but not successfully constructed during his lifetime, was a general-purpose programmable computing machine that presaged many of the architectural assumptions underlying the modern computer."

Ada Lovelace:

"In describing the analytical engine, Ada Lovelace (1961), Babbage's friend, supporter, and collaborator, said: 'We may say most aptly that the Analytical Engine weaves algebraical patterns just as the Jacquard loom weaves flowers and leaves. Here, it seems to us, resides much more of originality than the difference engine can be fairly entitled to claim.'"

"Babbage's inspiration was his desire to apply the technology of his day to liberate humans from the drudgery of making arithmetic calculations. In this sentiment, as well as with his conception of computers as mechanical devices, Babbage was thinking in purely nineteenth century terms. His analytical engine, however, also included many modern notions, such as the separation of memory and processor, the store and the mill in Babbage's terms, the concept of a digital rather than analog machine, and programmability based on the execution of a series of operations encoded on punched pasteboard cards. The most striking feature of Ada Lovelace's description, and of Babbage's work in general, is its treatment of the 'patterns' of algebraic relationships as entities that may be studied, characterized, and finally implemented and manipulated mechanically without concern for the particular values that are finally passed through the mill of the calculating machine. This is an example implementation of the 'abstraction and manipulation of form' first described by Aristotle and Liebniz."

Boole:

"The goal of creating a formal language for thought also appears in the work of George Boole, another nineteenth-century mathematician whose work must be included in any discussion of the roots of artificial intelligence (Boole 1847, 1854). Although he made contributions to a number of areas of mathematics, his best known work was in the mathematical formalization of the laws of logic, an accomplishment that forms the very heart of modern computer science. Though the role of Boolean algebra in the design of logic circuitry is well known, Boole's own goals in developing his system seem closer to those of contemporary AI researchers."

Luger quotes Boole's stated goals (1854):

"to investigate the fundamental laws of those operations of the mind by which reasoning is performed: to give expression to them in the symbolical language of a Calculus, and upon this foundation to establish the science of logic and instruct its method; …and finally to collect from the various elements of truth brought to view in the course of these inquiries some probable intimations concerning the nature and constitution of the human mind."

"The importance of Boole's accomplishment is in the extraordinary power and simplicity of the system he devised: three operations, 'AND' (denoted by * or ∧), 'OR' (denoted by + or ∨), and 'NOT' (denoted by ¬), formed the heart of his logical calculus. These operations have remained the basis for all subsequent developments in formal logic, including the design of modern computers. While keeping the meaning of these symbols nearly identical to the corresponding algebraic operations, Boole noted that 'the Symbols of logic are further subject to a special law, to which the symbols of quantity, as such, are not subject'. This law states that for any X, an element in the algebra, X*X = X (or that once something is known to be true, repetition cannot augment that knowledge). This led to the characteristic restriction of Boolean values to the only two numbers that may satisfy this equation: 1 and 0. The standard definitions of Boolean multiplication (AND) and addition (OR) follow from this insight."

"Boole's system not only provided the basis of binary arithmetic but also demonstrated that an extremely simple formal system was adequate to capture the full power of logic. This assumption and the system Boole developed to demonstrate it form the basis of all modern efforts to formalize logic, from Russell and Whitehead's Principia Mathematica (Whitehead and Russell 1950), through the work of Turing and Godel, up to modern automated reasoning systems."

Frege:

"Gottlob Frege, in his Foundations of Arithmetic (Frege 1879, 1884), created a mathematical specification language for describing the basis of arithmetic in a clear and precise fashion. With this language Frege formalized many of the issues first addressed by Aristotle's Logic. Frege's language, now called the first-order predicate calculus, offers a tool for describing the propositions and truth value assignments that make up the elements of mathematical reasoning and describes the axiomatic basis of 'meaning' for these expressions. The formal system of the predicate calculus, which includes predicate symbols, a theory of functions, and quantified variables, was intended to be a language for describing mathematics and its philosophical foundations. It also plays a fundamental role in creating a theory of representation for artificial intelligence (Chapter 2). The first-order predicate calculus offers the tools necessary for automating reasoning: a language for expressions, a theory for assumptions related to the meaning of expressions, and a logically sound calculus for inferring new true expressions."

Whitehead and Russell:

"Whitehead and Russell's (1950) work is particularly important to the foundations of AI, in that their stated goal was to derive the whole of mathematics through formal operations on a collection of axioms. Although many mathematical systems have been constructed from basic axioms, what is interesting is Russell and Whitehead's commitment to mathematics as a purely formal system. This meant that axioms and theorems would be treated solely as strings of characters: proofs would proceed solely through the application of well-defined rules for manipulating these strings. There would be no reliance on intuition or the meaning of theorems as a basis for proofs. Every step of a proof followed from the strict application of formal (syntactic) rules to either axioms or previously proven theorems, even where traditional proofs might regard such a step as 'obvious'. What 'meaning' the theorems and axioms of the system might have in relation to the world would be independent of their logical derivations. This treatment of mathematical reasoning in purely formal (and hence mechanical) terms provided an essential basis for its automation on physical computers. The logical syntax and formal rules of inference developed by Russell and Whitehead are still a basis for automatic theorem-proving systems, presented in Chapter 14, as well as for the theoretical foundations of artificial intelligence."

Tarski:

"Alfred Tarski is another mathematician whose work is essential to the foundations of AI. Tarski created a theory of reference wherein the well-formed formulae of Frege or Russell and Whitehead can be said to refer, in a precise fashion, to the physical world (Tarski 1944, 1956; see Chapter 2). This insight underlies most theories of formal semantics. In his paper The Semantic Conception of Truth and the Foundation of Semantics, Tarski describes his theory of reference and truth value relationships. Modern computer scientists, especially Scott, Strachey, Burstall (Burstall and Darlington 1977), and Plotkin have related this theory to programming languages and other specifications for computing."

The digital computer:

"Although in the eighteenth, nineteenth, and early twentieth centuries the formalization of science and mathematics created the intellectual prerequisite for the study of artificial intelligence, it was not until the twentieth century and the introduction of the digital computer that AI became a viable scientific discipline. By the end of the 1940s electronic digital computers had demonstrated their potential to provide the memory and processing power required by intelligent programs. It was now possible to implement formal reasoning systems on a computer and empirically test their sufficiency for exhibiting intelligence. An essential component of the science of artificial intelligence is this commitment to digital computers as the vehicle of choice for creating and testing theories of intelligence."

"Digital computers are not merely a vehicle for testing theories of intelligence. Their architecture also suggests a specific paradigm for such theories: intelligence is a form of information processing. The notion of search as a problem-solving methodology, for example, owes more to the sequential nature of computer operation than it does to any biological model of intelligence. Most AI programs represent knowledge in some formal language that is then manipulated by algorithms, honoring the separation of data and program fundamental to the von Neumann style of computing. Formal logic has emerged as an important representational tool for AI research, just as graph theory plays an indispensable role in the analysis of problem spaces as well as providing a basis for semantic networks and similar models of semantic meaning. These techniques and formalisms are discussed in detail throughout the body of this text; we mention them here to emphasize the symbiotic relationship between the digital computer and the theoretical underpinnings of artificial intelligence."

3.8 The Turing Test (1950)

"One of the earliest papers to address the question of machine intelligence specifically in relation to the modern digital computer was written in 1950 by the British mathematician Alan Turing. Computing Machinery and Intelligence (Turing 1950) remains timely in both its assessment of the arguments against the possibility of creating an intelligent computing machine and its answers to those arguments. Turing, known mainly for his contributions to the theory of computability, considered the question of whether or not a machine could actually be made to think. Noting that the fundamental ambiguities in the question itself (what is thinking? what is a machine?) precluded any rational answer, he proposed that the question of intelligence be replaced by a more clearly defined empirical test."

The test:

"The Turing test measures the performance of an allegedly intelligent machine against that of a human being, arguably the best and only standard for intelligent behavior. The test, which Turing called the imitation game, places the machine and a human counterpart in rooms apart from a second human being, referred to as the interrogator (Figure 1.1). The interrogator is not able to see or speak directly to either of them, does not know which entity is

actually the machine, and may communicate with them solely by use of a textual device such as a terminal. The interrogator is asked to distinguish the computer from the human being solely on the basis of their answers to questions asked over this device. If the interrogator cannot distinguish the machine from the human, then, Turing argues, the machine may be assumed to be intelligent."

"By isolating the interrogator from both the machine and the other human participant, the test ensures that the interrogator will not be biased by the appearance of the machine or any mechanical property of its voice. The interrogator is free, however, to ask any questions, no matter how devious or indirect, in an effort to uncover the computer's identity. For example, the interrogator may ask both subjects to perform a rather involved arithmetic calculation, assuming that the computer will be more likely to get it correct than the human; to counter this strategy, the computer will need to know when it should fail to get a correct answer to such problems in order to seem like a human. To discover the human's identity on the basis of emotional nature, the interrogator may ask both subjects to respond to a poem or work of art; this strategy will require that the computer have knowledge concerning the emotional makeup of human beings."

The three important features (Luger's numbered list):

"1. It attempts to give an objective notion of intelligence, i.e., the behavior of a known intelligent being in response to a particular set of questions. This provides a standard for determining intelligence that avoids the inevitable debates over its 'true' nature. 2. It prevents us from being sidetracked by such confusing and currently unanswerable questions as whether or not the computer uses the appropriate internal processes or whether or not the machine is actually conscious of its actions. 3. It eliminates any bias in favor of living organisms by forcing the interrogator to focus solely on the content of the answers to questions."

Evaluation of modern AI programs:

"Because of these advantages, the Turing test provides a basis for many of the schemes actually used to evaluate modern AI programs. A program that has potentially achieved intelligence in some area of expertise may be evaluated by comparing its performance on a given set of problems to that of a human expert. This evaluation technique is just a variation of the Turing test: a group of humans are asked to blindly compare the performance of a computer and a human being on a particular set of problems. As we will see, this methodology has become an essential tool in both the development and verification of modern expert systems."

Criticisms:

"The Turing test, in spite of its intuitive appeal, is vulnerable to a number of justifiable criticisms. One of the most important of these is aimed at its bias toward purely symbolic problem-solving tasks. It does not test abilities requiring perceptual skill or manual dexterity, even though these are important components of human intelligence. Conversely, it is sometimes suggested that the Turing test needlessly constrains machine intelligence to fit a human mold. Perhaps machine intelligence is simply different from human intelligence and trying to evaluate it in human terms is a fundamental mistake. Do we really wish a machine would do mathematics as slowly and inaccurately as a human? Shouldn't an intelligent machine capitalize on its own assets, such as a large, fast, reliable memory, rather than trying to emulate human cognition? In fact, a number of modern AI practitioners (e.g., Ford and Hayes 1995) see responding to the full challenge of Turing's test as a mistake and a major distraction to the more important work at hand: developing general theories to explain the mechanisms of intelligence in humans and machines and applying those theories to the development of tools to solve specific, practical problems. Although we agree with the Ford and Hayes concerns in the large, we still see Turing's test as an important component in the verification and validation of modern AI software."

Turing's two objections addressed:

"Two of the objections cited by Turing are worth considering further. Lady Lovelace's Objection, first stated by Ada Lovelace, argues that computers can only do as they are told and consequently cannot perform original (hence, intelligent) actions. This objection has become a reassuring if somewhat dubious part of contemporary technological folklore. Expert systems (Section 1.2.3 and Chapter 8), especially in the area of diagnostic reasoning, have reached conclusions unanticipated by their designers. Indeed, a number of researchers feel that human creativity can be expressed in a computer program."

"The other related objection, the Argument from Informality of Behavior, asserts the impossibility of creating a set of rules that will tell an individual exactly what to do under every possible set of circumstances. Certainly, the flexibility that enables a biological intelligence to respond to an almost infinite range of situations in a reasonable if not necessarily optimal fashion is a hallmark of intelligent behavior. While it is true that the control structure used in most traditional computer programs does not demonstrate great flexibility or originality, it is not true that all programs must be written in this fashion. Indeed, much of the work in AI over the past 25 years has been to develop programming languages and models such as production systems, object-based systems, neural network representations, and others discussed in this book that attempt to overcome this deficiency."

"Many modern AI programs consist of a collection of modular components, or rules of behavior, that do not execute in a rigid order but rather are invoked as needed in response to the structure of a particular problem instance. Pattern matchers allow general rules to apply over a range of instances. These systems have an extreme flexibility that enables relatively small programs to exhibit a vast range of possible behaviors in response to differing problems and situations."

Simon's Ant:

"Whether these systems can ultimately be made to exhibit the flexibility shown by a living organism is still the subject of much debate. Nobel laureate Herbert Simon has argued that much of the originality and variability of behavior shown by living creatures is due to the richness of their environment rather than the complexity of their own internal programs. In The Sciences of the Artificial, Simon (1981) describes an ant progressing circuitously along an uneven and cluttered stretch of ground. Although the ant's path seems quite complex, Simon argues that the ant's goal is very simple: to return to its colony as quickly as possible. The twists and turns in its path are caused by the obstacles it encounters on its way. Simon concludes that 'An ant, viewed as a behaving system, is quite simple. The apparent complexity of its behavior over time is largely a reflection of the complexity of the environment in which it finds itself.'"

"This idea, if ultimately proved to apply to organisms of higher intelligence as well as to such simple creatures as insects, constitutes a powerful argument that such systems are relatively simple and, consequently, comprehensible. It is interesting to note that if one applies this idea to humans, it becomes a strong argument for the importance of culture in the forming of intelligence. Rather than growing in the dark like mushrooms, intelligence seems to depend on an interaction with a suitably rich environment. Culture is just as important in creating humans as human beings are in creating culture. Rather than denigrating our intellects, this idea emphasizes the miraculous richness and coherence of the cultures that have formed out of the lives of separate human beings. In fact, the idea that intelligence emerges from the interactions of individual elements of a society is one of the insights supporting the approach to AI technology presented in the next section."

3.9 Biological and Social Models of Intelligence: Agents Theories

"So far, we have approached the problem of building intelligent machines from the viewpoint of mathematics, with the implicit belief of logical reasoning as paradigmatic of intelligence itself, as well as with a commitment to 'objective' foundations for logical reasoning. This way of looking at knowledge, language, and thought reflects the rationalist tradition of western philosophy, as it evolved through Plato, Galileo, Descartes, Leibniz, and many of the other philosophers discussed earlier in this chapter. It also reflects the underlying assumptions of the Turing test, particularly its emphasis on symbolic reasoning as a test of intelligence, and the belief that a straightforward comparison with human behavior was adequate to confirming machine intelligence."

Challenges to rationalism:

"The latter half of the twentieth century has, however, seen numerous challenges to rationalist philosophy. Various forms of philosophical relativism question the objective basis of language, science, society, and thought itself. Ludwig Wittgenstein's later philosophy (Wittgenstein 1953), has forced us to reconsider the basis on meaning in both natural and formal languages. The work of Godel (Nagel and Newman 1958) and Turing has cast doubt on the very foundations of mathematics itself. Post-modern thought has changed our understanding of meaning and value in the arts and society. Artificial intelligence has not been immune to these criticisms; indeed, the difficulties that AI has encountered in achieving its goals are often taken as evidence of the failure of the rationalist viewpoint (Winograd and Flores 1986, Lakoff and Johnson 1999, Dennett 2005)."

Wittgenstein and Husserl/Heidegger:

"Two philosophical traditions, that of Wittgenstein (1953) as well as that of Husserl (1970, 1972) and Heidegger (1962) are central to this reappraisal of the Western philosophical tradition. In his later work, Wittgenstein questioned many of the assumptions of the rationalist tradition, including the foundations of language, science, and knowledge. Human language was a major focus of Wittgenstein's analysis: he challenged the notion that language derived its meaning from any sort of objective foundation."

"For Wittgenstein, as well as the speech act theory developed by Austin (1962) and his followers (Grice 1975, Searle 1969), the meaning of any utterance depends on its being situated in a human, cultural context. Our understanding of the meaning of the word 'chair', for example, is dependent on having a physical body that conforms to a sitting posture and the cultural conventions for using chairs. When, for example, is a large, flat rock a chair? Why is it odd to refer to the throne of England as a chair? What is the difference between a human being's understanding of a chair and that of a dog or cat, incapable of sitting in the human sense? Based on his attacks on the foundations of meaning, Wittgenstein argued that we should view the use of language in terms of choices made and actions taken in a shifting cultural context. Wittgenstein even extended his criticisms to science and mathematics, arguing that they are just as much social constructs as is language use."

"Husserl (1970, 1972), the father of phenomenology, was committed to abstractions as rooted in the concrete Lebenswelt or life-world: a rationalist model was very much secondary to the concrete world that supported it. For Husserl, as well as for his student Heidegger (1962), and their proponent Merleau-Ponty (1962), intelligence was not knowing what was true, but rather knowing how to cope in a world that was constantly changing and evolving. Gadamer (1976) also contributed to this tradition. For the existentialist/phenomenologist, intelligence is seen as survival in the world, rather than as a set of logical propositions about the world (combined with some inferencing scheme)."

"Many authors, for example Dreyfus and Dreyfus (1985) and Winograd and Flores (1986), have drawn on Wittgenstein's and the Husserl/Heidegger work in their criticisms of AI. Although many AI practitioners continue developing the rational/logical agenda, also known as GOFAI, or Good Old Fashioned AI, a growing number of researchers in the field have incorporated these criticisms into new and exciting models of intelligence. In keeping with Wittgenstein's emphasis on the anthropological and cultural roots of knowledge, they have turned to social, sometimes referred to as agent-based or situated, models of intelligent behavior for their inspiration."

Alternative models:

"As an example of an alternative to a logic-based approach, research in connectionist learning (Section 1.2.9 and Chapter 11) de-emphasizes logic and the functioning of the rational mind in an effort to achieve intelligence by modeling the architecture of the physical brain. Neural models of intelligence emphasize the brain's ability to adapt to the world in which it is situated by modifying the relationships between individual neurons. Rather than representing knowledge in explicit logical sentences, they capture it implicitly, as a property of patterns of relationships."

"Another biologically based model of intelligence takes its inspiration from the processes by which entire species adapt to their surroundings. Work in artificial life and genetic algorithms (Chapter 12) applies the principles of biological evolution to the problems of finding solutions to difficult

problems. These programs do not solve problems by reasoning logically about them; rather, they spawn populations of competing candidate solutions and drive them to evolve ever better solutions through a process patterned after biological evolution: poor candidate solutions tend to die out, while those that show the promise for solving a problem survive and reproduce by constructing new solutions out of components of their successful parents."

Social systems as metaphor:

"Social systems provide another metaphor for intelligence in that they exhibit global behaviors that enable them to solve problems that would confound any of their individual members. For example, although no individual could accurately predict the number of loaves of bread to be consumed in New York City on a given day, the entire system of New York bakeries does an excellent job of keeping the city stocked with bread, and doing so with minimal waste. The stock market does an excellent job of setting the relative values of hundreds of companies, even though each individual investor has only limited knowledge of a few companies. A final example comes from modern science. Individuals located in universities, industry, or government environments focus on common problems. With conferences and journals as the main communication media, problems important to society at large are attacked and solved by individual agents working semi-independently, although progress in many instances is also driven by funding agencies."

"These examples share two themes: first, the view of intelligence as rooted in culture and society and, as a consequence, emergent. The second theme is that intelligence is reflected by the collective behaviors of large numbers of very simple interacting, semiautonomous individuals, or agents. Whether these agents are neural cells, individual members of a species, or a single person in a society, their interactions produce intelligence."

The five themes of the agent-oriented view (Luger's numbered list):

"1. Agents are autonomous or semi-autonomous. That is, each agent has certain responsibilities in problem solving with little or no knowledge of either what other agents do or how they do it. Each agent does its own independent piece of the problem solving and either produces a result itself (does something) or reports results back to others in the community (communicating agent). 2. Agents are 'situated.' Each agent is sensitive to its own surrounding environment and (usually) has no knowledge of the full domain of all agents. Thus, an agent's knowledge is limited to the tasks to hand: 'the-file-I'm-processing' or 'the-wall-next-to-me' with no knowledge of the total range of files or physical constraints in the problem solving task. 3. Agents are interactional. That is, they form a collection of individuals that cooperate on a particular task. In this sense they may be seen as a 'society' and, as with human society, knowledge, skills, and responsibilities, even when seen as collective, are distributed across the population of individuals. 4. The society of agents is structured. In most views of agent-oriented problem solving, each individual, although having its own unique environment and skill set, will coordinate with other agents in the overall problem solving. Thus, a final solution will not only be seen as collective, but also as cooperative. 5. Finally, the phenomenon of intelligence in this environment is 'emergent.' Although individual agents are seen as possessing sets of skills and responsibilities, the overall cooperative result can be viewed as greater than the sum of its individual contributors. Intelligence is seen as a phenomenon resident in and emerging from a society and not just a property of an individual agent."

Definition of an agent:

"Based on these observations, we define an agent as an element of a society that can perceive (often limited) aspects of its environment and affect that environment either directly or through cooperation with other agents."

Types of agents:

"Most intelligent solutions require a variety of agents. These include rote agents, that simply capture and communicate pieces of information, coordination agents that can support the interactions between other agents, search agents that can examine multiple pieces of information and return some chosen bit of it, learning agents that can examine collections of information and form concepts or generalizations, and decision agents that can both dispatch tasks and come to conclusions in the light of limited information and processing. Going back to an older definition of intelligence, agents can be seen as the mechanisms supporting decision making in the context of limited processing resources."

The three requisites:

"The main requisites for designing and building such a society are: 1. structures for the representation of information, 2. strategies for the search through alternative solutions, and 3. the creation of architectures that can support the interaction of agents."

Luger's caveat:

"Our preliminary discussion of the possibility of a theory of automated intelligence is in no way intended to overstate the progress made to date or minimize the work that lies ahead. As we emphasize throughout this book, it is important to be aware of our limitations and to be honest about our successes. For example, there have been only limited results with programs that in any interesting sense can be said to 'learn'. Our accomplishments in modeling the semantic complexities of a natural language such as English have also been very modest. Even fundamental issues such as organizing knowledge or fully managing the complexity and correctness of very large computer programs (such as large knowledge bases) require considerable further research. Knowledge-based systems, though they have achieved marketable engineering successes, still have many limitations in the quality and generality of their reasoning. These include their inability to perform commonsense reasoning or to exhibit knowledge of rudimentary physical reality, such as how things change over time."

4. OVERVIEW OF AI APPLICATION AREAS

Luger introduces this section:

"We now return to our goal of defining artificial intelligence through an examination of the ambitions and accomplishments of workers in the field. The two most fundamental concerns of AI researchers are knowledge representation and search."

"In this section we outline several of these major application areas and their contributions to artificial intelligence as a whole."

4.1 Game Playing

"Much of the early research in state space search was done using common board games such as checkers, chess, and the 15-puzzle. In addition to their inherent intellectual appeal, board games have certain properties that made them ideal subjects for research. Most games are played using a well-defined set of rules: this makes it easy to generate the search space and frees the researcher from many of the ambiguities and complexities inherent in less structured problems. The board configurations used in playing games are easily represented on a computer, requiring none of the complex formalisms needed to capture the semantic subtleties of more complex problem domains. As games are easily played, testing a game-playing program presents no financial or ethical burden. State space search, the paradigm underlying most game-playing research, is presented in Chapters 3 and 4."

"Games can generate extremely large search spaces. These are large and complex enough to require powerful techniques for determining what alternatives to explore in the problem space. These techniques are called heuristics and constitute a major area of AI research. A heuristic is a useful but potentially fallible problem-solving strategy, such as checking to make sure that an unresponsive appliance is plugged in before assuming that it is broken or to castle in order to try and protect your king from capture in a chess game. Much of what we commonly call intelligence seems to reside in the heuristics used by humans to solve problems."

"Because most of us have some experience with these simple games, it is possible to devise and test the effectiveness of our own heuristics. We do not need to find and consult an expert in some esoteric problem area such as medicine or mathematics (chess is an obvious exception to this rule). For these reasons, games provide a rich domain for the study of heuristic search. Chapter 4 introduces heuristics using these simple games. Game-playing programs, in spite of their simplicity, offer their own challenges, including an opponent whose moves may not be deterministically anticipated, Chapters 5 and 9, and the need to consider psychological as well as tactical factors in game strategy."

"Recent successes in computer-based game playing include world championships in backgammon and chess. It is also interesting to note that in 2007 the full state space for the game of checkers was mapped out, allowing it to be from the first move, deterministic!"

4.2 Automated Reasoning and Theorem Proving

"We could argue that automatic theorem proving is the oldest branch of artificial intelligence, tracing its roots back through Newell and Simon's Logic Theorist (Newell and Simon 1963a) and General Problem Solver (Newell and Simon 1963b), through Russell and Whitehead's efforts to treat all of mathematics as the purely formal derivation of theorems from basic axioms, to its origins in the writings of Babbage and Leibniz. In any case, it has certainly been one of the most fruitful branches of the field. Theorem-proving research was responsible for much of the early work in formalizing search algorithms and developing formal representation languages such as the predicate calculus (Chapter 2) and the logic programming language Prolog."

"Most of the appeal of automated theorem proving lies in the rigor and generality of logic. Because it is a formal system, logic lends itself to automation. A wide variety of problems can be attacked by representing the problem description and relevant background information as logical axioms and treating problem instances as theorems to be proved. This insight is the basis of work in automatic theorem proving and mathematical reasoning systems (Chapter 14)."

"Unfortunately, early efforts at writing theorem provers failed to develop a system that could consistently solve complicated problems. This was due to the ability of any reasonably complex logical system to generate an infinite number of provable theorems: without powerful techniques (heuristics) to guide their search, automated theorem provers proved large numbers of irrelevant theorems before stumbling onto the correct one. In response to this inefficiency, many argue that purely formal, syntactic methods of guiding search are inherently incapable of handling such a huge space and that the only alternative is to rely on the informal, ad hoc strategies that humans seem to use in solving problems. This is the approach underlying the development of expert systems (Chapter 8), and it has proved to be a fruitful one."

"Still, the appeal of reasoning based in formal mathematical logic is too strong to ignore. Many important problems such as the design and verification of logic circuits, verification of the correctness of computer programs, and control of complex systems seem to respond to such an approach. In addition, the theorem-proving community has enjoyed success in devising powerful solution heuristics that rely solely on an evaluation of the syntactic form of a logical expression, and as a result, reducing the complexity of the search space without resorting to the ad hoc techniques used by most human problem solvers."

"Another reason for the continued interest in automatic theorem provers is the realization that such a system does not have to be capable of independently solving extremely complex problems without human assistance. Many modern theorem provers function as intelligent assistants, letting humans perform the more demanding tasks of decomposing a large problem into subproblems and devising heuristics for searching the space of possible proofs. The theorem prover then performs the simpler but still demanding task of proving lemmas, verifying smaller conjectures, and completing the formal aspects of a proof outlined by its human associate (Boyer and Moore 1979, Bundy 1988, Veroff 1997, Veroff and Spinks 2006)."

4.3 Expert Systems

"One major insight gained from early work in problem solving was the importance of domain-specific knowledge. A doctor, for example, is not effective at diagnosing illness solely because she possesses some innate general problem-solving skill; she is effective because she knows a lot about medicine. Similarly, a geologist is effective at discovering mineral deposits because he is able to apply a good deal of theoretical and empirical knowledge about geology to the problem at hand. Expert knowledge is a combination of a theoretical understanding of the problem and a collection of heuristic problem-solving rules that experience has shown to be effective in the

domain. Expert systems are constructed by obtaining this knowledge from a human expert and coding it into a form that a computer may apply to similar problems."

"This reliance on the knowledge of a human domain expert for the system's problem solving strategies is a major feature of expert systems. Although some programs are written in which the designer is also the source of the domain knowledge, it is far more typical to see such programs growing out of a collaboration between a domain expert such as a doctor, chemist, geologist, or engineer and a separate artificial intelligence specialist. The domain expert provides the necessary knowledge of the problem domain through a general discussion of her problem-solving methods and by demonstrating those skills on a carefully chosen set of sample problems. The AI specialist, or knowledge engineer, as expert systems designers are often known, is responsible for implementing this knowledge in a program that is both effective and seemingly intelligent in its behavior. Once such a program has been written, it is necessary to refine its expertise through a process of giving it example problems to solve, letting the domain expert criticize its behavior, and making any required changes or modifications to the program's knowledge. This process is repeated until the program has achieved the desired level of performance."

Early expert systems (Luger's descriptions):

"One of the earliest systems to exploit domain-specific knowledge in problem solving was DENDRAL, developed at Stanford in the late 1960s (Lindsay et al. 1980). DENDRAL was designed to infer the structure of organic molecules from their chemical formulas and mass spectrographic information about the chemical bonds present in the molecules. Because organic molecules tend to be very large, the number of possible structures for these molecules tends to be huge. DENDRAL addresses the problem of this large search space by applying the heuristic knowledge of expert chemists to the structure elucidation problem. DENDRAL's methods proved remarkably effective, routinely finding the correct structure out of millions of possibilities after only a few trials. The approach has proved so successful that descendants extensions of DENDRAL are currently used in chemical and pharmaceutical laboratories throughout the world."

"Whereas DENDRAL was one of the first programs to effectively use domain-specific knowledge to achieve expert performance, MYCIN established the methodology of contemporary expert systems (Buchanan and Shortliffe 1984). MYCIN uses expert medical knowledge to diagnose and prescribe treatment for spinal meningitis and bacterial infections of the blood. MYCIN, developed at Stanford in the mid-1970s, was one of the first programs to address the problems of reasoning with uncertain or incomplete information. MYCIN provided clear and logical explanations of its reasoning, used a control structure appropriate to the specific problem domain, and identified criteria to reliably evaluate its performance. Many of the expert system development techniques currently in use were first developed in the MYCIN project (Chapter 8)."

"Other early expert systems include the PROSPECTOR program for determining the probable location and type of ore deposits based on geological information about a site (Duda et al. 1979a, 1979b), the INTERNIST program for performing diagnosis in the area of internal medicine, the Dipmeter Advisor for interpreting the results of oil well drilling logs (Smith and Baker 1983), and XCON for configuring VAX computers. XCON was developed in 1981, and at one time every VAX sold by Digital Equipment Corporation was configured by that software. Numerous other expert systems are currently solving problems in areas such as medicine, education, business, design, and science (Waterman 1986, Durkin 1994). See also current proceedings of the Inovative Applications of Artificial Intelligence (IAAI) Conferences."

The five current deficiencies (Luger's numbered list):

"1. Difficulty in capturing 'deep' knowledge of the problem domain. MYCIN, for example, lacks any real knowledge of human physiology. It does not know what blood does or the function of the spinal cord. Folklore has it that once, when selecting a drug for treatment of meningitis, MYCIN asked whether the patient was pregnant, even though it had been told that the patient was male. Whether this actually occurred or not, it does illustrate the potential narrowness of knowledge in expert systems. 2. Lack of robustness and flexibility. If humans are presented with a problem instance that they cannot solve immediately, they can generally return to an examination of first principles and come up with some strategy for attacking the problem. Expert systems generally lack this ability. 3. Inability to provide deep explanations. Because expert systems lack deep knowledge of their problem domains, their explanations are generally restricted to a description of the steps they took in finding a solution. For example, they often cannot tell 'why' a certain approach was taken. 4. Difficulties in verification. Though the correctness of any large computer system is difficult to prove, expert systems are particularly difficult to verify. This is a serious problem, as expert systems technology is being applied to critical applications such as air traffic control, nuclear reactor operations, and weapons systems. 5. Little learning from experience. Current expert systems are handcrafted; once the system is completed, its performance will not improve without further attention from its programmers, leading to doubts about the intelligence of such systems."

4.4 Natural Language Understanding and Semantics

"One of the long-standing goals of artificial intelligence is the creation of programs that are capable of understanding and generating human language. Not only does the ability to use and understand natural language seem to be a fundamental aspect of human intelligence, but also its successful automation would have an incredible impact on the usability and effectiveness of computers themselves. Much effort has been put into writing programs that understand natural language. Although these programs have achieved success within restricted contexts, systems that can use natural language with the flexibility and generality that characterize human speech are beyond current methodologies."

"Understanding natural language involves much more than parsing sentences into their individual parts of speech and looking those words up in a dictionary. Real understanding depends on extensive background knowledge about the domain of discourse and the idioms used in that domain as well as an ability to apply general contextual knowledge to resolve the omissions and ambiguities that are a normal part of human speech."

"Consider, for example, the difficulties in carrying on a conversation about baseball with an individual who understands English but knows nothing about the rules, players, or history of the game. Could this person possibly understand the meaning of the sentence: 'With none down in the top of the ninth and the go-ahead run at second, the manager called his relief from the bull pen'? Even though all of the words in the sentence may be individually understood, this sentence would be gibberish to even the most intelligent non-baseball fan."

"The task of collecting and organizing this background knowledge in such a way that it may be applied to language comprehension forms the major problem in automating natural language understanding. Responding to this need, researchers have developed many of the techniques for structuring semantic meaning used throughout artificial intelligence (Chapters 7 and 15)."

SHRDLU and the micro-world methodology:

"Because of the tremendous amounts of knowledge required for understanding natural language, most work is done in well-understood, specialized problem areas. One of the earliest programs to exploit this 'micro world' methodology was Winograd's SHRDLU, a natural language system that could 'converse' about a simple configuration of blocks of different shapes and colors (Winograd 1973). SHRDLU could answer queries such as 'what color block is on the blue cube?' as well as plan actions such as 'move the red pyramid onto the green brick'. Problems of this sort, involving the description and manipulation of simple arrangements of blocks, appeared with surprising frequency in early AI research and are known as 'blocks world' problems."

"In spite of SHRDLU's success in conversing about arrangements of blocks, its methods did not generalize from that domain. The representational techniques used in the program were too simple to capture the semantic organization of richer and more complex domains in a useful way. Much of the current work in natural language understanding is devoted to finding representational formalisms that are general enough to be used in a wide range of applications yet adapt themselves well to the specific structure of a given domain. A number of different techniques (many of which are extensions or modifications of semantic networks) are explored for this purpose and used in the development of programs that can understand natural language in constrained but interesting knowledge domains. Finally, in current research (Marcus 1980, Manning and Schutze 1999, Jurafsky and Martin 2009) stochastic models, describing how words and language structures 'occur' in use, are employed to characterize both syntax and semantics. Full computational understanding of language, however, remains beyond the current state of the art."

4.5 Modeling Human Performance

"Although much of the above discussion uses human intelligence as a reference point in considering artificial intelligence, it does not follow that programs should pattern themselves after the organization of the human mind. Indeed, many AI programs are engineered to solve some useful problem without regard for their similarities to human mental architecture. Even expert systems, while deriving much of their knowledge from human experts, do not really attempt to simulate human internal mental problem solving processes. If performance is the only criterion by which a system will be judged, there may be little reason to attempt to simulate human problem-solving methods; in fact, programs that take nonhuman approaches to solving problems (chess) are often more successful than their human counterparts. Still, the design of systems that explicitly model aspects of human performance is a fertile area of research in both AI and psychology."

"Human performance modeling, in addition to providing AI with much of its basic methodology, has proved to be a powerful tool for formulating and testing theories of human cognition. The problem-solving methodologies developed by computer scientists have given psychologists a new metaphor for exploring the human mind. Rather than casting theories of cognition in the vague language used in early research or abandoning the problem of describing the inner workings of the human mind entirely (as suggested by the behaviorists), many psychologists have adopted the language and theory of computer science to formulate models of human intelligence. Not only do these techniques provide a new vocabulary for describing human intelligence, but also computer implementations of these theories offer psychologists an opportunity to empirically test, critique, and refine their ideas (Luger 1994; See the Cognitive Science Society's journal and conferences). The relationship between artificial and human intelligence is summarized in Chapter 16."

4.6 Planning and Robotics

"Research in planning began as an effort to design robots that could perform their tasks with some degree of flexibility and responsiveness to the outside world. Briefly, planning assumes a robot that is capable of performing certain atomic actions. It attempts to find a sequence of those actions that will accomplish some higher-level task, such as moving across an obstacle-filled room."

"Planning is a difficult problem for a number of reasons, not the least of which is the size of the space of possible sequences of moves. Even an extremely simple robot is capable of generating a vast number of potential move sequences. Imagine, for example, a robot that can move forward, backward, right, or left, and consider how many different ways that robot can possibly move around a room. Assume also that there are obstacles in the room and that the robot must select a path that moves around them in some efficient fashion. Writing a program that can discover the best path under these circumstances, without being overwhelmed by the huge number of possibilities, requires sophisticated techniques for representing spatial knowledge and controlling search through possible environments."

Hierarchical problem decomposition:

"One method that human beings use in planning is hierarchical problem decomposition. If you are planning a trip from Albuquerque to London, you will generally treat the problems of arranging a flight, getting to the airport, making airline connections, and finding ground transportation in London separately, even

though they are all part of a bigger overall plan. Each of these may be further decomposed into smaller subproblems such as finding a map of the city, negotiating the subway system, and finding a decent pub. Not only does this approach effectively restrict the size of the space that must be searched, but also supports the saving of frequently used subplans for future use."

"While humans plan effortlessly, creating a computer program that can do the same is a difficult challenge. A seemingly simple task such as breaking a problem into independent subproblems actually requires sophisticated heuristics and extensive knowledge about the planning domain. Determining what subplans should be saved and how they may be generalized for future use is an equally difficult problem."

Robotics:

"A robot that blindly performs a sequence of actions without responding to changes in its environment or being able to detect and correct errors in its own plan could hardly be considered intelligent. A robot may not have adequate sensors to locate all obstacles in the way of a projected path. Such a robot must begin moving through the room based on what it has 'perceived' and correct its path as other obstacles are detected. Organizing plans in a fashion that allows response to changing environmental conditions is a major problem for planning (Lewis and Luger 2000, Thrun et al. 2007)."

"Finally, robotics was one of the research areas in AI that produced many of the insights supporting agent-oriented problem solving (Section 1.1.4). Frustrated by both the complexities of maintaining the large representational space as well as the design of search algorithms for traditional planning, researchers, including Agre and Chapman (1987), Brooks (1991a), Thrun et al. (2007), restated the problem in terms of the interaction of multiple semi-autonomous agents. Each agent was responsible for its own portion of the problem task and through their coordination the larger solution would emerge."

"Planning research now extends well beyond the domains of robotics, to include the coordination of any complex set of tasks and goals. Modern planners are applied to agents (Nilsson 1994) as well as to control of particle beam accelerators (Klein et al. 1999, 2000)."

4.7 Languages and Environments for AI

"Some of the most important by-products of artificial intelligence research have been advances in programming languages and software development environments. For a number of reasons, including the size of many AI application programs, the importance of a prototyping methodology, the tendency of search algorithms to generate huge spaces, and the difficulty of predicting the behavior of heuristically driven programs, AI programmers have been forced to develop a powerful set of programming methodologies."

"Programming environments include knowledge-structuring techniques such as object-oriented programming. High-level languages, such as Lisp and Prolog, which support modular development, help manage program size and complexity. Trace packages allow a programmer to reconstruct the execution of a complex algorithm and make it possible to unravel the complexities of heuristic search. Without such tools and techniques, it is doubtful that many significant AI systems could have been built."

"Many of these techniques are now standard tools for software engineering and have little relationship to the core of AI theory. Others, such as object-oriented programming, are of significant theoretical and practical interest. Finally, many AI algorithms are also now built in more traditional computing languages, such as C++ and Java."

"The languages developed for artificial intelligence programming are intimately bound to the theoretical structure of the field. We have built many of the representational structures presented in this book in Prolog, Lisp and Java and make them available in Luger…"

4.8 Machine Learning

"Learning has remained a challenging area for AI. The importance of learning, however, is beyond question, particularly as this ability is one of the most important components of intelligent behavior. An expert system may perform extensive and costly computations to solve a problem. Unlike a human being, however, if it is given the same or a similar problem a second time, it usually does not remember the solution. It performs the same sequence of computations again. This is true the second, third, fourth, and every time it solves that problem—hardly the behavior of an intelligent problem solver. The obvious solution to this problem is for programs to learn on their own, either from experience, analogy, examples, being 'told' what to do, or rewarded or punished depending on results."

Early learning programs (Luger's list):

"Although learning is a difficult area, there are several programs that suggest that it is not impossible. One early program is AM, the Automated Mathematician, designed to discover mathematical laws (Lenat 1977, 1982). Initially given the concepts and axioms of set theory, AM was able to induce such important mathematical concepts as cardinality, integer arithmetic, and many of the results of number theory. AM conjectured new theorems by modifying its current knowledge base and used heuristics to pursue the 'best' of a number of possible alternative theorems. More recently, Cotton et al. (2000) designed a program that automatically invents 'interesting' integer sequences."

"Early influential work also includes Winston's research on the induction of structural concepts such as 'arch' from a set of examples in the blocks world (Winston 1975a). The ID3 algorithm has proved successful in learning general patterns from examples (Quinlan 1986a). Meta-DENDRAL learns rules for interpreting mass spectrographic data in organic chemistry from examples of data on compounds of known structure. Teiresias, an intelligent 'front end' for expert systems, converts high-level advice into new rules for its knowledge base (Davis 1982). Hacker devises plans for performing blocks world manipulations through an iterative process of devising a plan, testing it, and correcting any flaws discovered in the candidate plan (Sussman 1975). Work in explanation-based learning has shown the effectiveness of prior knowledge in learning (Mitchell et al. 1986, DeJong and Mooney 1986). There are also now many important biological and sociological models of learning; we review these in the connectionist learning and emergent learning chapters."

"The success of machine learning programs suggests the existence of a set of general learning principles that will allow the construction of programs with the ability to learn in realistic domains. We present several approaches to learning in Section IV."

4.9 Alternative Representations: Neural Nets and Genetic Algorithms

"Most of the techniques presented in this AI book use explicitly represented knowledge and carefully designed search algorithms to implement intelligence. A very different approach seeks to build intelligent programs using models that parallel the structure of neurons in the human brain or the evolving patterns found in genetic algorithms and artificial life."

The neuron:

"A simple schematic of a neuron (Figure 1.2) consists of a cell body that has a number of branched protrusions, called dendrites, and a single branch called the axon. Dendrites receive signals from other neurons. When these combined impulses exceed a certain threshold, the neuron fires and an impulse, or spike, passes down the axon. Branches at the end of the axon form synapses with the dendrites of other neurons. The synapse is the point of contact between neurons; synapses may be either excitatory or inhibitory, either adding to the total of signals reaching the neuron or subtracting from that total."

"This description of a neuron is excessively simple, but it captures those features that are relevant to neural models of computation. In particular, each computational unit computes some function of its inputs and passes the result along to connected units in the network: the final results are produced by the parallel and distributed processing of this network of neural connections and threshold weights."

Why neural architectures are appealing:

"Neural architectures are appealing mechanisms for implementing intelligence for a number of reasons. Traditional AI programs can be brittle and overly sensitive to noise. Human intelligence is much more flexible and good at interpreting noisy input, such as a face in a darkened room or a conversation at a noisy party. Neural architectures, because they capture knowledge in a large number of fine-grained units distributed about a network, seem to have more potential for partially matching noisy and incomplete data."

Genetic algorithms:

"With genetic algorithms and artificial life we evolve new problem solutions from components of previous solutions. The genetic operators, such as crossover and mutation, much like their genetic equivalents in the natural world, work to produce, for each new generation, ever better potential problem solutions. Artificial life produces its new generation as a function of the 'quality' of its neighbors in previous generations."

"Both neural architectures and genetic algorithms provide a natural model for parallelism, because each neuron or segment of a solution is an independent unit. Hillis (1985) has commented on the fact that humans get faster at a task as they acquire more knowledge, while computers tend to slow down. This slowdown is due to the cost of sequentially searching a knowledge base; a massively parallel architecture like the human brain would not suffer from this problem. Finally, something is intrinsically appealing about approaching the problems of intelligence from a neural or genetic point of view. After all, the evolved brain achieves intelligence and it does so using a neural architecture. We present neural networks, genetic algorithms, and artificial life, in Chapters 10 and 11."

4.10 AI and Philosophy

"In Section 1.1 we presented the philosophical, mathematical, and sociological roots of artificial intelligence. It is important to realize that modern AI is not just a product of this rich intellectual tradition but also contributes to it."

"For example, the questions that Turing posed about intelligent programs reflect back on our understanding of intelligence itself. What is intelligence, and how is it described? What is the nature of knowledge? Can knowledge be represented? How does knowledge in an application area relate to problem-solving skill in that domain? How does knowing what is true, Aristotle's theoria, relate to knowing how to perform, his praxis?"

"Answers proposed to these questions make up an important part of what AI researchers and designers do. In the scientific sense, AI programs can be viewed as experiments. A design is made concrete in a program and the program is run as an experiment. The program designers observe the results and then redesign and rerun the experiment. In this manner we can determine whether our representations and algorithms are sufficient models of intelligent behavior. Newell and Simon (1976) proposed this approach to scientific understanding in their 1976 Turing Award lecture (Section VI). Newell and Simon (1976) also propose a stronger model for intelligence with their physical symbol system hypothesis: the necessary and sufficient condition for a physical system to exhibit intelligence is that it be a physical symbol system. We take up in Section VI what this hypothesis means in practice as well as how it has been criticized by many modern thinkers."

"A number of AI application areas also open up deep philosophical issues. In what sense can we say that a computer can understand natural language expressions? To produce or understand a language requires interpretation of symbols. It is not sufficient to be able to say that a string of symbols is well formed. A mechanism for understanding must be able to impute meaning or interpret symbols in context. What is meaning? What is interpretation? In what sense does interpretation require responsibility?"

"Similar philosophical issues emerge from many AI application areas, whether they be building expert systems to cooperate with human problem solvers, designing computer vision systems, or designing algorithms for machine learning. We look at many of these issues as they come up in the chapters of this book and address the general issue of relevance to philosophy again in Section VI."

5. ARTIFICIAL INTELLIGENCE — A SUMMARY (Section 1.3)

"We have attempted to define artificial intelligence through discussion of its major areas of research and application. This survey reveals a young and promising field of study whose primary concern is finding an effective way to understand and apply intelligent problem solving, planning, and communication skills to a wide range of practical problems. In spite of the variety of problems addressed in

artificial intelligence research, a number of important features emerge that seem common to all divisions of the field; these include:"

Luger's numbered list:

"1. The use of computers to do reasoning, pattern recognition, learning, or some other form of inference. 2. A focus on problems that do not respond to algorithmic solutions. This underlies the reliance on heuristic search as an AI problem-solving technique. 3. A concern with problem solving using inexact, missing, or poorly defined information and the use of representational formalisms that enable the programmer to compensate for these problems. 4. Reasoning about the significant qualitative features of a situation. 5. An attempt to deal with issues of semantic meaning as well as syntactic form. 6. Answers that are neither exact nor optimal, but are in some sense 'sufficient'. This is a result of the essential reliance on heuristic problem-solving methods in situations where optimal or exact results are either too expensive or not possible. 7. The use of large amounts of domain-specific knowledge in solving problems. This is the basis of expert systems. 8. The use of meta-level knowledge to effect more sophisticated control of problem solving strategies. Although this is a very difficult problem, addressed in relatively few current systems, it is emerging as an essential area of research."

"We hope that this introduction provides some feel for the overall structure and significance of the field of artificial intelligence. We also hope that the brief discussions of such technical issues as search and representation were not excessively cryptic and obscure; they are developed in proper detail throughout the remainder of the book, but included here to demonstrate their significance in the general organization of the field."

"As we mentioned in the discussion of agent-oriented problem solving, objects take on meaning through their relationships with other objects. This is equally true of the facts, theories, and techniques that constitute a field of scientific study. We have intended to give a sense of those interrelationships, so that when the separate technical themes of artificial intelligence are presented, they will find their place in a developing understanding of the overall substance and directions of the field. We are guided in this process by an observation made by Gregory Bateson (1979), the psychologist and systems theorist: 'Break the pattern which connects the items of learning and you necessarily destroy all quality.'"

6. EPILOGUE AND REFERENCES (Section 1.4)

Luger's own summary:

"The field of AI reflects some of the oldest concerns of Western civilization in the light of the modern computational model. The notions of rationality, representation, and reason are now under scrutiny as perhaps never before, because we workers in AI demand to understand them algorithmically! At the same time, the political, economic, and ethical situation of our species forces us to confront our responsibility for the effects of our artifices."

Luger's recommended sources (his own list):

Topic
Sources

Philosophy of AI
Mind Design (Haugeland 1997); *Artificial Intelligence: The Very Idea* (Haugeland 1985); *Brainstorms* (Dennett 1978); *Darwin's Dangerous Idea* (Dennett 1995a); *Sweet Dreams* (Dennett 2006)

Primary sources
Aristotle's *Physics*, *Metaphysics*, *Logic*; papers by Frege; writings of Babbage, Boole, Russell and Whitehead

Turing
Turing (1950); *On Computable Numbers, with an Application to the Entscheidungsproblem* (Turing 1937); *Alan Turing: The Enigma* (Hodges 1983)

Early learning
Selfridge's *Pandemonium* (1959)

Early AI papers
Webber and Nilsson (1981)

Critical perspectives
Computer Power and Human Reason (Weizenbaum 1976); *Understanding Computers and Cognition* (Winograd and Flores 1986)

Positive statement
The Sciences of the Artificial (Simon 1981)

AI applications
Handbook of Artificial Intelligence (Barr and Feigenbaum 1989); *Encyclopedia of Artificial Intelligence* (Shapiro 1992)

Natural language
Allen (1995); Winograd (1983); Schank and Colby (1973); Wilks (1972); Pinker (1994); Lakoff and Johnson (1999); Jurafsky and Martin (2009)

Human performance
Newell and Simon (1972); Pylyshyn (1984); Anderson (1978); Luger (1994); Luger et al. (2002); Doya et al. (2007)

Machine learning
Michalski et al. (1983, 1986); Kodratoff and Michalski (1990); *Journal of Artificial Intelligence*; *Journal of Machine Learning*

Society of mind
Minsky (1985); Ford et al. (1995b); Langton (1995)

7. EXERCISES (Section 1.5)

The twelve exercises exactly as Luger lists them:

"1. Create and justify your own definition of artificial intelligence. 2. Give several other examples of Aristotle's distinction between matter and form. Can you show how your examples might fit into a theory of abstraction? 3. Much traditional Western thought has dwelt on the mind-body relationship. Are the mind and body: a. distinct entities somehow interacting, or b. is mind an expression of 'physical processes', or c. is body just an illusion of the rational mind? Discuss your thoughts on the mind-body problem and its importance for a theory of artificial intelligence. 4. Criticize Turing's criteria for computer software being 'intelligent'. 5. Describe your own criteria for computer software to be considered 'intelligent'. 6. Although computing is a relatively new discipline, philosophers and mathematicians have been thinking about the issues involved in automating problem solving for thousands of years. What is your opinion of the relevance of these philosophical issues to the design of a device for intelligent problem solving? Justify your answer. 7. Given the differences between the architectures of modern computers and that of the human brain, what relevance does research into the physiological structure and function of biological systems have for the engineering of AI programs? Justify your answer. 8. Pick one problem area that you feel would justify the energy required to design an expert system solution. Spell the problem out in some detail. Based on your own intuition, which aspects of this solution would be most difficult to automate? 9. Add two more benefits for expert systems to those already listed in the text. Discuss these in terms of intellectual, social, or financial results. 10. Discuss why you think the problem of machines 'learning' is so difficult. 11. Discuss whether or not you think it is possible for a computer to understand and use a natural (human) language. 12. List and discuss two potentially negative effects on society of the development of artificial intelligence technologies."

8. QUICK-REFERENCE TABLE OF PERSONS AND CONTRIBUTIONS

Person / Event	Contribution (as stated by Luger)
Prometheus	Stole fire (knowledge/intelligence/nous) from the gods; punished by Zeus
Aristotle	Matter/form distinction; syllogism (modus ponens); epistemology; Metaphysics
Copernicus	Earth-centered → sun-centered model; split between mind and world
Galileo	Scientific observations contradicted “obvious” truths; mathematics as tool
Francis Bacon	*Novum Organum*; concept learning algorithm (positive instances, negative near instances, essential features)
Chinese	Abacus — first calculating machine
John Napier	Logarithms, Napier’s Bones
Wilhelm Schickard	Calculating Clock (addition/subtraction; chiming on overflow)
Blaise Pascal	Pascaline (addition/subtraction with carries and borrows)
Leibniz	Leibniz Wheel (multiplication/division); *Calculus Philosophicus* (first formal logic system)
Descartes	Mind–body dualism; structure of ideas ≠ structure of subject matter
Hobbes	“By ratiocination, I mean computation”
Plato	Early rationalist
Pascal	Rationalist
Locke	Empiricist
Hume	Impressions vs ideas; skepticism about causality
Bayes	Bayes’ theorem (1763) addressed Hume’s questions mathematically
Kant	Synthesis: a priori + a posteriori; experience meaningful through subject’s contribution
Euler	Graph theory (Königsberg bridges)
Babbage	Difference Engine; Analytical Engine (memory/processor separation, digital, punched cards)
Ada Lovelace	“Analytical Engine weaves algebraical patterns...”
Boole	Boolean algebra (AND, OR, NOT; X*X=X); binary arithmetic; logic circuitry
Frege	First-order predicate calculus
Whitehead & Russell	*Principia Mathematica* — mathematics as purely formal system
Tarski	Theory of reference; formal semantics
Turing	Turing Test (1950); *Computing Machinery and Intelligence*
Newell & Simon	Logic Theorist; General Problem Solver; Physical Symbol System Hypothesis
Simon	Simon’s Ant (complexity in environment, not ant)
Wittgenstein	Meaning depends on human, cultural context
Husserl	Phenomenology; *Lebenswelt* (life-world)
Heidegger	Intelligence as coping in a changing world
Merleau-Ponty	Proponent of Husserl/Heidegger
Dreyfus & Dreyfus	Critics of AI
Winograd & Flores	Critics of AI (rationalist bias)
Brooks	Critic of rationalist AI; agent-based approach
Searle	Critic of AI (rationalist bias)
Hillis	Observation: humans speed up with knowledge; computers slow down
Langley et al.	Bacon program (induces physical laws from data)
Lenat	AM (Automated Mathematician)
Quinlan	ID3 algorithm
Winston	Learning structural concepts (arch)
Sussman	Hacker (blocks world plans)
Mitchell et al.	Explanation-based learning
DeJong & Mooney	Explanation-based learning
Lindsay et al.	DENDRAL
Buchanan & Shortliffe	MYCIN
Duda et al.	PROSPECTOR
Smith & Baker	Dipmeter Advisor
XCON	VAX computer configuration (1981)
Waterman	Expert systems survey
Durkin	Expert systems survey
Marcus	Stochastic models in NLP
Manning & Schutze	Stochastic models in NLP
Jurafsky & Martin	Stochastic models in NLP
Winograd	SHRDLU
Newell & Simon (1976)	Turing Award lecture; AI as empirical enquiry
Bateson	“Break the pattern which connects the items of learning…”

9. WHAT IS NOT IN CHAPTER 1

Topic	Belongs to
Predicate calculus syntax and semantics	Chapter 2
State space search algorithms (BFS, DFS, backtracking)	Chapter 3
Heuristic search (f = g + h, A*, admissibility)	Chapter 4
Probability, Bayes’ theorem calculations	Chapter 5
Production systems, blackboard architecture	Chapter 6
Semantic networks, frames, scripts, conceptual graphs	Chapter 7
Expert system architecture (detail), forward/backward chaining	Chapter 8
Reasoning under uncertainty (certainty factors, fuzzy logic, Dempster-Shafer)	Chapter 9
Machine learning algorithms (version space, ID3, backpropagation, genetic algorithms)	Chapters 10–13
Resolution theorem proving	Chapter 14
Natural language parsing (ATN, Earley, stochastic)	Chapter 15
Constructivist epistemology, science of intelligent systems	Chapter 16

End of Chapter 1 notes, based strictly on Luger’s textbook.

Chapter 2: The Predicate Calculus — Complete Study Notes

1. WHY CHAPTER 2 EXISTS

Chapter 2 introduces the predicate calculus as the **representation language** for AI. It is the formal system that gives AI:
A **well-defined syntax** (what expressions are legal).
A **formal semantics** (what expressions mean).
Sound and complete inference rules (how to derive new truths from old).
Without a representation language, there is nothing to search through. Chapter 2 is the foundation for everything that follows — search (Ch 3–4), knowledge representation (Ch 7), expert systems (Ch 8), theorem proving (Ch 14), and Prolog (Ch 14).

2. THE PROPOSITIONAL CALCULUS (Section 2.1, optional)

2.1.1 Symbols and sentences

Symbols of the propositional calculus:

Symbol type	Symbols	Meaning
Propositional symbols	P, Q, R, S, … (uppercase, near end of alphabet)	Statements about the world that may be true or false
Truth symbols	true, false	Fixed truth values
Connectives	∧, ∨, ¬, →, ≡	Logical operators
Grouping	( ), [ ]	Control order of evaluation

#	Rule
1	Every propositional symbol and truth symbol is a sentence
2	The negation of a sentence is a sentence
3	The conjunction of two sentences is a sentence
4	The disjunction of two sentences is a sentence
5	The implication of one sentence from another is a sentence
6	The equivalence of two sentences is a sentence

Term	Meaning
Well-formed formula (WFF)	A legal sentence
Conjuncts	The two parts of P ∧ Q
Disjuncts	The two parts of P ∨ Q
Premise / antecedent	The P in P → Q
Conclusion / consequent	The Q in P → Q

Worked example — is ((P∧Q)→R) ≡ ¬P∨¬Q∨R well formed?

Step	Justification
1	P, Q, R are propositions → sentences
2	P ∧ Q is a conjunction of two sentences → sentence
3	(P ∧ Q) → R is an implication → sentence
4	¬P and ¬Q are negations → sentences
5	¬P ∨ ¬Q is a disjunction → sentence
6	¬P ∨ ¬Q ∨ R is a disjunction → sentence
7	((P∧Q)→R) ≡ ¬P∨¬Q∨R is an equivalence of two sentences → sentence

2.1.2 The semantics of the propositional calculus

Definition — Interpretation: An interpretation is a mapping from the propositional symbols into the set {T, F}.
Important distinction: The symbols true and false are *sentence symbols*. The symbols T and F are *truth value assignments*. They are different things.

#	Rule
1	¬P is F if P is T; T if P is F
2	P ∧ Q is T only when both conjuncts are T; otherwise F
3	P ∨ Q is F only when both disjuncts are F; otherwise T
4	P → Q is F only when P is T and Q is F; otherwise T
5	P ≡ Q is T only when both have the same value

P	Q	P ∧ Q
T	T	T
T	F	F
F	T	F
F	F	F

P	Q	¬P	¬P ∨ Q	P → Q
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Name	
Identity	P ≡ P
Double negation	¬¬P ≡ P
Material implication	(P → Q) ≡ (¬P ∨ Q)
Contrapositive law	(P → Q) ≡ (¬Q → ¬P)
De Morgan (OR)	¬(P ∨ Q) ≡ (¬P ∧ ¬Q)
De Morgan (AND)	¬(P ∧ Q) ≡ (¬P ∨ ¬Q)
Commutative (AND)	(P ∧ Q) ≡ (Q ∧ P)
Commutative (OR)	(P ∨ Q) ≡ (Q ∨ P)
Associative (AND)	((P ∧ Q) ∧ R) ≡ (P ∧ (Q ∧ R))
Associative (OR)	((P ∨ Q) ∨ R) ≡ (P ∨ (Q ∨ R))
Distributive (OR over AND)	P ∨ (Q ∧ R) ≡ (P ∨ Q) ∧ (P ∨ R)
Distributive (AND over OR)	P ∧ (Q ∨ R) ≡ (P ∧ Q) ∨ (P ∧ R)

2.

Operation	Boolean algebra notation	Logical notation
AND	* or ×	∧
OR	+	∨
NOT	¬	¬

Key law
X * X = X
P ∧ P ≡ P
Why X*X = X matters: Repetition does not change truth. Only 0 and 1 satisfy this equation, which is why Boolean values are restricted to {0, 1} — the basis of binary arithmetic and logic circuits.

Historical note: The Logic Theorist (Newell and Simon, 1956) used transformations between equivalent forms of expressions to prove many theorems in Whitehead and Russell's *Principia Mathematica*.

3. THE PREDICATE CALCULUS (Section 2.2)

Why predicate calculus is more expressive than propositional calculus:

Aspect	Propositional	Predicate
Atomic unit	Whole proposition P	Predicate + terms
Access to components	No	Yes
Variables	No	Yes
Quantifiers	No	Yes (∀, ∃)
Example	P = "it rained on Tuesday"	weather(tuesday, rain)

Allowed character	Example
Letters (upper and lowercase)	a, R, p_z, George
Digits	0–9
Underscore	_

Character	Why illegal
%	Not in alphabet
@	Not in alphabet
/	Not in alphabet
&	Not in alphabet
*	Not in alphabet
!	Not in alphabet
(space)	Not allowed inside symbols

Symbol	Type
George	Variable (uppercase)
fire3	Constant (lowercase)
tom_and_jerry	Constant
bill	Constant
XXXX	Variable
friends_of	Predicate or function

Symbol	Why illegal
3jack	Starts with digit
ab%cd	Contains %
***71	Contains *
duck!!!	Contains !
no blanks allowed	Contains spaces
geORGE	Mixed case — constants must be all lowercase
bill	Lowercase — cannot be a variable

Symbol	Purpose
( )	Grouping
,	Separates arguments
.	Sentence terminator

These do **not** denote objects or relations in the world.

The four symbol types:

Symbol type	Constants	Variables	Functions	Predicates
First character	Lowercase	Uppercase	Lowercase	Lowercase
Meaning	Name specific objects	Designate general classes of objects	Denote a mapping; have arity	Name a relationship; have arity
Example	george, tree, tall, blue	George, BILL, KATE, X	father, plus, price	likes, equals, on, near

Reserved truth symbols: true and false (these are the only lowercase constants that are reserved).

A

Term	Definition
Arity	The number of arguments of a function or predicate
Unary function	Arity 1: father(david)
Binary function	Arity 2: plus(2, 3)
Unary predicate	Arity 1: clear(X)
Binary predicate	Arity 2: likes(george, kate)

Important: Predicates with the same name but different arities are considered **distinct** relations.

F

Example	Meaning
f(X, Y)	Function f of two arguments
father(david)	The father of David
price(bananas)	The price of bananas
plus(2, 3)	2 + 3 (evaluates to 5)

Definition — symbols and

terms:

Symbol type
Definition
Truth symbols
true, false (reserved)
Constant symbols
Symbol expressions with first character lowercase
Variable symbols
Symbol expressions beginning with uppercase
Function symbols
Symbol expressions with first character lowercase; have attached arity
Function expression
Function constant of arity n followed by n terms in parentheses
Term
A constant, variable, or function expression
Examples of terms: cat, times(2,3), X, blue, mother(sarah), kate

Predicates and atomic sentences:

Term
Definition
Predicate symbol
Symbol beginning with lowercase
Arity
Number of objects related by the predicate
Atomic sentence
Predicate constant of arity n followed by n terms in parentheses
Atomic expression / atom / proposition
Another name for an atomic sentence

Examples of atomic sentences:

Sentence
Predicate
Arity
likes(george, kate)
likes
2
likes(X, george)
likes
2
likes(george, susie)
likes
2
likes(X, X)
likes
2
likes(george, sarah, tuesday)
likes
3 (different relation from the 2-ary likes)
friends(bill, richard)
friends
2
friends(bill, george)
friends
2
friends(father_of(david), father_of(andrew))
friends
2 (arguments are function expressions)
helps(bill, george)
helps
2
helps(richard, bill)
helps
2
Key point: likes(george, kate) and likes(george, sarah, tuesday) use the same predicate name but different arities → **two different relations**.

The seven predicate calculus sentence formation rules:

#
Rule
1
Every atomic sentence is a sentence
2
If S is a sentence, then so is its negation ¬S
3
If S_1 and S_2 are sentences, then so is $S_1 \wedge S_2$
4
If S_1 and S_2 are sentences, then so is $S_1 \vee S_2$
5
If S_1 and S_2 are sentences, then so is $S_1 \rightarrow S_2$
6
If S_1 and S_2 are sentences, then so is $S_1 \equiv S_2$
7
If X is a variable and S a sentence, then ∀X S and ∃X S are sentences

Examples of well-formed sentences:

Expression
Why it is a sentence
plus(two,three)
Not a sentence — it is a function expression
equal(plus(two,three), five)
Atomic sentence
equal(plus(2,3), seven)
Atomic sentence (false given standard interpretation)
∃X foo(X,two,plus(two,three)) ∧ equal(plus(two,three),five)
Sentence (both conjuncts are sentences)
(foo(two,two,plus(two,three))) → (equal(plus(three,two),five) ≡ true)
Sentence
Key point: Well-formedness and truth value are independent issues. equal(plus(2,3), seven) is well-formed but false.

The quantifiers:

Symbol
Name
Meaning
Example
∀
Universal quantifier
True for all values of the variable
∀X likes(X, ice_cream) = everyone likes ice cream
∃
Existential quantifier
True for at least one value of the variable
∃Y friends(Y, peter) = someone is a friend of Peter
A quantifier is followed by a variable and a sentence.

The verify_sentence algorithm:

```
function verify_sentence(expression);
begin
   case expression is an atomic sentence:
      return SUCCESS;
   expression is of the form Q X s, where Q is either ∀ or ∃, X is a variable:
      if verify_sentence(s) returns SUCCESS then return SUCCESS
      else return FAIL;
   expression is of the form ¬s:
      if verify_sentence(s) returns SUCCESS then return SUCCESS
      else return FAIL;
   expression is of the form s1 op s2, where op is a binary logical operator:
      if verify_sentence(s1) returns SUCCESS and verify_sentence(s2) returns SUCCESS
      then return SUCCESS
      else return FAIL;
   otherwise:
      return FAIL
end
end.
```

Worked example — family relationships:

Fact / rule
Meaning
mother(eve, abel)
Eve is mother of Abel
mother(eve, cain)
Eve is mother of Cain
father(adam, abel)
Adam is father of Abel
father(adam, cain)
Adam is father of Cain
∀X ∀Y father(X, Y) ∨ mother(X, Y) → parent(X, Y)
A parent is a father or mother
∀X ∀Y ∀Z parent(X, Y) ∧ parent(X, Z) → sibling(Y, Z)
Siblings share a parent
Purpose of this example: It shows how to define general relationships (parent, sibling) in terms of primitive predicates. It leads into the semantics and inference rules that follow.

3.2.2 A semantics for the predicate calculus

Definition — Interpretation over a domain D:

Let D be a nonempty set. An interpretation over D assigns:

#
Rule
1
Each constant is assigned an element of D
2
Each variable is assigned a nonempty subset of D (the allowable substitutions)
3
Each function f of arity m is defined on m arguments of D and maps $D^m \rightarrow D$
4

Each predicate p of arity n is defined on n arguments of D and maps $D^n \rightarrow \{T, F\}$

Calculus
Variables
Decidable?
Why
Propositional
None
Yes
Finitely many interpretations (2^n)
Predicate
Yes
No
Infinite domain → exhaustive testing impossible

Key knowledge about translation:

Aspect
Detail
No unique mapping
An English sentence may have many predicate calculus representations
The challenge
Optimise expressiveness and efficiency
Variables as placeholders
A variable stands for all constants that might appear in its position
Dummy variables
A variable name can be replaced by any other without changing meaning

Definition — Truth value of predicate calculus expressions:

#
Rule
1
The value of a constant is the element of D assigned by I
2
The value of a variable is the set of elements of D assigned by I
3
The value of a function expression is the element of D obtained by evaluating
4
The value of true is T; false is F
5
The value of an atomic sentence is T or F as determined by I
6
The value of ¬S is T if S is F, F if S is T
7
The value of $S_1 \wedge S_2$ is T if both are T, F otherwise
8
The value of $S_1 \vee S_2$ is F if both are F, T otherwise
9
The value of $S_1 \rightarrow S_2$ is F if S_1 is T and S_2 is F, T otherwise
10
The value of $S_1 \equiv S_2$ is T if both have the same value, F otherwise
11
∀X S is T if S is T for all assignments to X under I, F otherwise
12
∃X S is T if S is T for some assignment to X under I, F otherwise

Equivalence
Meaning
$\neg\exists X\ p(X) \equiv \forall X\ \neg p(X)$
“No X is p” = “All X are not p”
$\neg\forall X\ p(X) \equiv \exists X\ \neg p(X)$
“Not all X are p” = “Some X is not p”
$\exists X\ p(X) \equiv \exists Y\ p(Y)$
Dummy variable renaming
$\forall X\ q(X) \equiv \forall Y\ q(Y)$
Dummy variable renaming
$\forall X\ (p(X) \wedge q(X)) \equiv \forall X\ p(X) \wedge \forall Y\ q(Y)$
Distribution of ∀ over ∧
$\exists X\ (p(X) \vee q(X)) \equiv \exists X\ p(X) \vee \exists Y\ q(Y)$
Distribution of ∃ over ∨

3.2.3 A “blocks world” example of semantic meaning

The blocks world (Figure 2.3):
Element
Detail
Blocks
a, b, c, d, e
Configuration
c on a; b on d; a, d on table; b, c, e clear; hand empty

First-order predicate calculus:

Aspect
First-order
Higher-order
Quantifiers range over
Objects only
Predicates and functions too
Example
∀X likes(X, kate)
∀ (Likes) Likes(george, kate)
Status in AI
Standard representation
Used in some natural language systems (McCarthy 1968, Appelt 1985)

Th

Predicate
Meaning
on(c, a)
Block c is on block a
on(b, d)
Block b is on block d
ontable(a)
Block a is on the table
ontable(d)
Block d is on the table
clear(b)
Block b is clear (nothing on top)
clear(c)
Block c is clear
hand_empty
The robot hand is empty
clear(e)
Block e is clear
gripping( )
The robot hand is empty (another notation)

Quantification terminology:

Term
Definition
Free variable
A variable not within the scope of any quantifier
Closed expression
All variables are quantified
Ground expression
No variables at all
Dummy variable
A variable name that can be replaced by any other without changing meaning
Scope
The instances of a variable over which a quantifier holds

The English-to-predicate calculus translations (six standard examples):

English sentence
Predicate calculus form
If it doesn’t rain on Monday, Tom will go to the mountains
$\neg$weather(rain, monday) → go(tom, mountains)
Emma is a Doberman pinscher and a good dog
gooddog(emma) ∧ isa(emma, doberman)
All basketball players are tall
∀X (basketball_player(X) → tall(X))
Some people like anchovies
∃X (person(X) ∧ likes(X, anchovies))
If wishes were horses, beggars would ride
equal(wishes, horses) → ride(beggars)
Nobody likes taxes
$\neg\exists X$ likes(X, taxes)

The three rules:

Rule
Formal form
Meaning
Clear
$\forall X\ (\neg\exists Y\ on(Y, X) \rightarrow clear(X))$
X is clear if nothing is on X
Ontable
$\forall Y\ (ontable(Y) \rightarrow \neg\exists X\ on(Y, X))$
If Y is on the table, nothing is on Y
Gripping
$\forall Y\ (gripping( ) \leftrightarrow \neg gripping(Y))$
The hand is empty iff it is not gripping anything

Undecidability:

8.4) introduces the frame problem and non-monotonicity, which are addressed in later chapters.

4. USING INFERENCE RULES TO PRODUCE PREDICATE CALCULUS EXPRESSIONS (Section 2.3)

The four operators:

Operator	Preconditions	Add list	Delete list
pickup(X)	gripping( ) ∧ clear(X) ∧ ontable(X)	gripping(X)	ontable(X) ∧ gripping( )
putdown(X)	gripping(X)	ontable(X) ∧ gripping( ) ∧ clear(X)	gripping(X)
stack(X,Y)	clear(Y) ∧ gripping(X)	on(X,Y) ∧ gripping( ) ∧ clear(X)	clear(Y) ∧ gripping(X)
unstack(X,Y)	clear(X) ∧ gripping( ) ∧ on(X,Y)	gripping(X) ∧ clear(Y)	gripping( ) ∧ on(X,Y)

The blocks world:

Predicate	Change
ontable(a)	Unchanged
on(b, a)	Unchanged
clear(b)	Unchanged
ontable(c)	Unchanged
clear(c)	Unchanged
clear(d)	**New** (e removed from d)
ontable(d)	Unchanged
gripping(e)	**New** (e in hand)
clear(e)	**New** (e clear)

Concepts introduced by the blocks world:

Concept	Meaning
Declarative	No assumed timing or order for considering expressions
Procedural semantics	A specified methodology for evaluating expressions over time (e.g., Prolog)
Frame problem	The issue of representing what does **not** change when an action occurs
Non-monotonicity	Interpretations that may change as new facts are added

Important note: The blocks world is a **static snapshot**. Adding procedural semantics (Ch

4.3.1 Inference rules

Key definitions:

Term	Definition
Satisfy	An interpretation I satisfies X if X is T under I and a particular variable assignment
Model	An interpretation I is a model of X if I satisfies X for all variable assignments
Satisfiable	There exists an interpretation and variable assignment that satisfy X
Unsatisfiable / inconsistent	No interpretation or variable assignment satisfies X
Valid	X is T for all possible interpretations
Logically follows	Every interpretation satisfying S also satisfies X
Sound	Every expression produced by the rule logically follows from S
Complete	The rule can infer every expression that logically follows from S
Proof procedure	An inference rule plus an algorithm for applying it

Examples of inconsistency and validity:

Expression	Status	Why
∃X (p(X) ∧ ¬p(X))	**Inconsistent**	Cannot be satisfied under any interpretation
∀X (p(X) ∨ ¬p(X))	**Valid**	True for all interpretations

The five inference rules:

Rule	Form	Meaning
Modus ponens	P, P → Q ⊢ Q	If P and P implies Q, infer Q
Modus tollens	P → Q, ¬Q ⊢ ¬P	If P implies Q and not Q, infer not P
And elimination	P ∧ Q ⊢ P (and Q)	From a conjunction, infer either conjunct
And introduction	P, Q ⊢ P ∧ Q	From both, infer the conjunction
Universal instantiation	∀X p(X) ⊢ p(a)	Substitute a domain term for a universally quantified variable

Th

Step	Expression
1	If it is raining, then the ground is wet: P → Q
2	It is raining: P
3	By modus ponens: Q (the ground is wet)

The Socrates syllogism (the standard worked example):

Step	Expression	Justification
1	∀X (man(X) → mortal(X))	All men are mortal
2	man(socrates)	Socrates is a man
3	man(socrates) → mortal(socrates)	Universal instantiation with {socrates/X}
4	mortal(socrates)	Modus ponens on 2 and 3

Why unification is needed: An automated problem solver uses unification to determine that Socrates may be substituted for X in order to apply modus ponens.

Resolution: Chapter 14 discusses a more powerful rule of inference called resolution, the basis of many automated reasoning systems.

4.3.2 Unification

Definition: Unification is an algorithm for determining the substitutions needed to make two predicate calculus expressions match.

Key knowledge about unification:

Aspect	Rule
Constants	A constant is a ground instance; cannot be replaced
	Two different constants Cannot unify
Variable binding	A variable may be bound to a constant, term, or another variable
Occurs check	A variable cannot be unified with a term containing that variable

Consistency
Substitutions must be applied consistently across all occurrences
Composition
Substitutions compose; composition is **associative but not commutative**
Generality
Must return the most general unifier (mgu)
mgu uniqueness
Unique except for alphabetic variations of variable names
Why all variables must be universally quantified: This allows full freedom in computing substitutions. Existentially quantified variables are eliminated by **Skolemization** — replacing them with constants or with Skolem functions of the other variables.

Example of Skolemization: ∀X ∃Y mother(X, Y) becomes ∀X mother(X, f(X)) where f is a Skolem function.

Expression 1
Expression 2
Unifier
foo(X, a, goo(Y))
foo(fred, a, goo(Z))
{fred/X, Z/Y}
foo(X, a, goo(Y))
foo(W, a, goo(jack))
{W/X, jack/Y}
foo(X, a, goo(Y))
foo(Z, a, goo(moo(Z)))
{Z/X, moo(Z)/Y}
man(X)
man(socrates)
{socrates/X}
likes(X, Y)
likes(george, kate)
{george/X, kate/Y}
p(X)
p(Y)
{Z/X, Z/Y} (most general)
X
f(X)
FAIL (occurs check)

The most general unifier (mgu) definition:

If s is any unifier of expressions E, and g is the most general unifier of that set of expressions, then for s applied to E there exists another unifier s' such that Es = Egs'.

Why generality matters — the {fred/X} vs {Z/X} example:

Unifier
Generality
Effect
{fred/X, fred/Y}
Less general
Restricts all solutions to those involving fred
{Z/X, Z/Y}
Most general
Allows any value for Z
A less general unifier can eliminate possible solutions.

Substitution composition:

Composition of S and S′ (written SS′) is obtained by applying S′ to the elements of S and adding the result to S.

Step
Set
First set
{X/Y, W/Z}
Second set
{V/X}
Third set
{a/V, f(b)/W}
Compose third with second
{a/X, a/V, f(b)/W}
Compose result with first
{a/Y, a/X, a/V, f(b)/Z, f(b)/W}

Why composition is not commutative: The order in which substitutions are applied matters.

The unify algorithm pseudo-code:

```
function unify(E1, E2);
begin
   case
      both E1 and E2 are constants or the empty
list:
         if E1 = E2 then return {} else return
FAIL;
      E1 is a variable:
         if E1 occurs in E2 then return FAIL else
return {E2/E1};
      E2 is a variable:
         if E2 occurs in E1 then return FAIL else
return {E1/E2};
      either E1 or E2 are empty then return
FAIL;
      otherwise:
         begin
            HE1 := first element of E1;
            HE2 := first element of E2;
            SUBS1 := unify(HE1, HE2);
            if SUBS1 := FAIL then return FAIL;
            TE1 := apply(SUBS1, rest of E1);
            TE2 := apply(SUBS1, rest of E2);
            SUBS2 := unify(TE1, TE2);
            if SUBS2 := FAIL then return FAIL
            else return composition(SUBS1,
SUBS2)
         end
   end
end.
```

4.3.3 A unification example

The trace:

unify((parents X (father X) (mother bill)), (parents bill (father bill) Y))

Step
Call
Result
1
unify(parents, parents)
{}
2
unify((X (father X) (mother bill)), (bill (father bill) Y))
—
3
unify(X, bill)
{bill/X}
4
unify((father bill), (father bill))
{}
5
unify(father, father)
{}
6
unify(bill, bill)
{}
7
unify((), ())
{}
8
unify((mother bill), Y)
{(mother bill)/Y}
9
Final composition
{bill/X, (mother bill)/Y}

Final answer: {bill/X, (mother bill)/Y}

Why this works: Unification succeeds on atomic symbols (both constants or one variable), then recursively unifies the rest of each list, composing substitutions as it returns.

5. THE FINANCIAL ADVISOR APPLICATION (Section 2.4)

5.1 The problem

The advisor helps a user decide whether to invest in a savings account or the stock market. The recommendation depends on:

Rule
Recommendation
1
Individuals with inadequate savings should make increasing savings their first priority
2
Individuals with adequate savings and adequate income should consider stocks
3
Individuals with adequate savings but lower income may split between savings and stocks

5.2 The adequacy criteria

Function
Definition
Meaning
minsavings(X)
5000 * X
At least $5,000 in savings per dependent
minincome(X)
15000 + (4000 * X)
At least $15,000 plus $4,000 per dependent, and steady

Predicate
Arity
Values
savings_account(X)
1
adequate, inadequate
income(X)
1
adequate, inadequate
investment(X)
1
stocks, savings, combination
amount_saved(X)

1
numeric
earnings(X, Y)
2
amount and steady/unsteady
dependents(Y)
1
number of dependents
greater(X, Y)
2
standard arithmetic test

5.4 The investment rules

#
Rule
1
savings_account(inadequate) → investment(savings)
2
savings_account(adequate) ∧ income(adequate) → investment(stocks)
3
savings_account(adequate) ∧ income(inadequate) → investment(combination)

5.5 The adequacy rules

#
Rule
4
∀X amount_saved(X) ∧ ∃Y (dependents(Y) ∧ greater(X, minsavings(Y))) → savings_account(adequate)
5
∀X amount_saved(X) ∧ ∃Y (dependents(Y) ∧ ¬greater(X, minsavings(Y))) → savings_account(inadequate)
6
∀X earnings(X, steady) ∧ ∃Y (dependents(Y) ∧ greater(X, minincome(Y))) → income(adequate)
7
∀X earnings(X, steady) ∧ ∃Y (dependents(Y) ∧ ¬greater(X, minincome(Y))) → income(inadequate)

Chapter 3: Structures and Strategies for State Space Search

Complete Study Notes — Based strictly on Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th ed., Chapter 3 and the Introduction to Part II

1. WHY CHAPTER 3 EXISTS

Luger introduces Chapter 3:

"Chapter 2 introduced predicate calculus as an example of an artificial intelligence representation language. Well-formed predicate calculus expressions provide a means of describing objects and relations in a problem domain, and inference rules such as modus ponens allow us to infer new knowledge from these descriptions. These inferences define a space that is searched to find a problem solution. Chapter 3 introduces the theory of state space search."

"To successfully design and implement search algorithms, a programmer must be able to analyze and predict their behavior. Questions that need to be answered include:

Is the problem solver guaranteed to find a solution?

Will the problem solver always terminate? Can it become caught in an infinite loop?

When a solution is found, is it guaranteed to be optimal?

What is the complexity of the search process in terms of time usage? Memory usage?

How can the interpreter most effectively reduce search complexity?

How can an interpreter be designed to most effectively utilize a representation language?"

"The theory of state space search is our primary tool for answering these questions. By representing a problem as a state space graph, we can use graph theory to analyze the structure and complexity of both the problem and the search procedures that we employ to solve it."

2. INTRODUCTION TO REPRESENTATION AND SEARCH (from Part II)

Luger opens Part II with the Dartmouth proposal (1955) and the following:

"From an engineering perspective, the description of artificial intelligence presented in Section 1.3 may be summarized as the study of representation and search through which intelligent activity can be enacted on a mechanical device. This perspective has dominated the origins and growth of AI."

"The first modern workshop/conference for AI practitioners was held at Dartmouth College in the summer of 1956. The proposal for this workshop is presented as the introductory quotation for Part II. This workshop, where the name artificial intelligence itself was chosen, brought together many of the then current researchers focused on the integration of computation and intelligence."

The main topics for discussion at this conference:

"1. Automatic Computers — If a machine can do a job, then an automatic calculator can be programmed to simulate the machine. 2. How Can a Computer be Programmed to Use a Language — It may be speculated that a large part of human thought consists of manipulating words according to rules of reasoning and rules of conjecture. 3. Neuron Nets — How can a set of (hypothetical) neurons be arranged so as to form concepts? 4. Theory of the Size of a Calculation — If we are given a well-defined problem (one for which it is possible to test mechanically whether or not a proposed answer is a valid answer) one way of solving it is to try all possible answers in order. This method is inefficient, and to exclude it one must have some criterion for efficiency of calculation. 5. Self-improvement (Machine Learning) — Probably a truly intelligent machine will carry out activities which may best be described as self-improvement. 6. Abstractions — A number of types of 'abstraction' can be distinctly defined and several others less distinctly. A direct attempt to classify these and to describe machine methods of forming abstractions from sensory and other data would seem worthwhile. 7. Randomness and Creativity — A fairly attractive and yet clearly incomplete conjecture is that the difference between creative thinking and unimaginative competent thinking lies in the injection of some randomness."

"It is interesting to note that the topics proposed for this first conference on artificial intelligence capture many of the issues, such as complexity theory, methodologies for abstraction, language design, and machine learning that make up the focus of modern computer science. In fact, many of the defining characteristics of computer science as we know it today have their roots in AI. AI has also had its own historical and political struggles, with several of these early topics proposed for research, such as 'neuron nets' and 'randomness and creativity' were put into background mode for decades."

"A powerful new computational tool, the Lisp language, emerged at about this time, built under the direction of John McCarthy, one of the original proposers of the Dartmouth Workshop. Lisp addressed several of the topics of the Workshop, supporting the ability to create relationships that could themselves be manipulated by other structures of the language. Lisp gave artificial intelligence both a highly expressive language, rich in abstraction, as well as a medium for interpretation of these expressions."

"The availability of the Lisp programming language did shape much of the early development of AI, in particular, the use of the predicate calculus as a representational medium as well as search to explore the efficacy of different logical alternatives, what we now call graph search. Prolog, created in the late 1970s, would offer AI a similar powerful computational tool."

Representational systems:

"The function of any representation scheme is to capture - often called abstract out - the critical features of a problem domain and make that information accessible to a problem-solving procedure. Abstraction is an essential tool for managing complexity as well as an important factor in assuring that the resulting programs are computationally efficient. Expressiveness (the result of the features abstracted) and efficiency (the computational complexity of the algorithms used on the abstracted features) are major dimensions for evaluating knowledge representation languages. Sometimes, expressiveness must be sacrificed to improve an algorithm's efficiency. This must be done without limiting the representation's ability to capture essential problem-solving knowledge. Optimizing the trade-off between efficiency and expressiveness is a major task for designers of intelligent programs."

The floating-point number example:

"The computer representation of floating-point numbers illustrates these trade-offs (see Fig. II.1). To be precise, real numbers require an infinite string of digits to be fully described; this cannot be accomplished on a finite device such as a computer. One answer to this dilemma is to represent the number in two pieces: its significant digits and the location within those digits of the decimal point. Although it is not possible to actually store a real number in a computer, it is possible to create a representation that functions adequately in most practical applications. Floating-point representation thus sacrifices full expressive power to make the representation efficient, in this case to make it possible. This representation also supports algorithms for multiple-precision arithmetic, giving effectively infinite precision by limiting round-off error to any pre-specified tolerance. It also guarantees well-behaved round-off errors. Like all representations, it is only an abstraction, a symbol pattern that designates a desired entity and not the entity itself."

The array example:

"The array is another representation common in computer science. For many problems, it is more natural and efficient than the memory architecture implemented in computer hardware. This gain in naturalness and efficiency involves compromises in expressiveness, as illustrated by the following example from image processing. Figure II.2 is a digitized image of human chromosomes in a stage called metaphase. The image is processed to determine the number and structure of the chromosomes, looking for breaks, missing pieces, and other abnormalities."

"The visual scene is made up of a number of picture points. Each picture point, or pixel, has both a location and a number value representing its intensity or gray level. It is natural, then, to collect the entire scene into a two-dimensional array where the row and column address gives the location of a pixel (X and Y coordinates) and the content of the array element is the gray level at that point. Algorithms are designed to perform operations like looking for isolated points to remove noise from the image, finding threshold levels for discerning objects and edges, summing contiguous elements to determine size or density, and in various other ways transforming the picture point data. Implementing these algorithms is straightforward, given the array representation and the FORTRAN language, for example. This task would be quite cumbersome using other representations such as the predicate calculus, records, or assembly code, because these do not have a natural fit with the material being represented."

"When we represent the picture as an array of pixel points, we sacrifice fineness of resolution (compare a photo in a newspaper to the original print of the same picture). In addition, pixel arrays cannot express the deeper semantic organization of the image. For example, a pixel array cannot represent the organization of chromosomes in a single cell nucleus, their genetic function, or the role of metaphase in cell division. This knowledge is more easily captured using a representation such as predicate calculus (Chapter 2) or semantic networks (Chapter 7). In summary, a representational scheme should be adequate to express all of the necessary information, support efficient execution of the resulting code, and provide a natural scheme for expressing the required knowledge."

Search:

"Given a representation, the second component of intelligent problem solving is search. Humans generally consider a number of alternative strategies on their way to solving a problem. A chess player typically reviews alternative moves, selecting the 'best' according to criteria such as the opponent's possible responses or the degree to which various moves support some global game strategy. A player also considers short-term gain (such as taking the opponent's queen), opportunities to sacrifice a piece for positional advantage, or conjectures concerning the opponent's psychological makeup and level of skill. This aspect of intelligent behavior underlies the problem-solving technique of state space search."

The tic-tac-toe example:

"Consider, for example, the game of tic-tac-toe. Given any board situation, there is only a finite number of moves that a player can make. Starting with an empty board, the first player may place an X in any one of nine places. Each of these moves yields a different board that will allow the opponent eight possible responses, and so on. We can represent this collection of possible moves and responses by regarding each board configuration as a node or state in a graph. The links of the graph represent legal moves from one board configuration to another. The resulting structure is a state space graph."

"The state space representation thus enables us to treat all possible games of tic-tac-toe as different paths through the state space graph. Given this representation, an effective game strategy will search through the graph for the paths that lead to the most wins and fewest losses and play in a way that always tries to force the game along one of these optimal paths, as in Figure II.5."

The automobile diagnosis example:

"As an example of how search is used to solve a more complicated problem, consider the task of diagnosing a mechanical fault in an automobile. Although this problem does not initially seem to lend itself to state space search as easily as tic-tac-toe or chess, it actually fits this strategy quite well. Instead of letting each node of the graph represent a 'board state,' we let it represent a state of partial knowledge about the automobile's mechanical problems. The process of examining the symptoms of the fault and inducing its cause may be thought of as searching through states of increasing knowledge. The starting node of the graph is empty, indicating that nothing is known about the cause of the problem. The first thing a mechanic might do is ask the customer which major system (engine, transmission, steering, brakes, etc.) seems to be causing the trouble. This is represented by a collection of arcs from the start state to states that indicate a focus on a single subsystem of the automobile, as in Figure II.6."

"Each of the states in the graph has arcs (corresponding to basic diagnostic checks) that

lead to states representing further accumulation of knowledge in the diagnostic process. For example, the engine trouble node has arcs to nodes labeled engine starts and engine won't start. From the won't start node we may move to nodes labeled turns over and won't turn over. The won't turn over node has arcs to nodes labeled battery dead and battery ok, see Figure II.7. A problem solver can diagnose car trouble by searching for a path through this graph that is consistent with the symptoms of a particular defective car. Although this problem is very different from that of finding an optimal way to play tic-tac-toe or chess, it is equally amenable to solution by state space search."

The limits of exhaustive search:

"In spite of this apparent universality, state space search is not, by itself, sufficient for automating intelligent problem-solving behavior; rather it is an important tool for the design of intelligent programs. If state space search were sufficient, it would be fairly simple to write a program that plays chess by searching through the entire space for the sequence of moves that brought a victory, a method known as exhaustive search. Though exhaustive search can be applied to any state space, the overwhelming size of the space for interesting problems makes this approach a practical impossibility. Chess, for example, has approximately 10^{120} different board states. This is a number larger than the number of molecules in the universe or the number of nanoseconds that have passed since the big bang. Search of this space is beyond the capabilities of any computing device, whose dimensions must be confined to the known universe and whose execution must be completed before the universe succumbs to the ravages of entropy."

Heuristics:

"Humans use intelligent search: a chess player considers a number of possible moves, a doctor examines several possible diagnoses, a computer scientist entertains different designs before beginning to write code. Humans do not use exhaustive search: the chess player examines only moves that experience has shown to be effective, the doctor does not require tests that are not somehow indicated by the symptoms at hand. Human problem solving seems to be based on judgmental rules that guide search to those portions of the state space that seem most 'promising'."

"These rules are known as heuristics, and they constitute one of the central topics of AI research. A heuristic (the name is taken from the Greek word 'to discover') is a strategy for selectively searching a problem space. It guides search along lines that have a high probability of success while avoiding wasted or apparently stupid efforts. Human beings use a large number of heuristics in problem solving. If you ask a mechanic why your car is overheating, she may say something like, 'Usually that means the thermostat is bad.' If you ask a doctor what could cause nausea and stomach pains, he might say it is 'probably either stomach flu or food poisoning.'"

"State space search gives us a means of formalizing the problem-solving process, and heuristics allow us to infuse that formalism with intelligence. These techniques are discussed in detail in the early chapters of this book and remain at the heart of most modern work in AI. In summary, state space search is a formalism, independent of any particular search strategies, and used as a launch point for many problem solving approaches."

3. INTRODUCTION (Section 3.0)

"Chapter 2 introduced predicate calculus as an example of an artificial intelligence representation language. Well-formed predicate calculus expressions provide a means of describing objects and relations in a problem domain, and inference rules such as modus ponens allow us to infer new knowledge from these descriptions. These inferences define a space that is searched to find a problem solution. Chapter 3 introduces the theory of state space search."

"To successfully design and implement search algorithms, a programmer must be able to analyze and predict their behavior. Questions that need to be answered include:

Is the problem solver guaranteed to find a solution?

Will the problem solver always terminate? Can it become caught in an infinite loop?

When a solution is found, is it guaranteed to be optimal?

What is the complexity of the search process in terms of time usage? Memory usage?

How can the interpreter most effectively reduce search complexity?

How can an interpreter be designed to most effectively utilize a representation language?"

"The theory of state space search is our primary tool for answering these questions. By representing a problem as a state space graph, we can use graph theory to analyze the structure and complexity of both the problem and the search procedures that we employ to solve it."

Definitions:

"A graph consists of a set of nodes and a set of arcs or links connecting pairs of nodes. In the state space model of problem solving, the nodes of a graph are taken to represent discrete states in a problem-solving process, such as the results of logical inferences or the different configurations of a game board. The arcs of the graph represent transitions between states. These transitions correspond to logical inferences or legal moves of a game. In expert systems, for example, states describe our knowledge of a problem instance at some stage of a reasoning process. Expert knowledge, in the form of if…then rules, allows us to generate new information; the act of applying a rule is represented as an arc between states."

The Königsberg bridges problem:

"Graph theory is our best tool for reasoning about the structure of objects and relations; indeed, this is precisely the need that led to its creation in the early eighteenth century. The Swiss mathematician Leonhard Euler invented graph theory to solve the 'bridges of Konigsberg problem.' The city of Konigsberg occupied both banks and two islands of a river. The islands and the riverbanks were connected by seven bridges, as indicated in Figure 3.1."

"The bridges of Königsberg problem asks if there is a walk around the city that crosses each bridge exactly once. Although the residents had failed to find such a walk and doubt that it was possible, no one had proved its impossibility. Devising a form of graph theory, Euler created an alternative representation for the map, presented in Figure 3.2. The riverbanks (rb1 and rb2) and islands (i1 and i2) are described by the nodes of a graph; the bridges are represented by labeled arcs between nodes (b1, b2, …, b7). The graph representation preserves the essential structure of the bridge system, while ignoring extraneous features such as bridge lengths, distances, and order of bridges in the walk."

The predicate calculus representation:

"Alternatively, we may represent the Königsberg bridge system using predicate calculus. The connect predicate corresponds to an arc of the graph, asserting that two land masses are connected by a particular bridge. Each bridge requires two connect predicates, one for each direction in which the bridge may be crossed. A predicate expression, connect(X, Y, Z) = connect(Y, X, Z), indicating that any bridge can be crossed in either direction, would allow removal of half the following connect facts:

connect(i1, i2, b1) connect(i2, i1, b1)
connect(rb1, i1, b2) connect(i1, rb1, b2)
connect(rb1, i1, b3) connect(i1, rb1, b3)
connect(rb1, i2, b4) connect(i2, rb1, b4)
connect(rb2, i1, b5) connect(i1, rb2, b5)
connect(rb2, i1, b6) connect(i1, rb2, b6)
connect(rb2, i2, b7) connect(i2, rb2, b7)"

"The predicate calculus representation is equivalent to the graph representation in that the connectedness is preserved. Indeed, an algorithm could translate between the two representations with no loss of information. However, the structure of the problem can be visualized more directly in the graph representation, whereas it is left implicit in the predicate calculus version. Euler's proof illustrates this distinction."

Euler's proof:

"In proving that the walk was impossible, Euler focused on the degree of the nodes of the graph, observing that a node could be of either even or odd degree. An even degree node has an even number of arcs joining it to neighboring nodes. An odd degree node has an odd number of arcs. With the exception of its beginning and ending nodes, the desired walk would have to leave each node exactly as often as it entered it. Nodes of odd degree could be used only as the beginning or ending of the walk, because such nodes could be crossed only a certain number of times before they proved to be a dead end. The traveler could not exit the node without using a previously traveled arc."

"Euler noted that unless a graph contained either exactly zero or two nodes of odd degree, the walk was impossible. If there were two odd-degree nodes, the walk could start at the first and end at the second; if there were no nodes of odd degree, the walk could begin and end at the same node. The walk is not possible for graphs containing any other number of nodes of odd degree, as is the case with the city of Konigsberg. This problem is now called finding an Euler path through a graph."

"Note that the predicate calculus representation, though it captures the relationships between bridges and land in the city, does not suggest the concept of the degree of a node. In the graph representation there is a single instance of each node with arcs between the nodes, rather than multiple occurrences of constants as arguments in a set of predicates. For this reason, the graph representation suggests the concept of node degree and the focus of Euler's proof. This illustrates graph theory's power for analyzing the structure of objects, properties, and relationships."

4. STRUCTURES FOR STATE SPACE SEARCH (Section 3.1)

4.1 Graph Theory (Section 3.1.1)

"A graph is a set of nodes or states and a set of arcs that connect the nodes. A labeled graph has one or more descriptors (labels) attached to each node that distinguish that node from any other node in the graph. In a state space graph, these descriptors identify states in a problem-solving process. If there are no descriptive differences between two nodes, they are considered the same. The arc between two nodes is indicated by the labels of the connected nodes."

"The arcs of a graph may also be labeled. Arc labels are used to indicate that an arc represents a named relationship (as in a semantic network) or to attach weights to arcs (as in the traveling salesperson problem). If there are different arcs between the same two nodes (as in Figure 3.2), these can also be distinguished through labeling."

"A graph is directed if arcs have an associated directionality. The arcs in a directed graph are usually drawn as arrows or have an arrow attached to indicate direction. Arcs that can be crossed in either direction may have two arrows attached but more often have no direction indicators at all. Figure 3.3 is a labeled, directed graph: arc (a, b) may only be crossed from node a to node b, but arc (b, c) is crossable in either direction."

"A path through a graph connects a sequence of nodes through successive arcs. The path is represented by an ordered list that records the nodes in the order they occur in the path. In Figure 3.3, [a, b, c, d] represents the path through nodes a, b, c, and d, in that order."

"A rooted graph has a unique node, called the root, such that there is a path from the root to all nodes within the graph. In drawing a rooted graph, the root is usually drawn at the top of the page, above the other nodes. The state space graphs for games are usually rooted graphs with the start of the game as the root. The initial moves of the tic-tac-toe game graph are represented by the rooted graph of Figure II.5. This is a directed graph with all arcs having a single direction. Note that this graph contains no cycles; players cannot (as much as they might sometimes wish to!) undo a move."

"A tree is a graph in which two nodes have at most one path between them. Trees often have roots, in which case they are usually drawn with the root at the top, like a rooted graph. Because each node in a tree has only one path of access from any other node, it is impossible for a path to loop or cycle through a sequence of nodes."

"For rooted trees or graphs, relationships between nodes include parent, child, and sibling. These are used in the usual familial fashion with the parent preceding its child along a directed arc. The children of a node are called siblings. Similarly, an ancestor comes before a descendant in some path of a directed graph. In Figure 3.4, b is a parent of nodes e and f (which are, therefore, children of b and siblings of each other). Nodes a and c are ancestors of states g, h, and i, and g, h, and i are descendants of a and c."

Formal definition:

"DEFINITION: GRAPH

A graph consists of: 1. A set of nodes N_1, N_2, N_3, …, N_n, …, which need not be finite. 2. A set of arcs that connect pairs of nodes.

Arcs are ordered pairs of nodes; i.e., the arc (N_3, N_4) connects node N_3 to node N_4. This indicates a direct connection from node N_3 to N_4 but not from N_4 to N_3 unless (N_4, N_3) is also an arc, and then the arc joining N_3 and N_4 is undirected.

If a directed arc connects N_j and N_k, then N_j is called the parent of N_k and N_k, the child of N_j. If the graph also contains an arc (N_j, N_i), then N_k and N_i are called siblings.

A rooted graph has a unique node N_s from which all paths in the graph originate. That is, the root has no parent in the graph.

A tip or leaf node is a node that has no children.

An ordered sequence of nodes [N_1, N_2, N_3, …, N_n], where each pair N_i, N_{i+1} in the sequence represents an arc, i.e., (N_i, N_{i+1}), is called a path of length $n - 1$.

On a path in a rooted graph, a node is said to be an ancestor of all nodes positioned after it (to its right) as well as a descendant of all nodes before it.

A path that contains any node more than once (some N_j in the definition of path above is repeated) is said to contain a cycle or loop.

A tree is a graph in which there is a unique path between every pair of nodes. (The paths in a tree, therefore, contain no cycles.)

The edges in a rooted tree are directed away from the root. Each node in a rooted tree has a unique parent.

Two nodes are said to be connected if a path exists that includes them both."

4.2 The Finite State Machine (Section 3.1.2)

"We can think of a machine as a system that accepts input values, possibly produces output values, and that has some sort of internal mechanism (states) to keep track of information about previous input values. A finite state machine (FSM) is a finite, directed, connected graph, having a set of states, a set of input values, and a state transition function that describes the effect that the elements of the input stream have on the states of the graph. The stream of input values produces a path within the graph of the states of this finite machine. Thus the FSM can be seen as an abstract model of computation."

"The primary use for such a machine is to recognize components of a formal language. These components are often strings of characters ('words' made from characters of an 'alphabet'). In Section 5.3 we extend this definition to a probabilistic finite state machine. These state machines have an important role in analyzing expressions in languages, whether computational or human, as we see in Sections 5.3, 9.3, and Chapter 15."

Formal definition:

"DEFINITION: FINITE STATE MACHINE (FSM)

A finite state machine is an ordered triple (S, I, F), where:

S is a finite set of states in a connected graph S_1, S_2, S_3, …, S_n

I is a finite set of input values i_1, i_2, i_3, …, i_m

F is a state transition function that for any i ∈ I, describes its effect on the states S of the machine, thus ∀i ∈ I, F_i: (S → S). If the machine is in state S_j and input i occurs, the next state of the machine will be $F_i(S_j)$."

The flip-flop example:

"For a simple example of a finite state machine, let S = {s_0, s_1}, I = {0, 1}, $f_0(s_0) = s_0$, $f_0(s_1) = (s_1)$, $f_1(s_0) = s_1$, and $f_1(s_1) = s_0$. With this device, sometimes called a flip-flop, an input value of zero leaves the state unchanged, while input 1 changes the state of the machine. We may visualize this machine from two equivalent perspectives, as a finite graph with labelled, directed arcs, as in Figure 3.5a, or as a transition matrix, Figure 3.5b. In the transition matrix, input values are listed along the top row, the states are in the leftmost column, and the output for an input applied to a state is at the intersection point."

The string recogniser example:

"A second example of a finite state machine is represented by the directed graph of Figure 3.6a and the (equivalent) transition matrix of Figure 3.6b. One might ask what the finite state machine of Figure 3.6 could represent. With two assumptions, this machine could be seen as a recognizer of all strings of characters from the alphabet {a, b, c, d} that contain the exact sequence 'abc'. The two assumptions are, first, that state s_0 has a special role as the starting state, and second, that s_3 is the accepting state. Thus, the input stream will present its first element to state s_0. If the stream later terminates with the machine in state s_3, it will have recognized that there is the sequence 'abc' within that input stream."

The finite state acceptor (Moore machine):

"What we have just described is a finite state accepting machine, sometimes called a Moore machine. We use the convention of placing an arrow from no state that terminates in the starting state of the Moore machine, and represent the accepting state (or states) as special, often using a doubled circle, as in Figure 3.6. We now present a formal definition of the Moore machine:"

"DEFINITION: FINITE STATE ACCEPTOR (MOORE MACHINE)

A finite state acceptor is a finite state machine (S, I, F), where:

∃ s_0 ∈ S such that the input stream starts at s_0, and

∃ s_n ∈ S, an accept state. The input stream is accepted if it terminates in that state. In fact, there may be a set of accept states.

The finite state acceptor is represented as (S, s_0, {s_n}, I, F)"

"We have presented two fairly simple examples of a powerful concept. As we will see in natural language understanding (Chapter 15), finite state recognizers are an important tool for determining whether or not patterns of characters, words, or sentences have desired properties. We will see that a finite state acceptor implicitly defines a formal language on the basis of the sets of letters (characters) and words (strings) that it accepts."

4.3 State Space Search (Section 3.1.3)

The definition:

"DEFINITION: STATE SPACE SEARCH

A state space is represented by a four-tuple [N, A, S, GD], where:

N is the set of nodes or states of the graph. These correspond to the states in a problem-solving process.

A is the set of arcs (or links) between nodes. These correspond to the steps in a problem-solving process.

S, a nonempty subset of N, contains the start state(s) of the problem.

GD, a nonempty subset of N, contains the goal state(s) of the problem. The states in GD are described using either:

A measurable property of the states encountered in the search.

A measurable property of the path developed in the search, for example, the sum of the transition costs for the arcs of the path.

A solution path is a path through this graph from a node in S to a node in GD."

The loop problem:

"One of the general features of a graph, and one of the problems that arise in the design of a graph search algorithm, is that states can sometimes be reached through different paths. For example, in Figure 3.3 a path can be made from state a to state d either through b and c or directly from a to d. This makes it important to choose the best path according to the needs of a problem. In addition, multiple paths to a state can lead to loops or cycles in a solution path that prevent the algorithm from reaching a goal. A blind search for goal state e in the graph of Figure 3.3 might search the sequence of states abcdabcdabcd … forever!"

"If the space to be searched is a tree, as in Figure 3.4, the problem of cycles does not occur. It is, therefore, important to distinguish between problems whose state space is a tree and those that may contain loops. General graph search algorithms must detect and eliminate loops from potential solution paths, whereas tree searches may gain efficiency by eliminating this test and its overhead."

4.4 Example 3.1.1: Tic-Tac-Toe

"The state space representation of tic-tac-toe appears in Figure II.5, pg 42. The start state is an empty board, and the termination or goal description is a board state having three Xs in a row, column, or diagonal (assuming that the goal is a win for X). The path from the start state to a goal state gives the series of moves in a winning game."

"The states in the space are all the different configurations of Xs and Os that the game can have. Of course, although there are 3^9 ways to arrange {blank, X, O} in nine spaces, most of them would never occur in an actual game. Arcs are generated by legal moves of the game, alternating between placing an X and an O in an unused location. The state space is a graph rather than a tree, as some states on the third and deeper levels can be reached by different paths. However, there are no cycles in the state space, because the directed arcs of the graph do not allow a move to be undone. It is impossible to 'go back up' the structure once a state has been reached. No checking for cycles in path generation is necessary. A graph structure with this property is called a directed acyclic graph, or DAG, and is common in state space search and in graphical models, Chapter 13."

"The state space representation provides a means of determining the complexity of the problem. In tic-tac-toe, there are nine first moves with eight possible responses to each of them, followed by seven possible responses to each of these, and so on. It follows that 9 × 8 × 7 × … or 9! different paths can be generated. Although it is not impossible for a computer to search this number of paths (362,880) exhaustively, many important problems also exhibit factorial or exponential complexity, although on a much larger scale. Chess has 10^{120} possible game paths; checkers has 10^{40}, some of which may never occur in an actual game. These spaces are difficult or impossible to search exhaustively. Strategies for searching such large spaces often rely on heuristics to reduce the complexity of the search (Chapter 4)."

4.5 Example 3.1.2: The 8-Puzzle

"In the 15-puzzle of Figure 3.7, 15 differently numbered tiles are fitted into 16 spaces on a grid. One space is left blank so that tiles can be moved around to form different patterns. The goal is to find a series of moves of tiles into the blank space that places the board in a goal configuration. This is a common game that most of us played as children. (The version I remember was about 3 inches square and had red and white tiles in a black frame.)"

"A number of interesting aspects of this game have made it useful to researchers in problem solving. The state space is large enough to be interesting but is not completely intractable (16! if symmetric states are treated as distinct). Game states are easy to represent. The game is rich enough to provide a number of interesting heuristics (see Chapter 4)."

"The 8-puzzle is a 3 × 3 version of the 15-puzzle in which eight tiles can be moved around in nine spaces. Because the 8-puzzle generates a smaller state space than the full 15-puzzle and its graph fits easily on a page, it is used for many examples in this book."

"Although in the physical puzzle moves are made by moving tiles ('move the 7 tile right, provided the blank is to the right of the tile' or 'move the 3 tile down'), it is much simpler to think in terms of 'moving the blank space'. This simplifies the definition of move rules because there are eight tiles but only a single blank. In order to apply a move, we must make sure that it does not move the blank off the board. Therefore, all four moves are not applicable at all times; for example, when the blank is in one of the corners only two moves are possible."

The legal moves:

"move the blank up ↑ move the blank right → move the blank down ↓ move the blank left ←"

"If we specify a beginning state and a goal state for the 8-puzzle, it is possible to give a state space accounting of the problem-solving process (Figure 3.8). States could be represented using a simple 3 × 3 array. A predicate calculus representation could use a 'state' predicate with nine parameters (for the locations of numbers in the grid). Four procedures, describing each of the possible moves of the blank, define the arcs in the state space."

"As with tic-tac-toe, the state space for the 8-puzzle is a graph (with most states having multiple parents), but unlike tic-tac-toe, cycles are possible. The GD or goal description of the state space is a particular state or board configuration. When this state is found on a path, the search terminates. The path from start to goal is the desired series of moves."

"It is interesting to note that the complete state space of the 8- and 15-puzzles consists of two disconnected (and in this case equal-sized) subgraphs. This makes half the possible states in the search space impossible to reach from any given start state. If we exchange (by prying loose!) two immediately adjacent tiles, states in the other component of the space become reachable."

4.6 Example 3.1.3: The Traveling Salesperson

"Suppose a salesperson has five cities to visit and then must return home. The goal of the problem is to find the shortest path for the salesperson to travel, visiting each city, and then returning to the starting city. Figure 3.9 gives an instance of this problem. The nodes of the graph represent cities, and each arc is labeled with a weight indicating the cost of traveling that arc. This cost might be a

representation of the miles necessary in car travel or cost of an air flight between the two cities. For convenience, we assume the salesperson lives in city A and will return there, although this assumption simply reduces the problem of N cities to a problem of (N − 1) cities."

"The path [A,D,C,B,E,A], with associated cost of 450 miles, is an example of a possible circuit. The goal description requires a complete circuit with minimum cost. Note that the goal description is a property of the entire path, rather than of a single state. This is a goal description of type 2 from the definition of state space search."

"Figure 3.10 shows one way in which possible solution paths may be generated and compared. Beginning with node A, possible next states are added until all cities are included and the path returns home. The goal is the lowest-cost path."

"As Figure 3.10 suggests, the complexity of exhaustive search in the traveling salesperson problem is (N − 1)!, where N is the number of cities in the graph. For 9 cities we may exhaustively try all paths, but for any problem instance of interesting size, for example with 50 cities, simple exhaustive search cannot be performed within a practical length of time. In fact complexity costs for an N! search grow so fast that very soon the search combinations become intractable."

"Several techniques can reduce this search complexity. One is called branch and bound (Horowitz and Sahni 1978). Branch and bound generates paths one at a time, keeping track of the best circuit found so far. This value is used as a bound on future candidates. As paths are constructed one city at a time, the algorithm examines each partially completed path. If the algorithm determines that the best possible extension to a path, the branch, will have greater cost than the bound, it eliminates that partial path and all of its possible extensions. This reduces search considerably but still leaves an exponential number of paths (1.26^n rather than N!)."

"Another strategy for controlling search constructs the path according to the rule 'go to the closest unvisited city.' The nearest neighbor path through the graph of Figure 3.11 is [A,E,D,B,C,A], at a cost of 375 miles. This method is highly efficient, as there is only one path to be tried! The nearest neighbor, sometimes called greedy, heuristic is fallible, as graphs exist for which it does not find the shortest path, see Figure 3.11, but it is a possible compromise when the time required makes exhaustive search impractical."

5. STRATEGIES FOR STATE SPACE SEARCH (Section 3.2)

5.1 Data-Driven and Goal-Driven Search (Section 3.2.1)

"A state space may be searched in two directions: from the given data of a problem instance toward a goal or from a goal back to the data."

"In data-driven search, sometimes called forward chaining, the problem solver begins with the given facts of the problem and a set of legal moves or rules for changing state. Search proceeds by applying rules to facts to produce new facts, which are in turn used by the rules to generate more new facts. This process continues until (we hope!) it generates a path that satisfies the goal condition."

"An alternative approach is possible: take the goal that we want to solve. See what rules or legal moves could be used to generate this goal and determine what conditions must be true to use them. These conditions become the new goals, or subgoals, for the search. Search continues, working backward through successive subgoals until (we hope!) it works back to the facts of the problem. This finds the chain of moves or rules leading from data to a goal, although it does so in backward order. This approach is called goal-driven reasoning, or backward chaining, and it recalls the simple childhood trick of trying to solve a maze by working back from the finish to the start."

"To summarize: data-driven reasoning takes the facts of the problem and applies the rules or legal moves to produce new facts that lead to a goal; goal-driven reasoning focuses on the goal, finds the rules that could produce the goal, and chains backward through successive rules and subgoals to the given facts of the problem."

"In the final analysis, both data-driven and goal-driven problem solvers search the same state space graph; however, the order and actual number of states searched can differ. The preferred strategy is determined by the properties of the problem itself. These include the complexity of the rules, the 'shape' of the state space, and the nature and availability of the problem data. All of these vary for different problems."

The Thomas Jefferson example:

"As an example of the effect a search strategy can have on the complexity of search, consider the problem of confirming or denying the statement 'I am a descendant of Thomas Jefferson.' A solution is a path of direct lineage between the 'I' and Thomas Jefferson. This space may be searched in two directions, starting with the 'I' and working along ancestor lines to Thomas Jefferson or starting with Thomas Jefferson and working through his descendants."

"Some simple assumptions let us estimate the size of the space searched in each direction. Thomas Jefferson was born about 250 years ago; if we assume 25 years per generation, the required path will be about length 10. As each person has exactly two parents, a search back from the 'I' would examine on the order of 2^{10} ancestors. A search that worked forward from Thomas Jefferson would examine more states, as people tend to have more than two children (particularly in the eighteenth and nineteenth centuries). If we assume an average of only three children per family, the search would examine on the order of 3^{10} nodes of the family tree. Thus, a search back from the 'I' would examine fewer nodes. Note, however, that both directions yield exponential complexity."

Goal-driven search is suggested if:

"1. A goal or hypothesis is given in the problem statement or can easily be formulated. In a mathematics theorem prover, for example, the goal is the theorem to be proved. Many diagnostic systems consider potential diagnoses in a systematic fashion, confirming or eliminating them using goal-driven reasoning. 2. There are a large number of rules that match the facts of the problem and thus produce an increasing number of conclusions or goals. Early selection of a goal can eliminate most of these branches, making goal-driven search more effective in pruning the space (Figure 3.12). In a theorem prover, for example, the total number of rules used to produce a given theorem is usually much smaller than the number of rules that may be applied to the entire set of axioms. 3. Problem data are not given but must be acquired by the problem solver. In this case, goal-driven search can help guide data acquisition. In a medical diagnosis program, for example, a wide range of diagnostic tests can be applied. Doctors order only those that are necessary to confirm or deny a particular hypothesis."

Data-driven search is appropriate for problems in which:

"1. All or most of the data are given in the initial problem statement. Interpretation problems often fit this mold by presenting a collection of data and asking the system to provide a high-level interpretation. Systems that analyze particular data (e.g., the PROSPECTOR or Dipmeter programs, which interpret geological data... 2. There are a large number of potential goals, but there are only a few ways to use the facts and given information of a particular problem instance. The DENDRAL program, an expert system that finds the molecular structure of organic compounds based on their formula, mass spectrographic data, and knowledge of chemistry, is an example of this. For any organic compound, there are an enormous number of possible structures. However, the mass spectrographic data on a compound allow DENDRAL to eliminate all but a few of these. 3. It is difficult to form a goal or hypothesis. In using DENDRAL, for example, little may be known initially about the possible structure of a compound."

"To summarize, there is no substitute for careful analysis of the particular problem to be solved, considering such issues as the branching factor of rule applications (see Chapter 4; on average, how many new states are generated by rule applications in both directions?), availability of data, and ease of determining potential goals."

"In solving a problem using either goal- or data-driven search, a problem solver must find a path from a start state to a goal through the state space graph. The sequence of arcs in this path corresponds to the ordered steps of the solution. If a problem solver were given an oracle or other infallible mechanism for choosing a solution path, search would not be required. The problem solver would move unerringly through the space to the desired goal, constructing the path as it went. Because oracles do not exist for interesting problems, a problem solver must consider different paths through the space until it finds a goal. Backtracking is a technique for systematically trying all paths through a state space."

"We begin with backtrack because it is one of the first search algorithms computer scientists study, and it has a natural implementation in a stack oriented recursive environment. We will present a simpler version of the backtrack algorithm with depth-first search (Section 3.2.3)."

The backtrack algorithm logic:

"Backtracking search begins at the start state and pursues a path until it reaches either a goal or a 'dead end.' If it finds a goal, it quits and returns the solution path. If it reaches a dead end, it 'backtracks' to the most recent node on the path having unexamined siblings and continues down one of these branches, as described in the following recursive rule:

If the present state S does not meet the requirements of the goal description, then generate its first descendant S_child1, and apply the backtrack procedure recursively to this node. If backtrack does not find a goal node in the subgraph rooted at S_child1, repeat the procedure for its sibling, S_child2. Continue until either some descendant of a child is a goal node or all the children have been searched. If none of the children of S leads to a goal, then backtrack 'fails back' to the parent of S, where it is applied to the siblings of S, and so on."

"The algorithm continues until it finds a goal or exhausts the state space. Figure 3.14 shows the backtrack algorithm applied to a hypothetical state space. The direction of the dashed arrows on the tree indicates the progress of search up and down the space. The number beside each node indicates the order in which it is visited. We now define an algorithm that performs a backtrack, using three lists to keep track of nodes in the state space:"

The three lists:

"SL, for state list, lists the states in the current path being tried. If a goal is found, SL contains the ordered list of states on the solution path.

NSL, for new state list, contains nodes awaiting evaluation, i.e., nodes whose descendants have not yet been generated and searched.

DE, for dead ends, lists states whose descendants have failed to contain a goal. If these states are encountered again, they will be detected as elements of DE and eliminated from consideration immediately."

"In defining the backtrack algorithm for the general case (a graph rather than a tree), it is necessary to detect multiple occurrences of any state so that it will not be reentered and cause (infinite) loops in the path. This is accomplished by testing each newly generated state for membership in any of these three lists. If a new state belongs to any of these lists, then it has already been visited and may be ignored."

The algorithm:

"function backtrack; begin SL := [Start]; NSL := [Start]; DE := []; CS := Start; % initialize while NSL ≠ [] do % while there are states to be tried begin if CS = goal (or meets goal description) then return SL; % on success, return list of

states in path. if CS has no children (excluding nodes already on DE, SL, and NSL) then begin while SL is not empty and CS = the first element of SL do begin add CS to DE; % record state as dead end remove first element from SL; %backtrack remove first element from NSL; CS := first element of NSL; end add CS to SL; end else begin place children of CS (except nodes already on DE, SL, or NSL) on NSL; CS := first element of NSL; add CS to SL end end; return FAIL; end."

"In backtrack, the state currently under consideration is called CS for current state. CS is always equal to the state most recently added to SL and represents the 'frontier' of the solution path currently being explored. Inference rules, moves in a game, or other appropriate problem-solving operators are ordered and applied to CS. The result is an ordered set of new states, the children of CS. The first of these children is made the new current state and the rest are placed in order on NSL for future examination. The new current state is added to SL and search continues. If CS has no children, it is removed from SL (this is where the algorithm 'backtracks') and any remaining children of its predecessor on SL are examined."

The trace of backtrack on Figure 3.14:

"A trace of backtrack on the graph of Figure 3.14 is given by:

Initialize: SL = [A]; NSL = [A]; DE = []; CS = A;
AFTER ITERATION
CS
SL
NSL
DE
0
A
[A]
[A]
[]
1
B
[B A]
[B C D A]
[]
2
E
[E B A]
[E F B C D A]
[]
3
H
[H E B A]
[H I E F B C D A]
[]
4
I
[I E B A]
[I E F B C D A]
[H]
5
F
[F B A]
[F B C D A]
[E I H]
6
J
[J F B A]
[J F B C D A]
[E I H]
7
C
[C A]
[C D A]
[B F J E I H]
8
G
[G C A]
[G C D A]
[B F J E I H]

5.3 Depth-First and Breadth-First Search (Section 3.2.3)

"As presented here, backtrack implements data-driven search, taking the root as a start state and evaluating its children to search for the goal. The algorithm can be viewed as a goal-driven search by letting the goal be the root of the graph and evaluating descendants back in an attempt to find a start state. If the goal description is of type 2 (see Section 3.1.3), the algorithm must determine a goal state by examining the path on SL."

"backtrack is an algorithm for searching state space graphs. The graph search algorithms that follow, including depth-first, breadth-first, and best-first search, exploit the ideas used in backtrack, including:

The use of a list of unprocessed states (NSL) to allow the algorithm to return to any of these states.

A list of 'bad' states (DE) to prevent the algorithm from retrying useless paths.

A list of nodes (SL) on the current solution path that is returned if a goal is found.

Explicit checks for membership of new states in these lists to prevent looping."

"The next section introduces search algorithms that, like backtrack, use lists to keep track of states in a search space. These algorithms, including depth-first, breadth-first, and best-first (Chapter 4) search, differ from backtrack in providing a more flexible basis for implementing alternative graph search strategies."

"In addition to specifying a search direction (data-driven or goal-driven), a search algorithm must determine the order in which states are examined in the tree or the graph. This section considers two possibilities for the order in which the nodes of the graph are considered: depth-first and breadth-first search."

The graph (Figure 3.15):

"Consider the graph represented in Figure 3.15. States are labeled (A, B, C, ...) so that they can be referred to in the discussion that follows. In depth-first search, when a state is examined, all of its children and their descendants are examined before any of its siblings. Depth-first search goes deeper into the search space whenever this is possible. Only when no further descendants of a state can be found are its siblings considered. Depth-first search examines the states in the graph of Figure 3.15 in the order A, B, E, K, S, L, T, F, M, C, G, N, H, O, P, U, D, I, Q, J, R. The backtrack algorithm of Section 3.2.2 implemented depth-first search."

"Breadth-first search, in contrast, explores the space in a level-by-level fashion. Only when there are no more states to be explored at a given level does the algorithm move on to the next deeper level. A breadth-first search of the graph of Figure 3.15 considers the states in the order A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U."

Breadth-first search:

"We implement breadth-first search using lists, open and closed, to keep track of progress through the state space. open, like NSL in backtrack, lists states that have been generated but whose children have not been examined. The order in which states are removed from open determines the order of the search. closed records states already examined. closed is the union of the DE and SL lists of the backtrack algorithm."

"function breadth_first_search; begin open := [Start]; % initialize closed := [] while open ≠ [] do % states remain begin remove leftmost state from open, call it X; if X is a goal then return SUCCESS % goal found else begin generate children of X; put X on closed; discard children of X if already on open or closed; % loop check put remaining children on right end of open % queue end end return FAIL % no states left end."

"Child states are generated by inference rules, legal moves of a game, or other state transition operators. Each iteration produces all children of the state X and adds them to open. Note that open is maintained as a queue, or first-in-first-out (FIFO) data structure. States are added to the right of the list and removed from the left. This biases search toward the states that have been on open the longest, causing the search to be breadth-first. Child states that have already been discovered (already appear on either open or closed) are discarded. If the algorithm terminates because the condition of the 'while' loop is no longer satisfied (open = []) then it has searched the entire graph without finding the desired goal: the search has failed."

The trace of breadth_first_search on Figure 3.15:

"A trace of breadth_first_search on the graph of Figure 3.15 follows. Each successive number, 2,3,4, ... , represents an iteration of the 'while' loop. U is the goal state.

open = [A]; closed = []
open = [B,C,D]; closed = [A]
open = [C,D,E,F]; closed = [B,A]
open = [D,E,F,G,H]; closed = [C,B,A]
open = [E,F,G,H,I,J]; closed = [D,C,B,A]
open = [F,G,H,I,J,K,L]; closed = [E,D,C,B,A]
open = [G,H,I,J,K,L,M] (as L is already on open); closed = [F,E,D,C,B,A]
open = [H,I,J,K,L,M,N]; closed = [G,F,E,D,C,B,A]
and so on until either U is found or open = []."

"Figure 3.16 illustrates the graph of Figure 3.15 after six iterations of breadth_first_search. The states on open and closed are highlighted. States not shaded have not been discovered by the algorithm. Note that open records the states on the 'frontier' of the search at any stage and that closed records states already visited."

"Because breadth-first search considers every node at each level of the graph before going deeper into the space, all states are first reached along the shortest path from the start state. Breadth-first search is therefore guaranteed to find the shortest path from the start state to the goal. Furthermore, because all states are first found along the shortest path, any states encountered a second time are found along a path of equal or greater length. Because there is no chance that duplicate states were found along a better path, the algorithm simply discards any duplicate states."

"It is often useful to keep other information on open and closed besides the names of the states. For example, note that breadth_first_search does not maintain a list of states on the current path to a goal as backtrack did on the list SL; all visited states are kept on closed. If a solution path is required, it can not be returned by this algorithm. The solution can be found by storing ancestor information along with each state. A state may be saved along with a record of its parent state, e.g., as a (state, parent) pair. If this is done in the search of Figure 3.15, the contents of open and closed at the fourth iteration would be:

open = [(D,A), (E,B), (F,B), (G,C), (H,C)]; closed = [(C,A), (B,A), (A,nil)]

The path (A, B, F) that led from A to F could easily be constructed from this information. When a goal is found, the algorithm can construct the solution path by tracing back along parents from the goal to the start state. Note that state A has a parent of nil, indicating that it is a start state; this stops reconstruction of the path. Because breadth-first search finds each state along the shortest path and retains the first version of each state, this is the shortest path from a start to a goal."

"Figure 3.17 shows the states removed from open and examined in a breadth-first search of the graph of the 8-puzzle. As before, arcs correspond to moves of the blank up, to the right, down, and to the left. The number next to each state indicates the order in which it was removed from open. States left on open when the algorithm halted are not shown."

Depth-first search:

"Next, we create a depth-first search algorithm, a simplification of the backtrack algorithm already presented in Section 3.2.3. In this algorithm, the descendant states are added and removed from the left end of open: open is maintained as a stack, or last-in-first-out (LIFO) structure. The organization of open as a stack directs search toward the most recently generated states, producing a depth-first search order:"

"function depth_first_search; begin open := [Start]; % initialize closed := [] while open ≠ [] do % states remain begin remove leftmost state from open, call it X; if X is a goal then return SUCCESS % goal found else begin generate children of X; put X on closed; discard children of X if already on open or closed; % loop check put remaining children on left end of open % stack end end; return FAIL % no states left end."

The trace of depth_first_search on Figure 3.15:

"A trace of depth_first_search on the graph of Figure 3.15 appears below. Each successive iteration of the 'while' loop is indicated by a single line (2, 3, 4, ...). The initial states of open and closed are given on line 1. Assume U is the goal state.

open = [A]; closed = []
open = [B,C,D]; closed = [A]
open = [E,F,C,D]; closed = [B,A]
open = [K,L,F,C,D]; closed = [E,B,A]
open = [S,L,F,C,D]; closed = [K,E,B,A]
open = [L,F,C,D]; closed = [S,K,E,B,A]
open = [T,F,C,D]; closed = [L,S,K,E,B,A]
open = [F,C,D]; closed = [T,L,S,K,E,B,A]
open = [M,C,D], (as L is already on closed); closed = [F,T,L,S,K,E,B,A]
open = [C,D]; closed = [M,F,T,L,S,K,E,B,A]
open = [G,H,D]; closed = [C,M,F,T,L,S,K,E,B,A]
and so on until either U is discovered or open = []."

"As with breadth_first_search, open lists all states discovered but not yet evaluated (the

current 'frontier' of the search), and closed records states already considered. Figure 3.18 shows the graph of Figure 3.15 at the sixth iteration of the depth_first_search. The contents of open and closed are highlighted. As with breadth_first_search, the algorithm could store a record of the parent along with each state, allowing the algorithm to reconstruct the path that led from the start state to a goal."

"Unlike breadth-first search, a depth-first search is not guaranteed to find the shortest path to a state the first time that state is encountered. Later in the search, a different path may be found to any state. If path length matters in a problem solver, when the algorithm encounters a duplicate state, the algorithm should save the version reached along the shortest path. This could be done by storing each state as a triple: (state, parent, length_of_path). When children are generated, the value of the path length is simply incremented by one and saved with the child. If a child is reached along multiple paths, this information can be used to retain the best version. This is treated in more detail in the discussion of algorithm A in Chapter 4. Note that retaining the best version of a state in a simple depth-first search does not guarantee that a goal will be reached along the shortest path."

"Figure 3.19 gives a depth-first search of the 8-puzzle. As noted previously, the space is generated by the four 'move blank' rules (up, down, left, and right). The numbers next to the states indicate the order in which they were considered, i.e., removed from open. States left on open when the goal is found are not shown. A depth bound of 5 was imposed on this search to keep it from getting lost deep in the space."

Breadth-first vs depth-first — the comparison:

"As with choosing between data- and goal-driven search for evaluating a graph, the choice of depth-first or breadth-first search depends on the specific problem being solved. Significant features include the importance of finding the shortest path to a goal, the branching factor of the space, the available compute time and space resources, the average length of paths to a goal node, and whether we want all solutions or only the first solution. In making these decisions, there are advantages and disadvantages for each approach."

"Breadth-First: Because it always examines all the nodes at level n before proceeding to level n + 1, breadth-first search always finds the shortest path to a goal node. In a problem where it is known that a simple solution exists, this solution will be found. Unfortunately, if there is a bad branching factor, i.e., states have a high average number of children, the combinatorial explosion may prevent the algorithm from finding a solution using available memory. This is due to the fact that all unexpanded nodes for each level of the search must be kept on open. For deep searches, or state spaces with a high branching factor, this can become quite cumbersome."

"The space utilization of breadth-first search, measured in terms of the number of states on open, is an exponential function of the length of the path at any time. If each state has an average of B children, the number of states on a given level is B times the number of states on the previous level. This gives B^n states on level n. Breadth-first search would place all of these on open when it begins examining level n. This can be prohibitive if solution paths are long, in the game of chess, for example."

"Depth-First: Depth-first search gets quickly into a deep search space. If it is known that the solution path will be long, depth-first search will not waste time searching a large number of 'shallow' states in the graph. On the other hand, depth-first search can get 'lost' deep in a graph, missing shorter paths to a goal or even becoming stuck in an infinitely long path that does not lead to a goal."

"Depth-first search is much more efficient for search spaces with many branches because it does not have to keep all the nodes at a given level on the open list. The space usage of depth-first search is a linear function of the length of the path."

5.4 Depth-First Search with Iterative Deepening (Section 3.2.4)

"A nice compromise on these trade-offs is to use a depth bound on depth-first search. The depth bound forces a failure on a search path once it gets below a certain level. This causes a breadth-first like sweep of the search space at that depth level. When it is known that a solution lies within a certain depth or when time constraints, such as those that occur in an extremely large space like chess, limit the number of states that can be considered; then a depth-first search with a depth bound may be most appropriate. Figure 3.19 showed a depth-first search of the 8-puzzle in which a depth bound of 5 caused the sweep across the space at that depth."

"This insight leads to a search algorithm that remedies many of the drawbacks of both depth-first and breadth-first search. Depth-first iterative deepening (Korf 1987) performs a depth-first search of the space with a depth bound of 1. If it fails to find a goal, it performs another depth-first search with a depth bound of 2. This continues, increasing the depth bound by one each time. At each iteration, the algorithm performs a complete depth-first search to the current depth bound. No information about the state space is retained between iterations."

"Because the algorithm searches the space in a level-by-level fashion, it is guaranteed to find a shortest path to a goal. Because it does only depth-first search at each iteration, the space usage at any level n is B × n, where B is the average number of children of a node."

"Interestingly, although it seems as if depth-first iterative deepening would be much less efficient than either depth-first or breadth-first search, its time complexity is actually of the same order of magnitude as either of these: $O(B^n)$. An intuitive explanation for this seeming paradox is given by Korf (1987): 'Since the number of nodes in a given level of the tree grows exponentially with depth, almost all the time is spent in the deepest level, even though shallower levels are generated an arithmetically increasing number of times.'"

"Unfortunately, all the search strategies discussed in this chapter—depth-first, breadth-first, and depth-first iterative deepening—may be shown to have worst-case exponential time complexity. This is true for all uninformed search algorithms. The only approaches to search that reduce this complexity employ heuristics to guide search. Best-first search is a search algorithm that is similar to the algorithms for depth- and breadth-first search just presented. However, best-first search orders the states on the open list according to some heuristic estimate of their 'closeness' to a goal. Thus, each iteration of the loop considers the most 'promising' state on the open list. The pseudo-code for the function best_first_search is presented in Chapter 4."

6. USING THE STATE SPACE TO REPRESENT REASONING WITH THE PROPOSITIONAL AND PREDICATE CALCULUS (Section 3.3)

6.1 State Space Description of a Logic System (Section 3.3.1)

"When we defined state space graphs in Section 3.1, we noted that nodes must be distinguishable from one another, with each node representing some state of the solution process. The propositional and predicate calculus can be used as the formal specification language for making these distinctions as well as for mapping the nodes of a graph onto the state space. Furthermore, inference rules can be used to create and describe the arcs between states. In this fashion, problems in the predicate calculus, such as determining whether a particular expression is a logical consequence of a given set of assertions, may be solved using search."

"The soundness and completeness of predicate calculus inference rules can guarantee the correctness of conclusions derived through this form of graph-based reasoning. This ability to produce a formal proof of the integrity of a solution through the same algorithm that produces the solution is a unique attribute of much artificial intelligence and theorem proving based problem solving."

"Although many problems' states, e.g., tic-tac-toe, can be more naturally described by other data structures, such as arrays, the power and generality of logic allow much of AI problem solving to use the propositional and predicate calculus descriptions and inference rules. Other AI representations such as rules (Chapter 8), semantic networks, or frames (Chapter 7) also employ search strategies similar to those presented in Section 3.2."

6.2 Example 3.3.1: The Propositional Calculus

"The first example of how a set of logic relationships may be viewed as defining a graph is from the propositional calculus. If p, q, r… are propositions, assume the assertions:

s t s → r r → p …"

"From this set of assertions and the inference rule modus ponens, certain propositions (p, r, and u) may be inferred; others (such as v and q) may not be so inferred and indeed do not logically follow from these assertions. The relationship between the initial assertions and these inferences is expressed in the directed graph in Figure 3.20."

"In Figure 3.20 the arcs correspond to logical implications (→). Propositions that are given as true (s and t) correspond to the given data of the problem. Propositions that are logical consequences of this set of assertions correspond to the nodes that may be reached along a directed path from a state representing a true proposition; such a path corresponds to a sequence of applications of modus ponens. For example, the path [s, r, p] corresponds to the sequence of inferences:

s and s → r yields r. r and r → p yields p."

"Given this representation, determining whether a given proposition is a logical consequence of a set of propositions becomes a problem of finding a path from a boxed node (the start node) to the proposition (the goal node). Thus, the task can be cast as a graph search problem. The search strategy used here is data-driven, because it proceeds from what is known (the true propositions) toward the goal. Alternatively, a goal-directed strategy could be applied to the same state space by starting with the proposition to be proved (the goal) and searching back along arcs to find support for the goal among the true propositions. We can also search this space of inferences in either a depth-first or breadth-first fashion."

6.3 And/Or Graphs (Section 3.3.2)

"In the propositional calculus example of Section 3.3.1, all of the assertions were implications of the form p → q. We did not discuss the way in which the logic operators and and or could be represented in such a graph. Expressing the logic relationships defined by these operators requires an extension to the basic graph model defined in Section 3.1 to what we call the and/or graph. And/or graphs are an important tool for describing the search spaces generated by many AI problems, including those solved by logic-based theorem provers and expert systems."

"In expressions of the form q ∧ r → p both q and r must be true for p to be true. In expressions of the form q ∨ r → p the truth of either q or r is sufficient to prove p is true. Because implications containing disjunctive premises may be written as separate implications, this expression is often written as q → p, r → p. To represent these different relationships graphically, and/or graphs distinguish between and nodes and or nodes. If the premises of an implication are connected by an ∧ operator, they are called and nodes in the graph and the arcs from this node are joined by a curved link. The expression q ∧ r → p is represented by the and/or graph of Figure 3.21."

"The link connecting the arcs in Figure 3.21 captures the idea that both q and r must be true to prove p. If the premises are connected by an or operator, they are regarded as or nodes in the graph. Arcs from or nodes to their parent node are not so connected (Figure 3.22). This captures the notion that the truth of any one of the premises is independently sufficient to determine the truth of the conclusion."

The hypergraph definition:

"An and/or graph is actually a specialization of a type of graph known as a hypergraph, which connects nodes by sets of arcs rather than by single arcs. A hypergraph is defined as follows:"

"DEFINITION: HYPERGRAPH

A hypergraph consists of: N, a set of nodes. H, a set of hyperarcs defined by ordered pairs in which the first element of the pair is a single node from N and the second element is a subset of N.

An ordinary graph is a special case of hypergraph in which all the sets of descendant nodes have a cardinality of 1."

"Hyperarcs are also known as k-connectors, where k is the cardinality of the set of descendant nodes. If k = 1, the descendant is thought of as an or node. If k > 1, the elements of the set of descendants may be thought of as and nodes. In this case, the connector is drawn between the individual edges from the parent to each of the descendant nodes; see, for example, the curved link in Figure 3.21."

6.4 Example 3.3.2: And/Or Graph Search

"The second example is also from the propositional calculus but generates a graph that contains both and and or descendants. Assume a situation where the following propositions are true:
a b c a ∧ b → d a ∧ c → e b ∧ d → f f → g a ∧ e → h"
"This set of assertions generates the and/or graph in Figure 3.23."
"Questions that might be asked (answers deduced by the search of this graph) are:
Is h true?
Is h true if b is no longer true?
What is the shortest path (i.e., the shortest sequence of inferences) to show that X (some proposition) is true?
Show that the proposition p (note that p is not supported) is false. What does this mean? What would be necessary to achieve this conclusion?"
"And/or graph search requires only slightly more record keeping than search in regular graphs, an example of which was the backtrack algorithm, previously discussed. The or descendants are checked as they were in backtrack: once a path is found connecting a goal to a start node along or nodes, the problem will be solved. If a path leads to a failure, the algorithm may backtrack and try another branch. In searching and nodes, however, all of the and descendants of a node must be solved (or proved true) to solve the parent node."
"In the example of Figure 3.23, a goal-directed strategy for determining the truth of h first attempts to prove both a and e. The truth of a is immediate, but the truth of e requires the truth of both c and a; these are given as true. Once the problem solver has traced all these arcs down to true propositions, the true values are recombined at the and nodes to verify the truth of h."
"A data-directed strategy for determining the truth of h, on the other hand, begins with the known facts (c, a, and b) and begins adding new propositions to this set of known facts according to the constraints of the and/or graph. e or d might be the first proposition added to the set of facts. These additions make it possible to infer new facts. This process continues until the desired goal, h, has been proved."
"One way of looking at and/or graph search is that the ∧ operator (hence the and nodes of the graph) indicates a problem decomposition in which the problem is broken into subproblems such that all of the subproblems must be solved to solve the original problem. An ∨ operator in the predicate calculus representation of the problem indicates a selection, a point in the problem solution at which a choice may be made between alternative problem-solving paths or strategies, any one of which, if successful, is sufficient to solve the problem."

6.5 Example 3.3.3: MACSYMA

"One natural example of an and/or graph is a program for symbolically integrating mathematical functions. MACSYMA is a well-known program that is used extensively by mathematicians. The reasoning of MACSYMA can be represented as an and/or graph. In performing integrations, one important class of strategies involves breaking an expression into sub-expressions that may be integrated independently of one another, with the result being combined algebraically into a solution expression. Examples of this strategy include the rule for integration by parts and for decomposing the integral of a sum into the sum of the integrals of the individual terms. These strategies, representing the decomposition of a problem into independent subproblems, can be represented by and nodes in the graph."
"Another class of strategies involves the simplification of an expression through various algebraic substitutions. Because any given expression may allow a number of different substitutions, each representing an independent solution strategy, these strategies are represented by or nodes of the graph. Figure 3.24 illustrates the space searched by such a problem solver. The search of this graph is goal-directed, in that it begins with the query 'find the integral of a particular function' and searches back to the algebraic expressions that define that integral. Note that this is an example in which goal-directed search is the obvious strategy. It would be practically impossible for a problem solver to determine the algebraic expressions that formed the desired integral without working back from the query."

6.6 Example 3.3.4: Goal-Driven And/Or Search

"This example is taken from the predicate calculus and represents a goal-driven graph search where the goal to be proved true in this situation is a predicate calculus expression containing variables. The axioms are the logical descriptions of a relationship between a dog, fred, and his master, sam."
The twelve clauses:
"1. Fred is a collie. collie(fred). 2. Sam is Fred's master. master(fred, sam). 3. The day is Saturday. day(saturday). 4. It is cold on Saturday. ¬(warm(saturday)). 5. Fred is trained. trained(fred). 6. Spaniels are good dogs and so are trained collies. ∀X[spaniel(X) ∨ (collie(X) ∧ trained(X)) → gooddog(X)] 7. If a dog is a good dog and has a master then he will be with his master. ∀(X,Y,Z)[gooddog(X) ∧ master(X,Y) ∧ location(Y,Z) → location(X,Z)] 8. If it is Saturday and warm, then Sam is at the park. (day(saturday) ∧ warm(saturday)) → location(sam,park). 9. If it is Saturday and not warm, then Sam is at the museum. (day(saturday) ∧ ¬(warm(saturday))) → location(sam,museum)."
The query and search:
"The goal is the expression ∃X location(fred,X), meaning 'where is fred?' A backward search algorithm examines alternative means of establishing this goal: if fred is a good dog and fred has a master and fred's master is at a location then fred is at that location also. The premises of this rule are then examined: what does it mean to be a gooddog, etc.? This process continues, constructing the and/or graph of Figure 3.25."
"Let us examine this search in more detail, particularly because it is an example of goal-driven search using the predicate calculus and it illustrates the role of unification in the generation of the search space. The problem to be solved is 'where is fred?' More formally, it may be seen as determining a substitution for the variable X, if such a substitution exists, under which location(fred,X) is a logical consequence of the initial assertions."
"When it is desired to determine Fred's location, clauses are examined that have location as their conclusion, the first being clause 7. This conclusion, location(X,Z), is then unified with location(fred, X) by the substitutions {fred/X, X/Z}. The premises of this rule, under the same substitution set, form the and descendants of the top goal:
gooddog(fred) ∧ master(fred,Y) ∧ location(Y,X)."
"This expression may be interpreted as meaning that one way to find Fred is to see if Fred is a good dog, find out who Fred's master is, and find out where the master is. The initial goal has thus been replaced by three subgoals. These are and nodes and all of them must be solved."
"To solve these subgoals, the problem solver first determines whether Fred is a gooddog. This matches the conclusion of clause 6 using the substitution {fred/X}. The premise of clause 6 is the or of two expressions:
spaniel(fred) ∨ (collie(fred) ∧ trained(fred))"
"The first of these or nodes is spaniel(fred). The database does not contain this assertion, so the problem solver must assume it is false. The other or node is (collie(fred) ∧ trained(fred)), i.e., is Fred a collie and is Fred trained. Both of these need to be true, which they are by clauses 1 and 5."
"This proves that gooddog(fred) is true. The problem solver then examines the second of the premises of clause 7: master(X,Y). Under the substitution {fred/X}, master(X,Y) becomes master(fred,Y), which unifies with the fact (clause 2) of master(fred,sam). This produces the unifying substitution of {sam/Y}, which also gives the value of sam to the third subgoal of clause 7, creating the new goal location(sam,X)."
"In solving this, assuming the problem solver tries rules in order, the goal location(sam,X) will first unify with the conclusion of clause 7. Note that the same rule is being tried with different bindings for X. Recall (Chapter 2) that X is a 'dummy' variable and could have any name (any string beginning with an uppercase letter). Because the extent of the meaning of any variable name is contained within the clause in which it appears, the predicate calculus has no global variables. Another way of saying this is that values of variables are passed to other clauses as parameters and have no fixed (memory) locations. Thus, the multiple occurrences of X in different rules in this example indicate different formal parameters (Section 14.3)."
"In attempting to solve the premises of rule 7 with these new bindings, the problem solver will fail because sam is not a gooddog. Here, the search will backtrack to the goal location(sam,X) and try the next match, the conclusion of rule 8. This will also fail, which will cause another backtrack and a unification with the conclusion of clause 9, at(sam,museum)."
"Because the premises of clause 9 are supported in the set of assertions (clauses 3 and 4), it follows that the conclusion of 9 is true. This final unification goes all the way back up the tree to finally answer ∃X location(fred,X) with location(fred, museum)."
"It is important to examine carefully the nature of the goal-driven search of a graph and compare it with the data-driven search of Example 3.3.2. Further discussion of this issue, including a more rigorous comparison of these two methods of searching a graph, continues in the next example, but is seen in full detail only in the discussion of production systems in Chapter 6 and in the application to expert systems in Part IV. Another point implicit in this example is that the order of clauses affects the order of search. In the example above, the multiple location clauses were tried in order, with backtracking search eliminating those that failed to be proved true."

6.7 Example 3.3.5: The Financial Advisor Revisited

"In the last example of Chapter 2 we used predicate calculus to represent a set of rules for giving investment advice. In that example, modus ponens was used to infer a proper investment for a particular individual. We did not discuss the way in which a program might determine the appropriate inferences. This is, of course, a search problem; the present example illustrates one approach to implementing the logic-based financial advisor, using goal-directed, depth-first search with backtracking. The discussion uses the predicates found in Section 2.4; these predicates are not duplicated here."
"Assume that the individual has two dependents, $20,000 in savings, and a steady income of $30,000. As discussed in Chapter 2, we can add predicate calculus expressions describing these facts to the set of predicate calculus expressions. Alternatively, the program may begin the search without this information and ask the user to add it as needed. This has the advantage of not requiring data that may not prove necessary for a solution. This approach, often taken with expert systems, is illustrated here."
"In performing a consultation, the goal is to find an investment; this is represented with the predicate calculus expression ∃X investment(X), where X in the goal variable we seek to bind. There are three rules (1, 2, and 3) that conclude about investments, because the query will unify with the conclusion of these rules. If we select rule 1 for initial exploration, its premise savings_account(inadequate) becomes the subgoal, i.e., the child node that will be expanded next."
"In generating the children of savings_account(inadequate), the only rule that may be applied is rule 5. This produces the and node:
amount_saved(X) ∧ dependents(Y) ∧ greater(X, minsavings(Y))."
"If we attempt to satisfy these in left-to-right order, amount_saved(X) is taken as the first subgoal. Because the system contains no rules that conclude this subgoal, it will query the user. When amount_saved(20000) is added the first subgoal will succeed, with unification substituting 20000 for X. Note that because an and node is being searched, a failure here would eliminate the need to examine the remainder of the expression."
"Similarly, the subgoal dependents(Y) leads to a user query, and the response, dependents(2), is added to the logical description. The subgoal matches this expression with the substitution {2/Y}. With these substitutions, the search next evaluates the truth of
greater(20000, minsavings(2))."
"This evaluates to false, causing failure of the entire and node. The search then backtracks to the parent node, savings_account(inadequate), and attempts to find an alternative way to prove that node true. This corresponds to the generation of the next child in the search. Because no other rules conclude this subgoal, search fails back to the top-level goal, investment(X). The next rule whose conclusions unify with this goal is rule 2, producing the new subgoals
savings_account(adequate) ∧ income(adequate)."

"Continuing the search, savings_account(adequate) is proved true as the conclusion of rule 4, and income(adequate) follows as the conclusion of rule 6. Although the details of the remainder of the search will be left to the reader, the and/or graph that is ultimately explored appears in Figure 3.26."

6.8 Example 3.3.6: An English Language Parser and Sentence Generator

"Our final example is not from the predicate calculus but consists of a set of rewrite rules for parsing sentences in a subset of English grammar. Rewrite rules take an expression and transform it into another by replacing the pattern on one side of the arrow (↔) with the pattern on the other side. For example, a set of rewrite rules could be defined to change an expression in one language, such as English, into another language (perhaps French or a predicate calculus clause). The rewrite rules given here transform a subset of English sentences into higher level grammatical constructs such as noun phrase, verb phrase, and sentence. These rules are used to parse sequences of words, i.e., to determine whether they are well-formed sentences (whether they are grammatically correct or not) and to model the linguistic structure of the sentences."

The five grammar rules:

"1. sentence ↔ np vp (A sentence is a noun phrase followed by a verb phrase.) 2. np ↔ n (A noun phrase is a noun.) 3. np ↔ art n (A noun phrase is an article followed by a noun.) 4. vp ↔ v (A verb phrase is a verb.) 5. vp ↔ v np (A verb phrase is a verb followed by a noun phrase.)"

The dictionary (terminals):

"6. art ↔ a 7. art ↔ the ('a' and 'the' are articles) 8. n ↔ man 9. n ↔ dog ('man' and 'dog' are nouns) 10. v ↔ likes 11. v ↔ bites ('likes' and 'bites' are verbs)"

"These rewrite rules define the and/or graph of Figure 3.27, where sentence is the root. The elements on the left of a rule correspond to and nodes in the graph. Multiple rules with the same conclusion form the or nodes. Notice that the leaf or terminal nodes of this graph are the English words in the grammar (hence, they are called terminals)."

"An expression is well formed in a grammar if it consists entirely of terminal symbols and there is a series of substitutions in the expression using rewrite rules that reduce it to the sentence symbol. Alternatively, this may be seen as constructing a parse tree that has the words of the expression as its leaves and the sentence symbol as its root."

The parsing trace:

"1. The first rule that will match is 7, rewriting the as art. This yields: art dog bites the man. 2. The next iteration would find a match for 7, yielding art dog bites art man. 3. Rule 8 will fire, producing art dog bites art n. 4. Rule 3 will fire to yield art dog bites np. 5. Rule 9 produces art n bites np. 6. Rule 3 may be applied, giving np bites np. 7. Rule 11 yields np v np. 8. Rule 5 yields np vp. 9. Rule 1 reduces this to sentence, accepting the expression as correct."

"The above example implements a data-directed depth-first parse, as it always applies the highest-level rule to the expression; e.g., art n reduces to np before bites reduces to v. Parsing could also be done in a goal-directed fashion, taking sentence as the starting string and finding a series of replacements of patterns that match left-hand sides of rules leading to a series of terminals that match the target sentence."

"Parsing is important, not only for natural language (Chapter 15) but also for constructing compilers and interpreters for computer languages (Aho and Ullman 1977). The literature is full of parsing algorithms for all classes of languages. For example, many goal-directed parsing algorithms look ahead in the input stream to determine which rule to apply next."

"In this example we have taken a very simple approach of searching the and/or graph in an uninformed fashion. One thing that is interesting in this example is the implementation of the search. This approach of keeping a record of the current expression and trying to match the rules in order is an example of using a production system to implement search. This is a major topic of Chapter 6."

"Rewrite rules are also used to generate legal sentences according to the specifications of the grammar. Sentences may be generated by a goal-driven search, beginning with sentence as the top-level goal and ending when no more rules can be applied. This produces a string of terminal symbols that is a legal sentence in the grammar. For example: A sentence is a np followed by a vp (rule 1). np is replaced by n (rule 2), giving n vp. man is the first n available (rule 8), giving man vp. Now np is satisfied and vp is attempted. Rule 3 replaces vp with v, man v. Rule 10 replaces v with likes. man likes is found as the first acceptable sentence."

"If it is desired to create all acceptable sentences, this search may be systematically repeated until all possibilities are tried and the entire state space has been searched exhaustively. This generates sentences including a man likes, the man likes, and so on. There are about 80 correct sentences that are produced by an exhaustive search. These include such semantic anomalies as the man bites the dog."

"Parsing and generation can be used together in a variety of ways to handle different problems. For instance, if it is desired to find all sentences to complete the string 'the man,' then the problem solver may be given an incomplete string the man… . It can work upward in a data-driven fashion to produce the goal of completing the sentence rule (rule 1), where np is replaced by the man, and then work in a goal-driven fashion to determine all possible vps that will complete the sentence. This would create sentences such as the man likes, the man bites the man, and so on. Again, this example deals only with syntactic correctness. The issue of semantics (whether the string has a mapping into some 'world' with 'truth') is entirely different. Chapter 2 examined the issue of constructing a logic-based semantics for expressions; for verifying sentences in natural language, the issue is much more difficult and is discussed in Chapter 15."

7. EPILOGUE AND REFERENCES (Section 3.4)

"Chapter 3 introduced the theoretical foundations of state space search, using graph theory to analyze the structure and complexity of problem-solving strategies. The chapter compared data-driven and goal-driven reasoning and depth-first and breadth-first search. And/or graphs allow us to apply state space search to the implementation of logic-based reasoning."

Luger's recommended references:

"Basic graph search is discussed in a number of textbooks on computer algorithms. These include Introduction to Algorithms by Thomas Cormen, Charles Leiserson, and Ronald Rivest (1990), Walls and Mirrors by Paul Helman and Robert Veroff (1986), Algorithms by Robert Sedgewick (1983), and Fundamentals of Computer Algorithms by Ellis Horowitz and Sartaj Sahni (1978). Finite automata are presented in Lindenmayer and Rosenberg (1976). More algorithms for and/or search are presented in Chapter 14, Automated Reasoning."

"The use of graph search to model intelligent problem solving is presented in Human Problem Solving by Alan Newell and Herbert Simon (1972). Artificial intelligence texts that discuss search strategies include Nils Nilsson's Artificial Intelligence (1998), Patrick Winston's Artificial Intelligence (1992), and Artificial Intelligence by Eugene Charniak and Drew McDermott (1985). Heuristics by Judea Pearl (1984) presents search algorithms and lays a groundwork for the material we present in Chapter 4. Developing new techniques for graph search are often topics at the annual AI conferences."

8. EXERCISES (Section 3.5)

The fifteen exercises exactly as Luger lists them:

"1. A Hamiltonian path is a path that uses every node of the graph exactly once. What conditions are necessary for such a path to exist? Is there such a path in the Königsberg map? 2. Give the graph representation for the farmer, wolf, goat, and cabbage problem: A farmer with his wolf, goat, and cabbage come to the edge of a river they wish to cross. There is a boat at the river's edge, but, of course, only the farmer can row. The boat also can carry only two things (including the rower) at a time. If the wolf is ever left alone with the goat, the wolf will eat the goat; similarly, if the goat is left alone with the cabbage, the goat will eat the cabbage. Devise a sequence of crossings of the river so that all four characters arrive safely on the other side of the river. Let nodes represent states of the world; e.g., the farmer and the goat are on the west bank and the wolf and cabbage on the east. Discuss the advantages of breadth-first and depth-first search for this problem. 3. Build a finite state acceptor that recognizes all strings of binary digits a) that contain '111', b) that end in '111', c) that contain '111' but not more that three consecutive '1's. 4. Give an instance of the traveling salesperson problem for which the nearest-neighbor strategy fails to find an optimal path. Suggest another heuristic for this problem. 5. 'Hand run' the backtrack algorithm on the graph in Figure 3.29. Begin from state A. Keep track of the successive values of NSL, SL, CS, etc. 6. Implement a backtrack algorithm in a programming language of your choice. 7. Determine whether goal-driven or data-driven search would be preferable for solving each of the following problems. Justify your answer. a. Diagnosing mechanical problems in an automobile. b. You have met a person who claims to be your distant cousin, with a common ancestor named John Doe. You would like to verify her claim. c. Another person claims to be your distant cousin. He does not know the common ancestor's name but knows that it was no more than eight generations back. You would like to either find this ancestor or determine that she did not exist. d. A theorem prover for plane geometry. e. A program for examining sonar readings and interpreting them, such as telling a large submarine from a small submarine from a whale from a school of fish. f. An expert system that will help a human classify plants by species, genus, etc. 8. Choose and justify a choice of breadth- or depth-first search for examples of Exercise 6. 9. Write a backtrack algorithm for and/or graphs. 10. Trace the goal-driven good-dog problem of Example 3.3.4 in a data-driven fashion. 11. Give another example of an and/or graph problem and develop part of the search space. 12. Trace a data-driven execution of the financial advisor of Example 3.3.5 for the case of an individual with four dependents, $18,000 in the bank, and a steady income of $25,000 per year. Based on a comparison of this problem and the example in the text, suggest a generally 'best' strategy for solving the problem. 13. Add rules for adjectives and adverbs to the English grammar of Example 3.3.6. 14. Add rules for (multiple) prepositional phrases to the English grammar of Example 3.3.6. 15. Add grammar rules to Example 3.3.6 that allow complex sentences such as, sentence ↔ sentence AND sentence."

9. QUICK-REFERENCE TABLE OF KEY DEFINITIONS

Term	Definition
Graph	A set of nodes $N_1, N_2, \ldots, N_n$ and a set of arcs connecting pairs of nodes
Arc (N_3, N_4)	Ordered pair connecting N_3 to N_4 (not back unless reverse arc exists)
Labeled graph	Nodes or arcs carry descriptors
Directed graph	Arcs have direction
Rooted graph	Unique root from which all paths originate; root has no parent
Tip / leaf	Node with no children
Path	Ordered sequence of nodes where each consecutive pair is an arc
Path length	$n - 1$ for a path of n nodes
Cycle / loop	Path containing a node more than once
Tree	Graph with a unique path between every pair of nodes
Connected	Path exists including both nodes
Parent	N_j is parent of N_k if arc (N_j, N_k) exists
Child	N_k is child of N_j if arc (N_j, N_k) exists
Sibling	Two nodes with the same parent
Ancestor	Node preceding another along a path
Descendant	Node following another along a path
DAG	Directed acyclic graph
FSM	Ordered triple (S, I, F) with state transition function
Finite state acceptor (Moore machine)	FSM with start state s_0 and accept state(s) $\{s_n\}$
State space	Four-tuple [N, A, S, GD]
Goal type 1	A measurable property of the states encountered in the search
Goal type 2	A measurable property of the path (e.g. sum of transition costs)
Solution path	Path from a node in S to a node in GD
Backtrack	Systematic trial of paths using SL, NSL, DE
SL	State List — current path
NSL	New State List — open list for backtracking
DE	Dead Ends — failed states
Breadth-first search	open as queue (FIFO); finds shortest path
Depth-first search	open as stack (LIFO); does not guarantee shortest path
Iterative deepening	DFS with increasing depth bounds; shortest path with linear memory
And node	All children must be satisfied (curved link)
Or node	Any child suffices (no link)
Hypergraph	Nodes + hyperarcs (single node, subset of N)
k-connector	Hyperarc with k descendants
Rewrite rule	Rule transforming one expression to another by pattern replacement
Terminals	The actual words at the leaves of a grammar graph
Nonterminals	Grammatical categories (sentence, np, vp)

Topic	Belongs to
Heuristic function f = g + h	Chapter 4
Admissibility, monotonicity, informedness	Chapter 4
Hill climbing	Chapter 4
Best-first search	Chapter 4
Minimax, alpha-beta pruning	Chapter 4
Probability measures	Chapter 5
Probabilistic FSM	Chapter 5
Production systems	Chapter 6
Blackboard architecture	Chapter 6
Resolution theorem proving	Chapter 14
Natural language parsing (ATN, Earley, stochastic)	Chapter 15

End of Chapter 3 notes, based strictly on Luger's textbook.

Chapter 4: Heuristic Search

Complete Study Notes — Based strictly on Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th ed., Chapter 4

1. WHY CHAPTER 4 EXISTS

Luger opens Chapter 4 with the Newell and Simon quote (1976 Turing Award Lecture):

"The task that a symbol system is faced with, then, when it is presented with a problem and a problem space, is to use its limited processing resources to generate possible solutions, one after another, until it finds one that satisfies the problem defining test. If the symbol system had some control over the order in which potential solutions were generated, then it would be desirable to arrange this order of generation so that actual solutions would have a high likelihood of appearing early. A symbol system would exhibit intelligence to the extent that it succeeded in doing this. Intelligence for a system with limited processing resources consists in making wise choices of what to do next…"

Chapter 3 showed that exhaustive search is impossible for interesting problems (chess ≈ 10^{120} states). Chapter 4 introduces the technique that makes search tractable: **heuristics**.

2. INTRODUCTION (Section 4.0)

Definition of heuristic:

"George Polya defines heuristic as 'the study of the methods and rules of discovery and invention' (Polya 1945). This meaning can be traced to the term's Greek root, the verb eurisco, which means 'I discover.' When Archimedes emerged from his famous bath clutching the golden crown, he shouted 'Eureka!' meaning 'I have found it!'. In state space search, heuristics are formalized as rules for choosing those branches in a state space that are most likely to lead to an acceptable problem solution."

Two basic situations where AI problem solvers use heuristics:

"1. A problem may not have an exact solution because of inherent ambiguities in the problem statement or available data. Medical diagnosis is an example of this. A given set of symptoms may have several possible causes; doctors use heuristics…"

"2. A problem may have an exact solution, but the computational cost of finding it may be prohibitive. In many problems (such as chess), state space growth is combinatorially explosive, with the number of possible states increasing exponentially or factorially with the depth of the search. In these cases, exhaustive, brute-force search techniques such as depth-first or breadth-first search may fail to find a solution within any practical length of time. Heuristics attack this complexity by guiding the search along the most 'promising' path through the space. By eliminating unpromising states and their descendants from consideration, a heuristic algorithm can (its designer hopes) defeat this combinatorial explosion and find an acceptable solution."

Heuristics are fallible:

"Unfortunately, like all rules of discovery and invention, heuristics are fallible. A heuristic is only an informed guess of the next step to be taken in solving a problem. It is often based on experience or intuition. Because heuristics use limited information, such as knowledge of the present situation or descriptions of states currently on the open list, they are not always able to predict the exact behavior of the state space farther along in the search. A heuristic can lead a search algorithm to a suboptimal solution or fail to find any solution at all. This is an inherent limitation of heuristic search. It cannot be eliminated by 'better' heuristics or more efficient search algorithms (Garey and Johnson 1979)."

Importance of heuristics:

"Heuristics and the design of algorithms to implement heuristic search have long been a core concern of artificial intelligence. Game playing and theorem proving are two of the oldest applications in artificial intelligence; both of these require heuristics to prune spaces of possible solutions. It is not feasible to examine every inference that can be made in a mathematics domain or every possible move that can be made on a chessboard. Heuristic search is often the only practical answer."

"Expert systems research has affirmed the importance of heuristics as an essential component of problem solving. When a human expert solves a problem, he or she examines the available information and makes a decision. The 'rules of thumb' that a human expert uses to solve problems efficiently are largely heuristic in nature. These heuristics are extracted and formalized by expert systems designers, as we see in Chapter 8."

The tic-tac-toe example:

"It is useful to think of heuristic search from two perspectives: the heuristic measure and an algorithm that uses heuristics to search the state space. In Section 4.1, we implement heuristics with hill-climbing and dynamic programming algorithms. In Section 4.2 we present an algorithm for best-first search. The design and evaluation of the effectiveness of heuristics is presented in Section 4.3, and game playing heuristics in Section 4.4."

"Consider heuristics in the game of tic-tac-toe, Figure II.5. The combinatorics for exhaustive search are high but not insurmountable. Each of the nine first moves has eight possible responses, which in turn have seven continuing moves, and so on. A simple analysis puts the total number of states for exhaustive search at 9 × 8 × 7 × … or 9!."

Symmetry reduction:

"Symmetry reduction decreases the search space. Many problem configurations are actually equivalent under symmetric operations of the game board. Thus, there are not nine but really three initial moves: to a corner, to the center of a side, and to the center of the grid. Use of symmetry on the second level further reduces the number of paths through the space to 12 × 7!, as seen in Figure 4.1. Symmetries in a game space such as this may be described as mathematical invariants, that, when they exist, can often be used to tremendous advantage in reducing search."

The "most wins" heuristic:

"A simple heuristic, however, can almost eliminate search entirely: we may move to the state in which X has the most winning opportunities. (The first three states in the tic-tac-toe game are so measured in Figure 4.2.) In case of states with equal numbers of potential wins, take the first such state found. The algorithm then selects and moves to the state with the highest number of opportunities. In this case X takes the center of the grid. Note that not only are the other two alternatives eliminated, but so are all their descendants. Two-thirds of the full space is pruned away with the first move, Figure 4.3."

"After the first move, the opponent can choose either of two alternative moves (as seen in Figure 4.3). Whichever is chosen, the heuristic can be applied to the resulting state of the game, again using 'most winning opportunities' to select among the possible moves. As search continues, each move evaluates the children of a single node; exhaustive search is not required. Figure 4.3 shows the reduced search after three steps in the game. States are marked with their heuristic values. Although not an exact calculation of search size for this 'most wins' strategy for tic-tac-toe, a crude upper bound can be computed by assuming a maximum of five moves in a game with five options per move. In reality, the number of states is smaller, as the board fills and reduces options. This crude bound of 25 states is an improvement of four orders of magnitude over 9!."

3. HILL-CLIMBING AND DYNAMIC PROGRAMMING (Section 4.1)

3.1 Hill-Climbing (Section 4.1.1)

"The simplest way to implement heuristic search is through a procedure called hill-climbing (Pearl 1984). Hill-climbing strategies expand the current state of the search and evaluate its children. The 'best' child is selected for further expansion; neither its siblings nor its parent are retained. Hill climbing is named for the strategy that might be used by an eager, but blind mountain climber: go uphill along the steepest possible path until you can go no farther up. Because it keeps no history, the algorithm cannot recover from failures of its strategy. An example of hill-climbing in tic-tac-toe was the 'take the state with the most possible wins' that we demonstrated in Section 4.0."

The local maximum problem:

"A major problem of hill-climbing strategies is their tendency to become stuck at local maxima. If they reach a state that has a better evaluation than any of its children, the algorithm falters. If this state is not a goal, but just a local maximum, the algorithm may fail to find the best solution. That is, performance might well improve in a limited setting, but because of the shape of the entire space, it may never reach the overall best. An example of local maxima in games occurs in the 8-puzzle. Often, in order to move a particular tile to its destination, other tiles already in goal position need be moved out. This is necessary to solve the puzzle but temporarily worsens the board state. Because 'better' need not be 'best' in an absolute sense, search methods without backtracking or some other recovery mechanism are unable to distinguish between local and global maxima."

"Figure 4.4 is an example of the local maximum dilemma. Suppose, exploring this search space, we arrive at state X, wanting to maximize state values. The evaluations of X's children, grand children, and great grandchildren demonstrate that hill-climbing can get confused even with multiple level look ahead. There are methods for getting around this problem, such as randomly perturbing the evaluation function, but in general there is no way of guaranteeing optimal performance with hill-climbing techniques. Samuel's (1959) checker program offers an interesting variant of the hill climbing algorithm."

Samuel's checker program:

"Samuel's program was exceptional for its time, 1959, particularly given the limitations of the 1950s computers. Not only did Samuel's program apply heuristic search to checker playing, but it also implemented algorithms for optimal use of limited memory, as well as a simple form of learning. Indeed, it pioneered many of the techniques still used in game-playing and machine learning programs."

"Samuel's program evaluated board states with a weighted sum of several different heuristic measures:

$\Sigma\ a_i x_i$"

"The x_i in this sum represented features of the game board such as piece advantage, piece location, control of center board, opportunities to sacrifice pieces for advantage, and even a calculation of moments of inertia of one player's pieces about an axis of the board. The a_i coefficients of these x_i were specially tuned weights that tried to model the importance of that factor in the overall board evaluation. Thus, if piece advantage was more important than control of the center, the piece advantage coefficient would reflect this."

"Samuel's program would look ahead in the search space the desired number of levels or plies (usually imposed by space and/or time limitations of the computer) and evaluate all the states at that level with the evaluation polynomial. Using a variation on minimax (Section 4.3), it propagated these values back up the graph. The checker player would then move to the best state; after the opponent's move, the process would be repeated for the new board state."

"If the evaluation polynomial led to a losing series of moves, the program adjusted its coefficients in an attempt to improve performance. Evaluations with large coefficients were given most of the blame for losses and had their weights decreased, while smaller weights were increased to give these evaluations more influence. If the program won, the opposite was done. The program trained by playing either against a human partner or against another version of itself."

"Samuel's program thus took a hill-climbing approach to learning, attempting to improve performance through local improvements on the evaluation polynomial. Samuel's checker player was able to improve its performance until it played a very good game of checkers. Samuel addressed some of the limitations of hill climbing by checking the effectiveness of the individual weighted heuristic measures, and replacing the less effective. The program also retained certain interesting limitations. For example, because of a limited global strategy, it was vulnerable to evaluation functions leading to traps. The learning component of the program was also vulnerable to inconsistencies in opponent's play; for example, if the opponent used widely varying strategies, or simply played foolishly, the weights on the evaluation polynomial might begin to take on 'random' values, leading to an overall degradation of performance."

3.2 Dynamic Programming (Section 4.1.2)

"Dynamic programming (DP) is sometimes called the forward-backward, or, when using

probabilities, the Viterbi algorithm. Created by Richard Bellman (1956), dynamic programming addresses the issue of restricted memory search in problems composed of multiple interacting and interrelated subproblems. DP keeps track of and reuses subproblems already searched and solved within the solution of the larger problem. An example of this would be the reuse the subseries solutions within the solution of the Fibonacci series. The technique of subproblem caching for reuse is sometimes called memoizing partial subgoal solutions. The result is an important algorithm often used for string matching, spell checking, and related areas in natural language processing (see Sections 9.4 and 14.4).”

The string alignment example:

“We demonstrate dynamic programming using two examples, both from text processing; first, finding an optimum global alignment of two strings of characters and, second, finding the minimum edit difference between two strings of characters. Suppose we wanted to find the best possible alignment for the characters in the strings BAADDCABDDA and BBADCBA. One optimal alignment, among several possible, would be:

BAADDCABDBA BDA DC B A”

“Dynamic programming requires a data structure to keep track of the subproblems related to the state currently being processed. We use an array. The size of the dimensions of this array, because of initialization requirements, is one more than the length of each string, in our example 12 by 8, as in Figure 4.5. The value of each row-column element of the array reflects the global alignment success to that point in the matching process. There are three possible costs for creating the current state: if a character is shifted along in the shorter string for better possible alignment, the cost is 1 and is recorded by the array’s column score. If a new character is inserted, the cost is 1 and reflected in the array’s row score. If the characters to be aligned are different, shift and insert, the cost is 2; or if they are identical the cost is 0; this is reflected in the array’s diagonal. Figure 4.5 shows initialization, the increasing +1s in the first row and column reflect the continued shifting or insertion of characters to the ‘—’ or empty string.”

“In the forward stage of the dynamic programming algorithm, we fill the array from the upper left corner by considering the partial matching successes to the current point of the solution. That is, the value of the intersection of row x and column y, (x,y), is a function (for the minimum alignment problem, the minimum cost) of one of the three values in row x − 1 column y, row x − 1 column y − 1, or row x column y − 1. These three array locations hold the alignment information up to the present point of the solution. If there is a match of characters at location (x,y) add 0 to the value at location (x − 1,y − 1), if there is no match add 2 (for shift and insert). We add 1 by either shifting the shorter character string (add to the previous value of column y) or inserting a character (add to the previous value of row x). Continuing this approach produces the filled array of Figure 4.6. It can be observed that the minimum cost match often takes place near the upper-left to lower-right diagonal of the array.”

“Once the array is filled, we begin the backward stage of the algorithm that produces particular solutions. That is, from the best alignment possible, we go back through the array to select a specific solution alignment. We begin this process at the maximum row column value, in our example the 6 in row 7 column 12. From there we move back through the array, at each step selecting one of the immediate state’s predecessors that produced the present state (from the forward stage), ie, one of either the diagonal, row, or column that produced that state. Whenever there is a forced decreasing difference, as in the 6 and 5 near the beginning of the trace back, we select the previous diagonal, as that is where the match came from; otherwise we use the value of the preceding row or column. The trace back of Figure 4.7, one of several possible, produces the optimal string alignment of the previous page.”

The minimum edit difference example:

“In the second example use of DP, we consider the idea of the minimum edit difference between two strings. If we were building an intelligent spell checker, for example, we might want to determine what words from our set of correctly spelled words best approximate a certain unrecognized string of characters. A similar approach could be used to determine what known (strings of phones representing) words most closely matched a given string of phones. The next example of establishing the edit distance between two strings of characters is adapted from Jurafsky and Martin (2009).”

“Suppose you produce an unrecognized string of characters. The task of the spell checker is to produce an ordered list of the most likely words from the dictionary that you meant to type. The question is then, how can a difference be measured between pairs of strings of characters, ie, the string you just typed and the character strings of a dictionary. For your string, we want to produce an ordered list of possibly correct words in the dictionary. For each of these words, we want a numerical measure of how ‘different’ each word is from your string.”

“A minimum edit difference between two strings can be specified as the number of character insertions, deletions, and replacements necessary to turn the first string, the source, into the second string, the target. This is sometimes called the Levenshtein distance (Jurafsky and Martin 2009). We now implement a dynamic programming search for determining the minimum edit difference between two strings of characters. We let intention be the source string and execution be the target. The edit cost for transforming the first string to the second is 1 for a character insertion or a deletion and 2 for a replacement (a deletion plus an insertion). We want to determine a minimum cost difference between these two strings.”

“Our array for subgoals will again be one character longer than each of the strings, in this case 10 by 10. The initialization is as in Figure 4.8 (a sequence of insertions is necessary, starting at the null string, to make either string resemble the other. The array location (2, 2) is 2, because a replacement (or making a delete plus an insert) is required to turn an i into an e.”

“Figure 4.9 gives the full result of applying the dynamic programming algorithm to transform intention into execution. The value at each location (x, y) in the array is the cost of the minimum editing to that point plus the (minimum) cost of either an insertion, deletion, or replacement. Thus the cost of (x, y) is the minimum of the cost of (x - 1, y) plus cost of insertion, or cost (x - 1, y - 1) plus cost of replacement, or cost of (x, y - 1) plus the deletion cost. Pseudocode for his algorithm is a function taking the two strings (a source and a target) and their lengths and returning the minimum edit difference cost:”

“function dynamic (source, sl, target, tl) return cost (i, j);

create array cost(sl + 1, tl + 1) cost (0,0) := 0 % initialize for i := 1 to sl + 1 do for j := 1 to tl + 1 do cost (i, j) := min [cost (i - 1, j) + insertion cost $target_{j-1}$ % add 1 cost (i - 1, j - 1) + replace cost $source_{i-1}$ $target_{j-1}$ % add 2 cost(i, j - 1) + delete cost $source_{j-1}$] % add 1”

“Using the results (bold) in Figure 4.9, the following edits will translate intention into execution with total edit cost of 8:

intention intention delete i, cost 1 etention replace n with e, cost 2 exention replace t with x, cost 2 exenution insert u, cost 1 execution replace n with c, cost 2”

“In the spell check situation of proposing a cost-based ordered list of words for replacing an unrecognized string, the backward segment of the dynamic programming algorithm is not needed. Once the minimum edit measure is calculated for the set of related strings a prioritized order of alternatives is proposed from which the user chooses an appropriate string.”

“The justification for dynamic programming is the cost of time/space in computation. Dynamic programming, as seen in our examples has cost of n^2, where n is the length of the largest string; the cost in the worse case is n^3, if other related subproblems need to be considered (other row/column values) to determine the current state. Exhaustive search for comparing two strings is exponential, costing between 2^n and 3^n.”

“There are a number of obvious heuristics that can be used to prune the search in dynamic programming. First, useful solutions will usually lie around the upper left to lower right diagonal of the array; this leads to ignoring development of array extremes. Second, it can be useful to prune the search as it evolves, e.g., for edit distances passing a certain threshold, cut that solution path or even abandon the whole problem, i.e., the source string will be so distant from the target string of characters as to be useless. There is also a stochastic approach to the pattern comparison problem that we will see in Section 5.3.”

4. THE BEST-FIRST SEARCH ALGORITHM (Section 4.2)

4.1 Implementing Best-First Search (Section 4.2.1)

“In spite of their limitations, algorithms such as backtrack, hill climbing, and dynamic programming can be used effectively if their evaluation functions are sufficiently informative to avoid local maxima, dead ends, and related anomalies in a search space. In general, however, use of heuristic search requires a more flexible algorithm: this is provided by best-first search, where, with a priority queue, recovery from these situations is possible.”

“Like the depth-first and breadth-first search algorithms of Chapter 3, best-first search uses lists to maintain states: open to keep track of the current fringe of the search and closed to record states already visited. An added step in the algorithm orders the states on open according to some heuristic estimate of their ‘closeness’ to a goal. Thus, each iteration of the loop considers the most ‘promising’ state on the open list. The pseudo-code for the function best_first_search appears below.”

“function best_first_search; begin open := [Start]; % initialize closed := []; while open ≠ [] do % states remain begin remove the leftmost state from open, call it X; if X = goal then return the path from Start to X else begin generate children of X; for each child of X do case the child is not on open or closed: begin assign the child a heuristic value; add the child to open end; the child is already on open: if the child was reached by a shorter path then give the state on open the shorter path the child is already on closed: if the child was reached by a shorter path then begin remove the state from closed; add the child to open end; end; % case put X on closed; re-order states on open by heuristic merit (best leftmost) end; return FAIL % open is empty end.”

“At each iteration, best_first_search removes the first element from the open list. If it meets the goal conditions, the algorithm returns the solution path that led to the goal. Note that each state retains ancestor information to determine if it had previously been reached by a shorter path and to allow the algorithm to return the final solution path. (See Section 3.2.3.)”

The trace:

“1. open = ; closed = 2. evaluate A5; open = [B4,C4,D6]; closed = 3. evaluate B4; open = [C4,E5,F5,D6]; closed = [B4,A5] 4. evaluate C4; open = [H3,G4,E5,F5,D6]; closed = [C4,B4,A5] 5. evaluate H3; open = [O2,P3,G4,E5,F5,D6]; closed = [H3,C4,B4,A5] 6. evaluate O2; open = [P3,G4,E5,F5,D6]; closed = [O2,H3,C4,B4,A5] 7. evaluate P3; the solution is found!”

“Figure 4.11 shows the space as it appears after the fifth iteration of the while loop. The states contained in open and closed are indicated. open records the current frontier of the search and closed records states already considered. Note that the frontier of the search is highly uneven, reflecting the opportunistic nature of best-first search.”

“The best-first search algorithm selects the most promising state on open for further expansion. However, as it is using a heuristic that may prove erroneous, it does not abandon all the other states but maintains them on open. In the event a heuristic leads the search down a path that proves incorrect, the algorithm will eventually retrieve some previously generated, ‘next best’ state from open and shift its focus to another part of the space. In the example of Figure 4.10, after the children of state B were found to have poor heuristic evaluations, the search shifted its focus to state C. The children of B were kept on open in case the algorithm needed to return to them later. In best_first_search, as in the algorithms of Chapter 3, the open list supports backtracking from paths that fail to produce a goal.”

4.2 Implementing Heuristic Evaluation Functions (Section 4.2.2)

"We next evaluate the performance of several different heuristics for solving the 8-puzzle. Figure 4.12 shows a start and goal state for the 8-puzzle, along with the first three states generated in the search."

Three heuristics:

"The simplest heuristic counts the tiles out of place in each state when compared with the goal. This is intuitively appealing, because it would seem that, all else being equal, the state that had fewest tiles out of place is probably closer to the desired goal and would be the best to examine next."

"However, this heuristic does not use all of the information available in a board configuration, because it does not take into account the distance the tiles must be moved. A 'better' heuristic would sum all the distances by which the tiles are out of place, one for each square a tile must be moved to reach its position in the goal state."

"Both of these heuristics can be criticized for failing to acknowledge the difficulty of tile reversals. That is, if two tiles are next to each other and the goal requires their being in opposite locations, it takes (several) more than two moves to put them back in place, as the tiles must 'go around' each other (Figure 4.13)."

"A heuristic that takes this into account multiplies a small number (2, for example) times each direct tile reversal (where two adjacent tiles must be exchanged to be in the order of the goal). Figure 4.14 shows the result of applying each of these three heuristics to the three child states of Figure 4.12."

Evaluation of the three heuristics:

Heuristic
Start state
First move
Second move
Third move
Goal
Tiles out of place
—
5
3
5
—
Sum of distances
—
6
4
6
—
2 × direct reversals
—
0
0
0
—

"In Figure 4.14's summary of evaluation functions, the sum of distances heuristic does indeed seem to provide a more accurate estimate of the work to be done than the simple count of the number of tiles out of place. Also, the tile reversal heuristic fails to distinguish between these states, giving each an evaluation of 0. Although it is an intuitively appealing heuristic, it breaks down since none of these states have any direct reversals. A fourth heuristic, which may overcome the limitations of the tile reversal heuristic, adds the sum of the distances the tiles are out of place and 2 times the number of direct reversals."

"This example illustrates the difficulty of devising good heuristics. Our goal is to use the limited information available in a single state descriptor to make intelligent choices. Each of the heuristics proposed above ignores some critical bit of information and is subject to improvement. The design of good heuristics is an empirical problem; judgment and intuition help, but the final measure of a heuristic must be its actual performance on problem instances."

The f(n) = g(n) + h(n) evaluation:

"If two states have the same or nearly the same heuristic evaluations, it is generally preferable to examine the state that is nearest to the root state of the graph. This state will have a greater probability of being on the shortest path to the goal. The distance from the starting state to its descendants can be measured by maintaining a depth count for each state. This count is 0 for the beginning state and is incremented by 1 for each level of the search. This depth measure can be added to the heuristic evaluation of each state to bias search in favor of states found shallower in the graph."

"This makes our evaluation function, f, the sum of two components:

f(n) = g(n) + h(n)

where g(n) measures the actual length of the path from any state n to the start state and h(n) is a heuristic estimate of the distance from state n to a goal."

"In the 8-puzzle, for example, we can let h(n) be the number of tiles out of place. When this evaluation is applied to each of the child states in Figure 4.12, their f values are 6, 4, and 6, respectively, see Figure 4.15."

"The full best-first search of the 8-puzzle graph, using f as defined above, appears in Figure 4.16. Each state is labeled with a letter and its heuristic weight, f(n) = g(n) + h(n). The number at the top of each state indicates the order in which it was taken off the open list. Some states (h, g, b, d, n, k, and i) are not so numbered, because they were still on open when the algorithm terminates."

The trace:

"1. open = ; closed = [] 2. open = [c4, b6, d6]; closed = 3. open = [e5, f5, b6, d6, g6]; closed = [a4, c4] 4. open = [f5, h6, b6, d6, g6, i7]; closed = [a4, c4, e5] 5. open = [j5, h6, b6, d6, g6, k7, i7]; closed = [a4, c4, e5, f5] 6. open = [l5, h6, b6, d6, g6, k7, i7]; closed = [a4, c4, e5, f5, j5] 7. open = [m5, h6, b6, d6, g6, n7, k7, i7]; closed = [a4, c4, e5, f5, j5, l5] 8. success, m = goal!"

"In step 3, both e and f have a heuristic of 5. State e is examined first, producing children, h and i. Although h, the child of e, has the same number of tiles out of place as f, it is one level deeper in the space. The depth measure, g(n), causes the algorithm to select f for evaluation in step 4. The algorithm goes back to the shallower state and continues to the goal. The state space graph at this stage of the search, with open and closed highlighted, appears in Figure 4.17. Notice the opportunistic nature of best-first search."

"In effect, the g(n) component of the evaluation function gives the search more of a breadth-first flavor. This prevents it from being misled by an erroneous evaluation: if a heuristic continuously returns 'good' evaluations for states along a path that fails to reach a goal, the g value will grow to dominate h and force search back to a shorter solution path. This guarantees that the algorithm will not become permanently lost, descending an infinite branch. Section 4.3 examines the conditions under which best-first search using this evaluation function can actually be guaranteed to produce the shortest path to a goal."

The three key summary points:

"To summarize: 1. Operations on states generate children of the state currently under examination. 2. Each new state is checked to see whether it has occurred before (is on either open or closed), thereby preventing loops. 3. Each state n is given an f value equal to the sum of its depth in the search space g(n) and a heuristic estimate of its distance to a goal h(n). The h value guides search toward heuristically promising states while the g value can prevent search from persisting indefinitely on a fruitless path."

4.3 Heuristic Search and Expert Systems (Section 4.2.3)

"Simple games such as the 8-puzzle are ideal vehicles for exploring the design and behavior of heuristic search algorithms for a number of reasons:

The search spaces are large enough to require heuristic pruning.

Most games are complex enough to suggest a rich variety of heuristic evaluations for comparison and analysis.

Games generally do not involve complex representational issues. A single node of the state space is just a board description and usually can be captured in a straightforward fashion. This allows researchers to focus on the behavior of the heuristic rather than the problems of knowledge representation.

Because each node of the state space has a common representation (e.g., a board description), a single heuristic may be applied throughout the search space. This contrasts with systems such as the financial advisor, where each node represents a different subgoal with its own distinct description."

"More realistic problems greatly complicate the implementation and analysis of heuristic search by requiring multiple heuristics to deal with different situations in the problem space. However, the insights gained from simple games generalize to problems such as those found in expert systems applications, planning, intelligent control, and machine learning. Unlike the 8-puzzle, a single heuristic may not apply to each state in these domains. Instead, situation specific problem-solving heuristics are encoded in the syntax and content of individual problem solving operators. Each solution step incorporates its own heuristic that determines when it should be applied; the pattern matcher matches the appropriate operation (heuristic) with the relevant state in the space."

Example 4.2.1: The Financial Advisor, Revisited:

"The use of heuristic measures to guide search is a general approach in AI. Consider again the financial advisor problem of Chapters 2 and 3, where the knowledge base was treated as a set of logical implications, whose conclusions are either true or false. In actuality, these rules are highly heuristic in nature. For example, one rule states that an individual with adequate savings and income should invest in stocks:

savings_account(adequate) ∧ income(adequate) ⇒ investment(stocks)."

"In reality, it is possible that such an individual may prefer the added security of a combination strategy or even that of placing all investment money in savings. Thus, the rule is a heuristic, and the problem solver should try to account for this uncertainty. We could take additional factors, such as the age of the investor and the long-term prospects for security and advancement in the investor's profession, into account to make the rules more informed and capable of finer distinctions. However, this does not change the fundamentally heuristic nature of financial advice."

"One way in which expert systems have addressed this issue is to attach a numeric weight (called a confidence measure or certainty factor) to the conclusion of each rule. This measures the confidence that may be placed in their conclusions."

"Each rule conclusion is given a confidence measure, say a real number between −1 and 1, with 1 corresponding to certainty (true) and −1 to a definite value of false. Values in between reflect varying confidence in the conclusion. For example, the preceding rule may be given a confidence of, say, 0.8, reflecting a small possibility that it may not be correct. Other conclusions may be drawn with different confidence weights:

savings_account(adequate) ∧ income(adequate) ⇒ investment(stocks) with confidence = 0.8 savings_account(adequate) ∧ income(adequate) ⇒ investment(combination) with confidence = 0.5

savings_account(adequate) ∧ income(adequate) ⇒ investment(savings) with confidence = 0.1"

"These rules reflect the common investment advice that although an individual with adequate savings and income would be most strongly advised to invest in stocks, there is some possibility that a combination strategy should be pursued and a slight chance that they may want to continue investing in savings. Heuristic search algorithms can use these certainty factors in a number of ways. For example, the results of all applicable rules could be produced along with their associated confidences. This exhaustive search of all possibilities may be appropriate in domains such as medicine. Alternatively, the program might return only the result with the strongest confidence value, if alternative solutions are not of interest. This can allow the program to ignore other rules, radically pruning the search space. A more conservative pruning strategy could ignore rules that draw a conclusion with a

confidence less than a certain value, 0.2 for example."

"A number of important issues must be addressed in using confidence measures to weight rule conclusions. What does it really mean to have a 'numeric confidence measure'? For example, how are the confidences handled if the conclusion of one rule is used as the premise of others? How are confidences combined in the event that more than one rule draws the same conclusion? How are the proper confidence measures assigned to rules in the first place? These issues are discussed in more detail in Chapter 8."

5. ADMISSIBILITY, MONOTONICITY, AND INFORMEDNESS (Section 4.3)

5.1 Admissibility Measures (Section 4.3.1)

"We may evaluate the behavior of heuristics along a number of dimensions. For instance, we may desire a solution and also require the algorithm to find the shortest path to the goal. This could be important when an application might have an excessive cost for extra solution steps, such as planning a path for an autonomous robot through a dangerous environment. Heuristics that find the shortest path to a goal whenever it exists are said to be admissible. In other applications a minimal solution path might not be as important as overall problem-solving efficiency."

"We may want to ask whether any better heuristics are available. In what sense is one heuristic 'better' than another? This is the informedness of a heuristic."

"When a state is discovered by using heuristic search, is there any guarantee that the same state won't be found later in the search at a cheaper cost (with a shorter path from the start state)? This is the property of monotonicity."

Algorithm A and A*:

"A search algorithm is admissible if it is guaranteed to find a minimal path to a solution whenever such a path exists. Breadth-first search is an admissible search strategy. Because it looks at every state at level n of the graph before considering any state at the level n + 1, any goal nodes are found along the shortest possible path. Unfortunately, breadth-first search is often too inefficient for practical use."

"Using the evaluation function f(n) = g(n) + h(n) that was introduced in the last section, we may characterize a class of admissible heuristic search strategies. If n is a node in the state space graph, g(n) measures the depth at which that state has been found in the graph, and h(n) is the heuristic estimate of the distance from n to a goal. In this sense f(n) estimates the total cost of the path from the start state through n to the goal state. In determining the properties of admissible heuristics, we define an evaluation function f*:

f*(n) = g*(n) + h*(n)

where g*(n) is the cost of the shortest path from the start to node n and h* returns the actual cost of the shortest path from n to the goal. It follows that f*(n) is the actual cost of the optimal path from a start node to a goal node that passes through node n."

"As we will see, when we employ best_first_search with the evaluation function f* the resulting search strategy is admissible. Although oracles such as f* do not exist for most real problems, we would like the evaluation function f to be a close estimate of f*. *In algorithm A, g(n), the cost of the current path to state n, is a reasonable estimate of g*, but they may not be equal: g(n) ≥ g*(n). These are equal only if the graph search has discovered the optimal path to state n."

"Similarly, we replace h*(n) with h(n), a heuristic estimate of the minimal cost to a goal state. Although we usually may not compute h*, it is often possible to determine whether or not the heuristic estimate, h(n), is bounded from above by h*(n), i.e., is always less than or equal to the actual cost of a minimal path. If algorithm A uses an evaluation function f in which h(n) ≤ h*(n), it is called algorithm A*."

"DEFINITION: ALGORITHM A, ADMISSIBILITY, ALGORITHM A*

Consider the evaluation function f(n) = g(n) + h(n), where

n is any state encountered in the search. g(n) is the cost of n from the start state. h(n) is the heuristic estimate of the cost of going from n to a goal.

If this evaluation function is used with the best_first_search algorithm of Section 4.1, the result is called algorithm A.

A search algorithm is admissible if, for any graph, it always terminates in the optimal solution path whenever a path from the start to a goal state exists.

If algorithm A is used with an evaluation function in which h(n) is less than or equal to the cost of the minimal path from n to the goal, the resulting search algorithm is called algorithm A* (pronounced 'A STAR')."

"It is now possible to state a property of A* algorithms:

All A* algorithms are admissible."

"The admissibility of A* algorithms is a theorem. An exercise at the end of the chapter gives directions for developing its proof (see also Nilsson 1980, p 76-78). The theorem says that any A* algorithm, i.e., one that uses a heuristic h(n) such that h(n) ≤ h*(n) for all n, is guaranteed to find the minimal path from n to the goal, if such a path exists."

"Note that breadth-first search may be characterized as an A* algorithm in which f(n) = g(n) + 0. The decision for considering a state is based solely on its distance from the start state. We will show (Section 4.3.3) that the set of nodes considered by an A* algorithm is a subset of the states examined in breadth-first search."

"Several heuristics from the 8-puzzle provide examples of A* algorithms. Although we may not be able to compute the value of h*(n) for the 8-puzzle, we may determine when a heuristic is bounded from above by the actual cost of the shortest path to a goal."

"For instance, the heuristic of counting the number of tiles not in the goal position is certainly less than or equal to the number of moves required to move them to their goal position. Thus, this heuristic is admissible and guarantees an optimal (or shortest) solution path when that path exists. The sum of the direct distances of tiles out of place is also less than or equal to the minimum actual path. Using small multipliers for direct tile reversals gives an admissible heuristic."

"This approach to proving admissibility of 8-puzzle heuristics may be applied to any heuristic search problem. Even though the actual cost of the shortest path to a goal may not always be computed, we can often prove that a heuristic is bounded from above by this value. When this can be done, the resulting search will terminate in the discovery of the shortest path to the goal, when such a path exists."

5.2 Monotonicity (Section 4.3.2)

"Recall that the definition of A* algorithms did not require that g(n) = g*(n). This means that admissible heuristics may initially reach non-goal states along a suboptimal path, as long as the algorithm eventually finds an optimal path to all states on the path to a goal. It is natural to ask if there are heuristics that are 'locally admissible,' i.e., that consistently find the minimal path to each state they encounter in the search. This property is called monotonicity."

"DEFINITION: MONOTONICITY

A heuristic function h is monotone if

For all states n_i and n_j, where n_j is a descendant of n_i, $h(n_i) - h(n_j) \leq cost(n_i, n_j)$, where $cost(n_i, n_j)$ is the actual cost (in number of moves) of going from state n_i to n_j.

The heuristic evaluation of the goal state is zero, or h(Goal) = 0."

"One way of describing the monotone property is that the search space is everywhere locally consistent with the heuristic employed. The difference between the heuristic measure for a state and any one of its successors is bound by the actual cost of going between that state and its successor. This is to say that the heuristic is everywhere admissible, reaching each state along the shortest path from its ancestors."

"If the graph search algorithm for best-first search is used with a monotonic heuristic, an important step may be omitted. Because the heuristic finds the shortest path to any state the first time that state is discovered, when a state is encountered a second time, it is not necessary to check whether the new path is shorter. It won't be! This allows any state that is rediscovered in the space to be dropped immediately without updating the path information retained on open or closed."

"When using a monotonic heuristic, as the search moves through the space, the heuristic measure for each state n is replaced by the actual cost for generating that piece of the path to n. Because the actual cost is equal to or larger than the heuristic in each instance, f will not decrease; i.e., f is monotonically nondecreasing (hence the name)."

The proof that monotone implies admissible:

"A simple argument can show that any monotonic heuristic is admissible. This argument considers any path in the space as a sequence of states $s_1, s_2, \ldots, s_9$, where s_1 is the start state and s_9 is the goal. For the sequence of moves in this arbitrarily selected path:

s_1 to s_2: $h(s_1) - h(s_2) \leq cost(s_1, s_2)$ by monotone property

s_2 to s_3: $h(s_2) - h(s_3) \leq cost(s_2, s_3)$ by monotone property

s_3 to s_4: $h(s_3) - h(s_4) \leq cost(s_3, s_4)$ by monotone property

… s_{9-1} to s_9: $h(s_{9-1}) - h(s_9) \leq cost(s_{9-1}, s_9)$ by monotone property"

"Summing each column and using the monotone property of $h(s_9) = 0$:

path s_1 to s_9: $h(s_1) \leq cost(s_1, s_9)$"

"This means that monotone heuristic h is A* and admissible. It is left as an exercise whether or not the admissibility property of a heuristic implies monotonicity."

5.3 When One Heuristic Is Better: More Informed Heuristics (Section 4.3.3)

"The final issue of this subsection compares two heuristics' ability to find the minimal path. An interesting case occurs when the heuristics are A*."

"DEFINITION: INFORMEDNESS

For two A* heuristics h_1 and h_2, if $h_1(n) \leq h_2(n)$, for all states n in the search space, heuristic h_2 is said to be more informed than h_1."

"We can use this definition to compare the heuristics proposed for solving the 8-puzzle. As pointed out previously, breadth-first search is equivalent to the A* algorithm with heuristic h_1 such that $h_1(x) = 0$ for all states x. This is, trivially, less than h. *We have also shown that h_2, the number of tiles out of place with respect to the goal state, is a lower bound for h*. In this case $h_1 \leq h_2 \leq h^*$. It follows that 'the number of tiles out of place' heuristic is more informed than breadth-first search. Figure 4.18 compares the spaces searched by these two heuristics. Both h_1 and h_2 find the optimal path, but h_2 evaluates many fewer states in the process."

"Similarly, we can argue that the heuristic that calculates the sum of the direct distances by which all the tiles are out of place is again more informed than the calculation of the number of tiles that are out of place with respect to the goal state, and indeed this is the case. One can visualize a sequence of search spaces, each smaller than the previous one, converging on the direct optimal path solution."

"If a heuristic h_2 is more informed than h_1, then the set of states examined by h_2 is a subset of those expanded by h_1. This can be verified by assuming the opposite (that there is at least one state expanded by h_2 and not by h_1). But since h_2 is more informed than h_1, for all n, $h_2(n) \uparrow h_1(n)$, and both are bounded above by h*, our assumption is contradictory."

"In general, then, the more informed an A* algorithm, the less of the space it needs to expand to get the optimal solution. We must be careful, however, that the computations necessary to employ the more informed heuristic are not so inefficient as to offset the gains from reducing the number of states searched."

"Computer chess programs provide an interesting example of this trade-off. One school of thought uses simple heuristics and relies on computer speed to search deeply into the search space. These programs often use specialized hardware for state evaluation to increase the depth of the search. Another school relies on more sophisticated heuristics to reduce the number of board states searched. These heuristics include calculations of piece advantages, control of board geography, possible attack strategies, passed pawns, and so on. Calculation of heuristics itself may involve exponential complexity (an issue discussed in Section 4.5). Since the total time for the first 40 moves of the game is limited it is important to optimize this trade-off between search and heuristic evaluation. The optimal blend of blind search and heuristics remains an open empirical question in computer chess, as can be seen in the Gary Kasparov Deep Blue battle (Hsu 2002)."

6. USING HEURISTICS IN GAMES (Section 4.4)

"Games have always been an important application area for heuristic algorithms. Two-person games are more complicated than simple puzzles because of the existence of a 'hostile' and essentially unpredictable opponent. Thus, they provide some interesting

opportunities for developing heuristics, as well as greater difficulties in developing search algorithms."

6.1 The Minimax Procedure on Exhaustively Searchable Graphs (Section 4.4.1)

"First we consider games whose state space is small enough to be exhaustively searched; here the problem is systematically searching the space of possible moves and countermoves by the opponent. Then we look at games in which it is either impossible or undesirable to exhaustively search the game graph. Because only a portion of the state space can be generated and searched, the game player must use heuristics to guide play along a path to a winning state."

The game of nim:

"We first consider a variant of the game nim, whose state space may be exhaustively searched. To play this game, a number of tokens are placed on a table between the two opponents; at each move, the player must divide a pile of tokens into two nonempty piles of different sizes. Thus, 6 tokens may be divided into piles of 5 and 1 or 4 and 2, but not 3 and 3. The first player who can no longer make a move loses the game. For a reasonable number of tokens, the state space can be exhaustively searched. Figure 4.19 illustrates the space for a game with 7 tokens."

MIN and MAX:

"In playing games whose state space may be exhaustively delineated, the primary difficulty is in accounting for the actions of the opponent. A simple way to handle this assumes that your opponent uses the same knowledge of the state space as you use and applies that knowledge in a consistent effort to win the game. Although this assumption has its limitations (which are discussed in Section 4.4.2), it provides a reasonable basis for predicting an opponent's behavior. Minimax searches the game space under this assumption."

"The opponents in a game are referred to as MIN and MAX. Although this is partly for historical reasons, the significance of these names is straightforward: MAX represents the player trying to win, or to MAXimize her advantage. MIN is the opponent who attempts to MINimize MAX's score. We assume that MIN uses the same information and always attempts to move to a state that is worst for MAX."

"In implementing minimax, we label each level in the search space according to whose move it is at that point in the game, MIN or MAX. In the example of Figure 4.20, MIN is allowed to move first. Each leaf node is given a value of 1 or 0, depending on whether it is a win for MAX or for MIN. Minimax propagates these values up the graph through successive parent nodes according to the rule:

If the parent state is a MAX node, give it the maximum value among its children. If the parent is a MIN node, give it the minimum value of its children."

"The value that is thus assigned to each state indicates the value of the best state that this player can hope to achieve (assuming the opponent plays as predicted by the minimax algorithm). These derived values are used to choose among possible moves. The result of applying minimax to the state space graph for nim appears in Figure 4.20."

"The values of the leaf nodes are propagated up the graph using minimax. Because all of MIN's possible first moves lead to nodes with a derived value of 1, the second player, MAX, always can force the game to a win, regardless of MIN's first move. MIN can win only if MAX played foolishly. In Figure 4.20, MIN may choose any of the first move alternatives, with the resulting win paths for MAX in bold arrows."

"Although there are games where it is possible to search the state space exhaustively, most interesting games do not. We examine fixed depth search next."

6.2 Minimaxing to Fixed Ply Depth (Section 4.4.2)

"In applying minimax to more complicated games, it is seldom possible to expand the state space graph out to the leaf nodes. Instead, the state space is searched to a predefined number of levels, as determined by available resources of time and memory. This strategy is called an n-ply look-ahead, where n is the number of levels explored. As the leaves of this subgraph are not final states of the game, it is not possible to give them values that reflect a win or a loss. Instead, each node is given a value according to some heuristic evaluation function. The value that is propagated back to the root node is not an indication of whether or not a win can be achieved (as in the previous example) but is simply the heuristic value of the best state that can be reached in n moves from the root. Look-ahead increases the power of a heuristic by allowing it to be applied over a greater area of the state space. Minimax consolidates these separate evaluations for the ancestor state."

"In a game of conflict, each player attempts to overcome the other, so many game heuristics directly measure the advantage of one player over another. In checkers or chess, piece advantage is important, so a simple heuristic might take the difference in the number of pieces belonging to MAX and MIN and try to maximize the difference between these piece measures. A more sophisticated strategy might assign different values to the pieces, depending on their value (e.g., queen vs. pawn or king vs. ordinary checker) or location on the board. Most games provide limitless opportunities for designing heuristics."

"Game graphs are searched by level, or ply. As we saw in Figure 4.20, MAX and MIN alternately select moves. Each move by a player defines a new ply of the graph. Game playing programs typically look ahead a fixed ply depth, determined by the space/time limitations of the computer. The states on that ply are measured heuristically and the values are propagated back up the graph using minimax. The search algorithm then uses these derived values to select among possible next moves."

"After assigning an evaluation to each state on the selected ply, the program propagates a value up to each parent state. If the parent is on a MIN level, the minimum value of the children is backed up. If the parent is a MAX node, minimax assigns it the maximum value of its children."

"Maximizing for MAX parents and minimizing for MIN, the values go back up the graph to the children of the current state. These values are then used by the current state to select among its children. Figure 4.21 shows minimax on a hypothetical state space with a four-ply look-ahead."

The horizon effect:

"We can make several final points about the minimax procedure. First, and most important, evaluations to any (previously decided) fixed ply depth may be seriously misleading. When a heuristic is applied with limited look-ahead, it is possible the depth of the look-ahead may not detect that a heuristically promising path leads to a bad situation later in the game. If your opponent in chess offers a rook as a lure to take your queen, and the evaluation only looks ahead to the ply where the rook is offered, the evaluation is going to be biased toward this state. Unfortunately, selection of the state may cause the entire game to be lost! This is referred to as the horizon effect. It is usually countered by searching several plies deeper from states that look exceptionally good. This selective deepening of search in important areas will not make the horizon effect go away, however. The search must stop somewhere and will be blind to states beyond that point."

"There is another effect that occurs in minimaxing on the basis of heuristic evaluations. The evaluations that take place deep in the space can be biased by their very depth (Pearl 1984). In the same way that the average of products differs from the product of averages, the estimate of minimax (which is what we desire) is different from the minimax of estimates (which is what we are doing). In this sense, deeper search with evaluation and minimax normally does, but need not always, mean better search. Further discussion of these issues and possible remedies may be found in Pearl (1984)."

The tic-tac-toe heuristic:

"In concluding the discussion of minimax, we present an application to tic-tac-toe (Section 4.0), adapted from Nilsson (1980). A slightly more complex heuristic is used, one that attempts to measure the conflict in the game. The heuristic takes a state that is to be measured, counts the total winning lines open to MAX, and then subtracts the total number of winning lines open to MIN. The search attempts to maximize this difference. If a state is a forced win for MAX, it is evaluated as $+\infty$; a forced win for MIN as $-\infty$. Figure 4.22 shows this heuristic applied to several sample states."

"Figures 4.23, 4.24, and 4.25 demonstrate the heuristic of Figure 4.22 in a two-ply minimax. These figures show the heuristic evaluation, minimax backup, and MAX's move, with some type of tiebreaker applied to moves of equal value, from Nilsson (1971)."

6.3 The Alpha-Beta Procedure (Section 4.4.3)

"Straight minimax requires a two-pass analysis of the search space, the first to descend to the ply depth and there apply the heuristic and the second to propagate values back up the tree. Minimax pursues all branches in the space, including many that could be ignored or pruned by a more intelligent algorithm. Researchers in game playing developed a class of search techniques called alpha-beta pruning, first proposed in the late 1950s (Newell and Simon 1976), to improve search efficiency in two-person games (Pearl 1984)."

"The idea for alpha-beta search is simple: rather than searching the entire space to the ply depth, alpha-beta search proceeds in a depth-first fashion. Two values, called alpha and beta, are created during the search. The alpha value, associated with MAX nodes, can never decrease, and the beta value, associated with MIN nodes, can never increase. Suppose a MAX node's alpha value is 6. Then MAX need not consider any backed-up value less than or equal to 6 that is associated with any MIN node below it. Alpha is the worst that MAX can 'score' given that MIN will also do its 'best'. Similarly, if MIN has beta value 6, it does not need to consider any MAX node below that has a value of 6 or more."

"To begin alpha-beta search, we descend to full ply depth in a depth-first fashion and apply our heuristic evaluation to a state and all its siblings. Assume these are MIN nodes. The maximum of these MIN values is then backed up to the parent (a MAX node, just as in minimax). This value is then offered to the grandparent of these MINs as a potential beta cutoff."

"Next, the algorithm descends to other grandchildren and terminates exploration of their parent if any of their values is equal to or larger than this beta value. Similar procedures can be described for alpha pruning over the grandchildren of a MAX node."

"Two rules for terminating search, based on alpha and beta values, are:

Search can be stopped below any MIN node having a beta value less than or equal to the alpha value of any of its MAX node ancestors.

Search can be stopped below any MAX node having an alpha value greater than or equal to the beta value of any of its MIN node ancestors."

"Alpha-beta pruning thus expresses a relation between nodes at ply n and nodes at ply n + 2 under which entire subtrees rooted at level n + 1 can be eliminated from consideration. As an example, Figure 4.26 takes the space of Figure 4.21 and applies alpha-beta pruning. Note that the resulting backed-up value is identical to the minimax result and the search saving over minimax is considerable."

"With a fortunate ordering of states in the search space, alpha-beta can effectively double the depth of the search space considered with a fixed space/time computer commitment (Nilsson 1980). If there is a particular unfortunate ordering, alpha-beta searches no more of the space than normal minimax; however, the search is done in only one pass."

7. COMPLEXITY ISSUES (Section 4.5)

"The most difficult aspect of combinatorial problems is that the 'explosion' often takes place without program designers realizing that it is happening. Because most human activity, computational and otherwise, takes place in a linear-time world, we have difficulty appreciating exponential growth. We hear the complaint: 'If only I had a larger (or faster or highly parallel) computer my problem would be solved.' Such claims, often made in the aftermath of the explosion, are usually rubbish. The problem wasn't understood properly and/or appropriate steps were not taken to address the combinatorics of the situation."

"The full extent of combinatorial growth staggers the imagination. It has been estimated that the number of states produced by a full search of the space of possible chess moves is about 10^{120}. This is not 'just another large number,' it is comparable to the number of molecules in the universe or the number of nanoseconds since the 'big bang'."

The branching factor:

"Several measures have been developed to help calculate complexity. One of these is the branching factor of a space. We define branching factor as the average number of branches (children) that are expanded from any state in the space. The number of states at depth n of the search is equal to the branching factor raised to the nth power. Once the

branching factor is computed for a space, it is possible to estimate the search cost to generate a path of any particular length. Figure 4.27 gives the relationship between B (branching), L (path length), and T (total states in the search) for small values. The figure is logarithmic in T, so L is not the 'straight' line it looks in the graph."

"Several examples using this figure show how bad things can get. If the branching factor is 2, it takes a search of about 100 states to examine all paths that extend six levels deep into the search space. It takes a search of about 10,000 states to consider paths 12 moves deep. If the branching can be cut down to 1.5 (by some heuristic), then a path twice as long can be examined for the same number of states searched."

The formula:

"The mathematical formula that produced the relationships of Figure 4.27 is:

$T = B + B^2 + B^3 + \ldots + B^L$

with T total states, L path length, and B branching factor. This equation reduces to:

$T = B(B^L - 1)/(B - 1)$"

The 8-puzzle branching factor:

"Measuring a search space is usually an empirical process done by considerable playing with a problem and testing its variants. Suppose, for example, we wish to establish the branching factor of the 8-puzzle. We calculate the total number of possible moves: 2 from each corner for a total of 8 corner moves, 3 from the center of each side for a total of 12, and 4 from the center of the grid for a grand total of 24. This divided by 9, the different number of possible locations of the blank, gives an average branching factor of 2.67. As can be seen in Figure 4.27, this is not very good for a deep search. If we eliminate moves directly back to a parent state (already built into the search algorithms of this chapter) there is one move fewer from each state. This gives a branching factor of 1.67, a considerable improvement, which might (in some state spaces) make exhaustive search possible."

Beam search:

"As we considered in Chapter 3, the complexity cost of an algorithm can also be measured by the sizes of the open and closed lists. One method of keeping the size of open reasonable is to save on open only a few of the (heuristically) best states. This can produce a better focused search but has the danger of possibly eliminating the best, or even the only, solution path. This technique of maintaining a size bound on open, often a function of the number of steps expected for the search, is called beam search."

The informedness trade-off:

"In the attempt to bring down the branching of a search or otherwise constrain the search space, we presented the notion of more informed heuristics. The more informed the search, the less the space that must be searched to get the minimal path solution. As we pointed out in Section 4.4, the computational costs of the additional information needed to further cut down the search space may not always be acceptable. In solving problems on a computer, it is not enough to find a minimum path. We must also minimize total cpu costs."

"Figure 4.28, taken from an analysis by Nilsson (1980), is an informal attempt to get at these issues. The 'informedness' coordinate marks the amount of information costs included in the evaluation heuristic that are intended to improve performance. The cpu coordinate marks the cpu costs for implementing state evaluation and other aspects of the search. As the information included in the heuristic increases, the cpu cost of the heuristic increases. Similarly, as the heuristic gets more informed, the cpu cost of evaluating states gets smaller, because fewer states are considered. The critical cost, however, is the total cost of computing the heuristic PLUS evaluating states, and it is usually desirable that this cost be minimized."

And/or graph heuristics:

"Finally, heuristic search of and/or graphs is an important area of concern, as the state spaces for expert systems are often of this form. The fully general search of these structures is made up of many of the components already discussed in this and the preceding chapter. Because all and children must be searched to find a goal, the heuristic estimate of the cost of searching an and node is the sum of the estimates of searching the children."

"There are many further heuristic issues, however, besides the numerical evaluation of individual and states, in the study of and/or graphs, such as are used in knowledge based systems. For instance, if the satisfaction of a set of and children is required for solving a parent state, which child should be considered first? The state most costly to evaluate? The state most likely to fail? The state the human expert considers first? The decision is important both for computational efficiency as well as overall cost, e.g., in medical or other diagnostic tests, of the knowledge system. These, as well as other related heuristic issues are visited again in Chapter 8."

8. EPILOGUE AND REFERENCES (Section 4.6)

"The search spaces for interesting problems tend to grow exponentially; heuristic search is a primary tool for managing this combinatorial complexity. Various control strategies for implementing heuristic search were presented in this chapter."

"We began the chapter with two traditional algorithms, both of them inherited from the discipline of operations research, hill-climbing and dynamic programming. We recommend reading the paper by Arthur Samuel (1959) discussing his checker playing program and its sophisticated use of hill-climbing and minimax search. Samuel also presents early interesting examples of a sophisticated memory management system and a program that is able to learn. Bellman's (1956) design of algorithms for dynamic programming remains important in areas such as natural language processing where it is necessary to compare strings of characters, words, or phonemes. Dynamic programming is often called the forward/backward or Viterbi algorithms. For important examples of use of dynamic programming for language analysis see Jurafsky and Martin (2009) and Chapter 15."

"We next presented heuristics in the context of traditional state space search. We presented the A and A* algorithms for implementing best-first search. Heuristic search was demonstrated using simple games such as the 8-puzzle and extended to the more complex problem spaces generated by rule-based expert systems (Chapter 8). The chapter also applied heuristic search to two-person games, using look-ahead with minimax and alpha-beta pruning to try to predict the behavior of the opponent. After discussing A* algorithms, we analyzed their behavior, considering properties including admissibility, monotonicity, and informedness."

"The discipline of complexity theory has essential ramifications for virtually every branch of computer science, especially the analysis of state space growth and heuristic pruning. Complexity theory examines the inherent complexity of problems (as opposed to algorithms applied to these problems). The key conjecture in complexity theory is that there exists a class of inherently intractable problems. This class, referred to as NP-hard (Nondeterministically Polynomial), consists of problems that may not be solved in less than exponential time without resorting to the use of heuristics. Almost all interesting search problems belong to this class. We especially recommend Computers and Intractability by Michael R. Garey and David S. Johnson (1979) and Algorithms from P to NP, Vol. I: Design and Efficiency by Bernard Moret and Henry Shapiro (1991) for discussing these issues."

"The book Heuristics by Judea Pearl (1984) provides a comprehensive treatment of the design and analysis of heuristic algorithms. R. E. Korf (1987, 1998, 1999, 2004) continues research on search algorithms, including an analysis of iterative deepening and the development of the IDA* algorithm. IDA* integrates iterative deepening with A* to obtain linear bounds on open for heuristic search. Chess and other game playing programs have held an abiding interest across the history of AI (Hsu 2002), with results often presented and discussed at the annual conferences."

"We are indebted to Nils Nilsson (1980) for the approach and many of the examples of this chapter."

9. EXERCISES (Section 4.7)

The seventeen exercises exactly as Luger lists them:

"1. Extend the 'most wins' heuristic for tic-tac-toe two plys deeper in the search space of Figure 4.3. What is the total number of states examined using this heuristic? Would the traditional hill-climbing algorithm work in this situation? Why? 2. Use the backward component of the dynamic programming algorithm to find another optimal alignment of the characters of Figure 4.6. How many optimal alignments are there? 3. With the Levenshtein metric of Section 4.1.2, use dynamic programming to determine the minimum edit distance from source strings sensation and excitation to target string execution. 4. Give a heuristic that a block-stacking program might use to solve problems of the form 'stack block X on block Y.' Is it admissible? Monotonic? 5. The sliding-tile puzzle consists of three black tiles, three white tiles, and an empty space in the configuration shown in Figure 4.29. The puzzle has two legal moves with associated costs: A tile may move into an adjacent empty location. This has a cost of 1. A tile can hop over one or two other tiles into the empty position. This has a cost equal to the number of tiles jumped over. The goal is to have all the white tiles to the left of all the black tiles. The position of the blank is not important. a. Analyze the state space with respect to complexity and looping. b. Propose a heuristic for solving this problem and analyze it with respect to admissibility, monotonicity, and informedness. 6. Compare the three 8-puzzle heuristics of Figure 4.14 with the heuristic of adding the sum of distances out of place to 2 times the number of direct reversals. Compare them in terms of: a. Accuracy in estimating distance to a goal. This requires that you first derive the shortest path solution and use it as a standard. b. Informedness. Which heuristic most effectively prunes the state space? c. Are any of these three 8-puzzle heuristics monotonic? d. Admissibility. Which of these heuristics are bounded from above by the actual cost of a path to the goal? Either prove your conclusions for the general case or give a counterexample. 7. a. As presented in the text, best-first search uses the closed list to implement loop detection. What would be the effect of eliminating this test and relying on the depth test, g(n), to detect loops? Compare the efficiencies of the two approaches. b. best_first_search does not test a state to see whether it is a goal until it is removed from the open list. This test could be performed when new states are generated. What effect would doing so have on the efficiency of the algorithm? Admissibility? 8. Prove A* is admissible. Hint: the proof should show that: a. A* search will terminate. b. During its execution there is always a node on open that lies on an optimal path to the goal. c. If there is a path to a goal, A* will terminate by finding the optimal path. 9. Does admissibility imply monotonicity of a heuristic? If not, can you describe when admissibility would imply monotonicity? 10. Prove that the set of states expanded by algorithm A* is a subset of those examined by breadth-first search. 11. Prove that more informed heuristics develop the same or less of the search space. Hint: formalize the argument presented in Section 4.3.3. 12. A Caesar cipher is an encryption scheme based on cyclic permutations of the alphabet, with the i-th letter of the alphabet replaced by the (i + n)-th letter of the alphabet. For example, in a Caesar cipher with a shift of 4, 'Caesar' would be encrypted as 'Geiwev.' a. Give three heuristics that might be used for solving Caesar ciphers. b. In a simple substitution cipher, each letter is replaced by another letter under some arbitrary one-to-one mapping. Which of the heuristics proposed for the Caesar cipher may be used to solve substitution ciphers? Explain. (Thanks to Don Morrison for this problem.) 13. Perform minimax on the tree shown in Figure 4.30. 14. Perform a left-to-right alpha-beta prune on the tree of Exercise 13. Perform a right-to-left prune on the same tree. Discuss why a different pruning occurs. 15. Consider three-dimensional tic-tac-toe. Discuss representational issues; analyze the complexity of the state space. Propose a heuristic for playing this game. 16. Perform alpha-beta pruning on the tic-tac-toe search of Figures 4.23, 4.24, and 4.25. How many leaf nodes can be eliminated in each case? 17. a. Create an algorithm for heuristically searching and/or graphs. Note that all descendants of an and node must be solved to solve the parent. Thus, in computing heuristic estimates of costs to a goal, the estimate of the cost to solve an and node must be at least the sum of the estimates to solve the different branches. b. Use this algorithm to search the graph in Figure 4.31."

10. QUICK-REFERENCE TABLE OF KEY DEFINITIONS

Term	Definition
Heuristic	A useful but potentially fallible problem-solving strategy; a rule of thumb
Hill-climbing	Expand current state, evaluate children, pick best; keeps no history; stuck at local maxima
Local maximum	A state better than all its neighbours but not the global best
Dynamic programming	Save and reuse subproblem solutions (memoization); $O(n^2)$ vs exponential
Levenshtein distance	Minimum edit distance (insertion/deletion cost 1, replacement cost 2)
Best-first search	open ordered by heuristic merit; can backtrack from failed paths
f(n) = g(n) + h(n)	g = actual cost from start; h = heuristic estimate to goal; f = total estimate
Admissibility	Guarantees finding a minimal path whenever one exists; $h(n) \leq h^*(n)$
Algorithm A	Best-first search with evaluation function f = g + h
Algorithm A*	Algorithm A with $h(n) \leq h$*(n); all A* algorithms are admissible
Monotonicity	$h(n_i) - h(n_j) \leq cost(n_i, n_j)$ and h(Goal) = 0; every monotone h is admissible
Informedness	h_2 more informed than h_1 if $h_1(n) \leq h_2(n)$ for all n; expands fewer states
Minimax	MAX maximises, MIN minimises; values propagated bottom-up
n-ply look-ahead	Search to fixed depth n, evaluate leaves with heuristic
Horizon effect	A promising ply hides a later disaster beyond the search depth
Alpha-beta pruning	Depth-first minimax with α (MAX, never decreases) and β (MIN, never increases); same result, fewer nodes
Branching factor (B)	Average number of children per state
$T = B(B^L - 1)/(B - 1)$	Total states generated for branching factor B and path length L
Beam search	Bound the size of open to a few best states
NP-hard	Problems that may not be solved in less than exponential time without heuristics

Topic	Belongs to
Probability and Bayes' theorem	Chapter 5
Probabilistic FSM, HMM	Chapter 5, 9, 13
Production systems, blackboard architecture	Chapter 6
Semantic networks, frames, scripts	Chapter 7
Expert system architecture (detail)	Chapter 8
Certainty factors (detail), fuzzy logic	Chapter 9
Neural networks (detail)	Chapter 11
Genetic algorithms (detail)	Chapter 12
Resolution theorem proving	Chapter 14
Natural language parsing (ATN, Earley, stochastic)	Chapter 15
Constructivist epistemology	Chapter 16

End of Chapter 4 notes, based strictly on Luger's textbook.

Chapter 5: Stochastic Methods

Complete Study Notes — Based strictly on Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th ed., Chapter 5

1. WHY CHAPTER 5 EXISTS

Luger opens the chapter with three quotations:
"God does not play dice.. — ALBERT EINSTEIN (His answer to the credibility of Quantum Theory)
God not only plays dice, but he sometimes throws them where they can't be seen.. — STEPHEN HAWKING
Probable impossibilities are to be preferred to improbable possibilities.. — ARISTOTLE As quoted by Bobby Wolfe in 'Aces on Bridge', 2003"
"Chapter 4 introduced heuristic search as an approach to problem solving in domains where either a problem does not have an exact solution or where the full state space may be too costly to calculate. In this chapter we propose the stochastic methodology as appropriate for these situations. Probabilistic reasoning is also suitable for situations where state information is found from sampling an information base and causal models are learned from data."
"One important application domain for the use of the stochastic methodology is diagnostic reasoning where cause/effect relationships are not always captured in a purely deterministic fashion, as is often possible in the knowledge-based approaches to problem solving that we saw in Chapters 2, 3, 4, and will see again in Chapter 8. A diagnostic situation usually presents evidence, such as fever or headache, without further causative justification. In fact, the evidence can often be indicative of several different causes, e.g., fever can be caused by either flu or an infection. In these situations probabilistic information can often indicate and prioritize possible explanations for the evidence."
"Another interesting application for the stochastic methodology is gambling, where supposedly random events such as the roll of dice, the dealing of shuffled cards, or the spin of a roulette wheel produce possible player payoff. In fact, in the 18th century, the..."
Applications listed in the chapter introduction:
"1. Diagnostic reasoning. In medical diagnosis, for example, there is not always an obvious cause/effect relationship between the set of symptoms presented by the patient and the causes of these symptoms. In fact, the same sets of symptoms often suggest multiple possible causes. Probabilistic models are also important in complex mechanical situations, such as monitoring aircraft or helicopter flight systems. Rule-based (Chapter 8) and probabilistic (Chapters 9 and 13) systems have both been applied in these and other diagnostic domains."
"2. Natural language understanding. If a computer is to understand and use a human language, that computer must be able to characterize how humans themselves use that language. Words, expressions, and metaphors are learned, but also change and evolve as they are used over time. The stochastic methodology supports the understanding of language; for example, when a computational system is trained on a database of specific language use (called a corpus linguistics). We consider these language issues further in this chapter as well as in Chapter 15."
"4. Learning. The three previous areas mentioned can also be seen as domains for automated learning. An important component of many stochastic systems is that they have the ability to sample situations and learn over time. Some sophisticated systems are able to both sample data and predict outcomes as well as learn new probabilistic relationships based on data and outcomes. We consider learning in Part IV, and stochastic approaches to learning in Chapter 13."
Road map:
"In Section 5.1 we introduce basic counting techniques, including the sum and product rules. Because of their importance in counting, we also present the permutations and combinations of discrete events. This is an optional section that can be skipped by readers with a sufficient background in discrete mathematics."
"In Section 5.2, we introduce a formal language for reasoning with the stochastic methodology. This includes the definitions of independence and different types of random variables. For example, for probabilistic propositions, random variables may be boolean (true or false), discrete, the integers 1 to 6 as in rolling a fair die, or continuous, a function defined on the real numbers."
"In Section 5.3, we present Bayes' theorem which supports most approaches to stochastic modeling and learning (Section 9.3 and Chapter 13). Bayes' rule is important for interpreting new evidence in the context of the prior knowledge or experience. In Section 5.4 we present two applications of stochastic methods, including probabilistic finite state automata and a methodology for predicting English word patterns based on sampled data."
"In Chapters 9 and 13 we continue presentation of probabilistic approaches to reasoning, including Bayesian belief networks (BBNs), an introduction to Markov models (HMMs), and first-order representation systems supporting stochastic modeling. These models use the directed acyclic graph (or DAG) formalism, and are referred to as graphical models. In Chapter 13 we present stochastic approaches to machine learning."

2. THE ELEMENTS OF COUNTING (Section 5.1, optional)

2.1 The Addition and Multiplication Rules (Section 5.1.1)

"If we have a set A, the number of the elements in set A is denoted by |A|, called the cardinality of A. Of course, A may be empty (the number of elements is zero), finite, countably infinite, or uncountably infinite. Each set is defined in terms of a domain of interest or universe, U, of elements that might be in that set. For example, the set of male people in a classroom may be defined in the context, or or in the universe, of all the people in that room. Similarly, the roll of a 3 on a fair die may be seen as one of a set of six possible outcomes."
"The domain or universe of a set A is thus also a set and is used to determine the complement of that set, A. For example, the complement of the set of all males in the classroom just mentioned is the set of all females, and the complement of the {3} roll of the fair die is 1,2,4,5,6. One set A is a subset of another set B, A ⊆ B, if every element of the set A is also an element of the set B. Thus, trivially, every set is a subset of itself, any set A is a subset of its universe, and the empty set, denoted {} or φ, is a subset of every set."
"The union of two sets A and B, A ∪ B, may be described as the set of all elements in either set. The number of elements in the union of two sets is the total of all the elements in each of the sets minus the number of elements that are in both sets. The justification for this, of course, is the fact that each distinct element in a set may only be counted once. Trivially, if the two sets have no elements in common, the number of the elements in their union is the sum of the number of elements in each set."
"The intersection of two sets A and B, A ∩ B, is the set of all elements common to both sets. We now give examples of a number of the concepts just defined."
The worked example:
"Suppose the universe, U, is the set {0,1,2,3,4,5,6,7,8,9}
Let A be the set {1,3,5,7,9}
Let B be the set {0,2,4,6,8}
Let C be the set {4,5,6}
Then |A| is 5, |B| is 5, |C| is 3, and |U| is 10.
Also, A ⊆ U, B ⊆ U, and A ∪ B = U, |B| = |A|, A = B.
Further, |A ∪ B| = |A| + |B| = 10, since A ∩ B = {}, but
|A ∪ C| = |A| + |C| − |A ∩ C| = 7, since A ∩ C = {5}"
The addition rule:
"We have just presented the major components for the addition rule for combining two sets. For any two sets A and C, the number of elements in the union of these sets is:
|A ∪ C| = |A| + |C| − |A ∩ C|"
"Note that this addition rule holds whether the two sets are disjoint or have elements in common. A similar addition rule holds for three sets A, B, and C, again whether or not they have elements in common:
|A ∪ B ∪ C| = |A| + |B| + |C| − |A ∩ B| − |A ∩ C| − |B ∩ C| + |A ∩ B ∩ C|"
"An argument similar to that made earlier can be used to justify this equation. Similar inclusion/exclusion equations are available, and easily demonstrated, for the addition of sets of elements of more than three sets."
The multiplication principle:
"The multiplication principle for counting states that if we have two sets of elements A and B of size a and b respectively, then there are a x b unique ways of combining the elements of the sets together. The justification for this, of course, is that for each of the a elements of A there are b pairings for that element. The multiplication principle supports many of the techniques used in counting, including the Cartesian product of sets, as well as permutations and combinations of sets."
"The Cartesian product of two sets A and B, denoted A × B, is the set of all ordered pairs (a, b) where a is an element of set A and b is an element of set B; or more formally:
A × B = {(a,b) | (a ∈ A) ∧ (b ∈ B)}
and by the multiplication principle of counting:
|A × B| = |A| × |B|"
"The Cartesian product can, of course, be defined across any number of sets. The product for n sets will be the set of n-tuples where the first component of the n-tuple is any element of the first set, the second component of the n-tuple is any element of the second set, and so on. Again, the number of unique n-tuples that result is the product of the number of elements in each set."

2.2 Permutations and Combinations (Section 5.1.2)

"A permutation of a set of elements is an arranged sequence of the elements of that set. In this arrangement of elements, each may be used only once. An example permutation is an arrangement or ordering of a set of ten books on a shelf that can hold all ten. Another example is the assignment of specific tasks to four of a group of six children."
"We often wish to know how many (unique) permutations there are of a set of n elements. We use multiplication to determine this. If there are n elements in set A, then the set of permutations of these elements is a sequence of length n, where the first element of the sequence is any of the n elements of A, the second element of the sequence is any of the (n - 1) remaining elements of A, the third element of the sequence is any of the (n - 2) remaining elements, and so on."
"The order in which the elements are placed in the permutation sequence is unimportant, i.e., any of the n elements of the set may be placed first in any location in the sequence, any of the n - 1 remaining elements may be placed second in any of the n - 1 remaining locations of the permutation sequence, and so on. Finally, by the multiplication principle, there are n! permutation sequences for this set of n elements."
"We can restrict the number of elements in a permutation of a set A to be any number greater than or equal to zero, and less than or equal to the number of elements n in the original set A. For example, we might want to know how many distinct orderings there are of ten possible books on a shelf that can only hold six of them at a time. If we wanted to determine the number of permutations of the n elements of A taken r at a time, where 0 ≤ r ≤ n, we use multiplication as before, except that now we only have r places in each permutation sequence:
n × (n - 1) × (n - 2) × (n - 3) × ... × (n - (r - 1))"
"Alternatively, we can represent this equation as:
[n × (n - 1) × (n - 2) × (n - 3) × ... × (n - (r - 1)) × (n - r) × (n - r - 1) × ... × 2 × 1] / [(n - r) × (n - r - 1) × ... × 2 × 1]"
"or equivalently, the number of permutations of n elements taken r at a time, which is symbolized as nPr, is:
nPr = n! / (n - r)!"
Combinations:
"The combination of a set of n elements is any subset of these elements that can be formed. As with permutations, we often want to count the number of combinations of items that can be formed, given a set of items. Thus, there is only one combination of the n elements of a set of n items. The key idea here is that the number of combinations represents the number of subsets of the full set of elements that can be created. In the bookshelf example, combinations represent the different subsets of six books, the books on the shelf, that can be formed from the full set of ten books. Another example of combinations is the task of forming four-member committees from a group of fifteen people. Each person is either on the committee or not, and it doesn't make any difference whether they are the first or last

member chosen. A further example is a five-card hand in a poker game. The order in which the cards are dealt makes no difference to the ultimate value of the hand. (It can make a huge difference to the value of the betting, if the last four cards are dealt face-up as in traditional stud poker, but the ultimate value of the hand is independent of the dealt order).”

“The number of combinations of n elements taken r at a time, where 0 ≤ r ≤ n, is symbolized by nCr. A straightforward method for determining the number of these combinations is to take the number of permutations, nPr, as we did already, and then divide out the number of duplicate sets. Since any r element subset of n elements has r! permutations, to get the number of combinations of n elements taken r at a time we divide the number of permutations of n elements taken r at a time by r!. Thus we have: nCr = nPr / r! = n! / [(n - r)! r!]”

“There are many other variations of the counting principles just presented, some of which will be found in the Chapter 5 exercises. We recommend any discrete mathematics textbook for further development of these counting techniques.”

3. ELEMENTS OF PROBABILITY THEORY (Section 5.2)

“With the foundation in the counting rules presented in Section 5.1, we can now introduce probability theory. First, in Section 5.2.1, we consider some fundamental definitions, such as the notion of whether two or more events are independent of each other. In Section 5.2.2 we demonstrate how to infer explanations for particular data sets. This will set us up to consider several examples of probabilistic inference in Section 5.3 and Bayes’ theorem in Section 5.4.”

3.1 The Sample Space, Probabilities, and Independence (Section 5.2.1)

“The following definitions, the foundation for a theory for probability, were first formalized by the French mathematician Laplace (1816) in the early nineteenth century. As mentioned in the introduction to Chapter 5, Laplace was in the process of creating a calculus for gambling!”

The definitions:

“DEFINITION: ELEMENTARY EVENT

An elementary or atomic event is a happening or occurrence that cannot be made up of other events.”

“DEFINITION: EVENT, E

An event is a set of elementary events.”

“DEFINITION: SAMPLE SPACE, S

The set of all possible outcomes of an event E is the sample space S or universe for that event.”

“DEFINITION: PROBABILITY, P

The probability of an event E in a sample space S is the ratio of the number of elements in E to the total number of possible outcomes of the sample space S of E. Thus, p(E) = |E| / |S|.”

The dice example:

“For example, what is the probability that a 7 or an 11 are the result of the roll of two fair dice? We first determine the sample space for this situation. Using the multiplication principle of counting, each die has 6 outcomes, so the total set of outcomes of the two dice is 36. The number of combinations of the two dice that can give a 7 is 1,6; 2,5; 3,4; 4,3; 5,2; and 6,1 - 6 altogether. The probability of rolling a 7 is thus 6/36 = 1/6. The number of combinations of the two dice that can give an 11 is 5,6; 6,5 - or 2, and the probability of rolling an 11 is 2/36 = 1/18. Using the additive property of distinct outcomes, there is 1/6 + 1/18 or 2/9 probability of rolling either a 7 or 11 with two fair dice.”

“In this 7/11 example, the two events are getting a 7 and getting an 11. The elementary events are the distinct results of rolling the two dice. Thus the event of a 7 is made up of the six atomic events (1,6), (2,5), (3,4), (4,3), (5,2), and (6,1). The full sample space is the union of all thirty-six possible atomic events, the set of all pairs that result from rolling the dice. As we see soon, because the events of getting a 7 and getting an 11 have no atomic events in common, they are independent, and the probability of their sum (union) is just the sum of their individual probabilities.”

The poker example:

“In a second example, how many four-of-a-kind hands can be dealt in all possible five-card poker hands? First, the set of atomic events that make up the full space of all five-card poker hands is the combination of 52 cards taken 5 at a time. To get the total number of four-of-a-kind hands we use the multiplication principle. We multiply the number of combinations of 13 cards taken 1 at a time (the number of different kinds of cards: ace, 2, 3..., king) times the number of ways to pick all four cards of the same kind (the combination of 4 cards taken 4 at a time) times the number of possible other cards that fill out the 5 card hand (48 cards remain). Thus, the probability of a four-of-a-kind poker hand is:

13C1 × 4C4 × 48C1 / 52C5 = 13 × 1 × 48 / 2,598,960 = 0.00024”

Results following from the definitions:

“Several results follow immediately from the definitions just made. First, the probability of any event E from the sample space S is:

0 ≤ p(E) ≤ 1, where E ⊆ S”

“A second result is that the sum of the probabilities of all possible outcomes in S is 1. To see this, note that the definition for sample space S indicates that it is made up of the union of all individual events E in the problem.”

“As a third result of the definitions, note that the probability of the complement of an event is:

p($\bar{E}$) = (|S| - |E|) / |S| = (|S| / |S|) - (|E| / |S|) = 1 - p(E).”

“The complement of an event is an important relationship. Many times it is easier to determine the probability of an event happening as a function of it not happening, for example, determining the probability that at least one element of a randomly generated bit string of length n is a 1. The complement is that all the bits in the string are 0, with probability 2^{-n} with n the length of the string. Thus, the probability of the original event is 1 - 2^{-n}.”

“Finally, from the probability of the complement of an event set we work out the probability when no event occurs, sometimes referred to as a contradictory or false outcome:

p({}) = 0”

The union of two sets:

“A final important relationship, the probability of the union of two sets of events, may be determined from the principle of counting presented in Section 5.1, namely that for any two sets A and B: |A ∪ B| = |A| + |B| - |A ∩ B|. From this relationship we can determine the probability of the union of any two sets taken from the sample space S:

p(A ∪ B) = p(A) + p(B) - p(A ∩ B)”

“Of course, this result may be extended to the union of any number of sets, along the line of the principle of inclusion/exclusion presented in Section 5.1.”

“We already presented an example of determining the probability of the union of two sets: The probability of rolling a 7 or an 11 with two fair dice. In this example, the formula just presented was used with the probability of the pairs of dice that gave a 7 disjoint from the pairs of dice that gave an 11. We may also use this formula in the more general case when the sets are not disjoint. Suppose we wanted to determine, rolling two fair dice, the probability of rolling an 8 or of rolling pairs of the same number. We would simply calculate the probability of this union where there is one elementary event - (4,4) - that is in the intersection of both desired outcome events.”

Independence:

“We next consider the probability of two independent events. Suppose that you are a player in a four-person card game where all the cards are dealt out equally. If you do not have the queen of spades, you can conclude that each of the other players has it with probability 1/3. Similarly, you can conclude that each player has the ace of hearts with probability 1/3 and that any one player has both cards with probability 1/3 × 1/3, or 1/9. In this situation we assumed that the events of getting these two cards are independent, even though this is only approximately true. We formalize this intuition with a definition.”

“DEFINITION: INDEPENDENT EVENTS

Two events A and B are independent if and only if the probability of their both occurring is equal to the product of their occurring individually. This independence relation is expressed:

p(A ∩ B) = p(A) * p(B)”

“We sometimes use the equivalent notation p(s,d) for p(s ∩ d). We clarify the notion of independence further in the context of conditional probabilities in Section 5.2.4.”

The bit-string example:

“Because the description of independence of events as just presented is an if and only if relationship, we can determine whether two events are independent by working out their probabilistic relationships. Consider the situation where bit strings of length four are randomly generated. We want to know whether the event of the bit string containing an even number of 1s is independent of the event where the bit string ends with a 0. Using the multiplication principle, each bit having 2 values, there are a total of 2^4 = 16 bit strings of length 4.”

“There are 8 bit strings of length four that end with a 0: {1110, 1100, 1010, 1000, 0010, 0100, 0110, 0000}. There are also 8 bit strings that have an even number of 1s: {1111, 1100, 1010, 1001, 0110, 0101, 0011, 0000}. The number of bit strings that have both an even number of 1s and end with a 0 is 4: {1100, 1010, 0110, 0000}. Now these two events are independent since

p({even number of 1s} ∩ {end with 0}) = p({even number of 1s}) × p({end with 0}) 4/16 = 8/16 × 8/16 = 1/4”

“Consider this same example of randomly generated bit strings of length four. Are the two following events independent: the bit strings have an even number of 1s, and the bit strings end in a 1? When two or more events are not independent, that is the probability of any one event affects the probability of the others, it requires the notion of conditional probability to work out their relationships. We see this in Section 5.2.4.”

Kolmogorov’s axioms:

“Before closing this section, we note that other axiom systems supporting the foundations of probability theory are possible, for instance, as an extension to the propositional calculus (Section 2.1). As an example of the set-based approach we have taken, the Russian mathematician Kolmogorov (1950) proposed a variant of the following axioms, equivalent to our definitions. From these three axioms Kolmogorov systematically constructed all of probability theory.”

“1. The probability of event E in sample space S is between 0 and 1, i.e., 0 ≤ p(E) ≤ 1 2. When the union of all E = S, p(S) = 1, and p($\bar{S}$) = 0 3. The probability of the union of two sets of events A and B is: p(A ∪ B) = p(A) + p(B) - p(A ∩ B)”

“We now demonstrate examples of reasoning with the ideas just presented. Suppose you are driving the interstate highway system and realize you are gradually slowing down because of increased traffic congestion. You begin to search for possible explanations of the slowdown. Could it be road construction? Has there been an accident? All you are aware of is that you are slowing down. But wait! You have access to the state highway statistics, and with your new automobile based GUI and inferencing system you can download to your car’s computer the relevant statistical information. Okay, so you have the data; what can you do with them?”

“For this example we assume we have three true or false parameters (we will define this type parameter as a boolean random variable in Section 5.2.4). First, there is whether or not the traffic - and you - are slowing down. This situation will be labeled S, with assignment of t or f. Second, there is the probability of whether or not there is an accident, A, with assignments t or f. Finally, the probability of whether or not there is road construction at the time, C; again either t or f. We can express these relationships for the interstate highway traffic, thanks to our car-based data download system, in Table 5.1.”

Table 5.1:

S
C
A
p
t
t

t
.16
t
f
t
.12
t
t
f
.01
t
f
f
.03
f
t
t
.16
f
f
t
.12
f
t
f
.05
f
f
f
.01

"The entries of Table 5.1 are interpreted, of course, just like the truth tables of Section 2.1, except that the right hand column gives the probability of the situation on the left hand side happening. Thus, the third row of the table gives the probability of the traffic slowing down and there being an accident but with no construction as 0.16:
S ∩ $\bar{C}$ ∩ A = 0.16"

"It should be noted that we have been developing our probabilistic calculus in the context of sets of events. Figure 5.1 demonstrates how the probabilities of Table 5.1 may be represented with the traditional Venn diagram. We could equally well have presented this situation as the probabilistic truth assignments of propositions, in which case the ∩ would be replaced by a ∧ and Table 5.1 would be interpreted as the truth values of the conjunction of propositions."

"Next, we note that the sum of all possible outcomes of the joint distribution of Table 5.1 is 1.0; this is as one would expect with the axioms of probability presented in Section 5.2.1. We can also work out the probability of any simple or complex set of events. For example, we can calculate the probability that there is traffic slowdown S. The value for slow traffic is 0.32, the sum of the first four lines of Table 5.1; that is, all the situations where S = t. This is sometimes called the unconditional or marginal probability of slow traffic, S. This process is called marginalization because all the probabilities other than slow traffic are summed out. That is, the distribution of a variable can be obtained by summing out all the other variables from the joint distribution containing that variable."

"In a like manner, we can calculate the probability of construction C with no slowdown S - a phenmenon not uncommon in the State of New Mexico! This situation is captured by p(C ∩ $\bar{S}$) = t as the sum of the 5th and the 6th lines of Table 5.1, or 0.06. If we consider the negation of the situation C ∩ $\bar{S}$, we would get (using deMorgan's laws), p($\bar{C}$ ∪ S). Calculating the probability of the union of two sets, as presented in Section 5.2.1, we obtain:
0.16 + 0.12 + 0.01 + 0.61 + 0.01 + 0.03 + 0.16 + 0.12 - (0.16 + 0.12) = .94"

"And again the total probability of C ∩ $\bar{S}$ and its complement (negation) is 1.0."

3.3 Random Variables (Section 5.2.3)

"In the theory of probability, individual probabilities are either computed analytically, through combinatorial methods, or empirically, by sampling a population of events. To this point most of our probabilities have been determined analytically. For example, there are six sides to a die, and two sides to a coin. When the die or coin is 'fair' we say that each outcome, from rolling or flipping, is equally likely. As a result, it is straightforward to determine the event space for these problem situations we later call parametric."

"More interesting probabilistic reasoning, however, results from the sampling based analysis of situations in the actual world of events. These situations often lack a well defined specification that supports the analytical calculation of probabilities. It is also the case that some situations, even when an analytic foundation exists, are so complex that time and computation costs are not sufficient for the deterministic calculation of probabilistic outcomes. In these situations we usually adopt an empirical sampling methodology."

"Most importantly, we assume that all outcomes of an experiment are not equally likely. We retain, however, the basic axioms or assumptions we have made in the previous sections; namely, that the probability of an event is a number between (and including) 0 and 1, and that the summed probabilities of all outcomes is 1. We also retain our rule for the probability of unioned sets of events. We define the idea of a random variable as a method for making this calculus precise."

The definitions:

"DEFINITION: RANDOM VARIABLE
A random variable is a function whose domain is a sample space and range a set of outcomes, most often real numbers. Rather than using a problem-specific event space, a random variable allows us to talk about probabilities as numerical values that are related to an event space."

"DEFINITION: BOOLEAN, DISCRETE, and CONTINUOUS RANDOM VARIABLES
A boolean random variable is a function from an event space to {true, false} or to the subset of real numbers {0.0, 1.0}. A boolean random variable is sometimes called a Bernoulli trial."

"A discrete random variable, which includes boolean random variables as a subset, is a function from the sample space to (a countable subset of) real numbers in [0.0, 1.0]."

"A continuous random variable has as its range the set of real numbers."

The Season example:

"An example using a discrete random variable on the domain of Season, where the atomic events of Season are {spring, summer, fall, winter}, assigns .75, say, to the domain element Season = spring. In this situation we say p(Season = spring) = .75. An example of a boolean random variable in the same domain would be the mapping p(Season = spring) = true. Most of the probabilistic examples that we consider will be of discrete random variables."

The binomial distribution:

"Another example of the use of a boolean random variable would be to calculate the probability of obtaining 5 heads in 7 flips of a fair coin. This would be the combination of 5 of 7 flips being heads times the 1/2 probability of heads to the 5th power times the 1/2 probability of not getting heads to the 2nd power, or:
$7C5 \times (1/2)^5 \times (1/2)^2$"

"This coin flip situation is an example of what is called the binomial distribution. In fact, the outcome of any situation where we want to measure r successes in n trials, where p is the known probability of success may be represented as:
$nCr \times p^r \times (1 - p)^{(n-r)}$"

Expectation:

"An important natural extension to associating probabilistic measures to events is the notion of the expected cost or payoff for that outcome. For example, we can calculate the prospective payback from betting specific money values of the draw of a card or the spin of a roulette wheel. We define the expectation of a random variable or event, ex(E):"

"DEFINITION: EXPECTATION OF AN EVENT
If the reward for the occurrence of an event E, with probability p(E), is r, and the cost of the event not occurring, 1 - p(E), is c, then the expectation derived from an event occurring, ex(E), is:
ex(E) = r × p(E) + c × (1 - p(E))"

"For example, suppose that a fair roulette wheel has integers 0 through 36 equally spaced on the slots of the wheel. In the game each player places $1 on any number she chooses: if the wheel stops on the number chosen, she wins $35; otherwise she loses the dollar. The reward of a win is $35; the cost of a loss, $1. Since the probability of winning is 1/37, of losing, 36/37, the expected value for this event, ex(E), is:
ex(E) = 35 (1/37) + (-1) (36/37) ≈ -0.027"

"Thus the player loses, on average, about $0.03 per play!"

Parametric vs non-parametric:

"We conclude this subsection with a brief discussion and summary of the origins of the values of probabilities used in stochastic reasoning. As noted above, in most of our examples so far, we considered probabilistic values that can be determined by reasoning about known situations such as the flip of a fair coin or the spin of a fair roulette wheel. When we have these situations, we can draw many conclusions about aspects of the probability space, such as its mean, the statistically-based probability measure, and how far from this mean the sampled values usually vary, the standard deviation of the outcomes in this domain."

"We refer to this well understood situation as the parametric approach to the generation of a sample outcome space. The parametric approach is justified in stochastic situations where there exist a priori expectations for the structure of the results of experimental trials. Our task is to 'fill in' the parameters of this well understood situation. An example is flipping a fair coin with the outcome as the binomial distribution. We then can build our expectations of the situation with the binomial model for possible outcomes."

"There are a number of advantages of parametric approaches. The first is that fewer data points are needed to calibrate the expected outcomes, since the shape of the outcome curve is known in advance. A further advantage is that it is often possible to determine a priori the number of outcomes or the amount of training data sufficient to make quality probability estimates. In fact, in the parametric situation, besides calculating the mean and standard deviation of the expectations, we can make an accurate determination of when certain data points are outside normal expectations."

"Of course, many, if not most, interesting situations do not have clear expected outcomes. One example is diagnostic reasoning, in medicine, say. A second example is the use and interpretation of a natural language expression. With language it is quite common to take a non parametric approach to expectations by sampling a large number of situations, as one might have for example, in a language corpus. By analyzing collected examples of language use in newspapers, say, or in conversations from a help-desk for computer support, it is possible to infer meaning for ambiguous expressions in these domains. We demonstrate this methodology analyzing possible phoneme relationships in Section 5.3."

"With sufficient data points, the resulting discrete distribution in non parametric environments can often be smoothed by interpolation to be continuous. Then new situations can be inferred in the context of this created distribution. A major disadvantage of non parametric methods is that, with the absence of the constraints from prior expectations, a large amount of training data is often required to compensate. We present examples of this type reasoning in Section 5.3."

3.4 Conditional Probability (Section 5.2.4)

"The probability measures discussed to this point in Chapter 5 are often called prior probabilities, because they are worked out prior to having any new information about the expected outcomes of events in a particular situation. In this present section we consider the conditional probability of an occurrence of an event, that is, the probability of the event, given some new information or constraint on that event."

"As seen earlier in this chapter, the prior probability of getting a 2 or a 3 on the roll of a fair die is the sum of these two individual results divided by the total number of possible outcomes of the roll of a fair die, or 2/6. The prior probability of a person having a disease is the number of people with the disease divided by the number of people in the domain of concern."

"An example of a conditional or posterior probability is the situation of a patient entering the doctor's office with a set of symptoms, headaches and nausea, say. The experienced doctor will know a set of prior expectations for different diseases based on symptoms, but will want to determine a specific diagnosis for this patient currently suffering from the headaches and nausea. To make these ideas more precise we make two important definitions."

"DEFINITION: PRIOR PROBABILITY
The prior probability, generally an unconditioned probability, of an event is the probability assigned based on all knowledge supporting its occurrence or absence, that is, the probability of the event prior to any new evidence. The prior probability of an event is symbolized: p(event)."

"DEFINITION: POSTERIOR PROBABILITY
The posterior (after the fact) probability, generally a conditional probability, of an event

is the probability of an event given some new evidence. The posterior probability of an event given some evidence is symbolized: p(event | evidence).”

Bayes’ theorem for one symptom and one disease:

“We next begin the presentation of Bayes’ theorem, who’s general form is seen in Section 5.4. The idea supporting Bayes is that the probability of a new (posterior) situation of an hypothesis given evidence can be seen as a function of known probabilities for the evidence given that hypothesis. We can say that we wish to determine the function f, such that p(h|e) = f(p(e|h)). We usually want to determine the value on the left side of this equation given that it is often much easier to compute the values on the right hand side.”

“We now prove Bayes’ theorem for one symptom and one disease. Based on the previous definitions, the posterior probability of a person having disease d, from a set of diseases D, with symptom or evidence, s, from a set of symptoms S, is:

p(d|s) = |d ∩ s| / |s|”

“As in Section 5.1, the ’| |’s surrounding a set is the cardinality or number of elements in that set. The right side of this equation is the number of people having both (intersection) the disease d and the symptom s divided by the total number of people having the symptom s. Figure 5.2 presents a Venn diagram of this situation. We expand the right hand side of this equation. Since the sample space for determining the probabilities of the numerator and denominator are the same, we get:

p(d|s) = p(d ∩ s) / p(s).”

“There is an equivalent relationship for p(s|d); again, see Figure 5.2:

p(s|d) = p(s ∩ d) / p(d).”

“We next solve the p(s|d) equation to determine the value for p(s ∩ d):

p(s ∩ d) = p(s|d) p(d).”

“Substituting this result in the previous equation for p(d|s) produces Bayes’ rule for one disease and one symptom:

p(d|s) = p(s|d) p(d) / p(s)”

“Thus, the posterior probability of the disease given the symptom is the product of the likelihood of the symptom given the disease and the likelihood of the disease, normalized by the probality of that symptom. We generalize this rule in Section 5.4.”

The chain rule:

“We next present the chain rule, an important technique used across many domains of stochastic reasoning, especially in natural language processing. We have just developed the equations for any two sets, A1 and A2:

p(A1 ∩ A2) = p(A1 | A2) p(A2) = p(A2 | A1) p(A1).”

“and now, the generalization to multiple sets Ai, called the chain rule:

p(A1 ∩ A2 ∩ … ∩ An) = p(A1) p(A2 | A1) p(A3 | A1 ∩ A2) …. p(An | $\cap_{i=1}^{n-1}$ Ai)”

“We make an inductive argument to prove the chain rule, consider the nth case:

p(A1 ∩ A2 ∩ … ∩ An-1 ∩ An) = p((A1 ∩ A2 ∩ … ∩ An-1) ∩ An),”

“We apply the intersection of two sets rule to get:

p((A1 ∩ A2 ∩ … ∩ An-1) ∩ An) = p(A1 ∩ A2 ∩ … ∩ An-1) p(An | A1 ∩ A2 ∩ … ∩ An-1)”

“and then reduce again, considering that:

p(A1 ∩ A2 ∩ … ∩ An-1) = p((A1 ∩ A2 ∩ … ∩ An-2) ∩ An-1)”

“until p(A1 ∩ A2) is reached, the base case, which we have already demonstrated.”

Conditional independence:

“We close this section with several definitions based on the use of the chain rule relationship. First, we redefine independent events (see Section 5.2.1) in the context of conditional probabilities, and then we define conditionally independent events, or the notion of how events can be independent of each other, given some third event.”

“DEFINITION: INDEPENDENT EVENTS

Two events A and B are independent of each other if and only if p(A ∩ B) = p(A) p(B). When p(B) ≠ 0 this is the same as saying that p(A) = p(A|B). That is, knowing that B is true does not affect the probability of A being true.”

“DEFINITION: CONDITIONALLY INDEPENDENT EVENTS

Two events A and B are said to be conditionally independent of each other, given event C if and only if p((A ∩ B) | C) = p(A | C) p(B | C).”

“As a result of the simplification of general chain rule offered by conditionally independent events, larger stochastic systems can be built with smaller computational cost; that is, conditionally independent events simplify joint distributions. An example from our slow traffic situation: suppose that as we slow down we notice orange traffic control barrels along the side of the road. Besides suggesting that the cause of our slowdown is now more likely to be from road construction than a traffic accident, the presence of orange barrels will have its own probabilistic measure. In fact, the variables representing traffic slowdown and the presence of orange barrels are conditionally independent since they are both caused by road construction. Thus, we say that the variable road construction separates traffic slowdown from orange barrels.”

“Because of the statistical efficiencies gained by conditional independence, a major task in the construction of large stochastic computational systems is to break out a complex problem into more weakly connected subproblems. The subproblem interaction relationships are then controlled by the various conditional separation relationships. We see this idea further formalized with the definition of d-separation in Section 9.3.”

4. BAYES’ THEOREM (Section 5.3)

“The Reverend Thomas Bayes was a mathematician and a minister. His famous theorem was published in 1763, four years after his death. His paper, entitled Essay towards Solving a Problem in the Doctrine of Chances was published in the Philosophical Transactions of the Royal Society of London. Bayes’ theorem relates cause and effect in such a way that by understanding the effect we can learn the probability of its causes. As a result Bayes’ theorem is important both for determining the causes of diseases, such as cancer, as well as useful for determining the effects of some particular medication on that disease.”

4.1 Introduction (Section 5.3.1)

“One of the most important results of probability theory is the general form of Bayes’ theorem. First, we revisit one of the results of Section 5.2.4, Bayes’ equation for one disease and one symptom. To help keep our diagnostic relationships in context, we rename the variables used previously to indicate individual hypotheses, h_i, from a set of hypotheses, H, and a set of evidence, E. Furthermore, we will now consider the set of individual hypotheses h_i as disjoint, and having the union of all h_i to be equal to H.

$p(h_i|E) = (p(E|h_i) \times p(h_i)) / p(E)$”

“This equation may be read, ‘The probability of an hypothesis h_i given a set of evidence E is . . .’ First, note that the denominator p(E) on the right hand side of the equation is very much a normalizing factor for any of the hypotheses h_i from the set H. It is often the case that Bayes’ theorem is used to determine which hypothesis out of a set of possible hypotheses is strongest, given a particular evidence set E. In this case we often drop the p(E) denominator which is identical for all the h_i, with the saving of a possibly large computational cost. Without the denominator, we have created the maximum a posteriori value for an hypothesis:

arg max(h_i) p(E|h_i) p(h_i)”

“We read this expression as ‘The maximum value over all h_i of p(E|h_i) p(h_i)’. The simplification just described is highly important for diagnostic reasoning as well as in natural language processing. Of course, the arg max, or maximum likelihood hypothesis, is no longer a random variable as defined previously.”

The denominator:

“Next consider the calculation of the denominator p(E) in the situation where the entire sample space is partitioned by the set of hypotheses h_i. The partition of a set is defined as the split of that set into disjoint non overlapping subsets, the union of which make up the entire set. Assuming that the set of hypotheses h_i partition the entire sample space, we get:

$p(E) = \Sigma_i\ p(E|h_i)\ p(h_i)$”

“This relationship is demonstrated by considering the fact that the set of hypotheses h_i forms a partition of the full set of evidence E along with the rule for the probability of the intersection of two sets. So:

E = (E ∩ h1) ∪ (E ∩ h2) ∪ … ∪ (E ∩ hn)”

“But by the generalized union of sets rule developed in Section 5.2.1:

p(E) = p((E ∩ h1) ∪ (E ∩ h2) ∪ … ∪ (E ∩ hn)) = p(E ∩ h1) + p(E ∩ h2) + … + p(E ∩ hn) - p(E ∩ h1 ∩ E ∩ h2 ∩ … ∩ hn) = p(E ∩ h1) + p(E ∩ h2) + … + p(E ∩ hn)”

“since the set of hypotheses h_i partition E and their intersection is empty.”

The general form of Bayes’ theorem:

“This calculation of p(E) produces the general form of Bayes’ theorem, where we assume that the set of hypotheses, h_i partition the evidence set E:

$p(h_i|E) = [p(E|h_i) \times p(h_i)] / [\Sigma_{k=1}^{n}\ p(E|h_k) \times p(h_k)]$”

“p(h_i|E) is the probability that h_i is true given evidence E. p(h_i) is the probability that h_i is true overall. p(E|h_i) is the probability of observing evidence E when h_i is true. n is the number of possible hypotheses.”

“Bayes’ theorem provides a way of computing the probability of a hypothesis h_i, given a particular piece of evidence, given only the probabilities with which the evidence follows from actual causes (the hypotheses).”

The PROSPECTOR example:

“As an example, suppose we want to examine the geological evidence at some location to see whether or not it is suited to finding copper. We must know in advance the probability of finding each of a set of minerals and the probability of certain evidence being present when any particular mineral is found. Then we can use Bayes’ theorem, with evidence found at the particular location, to determine the likelihood of copper. This approach is used by PROSPECTOR, built at Stanford University and SRI International and employed in mineral exploration (copper, molybdenum, and other minerals). PROSPECTOR has found commercially significant mineral deposits at several sites (Duda et al. 1979a).”

The dealer example:

“We next present a simple numerical example demonstrating Bayes’ theorem. Suppose that you go out to purchase an automobile. The probability that you will go to dealer 1, d1, is 0.2. The probability of going to dealer 2, d2, is 0.4. There are only three dealers you are considering and the probability that you go to the third, d3, is also 0.4. At d1 the probability of purchasing a particular automobile, a1, is 0.2; at dealer d2 the probability of purchasing a1 is 0.4. Finally, at dealer d3, the probability of purchasing a1 is 0.3. Suppose you purchase automobile a1. What is the probability that you purchased it at dealer d2?”

“First, we want to know, given that you purchased automobile a1, that you bought it from dealer d2, i.e., to determine p(d2|a1). We present Bayes’ theorem in variable form for determining p(d2|a1) and then with variables bound to the situation in the example.

p(d2|a1) = (p(a1|d2) p(d2)) / (p(a1|d1) p(d1) + p(a1|d2) p(d2) + p(a1|d3) p(d3)) = (0.4) (0.4) / ((0.2) (0.2) + (0.4) (0.4) + (0.4) (0.3)) = 0.16 / 0.32 = 0.5”

The commitments and the complexity problem:

“There are two major commitments in using Bayes’ theorem: first, all the probabilities on the relationships of the evidence with the various hypotheses must be known, as well as the probabilistic relationships among the pieces of evidence. Second, and sometimes more difficult to determine, all the relationships between evidence and hypotheses, or p(E|h_k), must be estimated or empirically sampled. Recall that calculation of p(E) for the general form of Bayes’ theorem also required that the hypothesis set h_i partitioned the set of evidence E. In general, and especially in areas such as medicine and natural language processing, an assumption of this partition cannot be justified a priori.”

“It is an interesting to note, however, that many situations that violate this assumption (that the individual pieces of evidence partition the evidence set) behave quite well! Using this partition assumption, even in situations where it is not justified, is called using naive Bayes or a Bayes classifier. With naive Bayes, the assumption is, for an hypothesis h_j:

$p(E|h_j) \approx \Pi_{i=1}^{n}\ p(e_i|h_j)$”

“i.e., we assume the pieces of evidence are independent, given a particular hypothesis.”

“Using Bayes’ theorem to determine the probability of some hypothesis h_i given a set of evidence E, p(h_i|E), the numbers on the right-hand side of the equation are often easily obtainable. This is especially true when compared to obtaining the values for the left-hand side of the equation, i.e., determining p(h_i|E) directly. For example, because the population is smaller, it is much easier to determine the number of meningitis patients who have headaches than it is to determine the percentage of headache sufferers with meningitis. Even more importantly, for the simple case of a single disease and a single

symptom, not very many numbers are needed. Troubles begin, however, when we consider multiple diseases h_i from the domain of diseases H and multiple symptoms e_n from a set E of possible symptoms. When we consider each disease from H and each symptom from E singly, we have m x n measures to collect and integrate. (Actually m x n posterior probabilities plus m + n prior probabilities.)"

"Unfortunately, our analysis is about to get much more complex. To this point, we considered each symptom e_i individually. In actual situations, single isolated symptoms are rarely the case. When a doctor is considering a patient, for instance, there are often many combinations of symptoms she must consider. We require a form of Bayes' theorem to consider any single hypothesis h_i in the context of the union of multiple possible symptoms e_i."

"$p(h_i|e1 \cup e2 \cup \ldots \cup e_n) = (p(h_i)\ p(e1 \cup e2 \cup \ldots \cup e_n|h_i)) / p(e1 \cup e2 \cup \ldots \cup e_n)$"

"With one disease and a single symptom we needed only m x n measurements. Now, for every pair of symptoms e_i and e_j and a particular disease hypothesis h_i, we need to know both $p(e_i \cup e_j \mid h_i)$ and $p(e_i \cup e_j)$. The number of such pairs is n x (n - 1), or approximately n^2, when there are n symptoms in E. Now, if we want to use Bayes, there will be about (m x n^2 conditional probabilities) + (n^2 symptom probabilities) + (m disease probabilities) or about m x n^2 + n^2 + m pieces of information to collect. In a realistic medical system with 200 diseases and 2000 symptoms, this value is over 800,000,000!"

"There is some hope, however. As was discussed when we presented conditional independence, many of these symptom pairs will be independent, that is $p(e_i|e_j) = p(e_i)$. Independence means, of course, that the probability of e_i is not affected by the presence of e_j. In medicine, for example, most symptoms are not related, e.g., hair loss and sore elbow. But even if only ten percent of our example symptoms are not independent, there are still about 80,000,000 relationships remaining to consider."

"In many diagnostic situations, we must also deal with negative information, e.g., when a patient does not have a symptom such as bad blood pressure. We require both:

$p(\bar{e}_i) = 1 - p(e_i)$ and $p(\bar{h}_i|e_i) = 1 - p(h_i|e_i)$."

"We also note that $p(e_i|h_i)$ and $p(h_i|e_i)$ are not the same and will almost always have different values. These relationships, and the avoidance of circular reasoning, is important for the design of Bayesian belief networks considered in Section 9.3.1."

"A final problem, which again makes keeping the statistics of complex Bayesian systems virtually intractable, is the need to rebuild probability tables when new relationships between hypotheses and evidence sets are discovered. In many active research areas such as medicine, new discoveries happen continuously. Bayesian reasoning requires complete and up-to-date probabilities, including joint probabilities, if its conclusions are to be correct. In many domains, such extensive data collection and verification are not possible, or if possible, quite expensive."

"Where these assumptions are met, however, Bayesian approaches offer the benefit of a mathematically well-founded handling of uncertainty. Most expert system domains do not meet these requirements and must rely on heuristic approaches, as presented in Chapter 8. Furthermore, due to complexity issues, we know that even fairly powerful computers cannot use full Bayesian techniques for successful real-time problem solving. We end this chapter with two examples that show how a Bayesian approach might work to organize hypothesis/evidence relationships. But first we must define probabilistic finite state machines/acceptors."

5. APPLICATIONS OF THE STOCHASTIC METHODOLOGY (Section 5.4)

"You say [t ow m ey t ow] and I say [t ow m aa t ow]. . . — IRA GERSHWIN, 'Let's Call the Whole Thing Off'"

"In this section we present two examples that use probability measures to reason about the interpretation of ambiguous information. First, we define an important modeling tool based on the finite state machine of Section 3.1, the probabilistic finite state machine."

"DEFINITION: PROBABILISTIC FINITE STATE MACHINE

A probabilistic finite state machine is a finite state machine where the next state function is a probability distribution over the full set of states of the machine."

"DEFINITION: PROBABILISTIC FINITE STATE ACCEPTOR

A probabilistic finite state machine is an acceptor, when one or more states are indicated as the start states and one or more as the accept states."

"It can be seen that these two definitions are simple extensions to the finite state and the Moore machines presented in Section 3.1. The addition for non-determinism is that the next state function is no longer a function in the strict sense. That is, there is no unique range state for each input value and each state of the domain. Rather, for any state, the next state function is a probability distribution over all possible next states."

"Figure 5.3 presents a probabilistic finite state acceptor that represents different pronunciations of the word 'tomato'. A particular acceptable pronunciation for the word tomato is characterized by a path from the start state to the accept state. The decimal values that label the arcs of the graph represent the probability that the speaker will make that particular transition in the state machine. For example, 60% of all speakers in this data set go directly from the t phone to the m without producing any vowel phone between."

"Besides characterizing the various ways people in the pronunciation database speak the word 'tomato', this model can be used to help interpret ambiguous collections of phonemes. This is done by seeing how well the phonemes match possible paths through the state machines of related words. Further, given a partially formed word, the machine can try to determine the probabilistic path that best completes that word."

The Brown and Switchboard corpora:

"In a second example, also adapted from Jurafsky and Martin (2009), we consider the phoneme recognition problem, often called decoding. Suppose a phoneme recognition algorithm has identified the phone ni (as in 'knee') that occurs just after the recognized word (phone) I, and we want to associate ni with either a word or the first part of a word. In this case we have linguistic corpora, the Brown and Switchboard corpora, to assist us. The Brown corpus is a one million word collection of sentences from 500 written texts, including newspapers, novels, and academic writings collected at Brown University in the 1960s (Kucera and Francis 1967, Francis 1979). The Switchboard corpus is a 1.4 million word collection of telephone conversations. These corpora together contain about 2,500,000 words that let us sample both written and spoken information bases."

Table 5.2:

Word
Frequency
Probability
knee
61
.000024
the
114834
.046
neat
338
.00013
need
1417
.00056
new
2625
.001

"There are a number of ways to proceed in identifying the most likely word to associate with the ni phone. First we can determine which word, with this phone first, is the most likely to be used. Table 5.2 presents the raw frequencies of these words along with the probability of their occurrence, that is, the frequency of the word divided by the total number of words in these combined corpora. (This table is adapted from Jurafsky and Martin (2009); see their book for a justification that 'the' belongs to this collection.) From this data, the word 'the' would seem to be the first choice for matching ni."

The Bayesian approach:

"We next apply a form of Bayes' theorem we presented in the previous section. Our second attempt at analyzing the phone ni following I, uses a simplification of that formula that ignores its denominator:

$p(word \mid [ni]) \propto p([ni] \mid word) \times p(word)$"

Table 5.3:

Word
p([ni] | word)
p(word)
p([ni] | word) × p(word)
new
0.36
0.001
0.00036
neat
0.52
0.00013
0.000068
need
0.11
0.00056
0.000062
knee
1.0
0.000024
0.000024
the
0.0
0.046
0.0

"The results of this calculation, ordered from most recommended to least, are found in Table 5.3 and explain why p(ni | the) is impossible. The results of Table 5.3 also suggest that new is the most likely word for decoding ni. But the two-word combination I new doesn't seem to make much sense, whereas other combinations, such as I need does. Part of the problem in this situation is that we are still reasoning on the phone level, that is, determining the probability p(ni | new). There is, in fact, a straightforward way of addressing this issue, and that is to look for explicit two word combinations in the corpora. Following this line of reasoning, it turns out that I need is a much more likely pair of consecutive words than is I new, or any of the other I-word combinations, for that matter."

"The methodology for deriving probabilities from pairs, or triples, of word combinations in corpora is called n-gram analysis. With two words, we were using bigrams, with three, trigrams. The probabilities derived from word combinations using n-grams is important, as we see again in Chapter 15."

"We present again and extend the example of Section 5.2.2. Suppose you are driving the interstate highway system and realize you are gradually slowing down because of increased traffic congestion. You begin to search for possible explanations of the slowdown. Could it be road construction? Has there been an accident? Perhaps there are other possible explanations. After a few minutes you come across orange barrels at the side of the road that begin to cut off the outside lane of traffic. At this point you determine that the best explanation of your slow down is road construction. At the same time the alternative hypothesis of accident is explained away.

Similarly if you would have seen flashing lights in the distance ahead, such as from a police vehicle or an ambulance, the best explanation given this evidence would be a traffic accident and road construction would have been explained away. When an hypothesis is explained away that does not mean that it is no longer possible. Rather, in the context of new evidence, it is simply less likely."

"Figure 5.4 presents a Bayesian account of what we have just seen. road construction is correlated with orange barrels and bad traffic. Similarly, accident correlates with flashing lights and bad traffic. We examine Figure 5.4 and build a joint probability distribution for the road construction and bad traffic relationship. We simplify both of these variables to be either true (t) or false (f) and represent the probability distribution in Table 5.4. Note that if construction is f there is not likely to be bad traffic and if it is t then bad traffic is likely. Note also that the probability of road construction on the interstate, C = true, is .5 and the probability of having bad traffic, T = true, is .4 (this is New Mexico!)."

Table 5.4:

C
T
p
t
t
.3
t
f
.2
f
t
.1
f
f
.4

"We next consider the change in the probability of road construction given the fact that we have bad traffic, or p(C|T) or p(C = t | T = t). p(C|T) = p(C = t , T = t) / (p(C = t , T = t) + p(C = f , T = t)) = .3 / (.3 + .1) = .75"

"So now, with the probability of road construction being .5, given that there actually is bad traffic, the probability for road construction goes up to .75. This probability will increase even further with the presence of orange barrels, explaining away the hypothesis of accident."

The complexity issue:

"Besides the requirement that we may have knowledge or measurements for any of our parameters being in a particular state, we also must address, as noted in Section 5.4.1, complexity issues. Consider the calculation of the joint probability of all the parameters of Figure 5.4 (using the chain rule and a topologically sorted order of variables): p(C,A,B,T,L) = p(C) × p(A|C) × p(B|C,A) × p(T|C,A,B) × p(L|C,A,B,T)"

"This result is a general decomposition of the probability measures that is always true. The cost of producing this joint probability table is exponential in the number of parameters involved, and in this case requires a table of size 2^5 or 32. We are considering a toy problem, of course, with only five parameters. A situation of interesting size, with thirty or more parameters say, requires a joint distribution table of roughly a billion elements. As we will see in Section 9.3, Bayesian belief networks and d-separation give us further tools for addressing this representational and computational complexity."

6. EPILOGUE AND REFERENCES (Section 5.5)

"Games of chance date back, at least, to the Greek and Roman civilizations. It wasn't until the European Renaissance, however, that the mathematical analysis of probability theory began. As noted in the chapter, probabilistic reasoning begins with determining principles of counting and combinatorics. Actually, one of the first combinatorial 'machines' is attributed to Ramon Llull (Ford et al. 1995), a Spanish philosopher and Franciscan monk, who created his device, said to automatically enumerate the attributes of God, with the intention of converting heathens. The first publication on probabilities, De Ratiociniis Ludo Aleae, was authored by Christian Huygens (1657). Huygens describes earlier results by Blaise Pascal, including a methodology for calculating probabilities, as well as conditional probabilities. Pascal targeted both the 'objective' analysis of the world of games as well as the more 'subjective' analysis of belief systems, including the existence of God."

"The definitions of probabilities presented in Section 5.2.1 are based on the formalism proposed by the French mathematician Pierre Simon Laplace. Laplace's book Theorie Analytique des Probabilitees (1816) documents this approach. Laplace's work was based on earlier results published by Gotlob Leibnitz and James Bernoulli."

"Thomas Bayes was a mathematician and a minister. His famous theorem was published in 1764, after his death. His paper, entitled Essay towards Solving a Problem in the Doctrine of Chances was published in the Philosophical Transactions of the Royal Society of London. Ironically, Bayes' theorem is never explicitly stated in this paper, although it is there! Bayes also has extensive discussion of the 'reality' of statistical measures."

"Bayes' research was partly motivated to answer the philosophical skepticism of the Scots philosopher David Hume. Hume's dismissal of causality destroyed any foundation for arguments through miracles for the existence of God. In fact, in a 1763 paper presented to the British Royal Society, the minister Richard Price used Bayes' theorem to show there was good evidence in favor of the miracles described in the New Testament."

"The mathematicians of the early twentieth century, including Fisher (1922), Popper (1959), and Carnap (1948) completed the foundation of modern probability theory continuing the 'subjective/objective' debates on the nature of probabilities. Kolmogorov (1950, 1965) axiomatized the foundations of probabilistic reasoning (see Section 5.2.1)."

"It is possible to present the topic of probabilistic reasoning from several vantage points. The two most popular approaches are based on the propositional calculus and set theory. For the propositional calculus, Section 2.1, propositions are assigned a confidence or probabilistic truth value in the range [0.0, 1.0]. This approach offers a natural extension to the semantics of propositional calculus. We have chosen the second approach to probabilistic reasoning, feeling this orientation is a bit more intuitive, bringing to bear all the counting and other techniques of set theory, as seen in Section 5.1. In Chapters 9 and 13, we extend our presentation of stochastic systems to first-order (variable based) representation schemes."

"Bayes' theorem has offered a foundation for several expert systems of the 1970s and 1980s, including an extensive analysis of acute abdominal pain at the University of Glasgow Hospital (de Dombal et al. 1974), and PROSPECTOR, the system from Stanford University supporting mineral exploration (Duda et al. 1979a). The naive Bayes approach has been used in a number of classification problems, including pattern recognition (Duda and Hart 1973), natural language processing (Mooney 1996), and elsewhere. Domingos and Pazzani (1997) offer justification for the successes of naive Bayes classifiers in situations which do not meet Bayesian independence assumptions."

"There is a large amount of current research in probabilistic reasoning in artificial intelligence, some presented in Chapters 9, 13, and 16. Uncertain reasoning makes up a component of AI conferences, including AAAI, IJCAI, NIPS, and UAI. There are several excellent introductory texts in probabilistic reasoning (Ross 1988, DeGroot 1989), as well as several introductions to the use of stochastic methods in artificial intelligence applications (Russell and Norvig 2003, Jurafsky and Martin 2009, Manning and Schutze 1999)."

"I am indebted to Dan Pless of Sandia National Laboratories for the general approach taken in this chapter, for several of the examples, and also for editorial suggestions."

7. EXERCISES (Section 5.6)

The seventeen exercises exactly as Luger lists them:

"1. A fair six-sided die is tossed five times and the numbers up are recorded in a sequence. How many different sequences are there? 2. Find the number of distinguishable permutations of the letters in the word MISSISSIPPI, of the letters of the word ASSOCIATIVE. 3. Suppose that an urn contains 15 balls, of which eight are red and seven are black. In how many ways can five balls be chosen so that: a. all five are red? all five are black? b. two are red and three are black c. at least two are black 4. How many ways can a committee of three faculty members and two students be selected from a group of five faculty and seven students? 5. In a survey of 250 television viewers, 88 like to watch news, 98 like to watch sports, and 94 like to watch comedy. 33 people like to watch news and sports, 31 like to watch sports and comedy, and 35 like to watch news and comedy. 10 people like to watch all three. Suppose a person from this group is picked at random: a. What is the probability that they watch news but not sports? b. What is the probability that they watch news or sports but not comedy? c. What is the probability that they watch neither sports nor news? 6. What is the probability that a four digit integer with no beginning zeros: a. Has 3, 5, or 7 as a digit? b. Begins with 3, ends with 5, or has 7 as a digit? 7. Two dice are tossed. Find the probability that their sum is: a. 4 b. 7 or an even number c. 10 or greater 8. A card is drawn from the usual fifty-two card deck. What is the probability of: a. drawing a face card (jack, queen, king or ace) b. drawing a queen or a spade c. drawing a face card or a club 9. What is the probability of being dealt the following hands in a five card poker game (from the normal deck of fifty-two cards)? a. A 'flush' or all cards from the same suit. b. A 'full house' or two cards of the same value and three cards of another value. c. A 'royal flush' or the ten, jack, queen, king, and ace all of the same suit. 10. The expectation is the mean or average of the value of a random variable. In throwing a die, for example, it can be calculated by totaling up the resulting values from a large number of throws and then dividing by the number of throws. a. What is the expectation from throwing a fair die? b. The value of a roulette wheel with 37 equally likely results? c. The value of a draw from a set of cards (ace is valued at 1, all other face cards as 10)? 11. Suppose that we are playing a game where we toss a die and then receive the amount of dollars equal to the value of the die. For example, if a 3 comes up we receive $3. If it costs us $4 to play this game, is this reasonable? 12. Consider the situation where bit strings of length four are randomly generated. Demonstrate whether or not the event of production of bit strings containing an even number of 1s is independent of the event of producing bit strings that end in a 1. 13. Show that the statement p(A,B|C) = p(A|C) p(B|C) is equivalent to both p(A|B,C) = p(A|C) and p(B|A,C) = p(B|C). 14. In manufacturing a product, 85% of the products that are produced are not defective. Of the products inspected, 10% of the good ones are seen as defective and not shipped whereas only 5% of the defective products are approved and shipped. If a product is shipped, what is the probability that it is defective? 15. A blood test is 90% effective in detecting a disease. It also falsely diagnoses that a healthy person has the disease 3% of the time. If 10% of those tested have the disease, what is the probability that a person who tests positive will actually have the disease? 16. Suppose an automobile insurance company classifies a driver as good, average, or bad. Of all their insured drivers, 25% are classified good, 50% are average, and 25% are bad. Suppose for the coming year, a good driver has a 5% chance of having an accident, and average driver has 15% chance of having an accident, and a bad driver has a 25% chance. If you had an accident in the past year what is the probability that you are a good driver? 17. Three prisoners, A, B, C are in their cells. They are told that one of them will be executed the next day and the others will be pardoned. Only the governor knows who will be executed. Prisoner A asks the guard a favor. 'Please ask the governor who will be executed, and then tell either prisoner B or C that they will be pardoned.' The guard does as was asked and then comes back and tells prisoner A that he has told prisoner B that he (B) will be pardoned. What are prisoner A's chances of being executed, given this message? Is there more information than before his request to the guard? (Adapted from Pearl 1988)."

Term	Definition
Elementary (atomic) event	A happening that cannot be made up of other events
Event E	A set of elementary events
Sample space S	The set of all possible outcomes of an event
Probability P	$p(E) = \|E\| / \|S\|$
Addition rule	$\|A \cup C\| = \|A\| + \|C\| - \|A \cap C\|$
Multiplication principle	If A has size a and B has size b, there are a × b pairings
Cartesian product	$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$; $\|A \times B\| = \|A\| \times \|B\|$
Permutation	Arranged sequence of all n elements: n!
Permutations taken r at a time	$nPr = n! / (n - r)!$
Combination	Any subset of n elements: $nCr = n! / [(n - r)! r!]$
Independent events	$p(A \cap B) = p(A)\, p(B)$
Complement	$p(\bar{E}) = 1 - p(E)$
Union of two events	$p(A \cup B) = p(A) + p(B) - p(A \cap B)$
Kolmogorov's axioms	$0 \le p(E) \le 1$; $p(S) = 1$; $p(A \cup B) = p(A) + p(B) - p(A \cap B)$
Marginalisation	Sum out other variables to get the distribution of one variable
Random variable	Function from a sample space to a set of outcomes (usually real numbers)
Boolean random variable	Function from event space to {true, false} or {0.0, 1.0}; also called a Bernoulli trial
Discrete random variable	Function to a countable subset of real numbers
Continuous random variable	Range is the real numbers
Binomial distribution	$nCr \times p^r \times (1 - p)^{(n-r)}$
Expectation ex(E)	$r \times p(E) + c \times (1 - p(E))$
Parametric approach	Probabilities determined analytically, with known distribution shape
Non-parametric approach	Probabilities determined by sampling, without prior expectations
Prior probability p(event)	Unconditioned probability before new evidence
Posterior probability p(event | evidence)	Probability after new evidence
Conditional probability	$p(d|s) = p(d \cap s) / p(s)$
Bayes' rule (one disease, one symptom)	$p(d|s) = p(s|d)\, p(d) / p(s)$
Chain rule	$p(A_1 \cap \ldots \cap A_n) = p(A_1)\, p(A_2|A_1) \ldots p(A_n|A_1 \cap \ldots \cap A_{n-1})$
Conditionally independent events	$p((A \cap B) \mid C) = p(A \mid C)\, p(B \mid C)$
Bayes' theorem (general form)	$p(h_i|E) = [p(E|h_i)\, p(h_i)] / [\Sigma_k\, p(E|h_k)\, p(h_k)]$
arg max hypothesis	Maximum a posteriori: $\arg\max p(E|h_i)\, p(h_i)$
Naive Bayes	$p(E|h_j) \approx \Pi\, p(e_i|h_j)$
Probabilistic FSM	FSM where the next-state function is a probability distribution
Probabilistic finite state acceptor	Probabilistic FSM with start state(s) and accept state(s)
Bigram	Two-word combination in n-gram analysis
Trigram	Three-word combination in n-gram analysis

8. QUICK-REFERENCE TABLE OF KEY DEFINITIONS

Topic	Belongs to
Bayesian belief networks (details)	Chapter 9
d-separation	Chapter 9
HMMs (details)	Chapter 9, 13
Markov models	Chapter 9, 13
Certainty factors	Chapter 9
Fuzzy logic	Chapter 9
Dempster-Shafer theory	Chapter 9
Resolution theorem proving	Chapter 14
Natural language parsing (ATN, Earley)	Chapter 15
Stochastic approaches to machine learning	Chapter 13

End of Chapter 5 notes, based strictly on Luger's textbook.

Chapter 6: Building Control Algorithms for State Space Search

Complete Study Notes — Based strictly on Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th ed., Chapter 6

1. WHY CHAPTER 6 EXISTS

Luger opens the chapter with two quotations:
"If we carefully factor out the influences of the task environments from the influences of the underlying hardware components and organization, we reveal the true simplicity of the adaptive system. For, as we have seen, we need postulate only a very simple information processing system in order to account for human problem solving in such tasks as chess, logic, and cryptarithmetic. The apparently complex behavior of the information processing system in a given environment is produced by the interaction of the demands of the environment with a few basic parameters of the system, particularly characteristics of its memories. —A. NEWELL AND H. A. SIMON, 'Human Problem Solving' (1972)"
"What we call the beginning is often the end / And to make an end is to make a beginning. / The end is where we start from . . . —T. S. ELIOT, 'Four Quartets'"
"To this point, Part II has represented problem solving as search through a set of problem situations or states. Chapter 2 presented the predicate calculus as a medium for describing states of a problem and sound inference as a method for producing new states. Chapter 3 introduced graphs to represent and link problem situations. The backtrack algorithm, as well as algorithms for depth-first and breadth-first search can explore these graphs. Chapter 4 presented algorithms for heuristic search. In Chapter 5, given probabilistic states of the world, stochastic inference was used to produce new states. To summarize, Part II has:
Represented a problem solution as a path in a graph from a start state to a goal.
Used search to test systematically alternative paths to goals.
Employed backtracking, or some other mechanism, to allow algorithms to recover from paths that failed to find a goal.
Used lists to keep explicit records of states under consideration.
The open list allows the algorithm to explore untried states if necessary.
The closed list of visited states allows the algorithm to implement loop detection and avoid repeating fruitless paths.
Implemented the open list as a stack for depth-first search, a queue for breadth-first search, and a priority queue for best-first search."
"Chapter 6 introduces further techniques for building search algorithms. In Section 6.1, recursive search implements depth-, breadth-, and best-first search in a more concise and natural fashion than was done in Chapter 3. Further, recursion is augmented with unification to search the state space generated by predicate calculus assertions. This pattern-directed search algorithm is the basis of PROLOG, see Section 14.3, and several of the expert systems discussed in Chapter 8. In Section 6.2 we introduce production systems, a general architecture for pattern-directed problem solving that has been used extensively to model human problem solving, Chapter 16, as well as in other AI applications, including expert systems, Chapter 7. Finally, in Section 6.3, we present another AI problem-solving control architecture, the blackboard."

2. RECURSION-BASED SEARCH (Section 6.1)

"In mathematics, a recursive definition uses the term being defined as part of its own definition. In computer science, recursion is used to define and analyze both data structures and procedures. A recursive procedure consists of:
A recursive step: the procedure calls itself to repeat a sequence of actions.
A terminating condition that stops the procedure from recurring endlessly (the recursive version of an endless loop).
Both these components are essential and appear in all recursive definitions and algorithms. Recursion is a natural control construct for data structures that have a regular structure and no definite size, such as lists, trees, and graphs, and is particularly appropriate for state space search."

2.1 Recursive Search (Section 6.1.1)

"A direct translation of the depth-first search algorithm of Chapter 3 into recursive form illustrates the equivalence of recursion and iteration. This algorithm uses global variables closed and open to maintain lists of states. Breadth-first and best-first search can be designed with virtually the same algorithm, that is, by retaining closed as a global data structure and by implementing open as a queue or a priority queue rather than as a stack (build stack becomes build queue or build priority queue):"
"function depthsearch; % open & closed global begin if open is empty then return FAIL; current_state := the first element of open; if current_state is a goal state then return SUCCESS else begin open := the tail of open; closed := closed with current_state added; for each child of current_state if not on closed or open % build stack then add the child to the front of open end; depthsearch % recur end."
"Depth-first search as just presented does not utilize the full power of recursion. It is possible to simplify the procedure further by using recursion itself (rather than an explicit open list) to organize states and paths through the state space. In this version of the algorithm, a global closed list is used to detect duplicate states and prevent loops, and the open list is implicit in the activation records of the recursive environment. Since the open list can no longer be explicitly manipulated, breadth-first and best-first search are no longer natural extensions of the following algorithm:"
"function depthsearch (current_state); % closed is global begin if current_state is a goal then return SUCCESS; add current_state to closed; while current_state has unexamined children begin child := next unexamined child; if child not member of closed then if depthsearch(child) = SUCCESS then return SUCCESS end; return FAIL % search exhausted end"
"Rather than generating all children of a state and placing them on an open list, this algorithm produces the child states one at a time and recursively searches the descendants of each child before generating its sibling. Note that the algorithm assumes an order to the state generation operators. In recursively searching a child state, if some descendant of that state is a goal, the recursive call returns success and the algorithm ignores the siblings."
"If the recursive call on the child state fails to find a goal, the next sibling is generated and all of its descendants are searched. In this fashion, the algorithm searches the entire graph in a depth-first order. The reader should verify that it actually searches the graph in the same order as the depth-first search algorithm of Section 3.2.3."
"The omission of an explicit open list is made possible through recursion. The mechanisms by which a programming language implements recursion include a separate activation record (Aho and Ullman 1977) for each recursive call. Each activation record captures the local variables and state of execution of each procedure call. When the procedure is called recursively with a new state, a new activation record stores its parameters (the state), any local variables, and the current state of execution. In a recursive search algorithm, the series of states on the current path are recorded in the sequence of activation records of the recursive calls. The record of each call also indicates the last operation used to generate a child state; this allows the next sibling to be generated when needed."
"Backtracking is effected when all descendants of a state fail to include a goal, causing the recursive call to fail. This returns fail to the procedure expanding the parent state, which then generates and recurs on the next sibling. In this situation, the internal mechanisms of recursion do the work of the open list used in the iterative version of the algorithm. The recursive implementation allows the programmer to restrict his or her point of view to a single state and its children rather than having to explicitly maintain an open list of states. The ability of recursion to express global concepts in a closed form is a major source of its power."
"As these two algorithms demonstrate, state space search is an inherently recursive process. To find a path from a current state to a goal, move to a child state and recur. If that child state does not lead to a goal, try its siblings in order. Recursion breaks a large and difficult problem (searching the whole space) into smaller, simpler pieces (generate the children of a single state) and applies this strategy (recursively) to each of them. This process continues until a goal state is discovered or the space is exhausted."
"In the next section, this recursive approach to problem solving is extended into a controller for a logic-based problem solver that uses unification and inference to generate and search a space of logical relations. The algorithm supports the and of multiple goals as well as back chaining from a goal to premises."

2.2 A Recursive Search Example: Pattern-Driven Reasoning (Section 6.1.2)

"In Section 6.1.2 we apply recursive search to a space of logical inferences; the result is a general search procedure for predicate calculus based problem specifications."
"Suppose we want to write an algorithm that determines whether a predicate calculus expression is a logical consequence of some set of assertions. This suggests a goal-directed search with the initial query forming the goal, and modus ponens defining the transitions between states. Given a goal (such as p(a)), the algorithm uses unification to select the implications whose conclusions match the goal (e.g., q(X) → p(X)). Because the algorithm treats implications as potential rules for solving the query, they are often simply called rules. After unifying the goal with the conclusion of the implication (or rule) and applying the resulting substitutions throughout the rule, the rule premise becomes a new goal (q(a)). This is called a subgoal. The algorithm then recurs on the subgoal. If a subgoal matches a fact in the knowledge base, search terminates. The series of inferences that led from the initial goal to the given facts prove the truth of the original goal."
The pattern_search algorithm:
"function pattern_search (current_goal); begin if current_goal is a member of closed % test for loops then return FAIL else add current_goal to closed; while there remain in data base unifying facts or rules do begin case current_goal unifies with a fact: return SUCCESS; current_goal is a conjunction (p ∧ ...): begin for each conjunct do call pattern_search on conjunct; if pattern_search succeeds for all conjuncts then return SUCCESS else return FAIL end; current_goal unifies with rule conclusion (p in q → p): begin apply goal unifying substitutions to premise (q); call pattern_search on premise; if pattern_search succeeds then return SUCCESS else return FAIL end; end; % end case end; return FAIL end."
"In the function pattern_search, search is performed by a modified version of the recursive search algorithm that uses unification, Section 2.3.2, to determine when two expressions match and modus ponens to generate the children of states. The current focus of the search is represented by the variable current_goal. If current_goal matches with a fact, the algorithm returns success. Otherwise the algorithm attempts to match current_goal with the conclusion of some rule, recursively attempting to solve the premise. If current_goal does not match any of the given assertions, the algorithm returns fail. This algorithm also handles conjunctive goals."
"For simplicity, the algorithm does not address the problem of maintaining consistency among the variable substitutions produced by unification. This is important when solving conjunctive queries with shared variables (as in p(X) ∧ q(X)). Both conjuncts succeed, but they must also succeed with the same unifiable binding for X, see Section 2.3.2."
"The major advantage of using general methods such as unification and modus ponens to generate states is that the resulting algorithm may search any space of logical inferences where the specifics of a problem are described using predicate calculus assertions. Thus, we have a means of separating problem-solving knowledge from its control and implementation on the computer. pattern_search provides our first example of the separation of problem knowledge from control of search."
"Although the initial version of pattern_search defined the behavior of a search algorithm for predicate calculus expressions, several subtleties must still be addressed. These include the order with which the algorithm tries alternative matches and proper handling of the full set of logical operators (∧, ∨, and ¬). Logic is declarative, and without a prescribed search strategy: it defines a space of possible inferences but does not tell a problem solver how to make the useful ones."

“To reason with predicate calculus, we need a control regime that systematically searches the space, avoiding meaningless paths and loops. A control algorithm such as pattern_search must try alternative matches in some sequential order. Knowing this order allows the program designer to control search by properly ordering rules in the knowledge base. A simple way to define such an order is to require that the algorithm try rules and facts in the order that they appear in the knowledge base.”

Handling logical connectives:

“A second issue is the existence of logical connectives in the rule premises: e.g., implications of the form ‘p ← q ∧ r’ or ‘p ← q ∨ (r ∧ s).’ As will be recalled from the discussion of and/or graphs, an ∧ operator indicates that both expressions must be shown to be true for the entire premise to be true. In addition, the conjuncts of the expression must be solved with consistent variable bindings. Thus, to solve p(X) ∧ q(X), it is not sufficient to solve p(X) with the substitution {a/X} and q(X) with the substitution {b/X}. Both must be solved with the same unifiable binding for X. An or operator, on the other hand, indicates that either expression must be found to be true. The search algorithm must take this into account.”

“The last addition to the algorithm is the ability to solve goals involving logical negation (¬). pattern_search handles negated goals by first solving the operand of the ¬. If this subgoal succeeds, then pattern_search returns fail. If the operand fails, then pattern_search returns an empty substitution set, indicating success. Note that even though a subgoal may contain variables, the result of solving its negation may not contain any substitutions. This is because ¬ can succeed only if its operand fails; hence, it cannot return any bindings for the operand.”

The complete pattern_search:

“Finally, the algorithm should not return success but should return the bindings involved in the solution. The complete version of pattern_search, which returns the set of unifications that satisfies each subgoal, is:”

“function pattern_search(current_goal); begin if current_goal is a member of closed % test for loops then return FAIL else add current_goal to closed; while there remain unifying facts or rules do begin case current_goal unifies with a fact: return unifying substitutions; current_goal is negated (¬ p): begin call pattern_search on p; if pattern_search returns FAIL then return {}; % negation is true else return FAIL; end; current_goal is a conjunction (p ∧ ...): begin for each conjunct do begin call pattern_search on conjunct; if pattern_search returns FAIL then return FAIL; else apply substitutions to other conjuncts; end; if pattern_search returns SUCCESS for all conjuncts then return composition of unifications; else return FAIL; end; current_goal is a disjunction (p ∨ ...): begin repeat for each disjunct call pattern_search on disjunct until no more disjuncts or SUCCESS; if pattern_search returns SUCCESS then return substitutions else return FAIL; end; current_goal unifies with rule conclusion (p in p ← q): begin apply goal unifying substitutions to premise (q); call pattern_search on premise; if pattern_search returns SUCCESS then return composition of p and q substitutions else return FAIL; end; end; %end case end %end while return FAIL end.”

“This pattern_search algorithm for searching a space of predicate calculus rules and facts is the basis of Prolog (where the Horn clause form of predicates is used, Section 14.3) and in many goal-directed expert system shells (Chapter 8). An alternative control structure for pattern-directed search is provided by the production system, discussed in the next section.”

3. PRODUCTION SYSTEMS (Section 6.2)

“The production system is a model of computation that has proved particularly important in AI, both for implementing search algorithms and for modeling human problem solving. A production system provides pattern-directed control of a problem-solving process and consists of a set of production rules, a working memory, and a recognize–act control cycle.”

“DEFINITION: PRODUCTION SYSTEM

A production system is defined by:

The set of production rules. These are often simply called productions. A production is a condition–action pair and defines a single chunk of problem-solving knowledge. The condition part of the rule is a pattern that determines when that rule may be applied to a problem instance. The action part defines the associated problem-solving step(s).

Working memory contains a description of the current state of the world in a reasoning process. This description is a pattern that is matched against the condition part of a production to select appropriate problem-solving actions. When the condition element of a rule is matched by the contents of working memory, the action(s) associated with that condition may then be performed. The actions of production rules are specifically designed to alter the contents of working memory.

The recognize–act cycle. The control structure for a production system is simple: working memory is initialized with the beginning problem description. The current state of the problem-solving is maintained as a set of patterns in working memory. These patterns are matched against the conditions of the production rules; this produces a subset of the production rules, called the conflict set, whose conditions match the patterns in working memory. The productions in the conflict set are said to be enabled. One of the productions in the conflict set is then selected (conflict resolution) and the production is fired. To fire a rule, its action is performed, changing the contents of working memory. After the selected production is fired, the control cycle repeats with the modified working memory. The process terminates when the contents of working memory do not match any rule’s conditions.”

“Conflict resolution chooses a rule from the conflict set for firing. Conflict resolution strategies may be simple, such as selecting the first rule whose condition matches the state of the world, or may involve complex rule selection heuristics. This is an important way in which a production system allows the addition of heuristic control to a search algorithm.”

“The pure production system model has no mechanism for recovering from dead ends in the search; it simply continues until no more productions are enabled and halts. Most practical implementations of production systems allow backtracking to a previous state of working memory in such situations.”

“A schematic drawing of a production system is presented in Figure 6.1.”

The string sorting example (Figure 6.2):

“A very simple example of production system execution appears in Figure 6.2. This is a production system program for sorting a string composed of the letters a, b, and c. In this example, a production is enabled if its condition matches a portion of the string in working memory. When a rule is fired, the substring that matched the rule condition is replaced by the string on the right-hand side of the rule. Production systems are a general model of computation that can be programmed to do anything that can be done on a computer. Their real strength, however, is as an architecture for knowledge-based systems.”

The production set:

Rule
Production
1
ba → ab
2
ca → ac
3
cb → bc

Iteration #
Working memory
Conflict set
Rule fired
0
cbaca
1, 2, 3
—
1
cabca
2
2
2
acbca
2, 3
2
3
acbac
1, 3
1
4
acabc
2
2
5
aacbc
1
1
6
aabcc
—
Halt

The history:

“The idea for the production-based design for computing came originally from writings of Post (1943), who proposed a production rule model as a formal theory of computation. The main construct of this theory was a set of rewrite rules for strings in many ways similar to the parsing rules in Example 3.3.6. It is also closely related to the approach taken by Markov algorithms (Markov 1954) and, like them, is equivalent in power to a Turing machine.”

“An interesting application of production rules to modeling human cognition is found in the work of Newell and Simon at the Carnegie Institute of Technology (now Carnegie Mellon University) in the 1960s and 1970s. The programs they developed, including the General Problem Solver, are largely responsible for the importance of production systems in AI. In this research, human subjects were monitored in various problem-solving activities such as solving problems in predicate logic and playing games like chess. The protocol (behavior patterns, including verbal descriptions of the problem-solving process, eye movements, etc.) of problem-solving subjects was recorded and broken down to its elementary components. These components were regarded as the basic bits of problem-solving knowledge in the human subjects and were composed as a search through a graph (called the problem behavior graph). A production system was then used to implement search of this graph.”

“The production rules represented the set of problem-solving skills of the human subject. The present focus of attention was represented as the current state of the world. In executing the production system, the ‘attention’ or ‘current focus’ of the problem solver would match a production rule, which would change the state of ‘attention’ to match another production-encoded skill, and so on.”

“It is important to note that in this work Newell and Simon used the production system not only as a vehicle for implementing graph search but also as an actual model of human problem-solving behavior. The productions corresponded to the problem-solving skills in the human’s long-term memory. Like the skills in long-term memory, these productions are not changed by the execution of the system; they are invoked by the ‘pattern’ of a particular problem instance, and new skills may be added without requiring ‘recoding’ of the previously existing knowledge. The production system’s working memory corresponds to short-term memory or current focus of attention in the human and describes the current stage of solving a problem instance. The contents of working memory are generally not retained after a problem has been solved.”

“These origins of the production system technology are further described in Human Problem Solving by Newell and Simon (1972) and in Luger (1978, 1994). Newell, Simon, and others have continued to use production rules to model the difference between novices and experts (Larkin et al. 1980; Simon and Simon 1978) in areas such as solving algebra word problems and physics problems. Production systems also form a basis for studying learning in both humans and computers (Klahr et

al. 1987); ACT* (Anderson 1983b) and SOAR (Newell 1990) build on this tradition."

"Production systems provide a model for encoding human expertise in the form of rules and designing pattern-driven search algorithms, tasks that are central to the design of the rule-based expert system. In expert systems, the production system is not necessarily assumed to actually model human problem-solving behavior; however, the aspects of production systems that make them useful as a potential model of human problem solving (modularity of rules, separation of knowledge and control, separation of working memory and problem-solving knowledge) make them an ideal tool for designing and building expert systems as we see in Sections 8.1 and 8.2."

"An important family of AI languages comes directly out of the production system language research at Carnegie Mellon. These are the OPS languages; OPS stands for Official Production System. Although their origins are in modeling human problem solving, these languages have proved highly effective for programming expert systems and for other AI applications. OPS5 was the implementation language for the VAX configurer XCON and other early expert systems developed at Digital Equipment Corporation (McDermott 1981, 1982; Soloway et al. 1987; Barker and O'Connor 1989). OPS interpreters are widely available for PCs and workstations. CLIPS, implemented in the C programming language, is a widely used, object-oriented version of a production system built by NASA. JESS, a production system implemented in Java, was created by Sandia National Laboratories."

Example 6.2.1: The 8-puzzle, Revisited:

"The search space generated by the 8-puzzle, introduced in Chapter 3, is both complex enough to be interesting and small enough to be tractable, so it is frequently used to explore different search strategies, such as depth-first and breadth-first search, as well as the heuristic strategies of Chapter 4. We now present a production system solution."

"Recall that we gain generality by thinking of 'moving the blank space' rather than moving a numbered tile. Legal moves are defined by the productions in Figure 6.3. Of course, all four of these moves are applicable only when the blank is in the center; when it is in one of the corners only two moves are possible. If a beginning state and a goal state for the 8-puzzle are now specified, it is possible to make a production system accounting of the problem's search space."

The production set:

#
Condition
Action
1
goal state in working memory
halt
2
blank is not on the left edge
move the blank left
3
blank is not on the top edge
move the blank up
4
blank is not on the right edge
move the blank right
5
blank is not on the bottom edge
move the blank down

"1. Try each production in order. 2. Do not allow loops. 3. Stop when goal is found."

"An example, taken from Nilsson (1980), of the space searched in finding a solution for the problem given in Figure 6.3 follows in Figure 6.4. Because this solution path can go very deep if unconstrained, a depth bound has been added to the search. (A simple means for adding a depth bound is to keep track of the length/depth of the current path and to force backtracking if this bound is exceeded.) A depth bound of 5 is used in the solution of Figure 6.4. Note that the number of possible states of working memory grows exponentially with the depth of the search."

Example 6.2.2: The Knight's Tour Problem:

"In the game of chess, a knight can move two squares either horizontally or vertically followed by one square in an orthogonal direction as long as it does not move off the board. There are thus at most eight possible moves that the knight may make (Figure 6.5)."

"As traditionally defined, the knight's tour problem attempts to find a series of legal moves in which the knight lands on each square of the chessboard exactly once. This problem has been a mainstay in the development and presentation of search algorithms. The example we use in this chapter is a simplified version of the knight's tour problem. It asks whether there is a series of legal moves that will take the knight from one square to another on a reduced-size (3 × 3) chessboard."

"Figure 6.6 shows a 3 × 3 chessboard with each square labeled with integers 1 to 9. This labeling scheme is used instead of the more general approach of giving each space a row and column number in order to further simplify the example. Because of the reduced size of the problem, we simply enumerate the alternative moves rather than develop a general move operator. The legal moves on the board are then described in predicate calculus using a predicate called move, whose parameters are the starting and ending squares of a legal move. For example, move(1,8) takes the knight from the upper left-hand corner to the middle of the bottom row. The predicates of Figure 6.6 enumerate all possible moves for the 3 × 3 chessboard."

The move predicates:

"move(1,8) move(6,1) move(1,6) move(6,7) move(2,9) move(7,2) move(2,7) move(7,6) move(3,4) move(8,3) move(3,8) move(8,1) move(4,9) move(9,2) move(4,3) move(9,4)"

The sixteen production rules:

Rule #
Condition
Action
1
knight on square 1
move knight to square 8
2
knight on square 1
move knight to square 6
3
knight on square 2
move knight to square 9
4
knight on square 2
move knight to square 7
5
knight on square 3
move knight to square 4
6
knight on square 3
move knight to square 8
7
knight on square 4
move knight to square 9
8
knight on square 4
move knight to square 3
9
knight on square 6
move knight to square 1
10
knight on square 6
move knight to square 7
11
knight on square 7
move knight to square 2
12
knight on square 7
move knight to square 6
13
knight on square 8
move knight to square 3
14
knight on square 8
move knight to square 1
15
knight on square 9
move knight to square 2
16
knight on square 9
move knight to square 4

The recursive path definition:

"We next specify a recursive procedure to implement a control algorithm for the production system. Because path(X,X) will unify only with predicates such as path(3,3) or path(5,5), it defines the desired terminating condition. If path(X,X) does not succeed we look at the production rules for a possible next state and then recur. The general recursive path definition is then given by two predicate calculus formulas:

∀X path(X,X) ∀X,Y [path(X,Y) ← ∃ Z [move(X,Z) ∧ path(Z,Y)"

The trace (Figure 6.7):

Iteration #
Working memory Current square
Goal square
Conflict set (rule #s)
Fire rule
0
1
2
1, 2
1
1
8
2
13, 14
13
2
3
2
5, 6
5
3
4
2
7, 8
7
4
9
2
15, 16
15
5
2
2
—
Halt

"This characterization of the path definition as a production system is given in Figure 6.8."

The loop problem:

"Production systems are capable of generating infinite loops when searching a state space graph. These loops are particularly difficult to spot in a production system because the rules can fire in any order. That is, looping may appear in the execution of the system, but it cannot easily be found from a syntactic inspection of the rule set. For example, with the 'move' rules of the knight's tour problem ordered as in Table 6.1 and a conflict resolution strategy of selecting the first match, the pattern move(2,X) would match with move(2,9), indicating a move to square 9. On the next iteration, the pattern move(9,X) would match with move(9,2), taking the search back to square 2, causing a loop."

"To prevent looping, pattern_search checked a global list (closed) of visited states. The actual conflict resolution strategy was therefore: select the first matching move that leads to an unvisited state. In a production system, the proper place for recording such case-specific data as a list of previously visited states is not a global closed list but the working memory itself. We can alter the path predicate to use working memory for loop detection. Assume that our predicate calculus language is augmented by the addition of a special construct, assert(X), which causes its argument X to be entered into the working memory. assert is not an ordinary predicate but an action that is performed; hence, it always succeeds."

"assert is used to place a 'marker' in working memory to indicate when a state has been visited. This marker is represented as a unary predicate, been(X), which takes as its argument a square on the board. been(X) is added to working memory when a new state X is visited. Conflict resolution may then require that been(Z) must not be in working memory before move(X,Z) can fire. For a specific value of Z this can be tested by matching a pattern against working memory."

"The modified recursive path controller for the production system is:

∀X path(X,X) ∀X,Y [path(X,Y) ← ∃ Z [move(X,Z) ∧ ¬ (been(Z)) ∧ assert(been(Z)) ∧ path(Z,Y)]]"

"In this definition, move(X,Z) succeeds on the first match with a move predicate. This binds a value to Z. If been(Z) matches with an entry in working memory, ¬(been (Z)) will cause a failure (i.e., it will be false). pattern_search will then backtrack and try another match for move(X,Z). If square Z is a new state, the search will continue, with been(Z) asserted to the working memory to prevent future loops. The actual firing of the production takes place when the path algorithm recurs. Thus, the presence of been predicates in working memory implements loop detection in this production system."

Example 6.2.3: The Full Knight's Tour:

"We may generalize the knight's tour solution to the full 8 × 8 chessboard. Because it makes little sense to enumerate moves for such a complex problem, we replace the 16 move facts with a set of 8 rules to generate legal knight moves. These moves (productions) correspond to the 8 possible ways a knight can move (Figure 6.5)."

"If we index the chessboard by row and column numbers, we can define a production rule for moving the knight down two squares and right one square:

CONDITION: current row < 6 ∧ current column < 7 ACTION: new row = current row + 2 ∧ new column = current column + 1"

"If we use predicate calculus to represent productions, then a board square could be defined by the predicate square(R,C), representing the Rth row and Cth column of the board. The above rule could be rewritten in predicate calculus as:

move(square(Row, Column), square(Newrow, Newcolumn)) ← less_than_or_equals(Row, 6) ∧ equals(Newrow, plus(Row, 2)) ∧ less_than_or_equals(Column, 7) ∧ equals(Newcolumn, plus(Column, 1))"

"plus is a function for addition; less_than_or_equals and equals have the obvious arithmetic interpretations. Seven additional rules can be designed that compute the remaining possible moves. These rules replace the move facts in the 3 × 3 version of the problem."

"The path definition from the 3 × 3 knight's example offers a control loop for this problem as well. As we have seen, when predicate calculus descriptions are interpreted procedurally, such as through the pattern_search algorithm, subtle changes are made to the semantics of predicate calculus. One such change is the sequential fashion in which goals are solved. This imposes an ordering, or procedural semantics, on the interpretation of predicate calculus expressions. Another change is the introduction of meta-logical predicates such as assert, which indicate actions beyond the truth value interpretation of predicate calculus expressions."

Example 6.2.4: The Financial Advisor as a Production System:

"In Chapters 2 and 3, we developed a small financial advisor, using predicate calculus to represent the financial knowledge and graph search to make the appropriate inferences in a consultation. The production system provides a natural vehicle for its implementation. The implications of the logical description form the productions. The case-specific information (an individual's salary, dependents, etc.) is loaded into working memory. Rules are enabled when their premises are satisfied. A rule is chosen from this conflict set and fired, adding its conclusion to working memory. This continues until all possible top-level conclusions have been added to the working memory. Indeed, many expert system 'shells', including JESS and CLIPS, are production systems with added features for supporting the user interface, handling uncertainty in the reasoning, editing the knowledge base, and tracing execution."

3.3 Control of Search in Production Systems (Section 6.2.3)

Control through choice of data-driven or goal-driven search strategy:

"Data-driven search begins with a problem description (such as a set of logical axioms, symptoms of an illness, or a body of data that needs interpretation) and infers new knowledge from the data. This is done by applying rules of inference, legal moves in a game, or other state-generating operations to the current description of the world and adding the results to that problem description. This process continues until a goal state is reached."

"This description of data-driven reasoning emphasizes its close fit with the production system model of computation. The 'current state of the world' (data that have been either assumed to be true or deduced as true with previous use of production rules) is placed in working memory. The recognize–act cycle then matches the current state against the (ordered) set of productions. When these data match (are unified with) the condition(s) of one of the production rules, the action of the production adds (by modifying working memory) a new piece of information to the current state of the world."

"All productions have the form CONDITION → ACTION. When the CONDITION matches some elements of working memory, its ACTION is performed. If the production rules are formulated as logical implications and the ACTION adds assertions to working memory, then the act of firing a rule can correspond to an application of the inference rule modus ponens. This creates a new state of the graph."

The data-driven example (Figure 6.9):

Rule
Production
1
p ∧ q → goal
2
r ∧ s → p
3
w ∧ r → q
4
t ∧ u → q
5
v → s
6
start → v ∧ r ∧ q

Iteration #
Working memory
Conflict set
Rule fired
0
start
6
—
1
start, v, r, q
6, 5
6
2
start, v, r, q, s
6, 5, 2
5
3
start, v, r, q, s, p
6, 5, 2, 1
2
4
start, v, r, q, s, p, goal
6, 5, 2, 1
1

halt

"To this point we have treated production systems in a data-driven fashion; however they may also be used to produce a goal-driven search. As defined in Chapter 3, goal-driven search begins with a goal and works backward to the facts of the problem to satisfy that goal. To implement this in a production system, the goal is placed in working memory and matched against the ACTIONs of the production rules. These ACTIONs are matched (by unification, for example) just as the CONDITIONs of the productions were matched in the data-driven reasoning. All production rules whose conclusions (ACTIONs) match the goal form the conflict set."

"When the ACTION of a rule is matched, the CONDITIONs are added to working memory and become the new subgoals (states) of the search. The new states are then matched to the ACTIONs of other production rules. The process continues until facts are found, usually in the problem's initial description or, as is often the case in expert systems, by directly asking the user for specific information. The search stops when CONDITIONs supporting the productions fired in this backward fashion that link to a goal are found to be true. The CONDITIONs and the chain of rule firings leading to the goal form a proof of its truth through successive inferences such as modus ponens. See Figure 6.10 for an instance of goal-driven reasoning on the same set of productions used in Figure 6.9. Note that the goal-driven search fires a different series of productions and searches a different space than the data-driven version."

The goal-driven example (Figure 6.10):

Iteration #
Working memory
Conflict set
Rule fired
0
goal
1
—
1

goal, p, q
1, 2, 3, 4
1
2
goal, p, q, r, s
1, 2, 3, 4, 5
2
3
goal, p, q, r, s, w
1, 2, 3, 4, 5
3
4
goal, p, q, r, s, w, t, u
1, 2, 3, 4, 5
4
5
goal, p, q, r, s, w, t, u, v
1, 2, 3, 4, 5
5
6
goal, p, q, r, s, w, t, u, v, start
1, 2, 3, 4, 5, 6
6

halt

"As this discussion illustrates, the production system offers a natural characterization of both goal-driven and data-driven search. The production rules are the encoded set of inferences (the 'knowledge' in a rule-based expert system) for changing state within the graph. When the current state of the world (the set of true statements describing the world) matches the CONDITIONs of the production rules and this match causes the ACTION part of the rule to create another (true) descriptor for the world, it is referred to as data-driven search."

"Alternatively, when the goal is matched against the ACTION part of the rules in the production rule set and their CONDITIONs are then set up as subgoals to be shown to be 'true' (by matching the ACTIONs of the rules on the next cycle of the production system), the result is goal-driven problem solving."

"Because a set of rules may be executed in either a data-driven or goal-driven fashion, we can compare and contrast the efficiency of each approach in controlling search. The complexity of search for either strategy is measured by such notions as branching factor or penetrance (Section 4.5). These measures of search complexity can provide a cost estimate for both the data-driven and goal-driven versions of a problem solver and therefore help in selecting the most effective strategy."

"We can also employ combinations of strategies. For example, we can search in a forward direction until the number of states becomes large and then switch to a goal directed search to use possible subgoals to select among alternative states. The danger in this situation is that, when heuristic or best-first search (Chapter 4) is used, the parts of the graphs actually searched may 'miss' each other and ultimately require more search than a simpler approach, as in Figure 6.11. However, when the branching of a space is constant and exhaustive search is used, a combined search strategy can cut back drastically the amount of space searched, as is seen in Figure 6.12."

Control of search through rule structure:

"The structure of rules in a production system, including the distinction between the condition and the action and the order in which conditions are tried, determines the fashion in which the space is searched. In introducing predicate calculus as a representation language, we emphasized the declarative nature of its semantics. That is, predicate calculus expressions simply define true relationships in a problem domain and make no assertion about their order of interpretation. Thus, an individual rule might be ∀X (foo(X) ∧ goo(X) → moo(X)). Under the rules of predicate calculus, an alternative form of the same rule is ∀X (foo(X) → moo(X) ∨ ¬ goo(X)). The equivalence of these two clauses is left as an exercise (Chapter 2)."

"Although these formulations are logically equivalent, they do not lead to the same results when interpreted as productions because the production system imposes an order on the matching and firing of rules. Thus, the specific form of the rules determines the ease (or possibility) of matching a rule against a problem instance. This is a result of differences in the way in which the production system interprets the rules. The production system imposes a procedural semantics on the declarative language used to form the rules."

"Because the production system tries each of its rules in a specific order, the programmer may control search through the structure and order of rules in the production set. Although logically equivalent, ∀X (foo(X) ∧ goo(X) → moo(X)) and ∀X (foo(X) → moo(X) ∨ ¬ goo(X)) do not have the same behavior in a search implementation."

"Human experts encode crucial heuristics within their rules of expertise. The order of premises encodes often critical procedural information for solving the problem. It is important that this form be preserved in building a program that 'solves problems like the expert'. When a mechanic says, 'If the engine won't turn over and the lights don't come on, then check the battery', she is suggesting a specific sequence of actions. This information is not captured by the logically equivalent statement 'the engine turns over or the lights come on or check the battery.' This form of the rules is critical in controlling search, making the system behave logically and making order of rule firings more understandable."

Control of search through conflict resolution:

"Though production systems (like all architectures for knowledge-based systems) allow heuristics to be encoded in the knowledge content of rules themselves, they offer other opportunities for heuristic control through conflict resolution. Although the simplest such strategy is to choose the first rule that matches the contents of working memory, any strategy may potentially be applied to conflict resolution. For example, conflict resolution strategies supported by OPS5 (Brownston et al. 1985) include:

Refraction. Refraction specifies that once a rule has fired, it may not fire again until the working memory elements that match its conditions have been modified. This discourages looping.

Recency. The recency strategy prefers rules whose conditions match with the patterns most recently added to working memory. This focuses the search on a single line of reasoning.

Specificity. This strategy assumes that it is appropriate to use a more specific problem-solving rule rather than to use a more general one. One rule is more specific than another if it has more conditions, which implies that it will match fewer working memory patterns."

3.4 Advantages of Production Systems for AI (Section 6.2.4)

"As illustrated by the preceding examples, the production system offers a general framework for implementing search. Because of its simplicity, modifiability, and flexibility in applying problem-solving knowledge, the production system has proved to be an important tool for the construction of expert systems and other AI applications. The major advantages of production systems for artificial intelligence include:"

"Separation of Knowledge and Control. The production system is an elegant model of separation of knowledge and control in a computer program. Control is provided by the recognize–act cycle of the production system loop, and the problem-solving knowledge is encoded in the rules themselves. The advantages of this separation include ease of modifying the knowledge base without requiring a change in the code for program control and, conversely, the ability to alter the code for program control without changing the set of production rules."

"A Natural Mapping onto State Space Search. The components of a production system map naturally into the constructs of state space search. The successive states of working memory form the nodes of a state space graph. The production rules are the set of possible transitions between states, with conflict resolution implementing the selection of a branch in the state space. These rules simplify the implementation, debugging, and documentation of search algorithms."

"Modularity of Production Rules. An important aspect of the production system model is the lack of any syntactic interactions between production rules. Rules may only effect the firing of other rules by changing the pattern in working memory; they may not 'call' another rule directly as if it were a subroutine, nor may they set the value of variables in other production rules. The scope of the variables of these rules is confined to the individual rule. This syntactic independence supports the incremental development of expert systems by successively adding, deleting, or changing the knowledge (rules) of the system."

"Pattern-Directed Control. The problems addressed by AI programs often require particular flexibility in program execution. This goal is served by the fact that the rules in a production system may fire in any sequence. The descriptions of a problem that make up the current state of the world determine the conflict set and, consequently, the particular search path and solution."

"Opportunities for Heuristic Control of Search. (Several techniques for heuristic control were described in the preceding section.)"

"Tracing and Explanation. The modularity of rules and the iterative nature of their execution make it easier to trace execution of a production system. At each stage of the recognize–act cycle, the selected rule can be displayed. Because each rule corresponds to a single 'chunk' of problem-solving knowledge, the rule content can provide a meaningful explanation of the system's current state and action. Furthermore, the chain of rules used within a solution process reflects both a path in the graph as well as a human expert's 'line of reasoning', as we see in detail in Chapter 8. In contrast, a single line of code or procedure in a traditional applicative language such as Pascal, FORTRAN, or Java is virtually meaningless."

"Language Independence. The production system control model is independent of the representation chosen for rules and working memory, as long as that representation supports pattern matching. We described production rules as predicate calculus implications of the form A ⇒ B, where the truth of A and the inference rule modus ponens allow us to conclude B. Although there are many advantages to using logic as both the basis for representation of knowledge and the source of sound inference rules, the production system model may be used with other representations, e.g., CLIPS and JESS."

"Although predicate calculus offers the advantage of logically sound inference, many problems require reasoning that is not sound in the logical sense. Instead, they involve probabilistic reasoning, use of uncertain evidence, and default assumptions. Later chapters (7, 8, and 9) discuss alternative inference rules that provide these capabilities. Regardless of the type of inference rules employed, however, the production system provides a vehicle for searching the state space."

"A Plausible Model of Human Problem Solving. Modeling human problem solving was among the first uses of production systems, see Newell and Simon (1972). They continue to be used as a model for human performance in many areas of cognitive science research (Chapter 16)."

"Pattern-directed search gives us the ability to explore the space of logical inferences in the predicate calculus. Many problems build on this technique by using predicate calculus to model specific aspects of the world such as time and change. In the next section blackboard systems are presented as a variation of the production system methodology, where task specific groups of production rules are combined into knowledge sources and cooperate in problem solving by communication through a global working memory or blackboard."

4. THE BLACKBOARD ARCHITECTURE FOR PROBLEM SOLVING (Section 6.3)

"The blackboard is the final control mechanism presented in this chapter. When we want to examine the states in a space of inferences in a very deterministic fashion, production systems provide great flexibility by allowing us to represent multiple partial solutions simultaneously in working memory and to select the next state through conflict resolution. Blackboards extend production systems by allowing us to organize production memory into separate modules, each of which corresponds to a different subset of the production rules. Blackboards integrate these separate sets of production rules and coordinate the actions of these multiple problem solving agents, sometimes called knowledge sources, within a single global structure, the blackboard."

"Many problems require the coordination of a number of different types of agents. For example, a speech understanding program may have to first manipulate an utterance represented as a digitized waveform. As the decoding process continues, it must find words in this utterance, form these into phrases and sentences, and finally produce a semantic representation of the utterance's meaning."

"A related problem occurs when multiple processes must cooperate to solve a single problem. An example of this is the sensor fusion problem (Lesser and Corkill 1983). Assume that we have a network of sensors, each of which is monitored by a separate process. Assume also that the processes can communicate and that proper interpretation of each sensor's data depends on the data received by other sensors in the network. This problem arises in situations as diverse as tracking airplanes across multiple radar sites to combining the readings of multiple sensors in a manufacturing process."

"The blackboard architecture is a model of control that has been applied to these and other problems requiring the coordination of multiple processes or knowledge sources. A blackboard is a central global data base for the communication of independent asynchronous knowledge sources focusing on related aspects of a particular problem. Figure 6.13 gives a schematic of the blackboard design."

"In Figure 6.13 each knowledge source KS_i gets its data from the blackboard, processes the data, and returns its results to the blackboard to be used by the other knowledge sources. Each KS_i is independent in that it is a separate process operating according to its own specifications and, when a multiprocessing or multiprocessor system is used, it is independent of the other processing in the problem. It is an asynchronous system in that each KS_i begins its operation whenever it finds appropriate input data posted on the blackboard. When it finishes its processing it posts its results to the blackboard and awaits new appropriate input data."

"The blackboard approach to organizing a large program was first presented in the HEARSAY-II research (Erman et al. 1980, Reddy 1976). HEARSAY-II was a speech understanding program; it was initially designed as the front end for a library database of computer science articles. The user of the library would address the computer in spoken English with queries such as, 'Are any by Feigenbaum and Feldman?' and the computer would answer the question with information from the library database. Speech understanding requires that we integrate a number of different processes, all of which require very different knowledge and algorithms, all of which can be exponentially complex. Signal processing; recognition of phonemes, syllables, and words; syntactic parsing; and semantic analysis mutually constrain each other in interpreting speech."

"The blackboard architecture allowed HEARSAY-II to coordinate the several different knowledge sources required for this complex task. The blackboard is usually organized along two dimensions. With HEARSAY-II these dimensions were time as the speech act was produced and the analysis level of the utterance. Each level of analysis was processed by a different class of knowledge sources. These analysis levels were:"

"KS1 The waveform of the acoustic signal. KS2 The phonemes or possible sound segments of the acoustic signal. KS3 The syllables that the phonemes could produce. KS4 The possible words as analyzed by one KS. KS5 The possible words as analyzed by a second KS (usually considering words from different parts of the data). KS6 A KS to try to generate possible word sequences. KS7 A KS that puts word sequences into possible phrases."

"We can visualize these processes as components of Figure 6.13. In processing spoken speech, the waveform of the spoken signal is entered at the lowest level. Knowledge sources for processing this entry are enabled and post their interpretations to the blackboard, to be picked up by the appropriate process. Because of the ambiguities of spoken language, multiple competing hypotheses may be present at each level of the blackboard. The knowledge sources at the higher levels attempt to disambiguate these competing hypotheses."

"The analysis of HEARSAY-II should not be seen as simply one lower level producing data that the higher levels can then analyze. It is much more complex than that. If a KS at one level cannot process (make sense of) the data sent to it, that KS can request the KS that sent it the data to go back for another try or to make another hypothesis about the data. Furthermore, different KSs can be working on different parts of the utterance at the same time. All the processes, as mentioned previously, are asynchronous and data-driven; they act when they have input data, continue acting until they have finished their task, and then post their results and wait for their next task."

"One of the KSs, called the scheduler, handles the 'consume-data post-result' communication between the KSs. This scheduler has ratings on the results of each KS's activity and is able to supply, by means of a priority queue, some direction in the problem solving. If no KS is active, the scheduler determines that the task is finished and shuts down."

"When the HEARSAY program had a database of about 1,000 words it worked quite well, although a bit slowly. When the database was further extended, the data for the knowledge sources got more complex than they could handle. HEARSAY-III (Balzer et al. 1980, Erman et al. 1981) is a generalization of the approach taken by HEARSAY-II. The time dimension of HEARSAY-II is no longer needed, but the multiple KSs for levels of analysis are retained. The blackboard for HEARSAY-III is intended to interact with a general-purpose relational database system. Indeed, HEARSAY-III is a shell for the design of expert systems; see Section 8.1.1."

"An important change in HEARSAY-III has been to split off the scheduler KS (as described above for HEARSAY-II) and to make it a separate blackboard controller for the first (or problem domain) blackboard. This second blackboard allows the scheduling process to be broken down, just as the domain of the problem is broken down, into separate KSs concerned with different aspects of the solution procedure (for example, when and how to apply the domain knowledge). The second blackboard can thus compare and balance different solutions for each problem (Nii and Aiello 1979, Nii 1986a, 1986b). An alternative model of the blackboard retains important parts of the knowledge base in the blackboard, rather than distributing them across knowledge sources (Skinner and Luger 1991, 1992)."

5. EPILOGUE AND REFERENCES (Section 6.4)

"Chapter 6 discussed the implementation of the search strategies of Chapters 3 and 4. Thus, the references listed in the epilogue to those chapters are also appropriate to this chapter."

"Section 6.1 presented recursion as an important tool for programming graph search, implementing the depth-first and backtrack algorithms of Chapter 3 in recursive form. Pattern-directed search with unification and inference rules, as in Chapter 2, simplifies the implementation of search through a space of logical inferences."

"In Section 6.2 the production system was shown as a natural architecture for modeling problem solving and implementing search algorithms. The section concluded with examples of production system implementations of data-driven and goal-driven search. In fact, the production system has always been an important paradigm for AI programming, beginning with work by Newell and Simon and their colleagues at Carnegie Mellon University (Newell and Simon 1976, Klahr et al. 1987, Neches et al. 1987, Newell et al. 1989). The production system has also been an important architecture supporting research in cognitive science (Newell and Simon 1972, Luger 1994; see also Chapter 16)."

Luger's recommended references:

"References on building production systems, especially the OPS languages, include Programming Expert Systems in OPS5 by Lee Brownston et al. (1985), and Pattern Directed Inference Systems by Donald Waterman and Frederick Hayes-Roth (1978). For modern production systems in C and Java, go to the web sites for CLIPS and JESS."

"Early work in blackboard models is described in HEARSAY-II research (Reddy 1976, Erman et al. 1980). Later developments in blackboards is described in the HEARSAY-III work (Lesser and Corkill 1983, Nii 1986a, Nii 1986b), and Blackboard Systems, edited by Robert Engelmore and Tony Morgan (1988)."

"Research in production systems, planning, and blackboard architectures remains an active part of artificial intelligence. We recommend that the interested reader consult recent proceedings of the American Association for Artificial Intelligence Conference and the International Joint Conference on Artificial Intelligence. Morgan Kaufmann and AAAI Press publish conference proceedings, as well as collections of readings on AI topics."

6. EXERCISES (Section 6.5)

The twelve exercises exactly as Luger lists them:

"1. a. Write a member-check algorithm to recursively determine whether a given element is a member of a list. b. Write an algorithm to count the number of elements in a list. c. Write an algorithm to count the number of atoms in a list. (The distinction between atoms and elements is that an element may itself be a list.) 2. Write a recursive algorithm (using open and closed lists) to implement breadth-first search. Does recursion allow the omission of the open list when implementing breadth-first search? Explain. 3. Trace the execution of the recursive depth-first search algorithm (the version that does not use an open list) on the state space of Figure 3.14. 4. In an ancient Hindu tea ceremony, there are three participants: an elder, a servant, and a child. The four tasks they perform are feeding the fire, serving cakes, pouring tea, and reading poetry; this order reflects the decreasing importance of the tasks. At the beginning of the ceremony, the child performs all four tasks. They are passed one at a time to the servant and the elder until, at the end of the ceremony, the elder is performing all four tasks. No one can take on a less important task than those they already perform. Generate a sequence of moves to transfer all the tasks from the child to the elder. Write a recursive algorithm to perform the move sequence. 5. Using the move and path definitions for the knight's tour of Section 6.2.2, trace the execution of pattern_search on the goals: a. path(1,9). b. path(1,5). c. path(7,6). When the move predicates are attempted in order, there is often looping in the search. Discuss loop detection and backtracking in this situation. 6. Write the pseudo-code definition for a breadth-first version of pattern_search (Section 6.1.2). Discuss the time and space efficiency of this algorithm. 7. Using the rule in Example 6.2.3 as a model, write the eight move rules needed for the full 8 × 8 version of the knight's tour. 8. Using the goal and start states of Figure 6.3, hand run the production system solution to the 8-puzzle: a. In goal-driven fashion. b. In data-driven fashion. 9. Consider the financial advisor problem discussed in Chapters 2, 3, and 4. Using predicate calculus as a representation language: a. Write the problem explicitly as a production system. b. Generate the state space and stages of working memory for the data-driven solution to the example in Chapter 3. 10. Repeat problem 9.b to produce a goal-driven solution. 11. Section 6.2.3 presented the general conflict resolution strategies of refraction, recency, and specificity. Propose and justify two more such strategies. 12. Suggest two applications appropriate for solution using the blackboard architecture. Briefly characterize the organization of the blackboard and knowledge sources for each implementation."

Term	Definition
Recursion	A procedure that calls itself, with a terminating condition
Activation record	Per-call record of parameters, local variables, and execution state
pattern_search	Goal-directed search over predicate calculus facts and rules using unification + modus ponens
Subgoal	The premise of a rule, which becomes a new goal to prove
Production system	A set of production rules + working memory + recognize–act cycle
Production rule	Condition–action pair; a single chunk of problem-solving knowledge
Working memory	Contains the current state of the world; matched against rule conditions
Recognize–act cycle	Match → conflict set → conflict resolution → fire → repeat
Conflict set	The subset of rules whose conditions match working memory
Enabled	A rule whose condition matches working memory
Conflict resolution	Choosing which enabled rule to fire
Refraction	Conflict resolution: a rule cannot fire again until its matching conditions are modified
Recency	Conflict resolution: prefer rules matching the most recently added patterns
Specificity	Conflict resolution: prefer rules with more conditions
assert(X)	Meta-logical action that adds X to working memory
been(Z)	Marker predicate in working memory indicating state Z has been visited
Procedural semantics	Imposing an order on the interpretation of declarative expressions
Declarative semantics	Expressions define true relationships without order of interpretation
Blackboard architecture	Central global data structure; independent asynchronous knowledge sources read/write it
Knowledge source (KS)	Independent process that reads from the blackboard, processes, and posts results back
Scheduler	Knowledge source that directs activity in HEARSAY-II via a priority queue
HEARSAY-II	Speech understanding program; origin of the blackboard architecture
HEARSAY-III	Generalization of HEARSAY-II; became an expert system shell
OPS	Official Production System — family of production-system languages
CLIPS	Production system implemented in C, built by NASA
JESS	Production system implemented in Java, created by Sandia National Laboratories
Data-driven search	Forward chaining: facts → conclusions
Goal-driven search	Backward chaining: goal → facts
Bidirectional search	Combining forward and backward search; risk of missing each other (Fig 6.11) or meeting in the middle (Fig 6.12)

7. QUICK-REFERENCE TABLE OF KEY DEFINITIONS

Topic	Belongs to
Expert system architecture (detail)	Chapter 8
Rule-based expert systems (detail)	Chapter 8
Forward/backward chaining in expert systems	Chapter 8
Certainty factors, fuzzy logic	Chapter 9
Bayesian belief networks	Chapter 9
Resolution theorem proving	Chapter 14
Natural language parsing	Chapter 15
Constructivist epistemology	Chapter 16

End of Chapter 6 notes, based strictly on Luger's textbook.

Chapter 7: Knowledge Representation

Complete Study Notes — Based strictly on Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th ed., Chapter 7 and the Introduction to Part III

1. WHY CHAPTER 7 EXISTS

Luger opens the Introduction to Part III with a quote from Marshall McLuhan and Andy Clark:

"Our age of anxiety is, in great part, the result of trying to do today's jobs with yesterday's tools... —MARSHALL MCLUHAN"

"Might it not be more fruitful to think of brains as controllers for embodied activity? That small shift in perspective has large implications for how we construct a science of the mind. It demands, in fact, a sweeping reform in our whole way of thinking about intelligent behavior. It requires us to abandon the ideas (common since Descartes) of the mental as a realm distinct from the realm of the body; to abandon the idea of neat dividing lines between perception, cognition and action; to abandon the idea of an executive center where the brain carries out high-level reasoning; and most of all to abandon research methods that artificially divorce thought from embodied action-taking... —ANDY CLARK, Being There (1997)."

"The question of representation, or how to best capture critical aspects of intelligent activity for use on a computer, or indeed, for communication with humans, has been a constant theme across the more than sixty-year history of AI."

"We begin Part III with a review of the three predominant approaches to representation taken by the AI research community over this time period. The first theme, articulated in the 1950s and 1960s by Newell and Simon in their work with the Logic Theorist (Newell and Simon 1956, 1963a), is known as **weak method problem-solving**. The second theme, common throughout the 1970s and 1980s, and espoused by the early expert system designers, is **strong method problem solving** (see Feigenbaum's quote at the beginning of Chapter 8). In more recent years, especially in the domains of robotics and the internet (Brooks 1987, 1989; Clark 1997), the emphasis is on **distributed and embodied or agent-based approaches** to intelligence. We next introduce each of these methodologies for representing intelligence."

Weak method problem solving:

"In the late 1950s and early 1960s, Alan Newell and Herbert Simon wrote several computer programs to test the hypothesis that intelligent behavior resulted from heuristic search. The Logic Theorist, developed with J. C. Shaw (Newell and Simon 1963a), proved theorems in elementary logic using the notation and axioms of Whitehead and Russell's (1950) Principia Mathematica."

"The authors describe their research as aimed at understanding 'the complex processes (heuristics) that are effective in problem solving. Hence, we are not interested in methods that guarantee solutions, but which require vast amounts of computation. Rather we wish to understand how a mathematician, for example, is able to prove a theorem even though he does not know when he starts how, or if, he is going to succeed.'"

"In a later program, the General Problem Solver or GPS, Newell and Simon (1963b, 1972) continued this effort to find general principles of intelligent problem solving. The GPS solved problems formulated as state space search; legal problem-solving steps were a set of operations for modifying state representations. GPS searched for a sequence of operations that would transform the start state into the goal state, searching in the same fashion as the algorithms discussed in earlier chapters of this book."

"GPS used means–ends analysis, a general heuristic for selecting among alternative state transformation operations, to guide search through the problem space. Means–ends analysis examines the syntactic differences between the current state and the goal state and selects an operator that can reduce these differences. Suppose, for example, that GPS is attempting to prove the equivalence of two logical expressions. If the current state contains an ∧ operator and the goal does not contain an ∧, then means–ends analysis would select a transformation such as de Morgan's law, to remove ∧ from expressions. (For a more detailed description and examples of this research see Section 14.1.)"

"By using a heuristic that examines only the syntactic form of states, it was hoped that GPS would be a general architecture for intelligent problem solving, no matter what the domain. Programs like GPS, which are restricted to syntax-based strategies and intended for a wide variety of applications, are known as weak method problem solvers."

"Unfortunately, there does not seem to be a single heuristic that can successfully be applied to all problem domains. In general, the methods we use to solve problems employ a great deal of knowledge about a situation. Doctors are able to diagnose illness because they have extensive knowledge of medicine, in addition to their general problem-solving abilities. Architects design houses because they know about architecture. Indeed, the heuristics used in medical diagnosis would be useless in designing an office building."

Strong method problem solving:

"Weak methods stand in sharp contrast to strong methods, which use explicit knowledge of a particular problem domain. Consider a rule for diagnostic analysis of automotive problems: if the engine does not turn over, and the lights do not come on then the problem is battery or cables (.8)"

"This heuristic, phrased as an if...then... rule, focuses search on the battery/cable subsystem of the automobile, eliminating other components and pruning the search space. Notice that this particular heuristic, unlike means–ends analysis, uses empirical knowledge of how automobiles function, such as knowledge of the relationships between the battery, lights, and starter. It is useless in any problem domain except vehicle repair."

"Not only do strong method problem solvers use domain-specific knowledge, but they generally require large amounts of such knowledge to be effective. The bad-battery heuristic, for example, would not be of much use in diagnosing a bad carburetor, and would likely be useless in other domains. Thus, a major challenge in designing a knowledge-based program is the acquisition and organization of large amounts of specific domain knowledge. Domain rules can also contain a measure, .8 in the rule above, to reflect the diagnostician's confidence in this piece of domain knowledge."

The knowledge representation hypothesis:

"In using the strong method approach, program designers are making certain assumptions about the nature of intelligent systems. These assumptions are formalized by Brian Smith (1985) as the knowledge representation hypothesis. This hypothesis states that any mechanically embodied intelligent process will be comprised of structural ingredients that (a) we as external observers naturally take to represent a propositional account of the knowledge that the overall process exhibits, and (b) independent of such external semantical attribution, play a formal role in engendering the behavior that manifests that knowledge."

"The important aspects of this hypothesis include the assumption that knowledge will be represented propositionally, that is, in a form that explicitly represents the knowledge in question and that may be seen by an outside observer as a 'natural' description of that knowledge. The second major assumption is that the behavior of the system can be seen as formally caused by the propositions in the knowledge base and that this behavior should be consistent with our perceived meaning of those propositions."

Agent-based problem solving:

"The final theme for AI representation is often described as agent-based, embodied, or emergent problem solving. Several researchers working in applied domains such as robotics, game design, and the internet, including Brooks (1987, 1989), Agre and Chapman (1987), Jennings (1995), Wooldridge (2000), Wooldridge et al. (2006, 2007), Fatima et al. (2004, 2005, 2006), and Vieira et al. (2007) have challenged the requirement of having any centralized knowledge base or general-purpose inferencing scheme. Problem solvers are designed as distributed agents: situated, autonomous, and flexible."

"On this viewpoint, problem solving is viewed as distributed, with agents performing tasks in different subcontexts of their domains, for example, the activity of an internet browser or security agent. The problem-solving task is laid out into its several components with little or no general coordination of tasks. The situated agent is able to receive sensory input from its particular environment, as well as react in that context without waiting for instructions from some general controller. In interactive game playing, for example, the agent would be addressing a particular local issue, e.g., defending against a particular attack or raising a specific alarm, without any general view of the entire problem situation."

"Agents are autonomous, in that they are often required to act without direct intervention of humans or some general control process. Autonomous agents thus have control over their own actions and internal state. Some agent systems are even capable of learning from their own experiences. Finally, agents are flexible in that they are responsive to situations in their local environment. They are also proactive, in that they can anticipate situations. Finally, they must be able to respond collaboratively with other agents in the problem domain, communicating about tasks, goals, and appropriate processes."

"Several researchers in the domain of robotics have built agent-based systems. Rodney Brooks (1987, 1989) in the MIT Robotics Laboratory has designed what he terms a subsumption architecture, a layered sequence of finite state machines, each one acting in its own context but also supporting functionality at higher levels. Manuela Veloso et al. (2000) working in the Robotics Lab at Carnegie Mellon, have designed a team of robotic soccer (football) agents that collaborate in the adversarial environment of soccer games. Finally, this situated and embodied approach to representation is described by several philosophers of science, including Dan Dennett (1991, 1995) and Andy Clark (1997) as appropriate characterizations of human intelligence. Later in this book we present further representational schemes, including connectionist (Chapter 11) and genetic (Chapter 12) approaches. General representational issues are discussed again in Chapter 16."

The chapter's roadmap:

"In Chapter 7 we examine in detail many of the major approaches the AI community has taken to representation. We begin with the early use of representations, including semantic networks, scripts, frames, and objects. We present these schemes from an evolutionary viewpoint, showing how modern tools grew from these early years of AI research. We next present John Sowa's conceptual graphs, a representation for use in natural language understanding. Finally, we present agent-based and situated approaches to representation, including Rodney Brooks' subsumption architecture, which calls into question the need for a central base of explicit knowledge used with a general purpose controller."

2. ISSUES IN KNOWLEDGE REPRESENTATION (Section 7.0)

"The representation of information for use in intelligent problem solving offers important and difficult challenges that lie at the core of AI. In Section 7.1, we present a brief historical retrospective of early research in representation; topics include semantic networks, conceptual dependencies, scripts, and frames. Section 7.2 offers a more modern representation used in natural language programs, John Sowa's conceptual graphs. In Section 7.3, we critique the requirement of creating centralized and explicit representational schemes. Brooks' alternative is the subsumption architecture for robots. Section 7.4 presents agents, another alternative to centralized control. In later chapters we extend our discussion of representations to include the stochastic (Section 9.3 and Chapter 13), the connectionist (Chapter 11), and the genetic/emergent (Chapter 12)."

The mapping view:

"We begin our discussion of representation from an historical perspective, where a knowledge base is described as a mapping between the objects and relations in a problem domain and the computational objects and relations of a program (Bobrow 1975). The results of inferences in the knowledge base are assumed to correspond to the results of actions or observations in the world. The computational objects, relations, and inferences available to programmers are mediated by the knowledge representation language."

The general principles:

"There are general principles of knowledge organization that apply across a variety of domains and can be directly supported by a representation language. For example, class hierarchies are found in both scientific and commonsense classification systems. How may we provide a general mechanism for representing them? How may we represent definitions? Exceptions? When should an intelligent system make default assumptions about missing information and how can it adjust its reasoning should these assumptions prove wrong? How may we best represent time? Causality? Uncertainty? Progress in building intelligent systems depends on discovering the principles of knowledge organization and supporting them through higher-level representational tools."

Scheme vs medium:

"It is useful to distinguish between a representational scheme and the medium of its implementation. This is similar to the distinction between data structures and programming languages. Programming languages are the medium of implementation; the data structure is the scheme. Generally, knowledge representation languages are more constrained than the predicate calculus or programming languages. These constraints take the form of explicit structures for representing categories of knowledge. Their medium of implementation might be Prolog, Lisp, or more common languages such as C++ or Java."

The challenge to traditional representation:

"Our discussion to this point illustrates the traditional view of AI representational schemes, a view that often includes a global knowledge base of language structures reflecting a static and 'preinterpreted bias' of a 'real world'. More recent research in robotics (Brooks 1991a, Lewis and Luger 2000), situated cognition (Agre and Chapman 1987, Lakoff and Johnson 1999), agent-based problem solving (Jennings et al. 1998, Wooldridge 2000; Fatima et al. 2005, 2006; Vieira et al. 2007), and philosophy (Clark 1997) has challenged this traditional approach. These problem domains require distributed knowledge, a world that can itself be used as a partial knowledge structure, the ability to reason with partial information, and even representations that can themselves evolve as they come to experience the invariants of a problem domain. These approaches are introduced in Sections 7.3 and 7.4."

3. A BRIEF HISTORY OF AI REPRESENTATIONAL SCHEMES (Section 7.1)

3.1 Associationist Theories of Meaning (Section 7.1.1)

"Logical representations grew out of the efforts of philosophers and mathematicians to characterize the principles of correct reasoning. The major concern of logic is the development of formal representation languages with sound and complete inference rules. As a result, the semantics of predicate calculus emphasizes truth-preserving operations on well-formed expressions. An alternative line of research has grown out of the efforts of psychologists and linguists to characterize the nature of human understanding. This work is less concerned with establishing a science of correct reasoning than with describing the way in which humans actually acquire, associate, and use knowledge of their world. This approach has proved particularly useful to the AI application areas of natural language understanding and commonsense reasoning."

The problem with logic for commonsense:

"There are many problems that arise in mapping commonsense reasoning into formal logic. For example, it is common to think of the operators ∨ and → as corresponding to the English 'or' and 'if … then …'. However, these operators in logic are concerned solely with truth values and ignore the fact that the English 'if … then …' suggests specific relationship (often more coorelational than causal) between its premises and its conclusion. For example, the sentence 'If a bird is a cardinal then it is red' (associating the bird cardinal with the color red) can be written in predicate calculus: ∀X (cardinal(X) → red(X)). This may be changed, through a series of truth-preserving operations, Chapter 2, into the logically equivalent expression ∀X (¬ red(X) → ¬ cardinal(X)). These two expressions have the same truth value; that is, the second is true if and only if the first is true. However, truth value equivalence is inappropriate in this situation. If we were to look for physical evidence of the truth of these statements, the fact that this sheet of paper is not red and also not a cardinal is evidence for the truth of the second expression. Because the two expressions are logically equivalent, it follows that it is also evidence for the truth of the first statement. This leads to the conclusion that the whiteness of the sheet of paper is evidence that cardinals are red."

"This line of reasoning strikes us as meaningless and rather silly. The reason for this incongruity is that logical implication only expresses a relationship between the truth values of its operands, while the English sentence implied a positive coorelation between membership in a class and the possession of properties of that class. In fact, the genetic makeup of a bird causes it to have a certain color. This relationship is lost in the second version of the expression. Although the fact that the paper is not red is consistent with the truth of both sentences, it is irrelevant to the causal nature of the color of birds."

Associationist theories:

"Associationist theories, following the empiricist tradition in philosophy, define the meaning of an object in terms of a network of associations with other objects. For the associationist, when humans perceive an object, that perception is first mapped into a concept. This concept is part of our entire knowledge of the world and is connected through appropriate relationships to other concepts. These relationships form an understanding of the properties and behavior of objects such as snow. For example, through experience, we associate the concept snow with other concepts such as cold, white, snowman, slippery, and ice. Our understanding of snow and the truth of statements such as 'snow is white' and 'the snowman is white' manifests itself out of this network of associations."

The Collins and Quillian experiment:

"There is psychological evidence that in addition to their ability to associate concepts, humans also organize their knowledge hierarchically, with information kept at the highest appropriate levels of the taxonomy. Collins and Quillian (1969) modeled human information storage and management using a semantic network (Figure 7.1). The structure of this hierarchy was derived from laboratory testing of human subjects. The subjects were asked questions about different properties of birds, such as, 'Is a canary a bird?' or 'Can a canary sing?' or 'Can a canary fly?'"

"As obvious as the answers to these questions may seem, reaction-time studies indicated that it took longer for subjects to answer 'Can a canary fly?' than it did to answer 'Can a canary sing?' Collins and Quillian explain this difference in response time by arguing that people store information at its most abstract level. Instead of trying to recall that canaries fly, and robins fly, and swallows fly, all stored with the individual bird, humans remember that canaries are birds and that birds have (usually) the property of flying. Even more general properties such as eating, breathing, and moving are stored at the 'animal' level, and so trying to recall whether a canary can breathe should take longer than recalling whether a canary can fly. This is, of course, because the human must travel further up the hierarchy of memory structures to get the answer."

"The fastest recall was for the traits specific to the bird, say, that it can sing or is yellow. Exception handling also seemed to be done at the most specific level. When subjects were asked whether an ostrich could fly, the answer was produced faster than when they were asked whether an ostrich could breathe. Thus the hierarchy ostrich → bird → animal seems not to be traversed to get the exception information: it is stored directly with ostrich. This knowledge organization has been formalized in inheritance systems."

Inheritance systems:

"Inheritance systems allow us to store information at the highest level of abstraction, which reduces the size of knowledge bases and helps prevent update inconsistencies. For example, if we are building a knowledge base about birds, we can define the traits common to all birds, such as flying or having feathers, for the general class bird and allow a particular species of bird to inherit these properties. This reduces the size of the knowledge base by requiring us to define these essential traits only once, rather than requiring their assertion for every individual. Inheritance also helps us to maintain the consistency of the knowledge base when adding new classes and individuals. Assume that we are adding the species robin to an existing knowledge base. When we assert that robin is a subclass of songbird; robin inherits all of the common properties of both songbirds and birds. It is not up to the programmer to remember (or forget!) to add this information."

Semantic networks:

"Graphs, by providing a means of explicitly representing relations using arcs and nodes, have proved to be an ideal vehicle for formalizing associationist theories of knowledge. A semantic network represents knowledge as a graph, with the nodes corresponding to facts or concepts and the arcs to relations or associations between concepts. Both nodes and links are generally labeled. For example, a semantic network that defines the properties of snow and ice appears in Figure 7.2. This network could be used (with appropriate inference rules) to answer a range of questions about snow, ice, and snowmen. These inferences are made by following the links to related concepts. Semantic networks also implement inheritance; for example, frosty inherits all the properties of snowman."

"The term 'semantic network' encompasses a family of graph-based representations. These differ chiefly in the names that are allowed for nodes and links and the inferences that may be performed. However, a common set of assumptions and concerns is shared by all network representation languages; these are illustrated by a discussion of the history of network representations. In Section 7.2 we examine conceptual graphs (Sowa 1984), a more modern network representation language that integrates many of these ideas."

3.2 Early Work in Semantic Nets (Section 7.1.2)

"Network representations have almost as long a history as logic. The Greek philosopher Porphyry created tree-based type hierarchies - with their roots at the top - to describe Aristotle's categories (Porphyry 1887). Frege developed a tree notation for logic expressions. Perhaps the earliest work to have a direct influence on contemporary semantic nets was Charles S. Peirce's system of existential graphs, developed in the nineteenth century (Roberts 1973). Peirce's theory had all the expressive power of first-order predicate calculus, with an axiomatic basis and formal rules of inference."

"Graphs have long been used in psychology to represent structures of concepts and associations. Selz (1913, 1922) pioneered this work, using graphs to represent concept hierarchies and the inheritance of properties. He also developed a theory of schematic anticipation that influenced AI work in frames and schemata. Anderson, Norman, Rumelhart, and others have used networks to model human memory and intellectual performance (Anderson and Bower 1973, Norman et al. 1975)."

"Much of the research in network representations has been done in the arena of natural language understanding. In the general case, language understanding requires an understanding of common sense, the ways in which physical objects behave, the interactions that occur between humans, and the ways in which human institutions are organized. A natural language program must understand intentions, beliefs, hypothetical reasoning, plans, and goals. Because of these requirements language understanding has always been a driving force for research in knowledge representation."

"The first computer implementations of semantic networks were developed in the early 1960s for use in machine translation. Masterman (1961) defined a set of 100 primitive concept types and used them to define a dictionary of 15,000 concepts. Wilks (1972) continued to build on Masterman's work in semantic network-based natural language systems. Shapiro's (1971) MIND program was the first implementation of a propositional calculus based semantic network. Other early AI workers exploring network representations include Ceccato (1961), Raphael (1968), Reitman (1965), and Simmons (1966)."

Quillian's program:

"An influential program that illustrates many of the features of early semantic networks was written by Quillian in the late 1960s (Quillian 1967). This program defined English words in much the same way that a dictionary does: a word is defined in terms of other words, and the components of the definition are defined in the same fashion. Rather than formally defining words in terms of basic axioms, each definition simply leads to other definitions in an unstructured and sometimes circular fashion. In looking up a word, we traverse this 'network'

until we are satisfied that we understand the original word."

"Each node in Quillian's network corresponded to a word concept, with associative links to other word concepts that formed its definition. The knowledge base was organized into planes, where each plane was a graph that defined a single word. Figure 7.3, taken from a paper by Quillian (1967), illustrates three planes that capture three different definitions of the word 'plant': a living organism (plant 1), a place where people work (plant 2), and the act of putting a seed in the ground (plant 3)."

"The program used this knowledge base to find relationships between pairs of English words. Given two words, it would search the graphs outward from each word in a breadth-first fashion, searching for a common concept or intersection node. The paths to this node represented a relationship between the word concepts. For example, Figure 7.4 from the same paper, shows the intersection paths between cry and comfort."

"Using this intersection path, the program was able to conclude: cry 2 is among other things to make a sad sound. To comfort 3 can be to make 2 something less sad (Quillian 1967). The numbers in the response indicate that the program has selected from among different meanings of the words."

Quillian's three proposed capabilities:

"Quillian (1967) suggested that this approach to semantics might provide a natural language understanding system with the ability to:

Determine the meaning of a body of English text by building up collections of these intersection nodes.

Choose between multiple meanings of words by finding the meanings with the shortest intersection path to other words in the sentence. For example, it could select a meaning for 'plant' in 'Tom went home to water his new plant' based on the intersection of the word concepts 'water' and 'plant.'

Answer a flexible range of queries based on associations between word concepts in the queries and concepts in the system."

"Although this and other early work established the power of graphs to model associative meaning, it was limited by the extreme generality of the formalism. Knowledge was generally structured in terms of specific relationships such as object/property, class/subclass, and agent/verb/object."

3.3 Standardization of Network Relationships (Section 7.1.3)

"In itself, a graph notation of relationships has little computational advantage over logic; it is just another notation for relationships between objects. In fact SNePS, Semantic Net Processing System (Shapiro 1979, Shapiro et al. 2006), was a theorem prover in the first-order predicate calculus. The power of his network representations comes from the definition of links and associated inference rules such as inheritance."

"Though Quillian's early work established most of the significant features of the semantic network formalism, such as labeled arcs and links, hierarchical inheritance, and inferences along associational links, it proved limited in its ability to deal with the complexities of many domains. One of the main reasons for this failure was the poverty of relationships (links) that captured the deeper semantic aspects of knowledge. Most of the links represented extremely general associations between nodes and provided no real basis for the structuring of semantic relationships. The same problem is encountered in efforts to use predicate calculus to capture semantic meaning. Although the formalism is highly expressive and can represent almost any kind of knowledge, it is too unconstrained and places the burden of constructing appropriate sets of facts and rules on the programmer."

"Much of the work in network representations that followed Quillian's focused on defining a richer set of link labels (relationships) that would more fully model the semantics of natural language. By implementing the fundamental semantic relationships of natural language as part of the formalism, rather than as part of the domain knowledge added by the system builder, knowledge bases require less handcrafting and achieve greater generality and consistency."

"Brachman (1979) has stated: 'The key issue here is the isolation of the primitives for semantic network languages. The primitives of a network language are those things that the interpreter is programmed in advance to understand, and that are not usually represented in the network language itself.'"

Case frames:

"Simmons (1973) addressed this need for standard relationships by focusing on the case structure of English verbs. In this verb-oriented approach, based on earlier work by Fillmore (1968), links define the roles played by nouns and noun phrases in the action of the sentence. Case relationships include agent, object, instrument, location, and time. A sentence is represented as a verb node, with various case links to nodes representing other participants in the action. This structure is called a case frame. In parsing a sentence, the program finds the verb and retrieves the case frame for that verb from its knowledge base. It then binds the values of the agent, object, etc., to the appropriate nodes in the case frame. Using this approach, the sentence 'Sarah fixed the chair with glue' might be represented by the network in Figure 7.5."

"Thus, the representation language itself captures much of the deep structure of natural language, such as the relationship between a verb and its subject (the agent relation) or that between a verb and its object. Knowledge of the case structure of the English language is part of the network formalism itself. When the individual sentence is parsed, these built-in relationships indicate that Sarah is the person doing the fixing and that glue is used to put the chair together. Note that these linguistic relationships are stored in a fashion that is independent of the actual sentence or even the language in which the sentence was expressed. A similar approach was also taken in network languages proposed by Norman (1972) and Rumelhart et al. (1972, 1973)."

Conceptual dependency theory:

"A number of major research endeavors attempted to standardize link names even further (Masterman 1961, Wilks 1972, Schank and Colby 1973, Schank and Nash-Webber 1975). Each effort worked to establish a complete set of primitives that could be used to represent the semantic structure of natural language expressions in a uniform fashion. These were intended to assist in reasoning with language constructs and to be independent of the idiosyncrasies of individual languages or phrasing."

"Perhaps the most ambitious attempt to model formally the deep semantic structure of natural language is Roger Schank's conceptual dependency theory (Schank and Rieger 1974). Conceptual dependency theory offers a set of four primitive conceptualizations from which the world of meaning is built. These are equal and independent. They are:

ACTs actions PPs objects (picture producers) AAs modifiers of actions (action aiders) PAs modifiers of objects (picture aiders)"

"For example, all actions are assumed to reduce to one or more of the primitive ACTs. The primitives listed below are taken as the basic components of action, with more specific verbs being formed through their modification and combination.

ATRANS transfer a relationship (give) PTRANS transfer physical location of an object (go) PROPEL apply physical force to an object (push) MOVE move body part by owner (kick) GRASP grab an object by an actor (grasp) INGEST ingest an object by an animal (eat) EXPEL expel from an animal's body (cry) MTRANS transfer mental information (tell) MBUILD mentally make new information (decide) CONC conceptualize or think about an idea (think) SPEAK produce sound (say) ATTEND focus sense organ (listen)"

"These primitives are used to define conceptual dependency relationships that describe meaning structures such as case relations or the association of objects and values. Conceptual dependency relationships are conceptual syntax rules and constitute a grammar of meaningful semantic relationships. These relationships can be used to construct an internal representation of an English sentence. A list of basic conceptual dependencies (Schank and Rieger 1974) appears in Figure 7.6. These capture, their creators feel, the fundamental semantic structures of natural language. For example, the first conceptual dependency in Figure 7.6 describes the relationship between a subject and its verb, and the third describes the verb–object relation. These can be combined to represent a simple transitive sentence such as 'John throws the ball' (see Figure 7.7)."

Tense and mode modifiers:

"Finally, tense and mode information may be added to the set of conceptualizations. Schank supplies a list of attachments or modifiers to the relationships. A partial list of these includes:

p past f future t transition k continuing ts start transition ? interrogative tf finish transition c conditional / negative nil present delta? timeless"

"These relations are the first-level constructs of the theory, the simplest semantic relationships out of which more complex structures can be built. Further examples of how these basic conceptual dependencies can be composed to represent the meaning of simple English sentences appear in Figure 7.8."

The example: "John ate the egg":

"Based on these primitives, the English sentence 'John ate the egg' is represented as shown in Figure 7.9, where the symbols have the following meanings:

← indicates the direction of dependency ⇔ indicates the agent—verb relationship p indicates past tense INGEST is a primitive act of the theory O object relation D indicates the direction of the object in the action"

Causality example:

"Another example of the structures that can be built using conceptual dependencies is the representation graph for 'John prevented Mary from giving a book to Bill' (Figure 7.10). This particular example is interesting because it demonstrates how causality can be represented."

Benefits and criticisms:

"Conceptual dependency theory offers a number of important benefits. By providing a formal theory of natural language semantics, it reduces problems of ambiguity. Second, the representation itself directly captures much of natural language semantics, by attempting to provide a canonical form for the meaning of sentences. That is, all sentences that have the same meaning will be represented internally by syntactically identical, not just semantically equivalent, graphs. This canonical representation is an effort to simplify the inferences required for understanding. For example, we can demonstrate that two sentences mean the same thing with a simple match of conceptual dependency graphs; a representation that did not provide a canonical form might require extensive operations on differently structured graphs."

"Unfortunately, it is questionable whether a program may be written to reliably reduce sentences to canonical form. As Woods (1985) and others have pointed out, reduction to a canonical form is provably uncomputable for monoids, a type of algebraic group that is far simpler than natural language. There is also no evidence that humans store their knowledge in any such sort of canonical form."

"Other criticisms of this point of view, besides objecting to the computational price paid in reducing everything to such low-level primitives, suggest that the primitives themselves are not adequate to capture many of the more subtle concepts that are important in natural language use. For example, the representation of 'tall' in the second sentence of Figure 7.8 does not address the ambiguity of this term as fully as is done in systems such as fuzzy logic (Zadeh 1983 and Section 9.2.2)."

"However, no one can say that the conceptual dependency model has not been extensively studied and well understood. More than a decade of research guided by Schank has focused on refining and extending the model. Important extensions of conceptual dependencies include research in scripts and memory organization packets, or MOPs. The research in scripts examines the organization of knowledge in memory and the role this organization plays in reasoning, see Section 7.1.4. MOPs provided one of the supporting research areas for the design of case-based reasoners, Section 8.3. Conceptual dependency theory is a fully developed model of natural language semantics with consistency of purpose and wide applicability."

3.4 Scripts (Section 7.1.4)

"A natural language understanding program must use a large amount of supporting knowledge to understand even the simplest conversation (Section 15.0). There is evidence that humans organize this knowledge into structures corresponding to typical situations (Bartlett 1932). If we are reading a story about restaurants, baseball, or politics, we resolve any ambiguities in the text in a way consistent with restaurants, baseball, or politics. If the subject of a story changes abruptly, there is evidence that people pause briefly in their reading, presumably to change knowledge structures. It is hard to understand a poorly

organized or structured story, possibly because we cannot easily fit it into any of our existing knowledge structures. There can also be errors in understanding when the subject of a conversation changes abruptly, presumably because we are confused over which context to use in resolving pronoun references and other ambiguities in the conversation."

Definition:

"A script is a structured representation describing a stereotyped sequence of events in a particular context. The script was originally designed by Schank and his research group (Schank and Abelson 1977) as a means of organizing conceptual dependency structures into descriptions of typical situations. Scripts are used in natural language understanding systems to organize a knowledge base in terms of the situations that the system is to understand."

The restaurant script:

"Most adults are quite comfortable (i.e., they, as customers, know what to expect and how to act) in a restaurant. They are either met at the restaurant entrance or see some sign indicating that they should continue in and find a table. If a menu is not available at the table or if it is not presented by the waiter, then the customer will ask for one. Similarly, customers understand the routines for ordering food, eating, paying, and leaving."

"In fact, the restaurant script is quite different from other eating scripts such as the 'fast-food' model or the 'formal family meal'. In the fast-food model the customer enters, gets in line to order, pays for the meal (before eating), waits for a tray with the food, takes the tray, tries to find a clean table, and so on. These are two different stereotyped sequences of events, and each has a potential script."

The script components:

"The components of a script are:

Entry conditions or descriptors of the world that must be true for the script to be called. In our example script, these include an open restaurant and a hungry customer that has some money.

Results or facts that are true once the script has terminated; for example, the customer is full and poorer, the restaurant owner has more money.

Props or the 'things' that support the content of the script. These will include tables, waiters, and menus. The set of props supports reasonable default assumptions about the situation: a restaurant is assumed to have tables and chairs unless stated otherwise.

Roles are the actions that the individual participants perform. The waiter takes orders, delivers food, and presents the bill. The customer orders, eats, and pays.

Scenes. Schank breaks the script into a sequence of scenes each of which presents a temporal aspect of the script. In the restaurant there is entering, ordering, eating, etc."

The restaurant script (Figure 7.11):

"The elements of the script, the basic 'pieces' of semantic meaning, are represented using conceptual dependency relationships. Placed together in a framelike structure, they represent a sequence of meanings, or an event sequence. The restaurant script taken from this research is presented in Figure 7.11."

"The program reads a small story about restaurants and parses it into an internal conceptual dependency representation. Because the key concepts in this internal description match with the entry conditions of the script, the program binds the people and things mentioned in the story to the roles and props mentioned in the script. The result is an expanded representation of the story contents, using the script to fill in any missing information and default assumptions. The program then answers questions about the story by referring to the script. The script allows the reasonable default assumptions that are essential to natural language understanding."

Three worked examples:

"EXAMPLE 7.1.1: John went to a restaurant last night. He ordered steak. When he paid he noticed he was running out of money. He hurried home since it had started to rain.

Using the script, the system can correctly answer questions such as: Did John eat dinner last night (the story only implied this)? Did John use cash or a credit card? How could John get a menu? What did John buy?"

"EXAMPLE 7.1.2: Amy Sue went out to lunch. She sat at a table and called a waitress, who brought her a menu. She ordered a sandwich.

Questions that might reasonably be asked of this story include: Why did the waitress bring Amy Sue a menu? Was Amy Sue in a restaurant (the example doesn't say so)? Who paid? Who was the 'she' who ordered the sandwich? This last question is difficult. The most recently named female is the waitress, an incorrect assumption for pronoun reference. Script roles help to resolve such references and other ambiguities."

"EXAMPLE 7.1.3: Kate went to a restaurant. She was shown to a table and ordered sushi from the waitress. She sat there and waited for a long time. Finally, she got mad and left.

Questions that can be answered from this story using the restaurant script include: Who is the 'she' who sat and waited? Why did she wait? Who was the 'she' who got mad and left? Why did she get mad? Note that there are other questions that the script cannot answer, such as why people get upset/mad when the waiter does not come promptly? Like any knowledge-based system, scripts require the knowledge engineer to correctly anticipate the knowledge required."

The two problems:

"Scripts, like frames and other structured representations, are subject to certain problems, including the script match problem and the between-the-lines problem. Consider Example 7.1.4, which could call either the restaurant or concert scripts. The choice is critical because 'bill' can refer to either the restaurant check or the playbill of the concert.

EXAMPLE 7.1.4: John visited his favorite restaurant on the way to the concert. He was pleased by the bill because he liked Mozart."

"Since script selection is usually based on matching 'key' words, it is often difficult to determine which of two or more potential scripts should be used. The script match problem is 'deep' in the sense that no algorithm exists for guaranteeing correct choices. It requires heuristic knowledge about the organization of the world, and perhaps even some backtracking; scripts assist only in the organization of that knowledge."

"The between-the-lines problem is equally difficult: it is not possible to know ahead of time the possible occurrences that can break a script. For instance:

EXAMPLE 7.1.5: Melissa was eating dinner at her favorite restaurant when a large piece of plaster fell from the ceiling and landed on her date.

Questions: Was Melissa eating a date salad? Was Melissa's date plastered? What did she do next? As this example illustrates, structured representations can be inflexible. Reasoning can be locked into a single script, even though this may not be appropriate."

MOPs:

"Memory organization packets (MOPs) address the problem of script inflexibility by representing knowledge as smaller components, MOPs, along with rules for dynamically combining them to form a schema that is appropriate to the current situation (Schank 1982). The organization of knowledge in memory is particularly important to implementations of case-based reasoning, in which the problem solver must efficiently retrieve a relevant prior problem solution from memory (Kolodner 1988a, Section 8.3)."

The Charniak example:

"The problems of organizing and retrieving knowledge are difficult and inherent to the modeling of semantic meaning. Eugene Charniak (1972) illustrated the amount of knowledge required to understand even simple children's stories. Consider a statement about a birthday party: Mary was given two kites for her birthday so she took one back to the store. We must know about the tradition of giving gifts at a party; we must know what a kite is and why Mary might not want two of them; we must know about stores and their exchange policies. In spite of these problems, programs using scripts and other semantic representations can understand natural language in limited domains. An example of this work is a program that interprets messages coming over the news wire services. Using scripts for natural disasters, coups, or other stereotypic stories, programs have shown remarkable success in this limited but realistic domain (Schank and Riesbeck 1981)."

3.5 Frames (Section 7.1.5)

"Another representational scheme, in many ways similar to scripts, that was intended to capture in explicitly organized data structures the implicit connections of information in a problem domain, was called frames. This representation supports the organization of knowledge into more complex units that reflect the organization of objects in the domain. In a 1975 paper, Minsky describes a frame: 'Here is the essence of the frame theory: When one encounters a new situation (or makes a substantial change in one's view of a problem) one selects from memory a structure called a 'frame'. This is a remembered framework to be adapted to fit reality by changing details as necessary (Minsky 1975).'"

"According to Minsky, a frame may be viewed as a static data structure used to represent well-understood stereotyped situations. Framelike structures seem to organize our own knowledge of the world. We adjust to every new situation by calling up information structured by past experiences. We then specially fashion or revise the details of these past experiences to represent the individual differences for the new situation."

The hotel room example:

"Anyone who has stayed in one or two hotel rooms has no trouble with entirely new hotels and their rooms. One expects to see a bed, a bathroom, a place to open a suitcase, a telephone, price and emergency evacuation information on the back of the entry door, and so on. The details of each room can be supplied when needed: color of the curtains, location and use of light switches, etc. There is also default information supplied with the hotel room frame: no sheets; call housekeeping; need ice: look down the hall; and so on. We do not need to build up our understanding for each new hotel room we enter. All of the pieces of a generic hotel room are organized into a conceptual structure that we access when checking into a hotel; the particulars of an individual room are supplied as needed."

The slot types:

"We could represent these higher-level structures directly in a semantic network by organizing it as a collection of separate networks, each of which represents some stereotypic situation. Frames, as well as object-oriented systems, provide us with a vehicle for this organization, representing entities as structured objects with named slots and attached values. Thus a frame or schema is seen as a single complex entity."

"For example, the hotel room and its components can be described by a number of individual frames. In addition to the bed, a frame could represent a chair: expected height is 20 to 40 cm, number of legs is 4, a default value, is designed for sitting. A further frame represents the hotel telephone: this is a specialization of a regular phone except that billing is through the room, there is a special hotel operator (default), and a person is able to use the hotel phone to get meals served in the room, make outside calls, and to receive other services. Figure 7.12 gives a frame representing the hotel room."

"Each individual frame may be seen as a data structure, similar in many respects to the traditional 'record', that contains information relevant to stereotyped entities. The slots in the frame contain information such as:

Frame identification information.

Relationship of this frame to other frames. The 'hotel phone' might be a special instance of 'phone,' which might be an instance of a 'communication device.'

Descriptors of requirements for a frame. A chair, for instance, has its seat between 20 and 40 cm from the floor, its back higher than 60 cm, etc. These requirements may be used to determine when new objects fit the stereotype defined by the frame.

Procedural information on use of the structure described. An important feature of frames is the ability to attach procedural instructions to a slot.

Frame default information. These are slot values that are taken to be true when no evidence to the contrary has been found. For instance, chairs have four legs, telephones are pushbutton, or hotel beds are made by the staff.

New instance information. Many frame slots may be left unspecified until given a value for a particular instance or when they are needed for some aspect of problem solving. For example, the color of the bedspread may be left unspecified."

How frames extend semantic networks:

"Frames extend semantic networks in a number of important ways. Although the frame description of hotel beds, Figure 7.12, might be equivalent to a network description, the frame version makes it much clearer that we are describing a bed with its various attributes. In the network version, there is simply a collection of nodes and we depend more on our

interpretation of the structure to see the hotel bed as the primary object being described. This ability to organize our knowledge into such structures is an important attribute of a knowledge base."

"Frames make it easier to organize our knowledge hierarchically. In a network, every concept is represented by nodes and links at the same level of specification. Very often, however, we may like to think of an object as a single entity for some purposes and only consider details of its internal structure for other purposes. For example, we usually are not aware of the mechanical organization of a car until something breaks down; only then do we pull up our 'car engine schema' and try to find the problem."

Procedural attachment:

"Procedural attachment is an important feature of frames because it supports the linking of specific pieces of code to appropriate entities in the frame representation. For example, we might want to include the ability to generate graphic images in a knowledge base. A graphics language is more appropriate for this than a network language. We use procedural attachment to create demons. A demon is a procedure that is invoked as a side effect of some other action in the knowledge base. For example, we may wish the system to perform type checks or to run consistency tests whenever a certain slot value is changed."

Inheritance:

"Frame systems support class inheritance. The slots and default values of a class frame are inherited across the class/subclass and class/member hierarchy. For instance, a hotel phone could be a subclass of a regular phone except that (1) all out-of-building dialing goes through the hotel switchboard (for billing) and (2) hotel services may be dialed directly. Default values are assigned to selected slots to be used only if other information is not available: assume that hotel rooms have beds and are, therefore, appropriate places to go if you want to sleep; if you don't know how to dial the hotel front desk try 'zero'; the phone may be assumed (no evidence to the contrary) to be pushbutton."

"When an instance of the class frame is created, the system will attempt to fill its slots, either by querying the user, accepting the default value from the class frame, or executing some procedure or demon to obtain the instance value. As with semantic nets, slots and default values are inherited across a class/subclass hierarchy. Of course, default information can cause the data description of the problem to be nonmonotonic, letting us make assumptions about default values that may not always prove correct (see Section 9.1)."

The cube example:

"Minsky's own work on vision provides an example of frames and their use in default reasoning: the problem of recognizing that different views of an object actually represent the same object. For example, the three perspectives of the one cube of Figure 7.13 actually look quite different. Minsky (1975) proposed a frame system that recognizes these as views of a single object by inferring the hidden sides as default assumptions."

"The frame system of Figure 7.13 represents four of the faces of a cube. The broken lines indicate that a particular face is out of view from that perspective. The links between the frames indicate the relations between the views represented by the frames. The nodes, of course, could be more complex if there were colors or patterns that the faces contained. Indeed, each slot in one frame could be a pointer to another entire frame. Also, because given information can fill a number of different slots (face E in Figure 7.13), there need be no redundancy in the information that is stored."

The summary:

"Frames add to the power of semantic nets by allowing complex objects to be represented as a single frame, rather than as a large network structure. This also provides a very natural way to represent stereotypic entities, classes, inheritance, and default values. Although frames, like logic and network representations, are a powerful tool, many of the problems of acquiring and organizing a complicated knowledge base must still be solved by the programmer's skill and intuition. Finally, this MIT research of the 1970s, as well as similar work at Xerox Palo Alto Research Center, led to the 'object-oriented' programming design philosophy as well as building important implementation languages, including Smalltalk, C++, and Java."

4. CONCEPTUAL GRAPHS: A NETWORK LANGUAGE (Section 7.2)

"Following on the early research work in AI that developed representational schemes (Section 7.1) a number of network languages were created to model the semantics of natural language and other domains. In this section, we examine a particular formalism in detail, to show how, in this situation, the problems of representing meaning were addressed. John Sowa's conceptual graphs (Sowa 1984) is an example of a network representation language. We define the rules for forming and manipulating conceptual graphs and the conventions for representing classes, individuals, and relationships. In Section 15.3.2 we show how this formalism may be used to represent meaning in natural language understanding."

4.1 Introduction to Conceptual Graphs (Section 7.2.1)

"A conceptual graph is a finite, connected, bipartite graph. The nodes of the graph are either concepts or conceptual relations. Conceptual graphs do not use labeled arcs; instead the conceptual relation nodes represent relations between concepts. Because conceptual graphs are bipartite, concepts only have arcs to relations, and vice versa. In Figure 7.14 dog and brown are concept nodes and color a conceptual relation. To distinguish these types of nodes, we represent concepts as boxes and conceptual relations as ellipses."

"In conceptual graphs, concept nodes represent either concrete or abstract objects in the world of discourse. Concrete concepts, such as a cat, telephone, or restaurant, are characterized by our ability to form an image of them in our minds. Note that concrete concepts include generic concepts such as cat or restaurant along with concepts of specific cats and restaurants. We can still form an image of a generic cat. Abstract concepts include things such as love, beauty, and loyalty that do not correspond to images in our minds."

"Conceptual relation nodes indicate a relation involving one or more concepts. One advantage of formulating conceptual graphs as bipartite graphs rather than using labeled arcs is that it simplifies the representation of relations of any arity. A relation of arity n is represented by a conceptual relation node having n arcs, as shown in Figure 7.14."

"Each conceptual graph represents a single proposition. A typical knowledge base will contain a number of such graphs. Graphs may be arbitrarily complex but must be finite. For example, one graph in Figure 7.14 represents the proposition 'A dog has a color of brown'. Figure 7.15 is a graph of somewhat greater complexity that represents the sentence 'Mary gave John the book'. This graph uses conceptual relations to represent the cases of the verb 'to give' and indicates the way in which conceptual graphs are used to model the semantics of natural language."

4.2 Types, Individuals, and Names (Section 7.2.2)

"Many early designers of semantic networks were careless in defining class/member and class/subclass relationships, with resulting semantic confusion. For example, the relation between an individual and its class is different from the relation between a class (such as dog) and its superclass (carnivore). Similarly, certain properties belong to individuals, and others belong to the class itself; the representation should provide a vehicle for making this distinction. The properties of having fur and liking bones belong to individual dogs; the class 'dog' does not have fur or eat anything. Properties that are appropriate to the class include its name and membership in a zoological taxonomy."

"In conceptual graphs, every concept is a unique individual of a particular type. Each concept box is labeled with a type label, which indicates the class or type of individual represented by that node. Thus, a node labeled dog represents some individual of that type. Types are organized into a hierarchy. The type dog is a subtype of carnivore, which is a subtype of mammal, etc. Boxes with the same type label represent concepts of the same type; however, these boxes may or may not represent the same individual concept."

Type labels:

"Each concept box is labeled with the names of the type and the individual. The type and individual labels are separated by a colon, ':'. The graph of Figure 7.16 indicates that the dog 'Emma' is brown. The graph of Figure 7.17 asserts that some unspecified entity of type dog has a color of brown. If the individual is not indicated, the concept represents an unspecified individual of that type."

Markers:

"Conceptual graphs also let us indicate specific but unnamed individuals. A unique token called a marker indicates each individual in the world of discourse. This marker is written as a number preceded by a #. Markers are different from names in that they are unique: individuals may have one name, many names, or no name at all, but they have exactly one marker. Similarly, different individuals may have the same name but may not have the same marker. This distinction gives us a basis for dealing with the semantic ambiguities that arise when we give objects names. The graph of Figure 7.17 asserts that a particular dog, #1352, is brown."

"Markers allow us to separate an individual from its name. If dog #1352 is named 'Emma,' we can use a conceptual relation called name to add this to the graph. The result is the graph of Figure 7.18. The name is enclosed in double quotes to indicate that it is a string. Where there is no danger of ambiguity, we may simplify the graph and refer to the individual directly by name. Under this convention, the graph of Figure 7.18 is equivalent to the graph of Figure 7.16."

"Although we frequently ignore it both in casual conversation and in formal representations, this distinction between an individual and its name is an important one that should be supported by a representation language. For example, if we say that 'John' is a common name among males, we are asserting a property of the name itself rather than of any individual named 'John'. This allows us to represent such English sentences as ''Chimpanzee' is the name of a species of primates'. Similarly, we may want to represent the fact that an individual has several different names. The graph of Figure 7.19 represents the situation described in the song lyric: 'Her name was McGill, and she called herself Lil, but everyone knew her as Nancy' (Lennon and McCartney 1968)."

Generic marker and variables:

"As an alternative to indicating an individual by its marker or name, we can also use the generic marker * to indicate an unspecified individual. By convention, this is often omitted from concept labels; a node given just a type label, dog, is equivalent to a node labeled dog:. *In addition to the generic marker, conceptual graphs allow the use of named variables. These are represented by an asterisk followed by the variable name (e.g.,* X or *foo). This is useful if two separate nodes are to indicate the same, but unspecified, individual. The graph of Figure 7.20 represents the assertion 'The dog scratches its ear with its paw'. Although we do not know which dog is scratching its ear, the variable* X indicates that the paw and the ear belong to the same dog that is doing the scratching."

"To summarize, each concept node can indicate an individual of a specified type. This individual is the referent of the concept. This reference is indicated either individually or generically. If the referent uses an individual marker, the concept is an individual concept; if the referent uses the generic marker, then the concept is generic."

4.3 The Type Hierarchy (Section 7.2.3)

"The type hierarchy, as illustrated by Figure 7.21, is a partial ordering on the set of types, indicated by the symbol <. If s and t are types and t < s, then t is said to be a subtype of s and s is a supertype of t. Because it is a partial ordering, a type may have one or more supertypes as well as one or more subtypes. If s, t, and u are types, with t < s and t < u, then t is said to be a common subtype of s and u. Similarly, if s < v and u < v then v is a common supertype of s and u."

"The type hierarchy of conceptual graphs forms a lattice, a common form of multiple inheritance system. In a lattice, types may have multiple parents and children. However, every pair of types must have a minimal common supertype and a maximal common subtype. For types s and u, v is a minimal common supertype if s < v, u < v, and for any w, a common supertype of s and u, v < w. Maximal common subtype has a corresponding definition. The minimal common supertype of a collection of types is the appropriate place to define properties common only to those types. Because many types, such as emotion and rock, have no obvious common supertypes or subtypes, it is necessary to add types that fill

these roles. To make the type hierarchy a true lattice, conceptual graphs include two special types. The universal type, indicated by T, is a supertype of all types. The absurd type, indicated by ⊥, is a subtype of all types."

4.4 Generalization and Specialization (Section 7.2.4)

"The theory of conceptual graphs includes a number of operations that create new graphs from existing graphs. These allow for the generation of a new graph by either specializing or generalizing an existing graph, operations that are important for representing the semantics of natural language. The four operations are copy, restrict, join, and simplify, as seen in Figure 7.22. Assume that g1 and g2 are conceptual graphs. Then:

The copy rule allows us to form a new graph, g, that is the exact copy of g1."

Restrict:

"Restrict allows concept nodes in a graph to be replaced by a node representing their specialization. There are two cases:

If a concept is labeled with a generic marker, the generic marker may be replaced by an individual marker.

A type label may be replaced by one of its subtypes, if this is consistent with the referent of the concept. In Figure 7.22 we can replace animal with dog."

Join:

"The join rule lets us combine two graphs into a single graph. If there is a concept node c1 in the graph s1 that is identical to a concept node c2 in s2, then we can form a new graph by deleting c2 and linking all of the relations incident on c2 to c1. Join is a specialization rule, because the resulting graph is less general than either of its components."

Simplify:

"If a graph contains two duplicate relations, then one of them may be deleted, along with all its arcs. This is the simplify rule. Duplicate relations often occur as the result of a join operation, as in graph g4 of Figure 7.22."

The uses of restrict and join:

"One use of the restrict rule is to make two concepts match so that a join can be performed. Together, join and restrict allow the implementation of inheritance. For example, the replacement of a generic marker by an individual implements the inheritance of the properties of a type by an individual. The replacement of a type label by a subtype label defines inheritance between a class and a superclass. By joining one graph to another and restricting certain concept nodes, we can implement inheritance of a variety of properties. Figure 7.23 shows how chimpanzees inherit the property of having a hand from the class primates by replacing a type label with its subtype. It also shows how the individual, Bonzo, inherits this property by instantiating a generic concept."

"Similarly, we can use joins and restrictions to implement the plausible assumptions that play a role in common language understanding. If we are told that 'Mary and Tom went out for pizza together,' we automatically make a number of assumptions: they ate a round Italian bread covered with cheese and tomato sauce. They ate it in a restaurant and must have had some way of paying for it. This reasoning can be done using joins and restrictions. We form a conceptual graph of the sentence and then join it with the conceptual graphs (from our knowledge base) for pizzas and restaurants. The resulting graph lets us assume that they ate tomato sauce and paid their bill."

Specialization vs generalization:

"Join and restrict are specialization rules. They define a partial ordering on the set of derivable graphs. If a graph g1 is a specialization of g2, then we may say that g2 is a generalization of g1. Generalization hierarchies are important in knowledge representation. Besides providing the basis for inheritance and other commonsense reasoning schemes, generalization hierarchies are used in many learning methods, allowing us, for instance, to construct a generalized assertion from a particular training instance."

Not truth-preserving:

"These rules are not rules of inference. They do not guarantee that true graphs will be derived from true graphs. For example, in the restriction of the graph of Figure 7.19, the result may not be true; Emma may be a cat. Similarly, the joining example of Figure 7.19 is not truth-preserving either: the dog on the porch and the dog that eats bones may be different animals. These operations are canonical formation rules, and although they do not preserve truth, they have the subtle but important property of preserving 'meaningfulness'. This is an important guarantee when we use conceptual graphs to implement natural language understanding. Consider the three sentences:

Albert Einstein formulated the theory of relativity. Albert Einstein plays center for the Los Angeles Lakers. Conceptual graphs are yellow flying popsicles.

The first of these sentences is true and the second is false. The third sentence, however, is meaningless; though grammatically correct, it makes no sense. The second sentence, although false, is meaningful. I can imagine Albert Einstein on a basketball court. The canonical formation rules enforce constraints on semantic meaning; that is, they do not allow us to form nonsensical graphs from meaningful ones. Although they are not sound inference rules, canonical formation rules form a basis for much of the plausible reasoning done in natural language understanding and common sense reasoning."

4.5 Propositional Nodes (Section 7.2.5)

"In addition to using graphs to define relations between objects in the world, we may also want to define relations between propositions. Consider, for example, the statement 'Tom believes that Jane likes pizza'. 'Believes' is a relation that takes a proposition as its argument."

"Conceptual graphs include a concept type, proposition, that takes a set of conceptual graphs as its referent and allows us to define relations involving propositions. Propositional concepts are indicated as a box that contains another conceptual graph. These proposition concepts may be used with appropriate relations to represent knowledge about propositions. Figure 7.24 shows the conceptual graph for the above assertion about Jane, Tom, and pizza. The experiencer relation is loosely analogous to the agent relation in that it links a subject and a verb. The experiencer link is used with belief states based on the notion that they are something one experiences rather than does."

"Figure 7.24 shows how conceptual graphs with propositional nodes may be used to express the modal concepts of knowledge and belief. Modal logics are concerned with the various ways propositions are entertained: believed, asserted as possible, probably or necessarily true, intended as a result of an action, or counterfactual (Turner 1984)."

4.6 Conceptual Graphs and Logic (Section 7.2.6)

"Using conceptual graphs, we can easily represent conjunctive concepts such as 'The dog is big and hungry', but we have not established any way of representing negation or disjunction. Nor have we addressed the issue of variable quantification."

"We may implement negation using propositional concepts and a unary operation called neg. neg takes as argument a proposition concept and asserts that concept as false. The conceptual graph of Figure 7.25 uses neg to represent the statement 'There are no pink dogs'."

"Using negation and conjunction, we may form graphs that represent disjunctive assertions according to the rules of logic. To simplify this, we may also define a relation or, which takes two propositions and represents their disjunction."

"In conceptual graphs, generic concepts are assumed to be existentially quantified. For example, the generic concept dog in the graph of Figure 7.14 actually represents an existentially quantified variable. This graph corresponds to the logical expression:

∃ X ∃Y (dog(X) ∧ color(X,Y) ∧ brown(Y))."

"Using negation and existential quantification, Section 2.2.2, we can also represent universal quantification. For example, the graph of Figure 7.25 could be thought of as representing the logical assertion:

∀ X ∀ Y (¬ (dog(X) ∧ color(X,Y) ∧ pink(Y)))."

"Conceptual graphs are equivalent to predicate calculus in their expressive power. As these examples suggest, there is a straightforward mapping from conceptual graphs into predicate calculus notation. The algorithm, taken from Sowa (1984), for changing a conceptual graph, g, into a predicate calculus expression is:

Assign a unique variable, x1, x2, . . . , xn, to each of the n generic concepts in g.

Assign a unique constant to each individual concept in g. This constant may simply be the name or marker used to indicate the referent of the concept.

Represent each concept node by a unary predicate with the same name as the type of that node and whose argument is the variable or constant given that node.

Represent each n-ary conceptual relation in g as an n-ary predicate whose name is the same as the relation. Let each argument of the predicate be the variable or constant assigned to the corresponding concept node linked to that relation.

Take the conjunction of all atomic sentences formed under 3 and 4. This is the body of the predicate calculus expressions. All the variables in the expression are existentially quantified."

"For example, the graph of Figure 7.16 may be transformed into the predicate calculus expression

∃ X1 (dog(emma) ∧ color(emma,X1) ∧ brown(X1))"

"Although we can reformulate conceptual graphs into predicate calculus syntax, conceptual graphs also support a number of special-purpose inferencing mechanisms, such as join and restrict, as presented in Section 7.2.4, not normally part of the predicate calculus."

"We have presented the syntax of conceptual graphs and defined the restriction operation as a means of implementing inheritance. We have not yet examined the full range of operations and inferences that may be performed on these graphs, nor have we addressed the problem of defining the concepts and relations needed for domains such as natural language. We address these issues again in Section 15.3.2 where we use conceptual graphs to implement a knowledge base for a simple natural language understanding program."

5. ALTERNATIVE REPRESENTATIONS AND ONTOLOGIES (Section 7.3)

"In recent years, AI researchers have continued to question the role of explicit representation in intelligence. Besides the connectionist and emergent approaches of Chapters 11 and 12, a further challenge to the role of traditional representation comes from Rodney Brooks' work at MIT (Brooks 1991a). Brooks questions, in designing a robot explorer, the need for any centralized representational scheme, and with his subsumption architecture, attempts to show how general intelligence might evolve from lower and supporting forms of intelligence."

"A second approach to the problem of explicit and static representations comes from the work of Melanie Mitchell and Douglas Hofstadter at Indiana University. The Copycat architecture is an evolving network which adjusts itself to the meaning relationships that it encounters through experiment with an external world."

"Finally, when building problem solvers in complex environments it is often necessary to employ multiple different representational schemes. Each representational scheme may be referred to as an ontology. It then becomes necessary to build communication and other links between these different representations to support knowledge management, communication, backup, and other aspects of the problem solving. We introduce these issues and possible solutions under the general heading of knowledge management technology. This technology can also support agent-based problem solving as presented in Section 7.4."

5.1 Brooks' Subsumption Architecture (Section 7.3.1)

"Brooks conjectures, and offers examples through his robotic creations, that intelligent behavior does not come from disembodied systems like theorem provers, or even from traditional expert systems (Section 8.2). Intelligence, Brooks claims, is the product of the interaction between an appropriately designed system and its environment. Furthermore, Brooks espouses the view that intelligent behavior emerges from the interactions of architectures of organized simpler behaviors: his subsumption architecture."

"The subsumption architecture is a layered collection of task-handlers. Each task is accomplished by a finite state machine that continually maps a perception-based input into an action-oriented output. This mapping is accomplished through simple sets of condition→action production rules (Section 6.2). These rules determine, in a fairly blind fashion, that is, with no global state knowledge, what actions are appropriate to the current state of

that subsystem. Brooks does allow some feedback to lower level systems."

"Figure 7.26, adapted from Brooks (1991a), shows a three-layered subsumption architecture. Each layer is composed of a fixed topology network of simple finite state machines, each having a few states, one or two internal registers, one or two internal timers, and access to simple computational devices, for example, to compute vector sums. These finite state machines run asynchronously, sending and receiving fixed length messages over wires. There is no central locus of control. Rather, each finite state machine is data-driven by the messages it receives. The arrival of a message or the expiration of a time period causes the machines to change state. There is no access to global data or to any dynamically created communication links. That is, there is no global control."

The three layers:

"Figure 7.26 presents a subset of the functions of the three-layered architecture that supported an early robot (Brooks 1991a). The robot had a ring of twelve sonar sensors around it. At every second these sensors gave twelve radial depth measurements. The lowest level layer of the subsumption architecture, AVOID, implements a behavior that keeps the robot from hitting objects, whether these are static or moving. The machine labeled sonar data emits an instantaneous map that is passed on to collide and feelforce, which in turn, are able to generate halt messages for the finite state machine in charge of running the robot forward. When feelforce is activated, it is able to generate either runaway or avoid instructions for object and threat avoidance."

"The lowest level network of finite state machines in the architecture generates all halt and avoid instructions for the entire system. The next layer, WANDER, makes the system move about by generating a random heading for the robot about every ten seconds. The AVOID (lower level) machine takes the heading from WANDER and couples it with the forces computed by the AVOID architecture. WANDER uses the result to suppress lower level behavior, forcing the robot to move in a direction close to what wander decided, but at the same time avoiding obstacles. Finally, if the turn and forward machines (top level architecture) are activated, they will suppress any new impulses sent from WANDER."

"The top level, EXPLORE, makes the robot explore its environment, looking for distant places and trying to reach them by planning a path. This layer is able to suppress the wander instructions and observes how the bottom layer diverts the robot due to obstacles. It corrects for these divergences and keeps the robot focused on its goal, inhibiting the wandering behavior but allowing the lowest level object avoidance to continue its function. When deviations generated at the lower level occur, the EXPLORE level calls plan to keep the system goal-oriented. The main point of Brook's subsumption architecture is that the system requires no centralized symbolic reasoning and takes all its actions without searching through possible next states. Although the behavior-based finite state machine is generating suggestions for actions based on its own current state, the global system acts based on the interactions of the systems layered below it."

Brooks' robots:

"The three-layer architecture just presented comes from Brooks' early design of a wandering, goal-seeking robot. More recently, his research group has built complex systems with further layers (Brooks 1991a, Brooks and Stein 1994). One system is able to wander around the MIT Robotics Lab looking for empty aluminum drink cans on people's desks. This requires layers for discovering offices, looking for desks, and recognizing drink cans. Further layers are able to guide the robot arm to collect these cans for recycling."

"Brooks insists that top level behavior emerges as a result of the design and testing of the individual lower level layers of the architecture. The design for coherent final behaviors, requiring both inter-layer and between-layer communications, is discovered through experiment. Despite this simplicity of design, the subsumption architecture has performed successfully in several applications (Brooks 1989, 1991a, 1997)."

The open questions:

"There remains, a number of important questions concerning the subsumption architecture and related approaches to the design of control systems (see also Section 16.2):

There is a problem of the sufficiency of information at each system level. Since at each level, purely reactive state machines make decisions on local information, it is difficult to see how such decisions can take account of information not at that level. By definition, it will be myopic.

If there exists absolutely no 'knowledge' or 'model' of the complete environment, how can the limited input on the local situation be sufficient for determination of globally acceptable actions? How can top level coherence result?

How can a purely reactive component with very limited state learn from its environment? At some level in the system, there must be sufficient state for the creation of learning mechanisms if the overall agent is to be called intelligent.

There is a problem of scale. Although Brooks and his associates claim to have built subsumption architectures of six and even ten layers, what design principles allow it to scale to further interesting behavior, i.e., to large complex systems?

Finally, we must ask the question what is 'emergence'? Is it magic? At this time in the evolution of science, 'emergence' seems to be a word to cover any phenomenon for which we can, as yet, make no other accounting. We are told to build systems and test them in the world and they will show intelligence. Unfortunately, without further design instruction, emergence just becomes a word to describe what we can't yet understand. As a result, it is very difficult to determine how we can use this technology to build ever more complex systems. In the next section we describe copycat, a hybrid architecture that is able by exploration to discover and use invariances found in a problem domain."

5.2 Copycat (Section 7.3.2)

"An often-heard criticism of traditional AI representation schemes is that they are static and cannot possibly reflect the dynamic nature of thought processes and intelligence. When a human perceives a new situation, for example, he or she is often struck by relationships with already known or analogous situations. In fact, it is often noted that human perception is both bottom up, that is, stimulated by new patterns in the environment, and top down, mediated by what the agent expects to perceive."

"Copycat is a problem-solving architecture built by Melanie Mitchell (1993) as a PhD dissertation under Douglas Hofstadter (1995) at Indiana University. Copycat builds on many of the representational techniques that preceded it, including blackboards, Section 6.3, semantic networks, Section 7.2, connectionist networks, Chapter 11, and classifier systems (Holland 1986). It also follows Brooks' approach to problem solving as active intervention in a problem domain. In contrast with Brooks and connectionist networks, however, copycat requires a global 'state' to be part of the problem solver. Secondly, representation is an evolving feature of that state. Copycat supports a semantic network-like mechanism that grows and changes with continuing experience within its environment."

"The original problem domain for copycat was the perception and building of simple analogies. In that sense it is building on earlier work by Evans (1968) and Reitman (1965). Examples of this domain are completing the patterns: hot is to cold as tall is to {wall, short, wet, hold} or bear is to pig as chair is to {foot, table, coffee, strawberry}. Copycat also worked to discover appropriate completions for alphabetic string patterns such as: abc is to abd as ijk is to ? or again, abc is to abd as iijjkk is to ?, Figure 7.27."

The three components:

"Copycat is made up of three components, the workspace, the slipnet, and the coderack. These three are mediated in their interactions by a temperature measure. The temperature captures the degree of perceptual organization in the system, and controls the degree of randomness used in making decisions. Higher temperatures reflect the fact that there is little information on which to base decisions and thus they are more random. A temperature drop indicates the system is building consensus and a low temperature indicates that an answer is emerging and reflects the program's 'confidence' in that solution."

"The workspace is a global structure, similar to the blackboard of Section 6.3, for creating structures that the other components of the system can inspect. In this sense it is also much like the message area in Holland's (1986) classifier system. The workspace is where perceptual structures are built hierarchically on top of the input (the three strings of alphabetic symbols of Figure 7.27) and gives possible states for the workspace, with bonds (the arrows) built between related components of the strings."

"The slipnet reflects the network of concepts or potential associations for the components of the analogy. One view of the slipnet is as a dynamically deformable semantic network, each of whose nodes has an activation level. Links in the network can be labeled by other nodes. Based on the activation level of the labelling nodes, the linked nodes grow or shrink. In this way the system changes the degree of association between the nodes as a function of context. The spreading activation among nodes is encouraged between nodes that (in the current context) are more closely related."

"The coderack is a priority based probabilistic queue containing codelets. Codelets are small pieces of executable code designed to interact with the objects in the workspace and to attempt to further some small part of the evolving solution, or, more simply, to explore different facets of the problem space. Again, the codelets are very much like the individual classifiers of Holland's (1986) system."

"Nodes in the slipnet generate codelets and post them to the coderack where they have a probabilistic chance of execution. The system maintains pieces of code in parallel that compete for the chance to find and build structures in the workspace. These pieces of code correspond to the nodes from which they came. This is a top-down activity, seeking more examples of the things that have already generated interest. Codelets also work bottom up, identifying and building on relationships already existing in the workspace."

"As structures are built in the workspace, an activation measure is added to the nodes that generated the codelets that built the structures. This allows the system's behavior in a context of the current state of the problem/solution to impact its future behavior."

"Finally, the temperature serves as a feedback mechanism calculating the 'cohesion' among the structures built in the workspace. With little cohesion, that is, few structures that offer promise of a solution, the biases of the probabilistic choices made by the system are less important: one choice is just about as useful as another. When the cohesion is high with an internally consistent solution evolving, biases become very important in the probabilistic choices: the choices made are more closely linked to the evolving solution."

"There are several follow-on projects in the copycat world that test the architecture in richer settings. Hofstadter and his students (Marshall 1999) continue to model analogical relationships. Lewis and Luger (2000) have expanded the copycat architecture for use as a control system for a mobile robot. The robotics domain gives copycat a concrete environment which supports learning-oriented interactions. Evolving from the robot's interaction with its 'world' a map of the space is generated and paths are planned and replanned."

5.3 Multiple Representations, Ontologies, and Knowledge Services (Section 7.3.3)

"Many complex AI applications require that multiple representational schemes be designed, each set up for specific casks, and then integrated into a user oriented software platform. A simple example of this might be the creation of a virtual meeting space. The various attendees, from their remote locations would interact with each other in this virtual 'space'. Besides voice technology, the virtual meeting space would require video streaming, text and presentation sharing, records access, and so on. One approach to this task would be to adapt various off-the-shelf components and then integrate them into a knowledge service that supports the interactions required."

"The automated virtual meeting service just described is an example of integrating multiple representational schemes to provide a service. We often use the term ontology to characterize the representation scheme of a particular component of that service. The resulting knowledge service requires the integration of these components. Ontology is derived from a Greek word, a form of the verb 'to be', that means being or existence. Thus the term refers to the components of a software module and the structure supporting the interactions of these components, that is, to their function or 'constituent being' as part of the module."

"When the modules are composed to build the knowledge service each of the modules have to

be appropriately linked to form the seamless service. We say that the ontologies of each module are merged to make a new more complex ontology that supports the full service. To create a new knowledge service, such as software to support the virtual meeting, the ontologies of each of its components must be understood and often exposed for the integration. There now exists a number of ontology languages designed explicitly for the creation of knowledge services and other related software products. These include the Owl family of languages (Mika et al. 2004) as well as the open source Java api SOFA, Simple Ontology Framework."

"There are if fact multiple ways that ontological information is used in modern computing, some straightforward, such as the optimized access to knowledge stored in a traditional database, and others quite novel. With the importance of the WWW, sophisticated web crawlers and search engines are critical support tools. Although most current web spiders apply their search to flat directly linked text files, a more sophisticated crawler can be envisioned that can traverse the classes and links of particular complex representations as well as 'interpret' more useful entities including graphs and images."

"Although full 'image interpretation' remains well beyond the current state of our capabilities, its creation will help support our desires for a more 'semantic' web. We may want to scan our holiday photographs to find all that have a particular person in them, or any animal, for example. In the larger situation of the WWW we may want to not only track and find any newspaper stories that describe natural disasters but also locate any images that present these disasters. Currently we are often limited to the search a figure's captions or other (textual) descriptive materials for a hint of their content."

"Besides the goal of interpreting graphical and image data, it is also desired to locate text data on specific topics and produce a summary of its content. For instance, we might want to search for on line research articles in a certain area and report back their contents. Although this is another current research topic in the area of natural language understanding, Chapter 15, its implementation remains at the boundary of our computer based 'semantic' skills. Currently we do things such as searching for a few key words that might be indicative of an article's content and perhaps print out its abstract, but other than this our summarization skills are limited (Ciravegna and Wilks 2004, Wilks 2004)."

"A further web service might be desired to search for 'help wanted' advertisement. It might be launched to search for software professionals, for example, with skills in Java and C++ and willing to locate in a specific city. This web crawler would need to know something about this type advertisement, including the facts that the company name and its location are usually included. Besides the skills required for the job, these ads often describe experience needed along with possible salaries and benefits. Thus the employment web service must have knowledge of and the ability to accesses the ontological structures used in the design of employment 'want ads'. We present an example of such a knowledge service in Section 15.5."

"Perhaps the most important application, however, for ontology based knowledge services is in agent-oriented problem solving. Agents by definition are intended to be autonomous, independent, flexible when addressing tasks, located in specific situations, and able to communicate appropriately. Agent-based problem solving is also highly distributed across environments, such as might be required for constructing an intelligent web service. The creation of specific agent skills can be seen as a question for building and linking ontologies. In the next section on agent-based problem solving, we consider further the distributed and component-oriented view of representation."

6. AGENT-BASED AND DISTRIBUTED PROBLEM SOLVING (Section 7.4)

"There were two insights in the AI research community in the 1980s that had important consequences for future research in the analysis of the role of representation in intelligent problem solving. The first was the research tradition of the 'Distributed Artificial Intelligence' or the 'DAI' community. The first DAI workshop was held at MIT in 1980 for discussion of issues related to intelligent problem solving with systems consisting of multiple problem solvers. It was decided at that time by the DAI community that they were not interested in low level parallelism issues, such as how to distribute processing over different machines or how to parallelize complex algorithms. Rather, their goal was to understand how distributed problem solvers could be effectively coordinated for the intelligent solution of problems. In fact, there had been an even earlier history of distributed processing in artificial intelligence, with the use and coordination of actors and demons and the design of blackboard systems, as seen in Section 6.3."

"The second AI research insight of the 1980s, already presented in Section 7.3, was that of Rodney Brooks and his group at MIT. Brooks' challenge to the AI community's traditional view of representation and reasoning had many important consequences (Section 7.3.1). First, the conjecture that intelligent problem solving does not require a centralized store of knowledge manipulated by some general-purpose inferencing scheme led to the notion of distributed and cooperative models of intelligence, where each element of the distributed representation was responsible for its own component of the problem solving process. Second, the fact that intelligence is situated and active in the context of particular tasks, allows the problem solver to offload aspects of the solution process into the environment itself. This allows, for example, an individual solver to address a task at hand and at the same time have no knowledge whatsoever of the progress towards solution within the general problem domain. Thus one web agent, say, can check inventory information, while another agent checks credit worthiness of a customer, both agents unaware of the higher level decision making, for example, whether or not to allow a purchase. Both of these research emphases of the 1980s brought on the current interest in the design and use of intelligent agents."

6.1 Agent-Oriented Problem Solving: A Definition (Section 7.4.1)

"Before proceeding further in the discussion of agent research, we define what we mean by 'agent', 'agent-based system', and 'multi-agent system'. There are problems here, however, as many different groups in the agent research community have differing opinions as to exactly what agent-based problem solving is about. Our definition and discussion is based on an extension of the work by Jennings, Sycara, and Wooldridge (1998), Wooldridge (2000), Wooldridge et al. (2006, 2007), and Lewis and Luger (2000)."

"For us, a multi-agent system is a computer program with problem solvers situated in interactive environments, which are each capable of flexible, autonomous, yet socially organized actions that can, but need not be, directed towards predetermined objectives or goals. Thus, the four criteria for an intelligent agent system include software problem solvers that are situated, autonomous, flexible, and social."

The four criteria:

"That an intelligent agent is situated means that the agent receives input from the environment in which it is active and can also effect changes within that environment. Examples of environments for situated agents include the internet, game playing, or a robotics application. A concrete example might be a soccer player in a ROBOCUP competition (Veloso et al. 2000) where an agent must interact appropriately with the ball and an opponent without full knowledge of the locations, challenges, and successes of other players in the contest. This situatedness can be contrasted with more traditional AI problem solvers, such as the STRIPS planner, Section 8.4, or the MYCIN expert system, Section 8.3, that maintain centrally located and exhaustive knowledge of application domains."

"An autonomous system is one that can interact with its environment without the direct intervention of other agents. To do this it must have control over its own actions and internal state. Some autonomous agents can also learn from their experience to improve their performance over time (see machine learning, Part IV). For example, on the internet, an autonomous agent could do a credit card authentication check independent of other issues in the purchasing transaction. In the ROBOCUP example, an agent could pass the ball to a teammate or kick it on goal depending on its individual situation."

"A flexible agent is both intelligently responsive as well as proactive depending on its current situation. A responsive agent receives stimuli from its environment and responds to them in an appropriate and timely fashion. A proactive agent does not simply respond to situations in its environment but is also able to plan, be opportunistic, goal directed, and have appropriate alternatives for various situations. A credit agent, for example, would be able to go back to the user with ambiguous results or find another credit agency if one alternative is not sufficient. The soccer agent could change its trajectory depending on the challenge pattern of an opponent."

"Finally, an agent is social that can interact, as appropriate, with other software or human agents. After all, an agent is only part of a complex problem solving process. The interactions of the social agent are oriented towards the goals of the larger multi-agent system. This social dimension of the agent system must address many difficult situations. These include: How can different agents bid for a subtask in problem solving? How can agents communicate with each other to facilitate the accomplishment of higher system-level tasks - in the ROBOCUP example, this might be to score a goal. How can one agent support another agent's goals, for example, to handle the security issues of an internet task? All these questions on the social dimension are the subject of ongoing research."

Multi-agent systems:

"We have described the basis for creating multi-agent systems. Multi-agent systems are ideal for representing problems that include many problem-solving methods, multiple viewpoints, and multiple entities. In these domains, multi-agent systems offer the advantages of distributed and concurrent problem solving along with the advantages of sophisticated schemes for interaction. Examples of interactions include cooperation in working towards a common goal, coordination in organizing problem-solving activity so that harmful interactions are avoided and beneficial possibilities exploited, and negotiation of subproblem constraints so that acceptable performance ensues. It is the flexibility of these social interactions that distinguishes multi-agent systems from more traditional software and which provides the power and excitement to the agent paradigm."

"In recent years, the term multi-agent system refers to all types of software systems composed of multiple semi-autonomous components. The distributed agent system considers how a particular problem can be solved by a number of modules (agents) which cooperate by dividing and sharing the knowledge about the problem and its evolving solution. Research in multi-agent systems is focused on the behaviors of collections of, sometimes already existing, autonomous agents aimed at solving a given problem. A multi-agent system can also be seen as a loosely coupled network of problem solvers that work together on problems beyond the scope of any of the agents individually (Durfee and Lesser 1989). See also the design of knowledge services and ontologies, Section 7.3.3."

The four characteristics of multi-agent problem solving:

"The problem solvers of a multi-agent system, besides being autonomous, may also be of heterogeneous design. Based on analysis by Jennings, Sycara, and Wooldridge (1998), Wooldridge et al. (2006, 2007) there are four important characteristics of multi-agent problem solving. First, each agent has incomplete information and insufficient capabilities for solving the entire problem, and thus can suffer from a limited viewpoint. Second, there is no global system controller for the entire problem solving. Third, the knowledge and input data for the problem is also decentralized, and fourth, the reasoning processes are often asynchronous."

Objects vs agents:

"Interestingly, traditional object-oriented programmers often fail to see anything new in an agent-based system. On consideration of the relative properties of agents and objects, this can be understandable. Objects are defined as computational systems with encapsulated state, they have methods associated with this state that support interactions in an environment, and they communicate by message passing."

"Differences between objects and agents include the fact that objects rarely exhibit control over their own behavior. We do not see agents as invoking methods on one another, but rather as requesting that service actions be performed. Further, agents are designed to have flexible, i.e., reactive, proactive, and

social behavior. Finally, interacting agents are often have their own individual threads of control. All these differences are not to indicate that object-oriented programming languages, such as C++, Java, or CLOS do not offer a suitable medium for building agent systems; quite to the contrary, their power and flexibility make them ideal for this task."

6.2 Examples of and Challenges to an Agent-Oriented Paradigm (Section 7.4.2)

"To make the ideas of the previous section more concrete we next describe a number of application domains where agent-based problem solving is appropriate. We also include references to research within these problem areas."

The seven application areas:

"Manufacturing The manufacturing domain can be modeled as a hierarchy of work areas. There may be work areas for milling, lathing, painting, assembling, and so on. These work areas may then be grouped into manufacturing subsystems, each subsystem a function within the larger manufacturing process. These manufacturing subsystems may then be grouped into a single factory. A larger entity, the company, may then control aspects of each of these factories, for instance, to manage orders, inventory, levels of duplication, profits, and so on. References for agent-based manufacturing include work in production sequencing (Chung and Wu 1997), manufacturing operations (Oliveira et al. 1997), and the collaborative design of products (Cutosky et al. 1993, Darr and Birmingham 1996, and Woldridge et al. 2007)."

"Automated Control Since process controllers are autonomous, reactive, and often distributed systems, it is not surprising that agent models can be important. There is research in controlling transportation systems (Corera et al. 1996), spacecraft control (Schwuttke and Quan 1993), particle beam accelerators (Perriolat et al. 1996, Klein et al. 2000), air traffic control (Ljunberg and Lucas 1992) and others."

"Telecommunications Telecommunication systems are large distributed networks of interacting components that require real-time monitoring and management. Agent-based systems have been used for network control and management (Schoonderwoerd et al. 1997, Adler et al. 1989, Fatima et al. 2006), transmission and switching (Nishibe et al. 1993), and service (Busuoic and Griffiths 1994). See (Veloso et al. 2000) for a comprehensive overview."

"Transportation Systems Traffic systems are almost by definition distributed, situated, and autonomous. Applications include coordinating commuters and cars for carpooling (Burmeister et al. 1997) and cooperative transportation scheduling (Fischer et al. 1996)."

"Information Management The richness, diversity, and complexity of information available to current society is almost overwhelming. Agent systems allow the possibility of intelligent information management, especially on the internet. Both human factors as well as information organization seem to conspire against comfortable access to information. Two critical agent tasks are information filtering, only a tiny portion of the information that we have access to do we actually want, and information gathering, the task of collecting and prioritizing the pieces of information that we actually do want. Applications include WEBMATE (Chen and Sycara 1998), electronic mail filtering (Maes 1994), a web browsing assistant (Lieberman 1995), and an expert locator agent (Kautz et al. 1997). See also knowledge services, Section 7.3.3."

"E-Commerce Commerce currently seems to be driven by human activity: we decide when to buy or sell, the appropriate quantities and price, even what information might be appropriate at the time. Certainly commerce is a domain appropriate for agent models. Although full development of e-commerce agents may be in the future, several systems are already available. For example, programs can now make many buy and sell decisions in the stock market, based on many diverse pieces of distributed information. Several agent systems are being developed for portfolio management (Sycara et al. 1996), shopping assistance (Doorenbos et al. 1997, Krulwich 1996), and interactive catalogues (Schrooten and van de Velde 1997, Takahashi et al. 1997)."

"Interactive Games and Theater Game and theater characters provide a rich interactive simulated environment. These agents can challenge us in war games, finance management scenarios, or even sports. Theater agents play roles analogous to their human counterparts and can offer the illusion of life for working with emotional situations, simulating medical emergencies, or training for diverse tasks. Research in this area includes computer games (Wavish and Graham 1996), interactive personalities (Hayes-Roth 1995, Trappl and Petta 1997), and negotiation (Fatima et al. 2006)."

The challenges:

"There are many other domains, of course, where agent-based approaches are appropriate. Even though the agent technology offers many potential advantages for intelligent problem solving it still has a number of challenges to overcome. The following research questions are based on ideas from Jennings et al. (1998) and Bond and Gasser (1988)."

"How can we systematically formalize, decompose, and allocate problems to agents? Furthermore, how do we appropriately synthesize their results?

How can we enable agents to communicate and interact? What communication languages and protocols are available? What and when is communication appropriate?

How can we ensure that agents act coherently in taking actions or making decisions? How can they address nonlocal effects and avoid harmful agent interactions?

How can individual agents represent and reason about the actions, plans, and knowledge of other agents in order to coordinate with them? How can agents reason about the state of their coordinated processes?

How can disparate viewpoints and conflicting intentions between agents be recognized and coordinated?

How can harmful overall system behavior, such as chaotic or oscillatory action, be recognized and avoided?

How can limited resources, both of the individual agent as well as for the full system, be allocated and managed?

Finally, what are the best hardware platforms and software technologies for the support and development of agent systems?"

"The intelligent software design skills necessary to support agent problem-solving technology are found throughout this book. First, the representational requirements for intelligent problem solving make up a constant theme of our presentation. Second, issues of search, especially heuristic search, may be found in Part II. Third, the area of planning, presented in Section 8.4, provides a methodology of ordering and coordinating subgoals in the process of organizing a problem solution. Fourth, we present the idea of stochastic agents reasoning under conditions of uncertainty in Section 9.3. Finally, issues in learning, automated reasoning, and natural language understanding are addressed in Parts V and IV. These subareas of traditional AI have their roles within the creation of agent architectures."

"There are a number of other design issues for the agent model that go beyond the scope of this book, for example agent communication languages, bidding schemes, and techniques for distributed control. These are addressed in the agent literature (Jennings, 1995; Jennings et al. 1998; Wooldridge 1998; Wooldridge et al. 2006, 2007; Fatima et al. 2005, 2006), and in particular in the appropriate conference proceedings (AAAI, IJCAI, and DAI)."

7. EPILOGUE AND REFERENCES (Section 7.5)

"In this chapter, we have examined many of the major alternatives for knowledge representation, including the use of logic, rules, semantic networks, and frames. We also considered systems with no centralized knowledge base or general-purpose reasoning scheme. Finally, we considered distributed problem-solving with agents. The results of careful study include an increased understanding of the advantages and limitations of each of these approaches to representation. Nonetheless, debate continues over the relative naturalness, efficiency, and appropriateness of each approach. We close this chapter with a brief discussion of several important issues in the area of knowledge representation."

The issues:

"The first of these is the selection and granularity of atomic symbols for representing knowledge. Objects in the world constitute the domain of the mapping; computational objects in the knowledge base are the range. The nature of the atomic elements in the language largely determines what can be described about the world. For example, if a 'car' is the smallest atom of the representation, then the system cannot reason about engines, wheels, or any of the component parts of a car. However, if the atoms correspond to these parts, then a larger structure may be required to represent 'car' as a single concept, possibly introducing a cost in efficiency in manipulating this larger structure."

"Another example of the trade-off in the choice of symbols comes from work in natural language understanding. Programs that use single words as elements of meaning may have difficulty in representing complex concepts that do not have a one-word denotation. There is also difficulty in distinguishing between different meanings of the same word or different words with the same meaning. One approach to this problem is to use semantic primitives or language-independent conceptual units, as the basis for representing the meaning of language. Although this viewpoint avoids the problem of using single words as units of meaning, it involves other trade-offs: many words require complex structures for their definitions; also, by relying on a small set of primitives, many subtle distinctions, such as push vs. shove or yell vs. scream, are difficult to express."

"Exhaustiveness is a property of a knowledge base that is assisted by an appropriate representation. A mapping is exhaustive with respect to a property or class of objects if all occurrences correspond to an explicit element of the representation. Geographic maps are assumed to be exhaustive to some level of detail; a map with a missing city or river would not be well regarded as a navigational tool. Although most knowledge bases are not exhaustive, exhaustiveness with respect to certain properties or objects is a desirable goal. For example, the ability to assume that a representation is exhaustive may allow a planner to ignore possible effects of the frame problem."

"When we describe problems as a state of the world that is changed by a series of actions or events, these actions or events generally change only a few components of the description; the program must be able to infer side effects and implicit changes in the world description. The problem of representing the side effects of actions is called the frame problem. For example, a robot stacking heavy boxes on a truck may have to compensate for the lowering of the truck bed due to the weight of each new box. If a representation is exhaustive, there will be no unspecified side effects, and the frame problem effectively disappears. The difficulty of the frame problem results from the fact that it is impossible to build a completely exhaustive knowledge base for most domains. A representation language should assist the programmer in deciding what knowledge may safely be omitted and help deal with the consequences of this omission. (Section 8.4 discusses the frame problem in planning.)"

"Related to exhaustiveness is the plasticity or modifiability of the representation: the addition of knowledge in response to deficiencies is the primary solution to a lack of exhaustiveness. Because most knowledge bases are not exhaustive, it should be easy to modify or update them. In addition to the syntactic ease of adding knowledge, a representation should help to guarantee the consistency of a knowledge base as information is added or deleted. Inheritance, by allowing properties of a class to be inherited by new instances, is an example of how a representational scheme may help ensure consistency."

"Several systems, including Copycat (Mitchell 1993) and Brooks' robot, Section 7.3, have addressed the plasticity issue by designing network structures that change and evolve as they meet the constraints of the natural world. In these systems, the representation is the result of bottom-up acquisition of new data, constrained by the expectation of the perceiving system. An application of this type of system is analogical reasoning."

"Another useful property of representations concerns the extent to which the mapping between the world and the knowledge base is homomorphic. Here, homomorphic implies a one-to-one correspondence between objects and actions in the world and the computational objects and operations of the language. In a homomorphic mapping the knowledge base reflects the perceived organization of the

domain and can be organized in a more natural and intuitive fashion."

"In addition to naturalness, directness, and ease of use, representational schemes may also be evaluated by their computational efficiency. Levesque and Brachman (1985) discuss the trade-off between expressiveness and efficiency. Logic, when used as a representational scheme, is highly expressive as a result of its completeness; however, systems based on unconstrained logic pay a considerable price in efficiency, see Chapter 14."

"Most of the representation issues just presented relate to any information that is to be captured and used by a computer. There are further issues that the designers of distributed and agent systems must address. Many of these issues relate to making decisions with partial (local) information, distributing the responsibility for accomplishing tasks, agent communication languages, and developing algorithms for the cooperation and information sharing of agents. Many of these issues are presented in Section 7.4."

"Finally, if the philosophical approaches of a distributed environment-based intelligence proposed by Clark (1997), Haugeland (1997), and Dennett (1991, 1995, 2006) are to be realized, where the so-called 'leaky' system utilizes both the environment and other agents as critical media for knowledge storage and use, it may be that entirely new representation languages await invention. Where are the representational tools and support to, as Brooks (1991a) proposes, 'use the world as its own model'?"

Luger's recommended references:

"We conclude with further references for the material presented in Chapter 7. Associationist theories have been studied as models of computer and human memory and reasoning (Selz 1913, 1922; Anderson and Bower 1973; Sowa 1984; Collins and Quillian 1969)."

"SNePS, (Shapiro 1979, Shapiro et al. 2007), one of the earliest associationist/logic based representations, see Section 7.1, has been recently extended as a tool for metacognition, where predicates are allowed to take other predicates as arguments, thus supporting the ability to reason about the structures of a knowledge base."

"Important work in structured knowledge representation languages includes Bobrow and Winograd's representation language KRL (Bobrow and Winograd 1977) and Brachman's (1979) representation language KL-ONE, which pays particular attention to the semantic foundations of structured representations."

"Our overview of conceptual graphs owes a considerable debt to John Sowa's book Conceptual Structures (1984). The reader is referred to this book for details that we have omitted. In its full treatment, conceptual graphs combine the expressive power of predicate calculus, as well as that of modal and higher-order logics, with a rich set of built-in concepts and relations derived from epistemology, psychology, and linguistics."

"There are a number of other approaches of interest to the representation problem. For example, Brachman, Fikes, and Levesque have proposed a representation that emphasizes functional specifications; that is, what information can be asked of or told to a knowledge base (Brachman 1985, Brachman et al. 1985, Levesque 1984)."

"A number of books can help with an advanced study of these issues. Readings in Knowledge Representation by Brachman and Levesque (1985) is a compilation of important, but dated articles in this area. Many of the articles referred to in this chapter may be found there, although we have referenced them in their original source. Representation and Understanding by Bobrow and Collins (1975), Representations of Commonsense Knowledge by Davis (1990), Readings in Qualitative Reasoning about Physical Systems by Weld and deKleer (1990) are all important. Principles of Knowledge Representation and Reasoning, (Brachman et al. 1990), An Overview of Knowledge Representation (Mylopoulos and Levesque 1984), and the proceedings of the annual conferences on AI are helpful resources."

"There is now a considerable number of papers following the directions proposed by Brooks with his subsumption architecture; see especially Brooks (1991a), Brooks and Stein (1994), Maes (1994), and Veloso et al. (2000). There are also contrasting positions, see McGonigle (1990, 1998) and Lewis and Luger (2000). For a philosophical view of distributed and embodied knowledge and intelligence see Clark's Being There (1997)."

"Agent-based research is widespread in modern AI. See Jennings et al. (1998) for an introduction to the area. There are now entire sections of the annual AI conferences (IAAI and IJCAI) devoted to agent research. We recommend reading the recent proceedings of these conferences for more up-to-date discussions of current research issues. We also recommend the conference proceedings and readings in the area of distributed artificial intelligence, or DAI. A summary and references of important application areas for agent-based research was presented in Section 7.4."

"Issues in knowledge representation also lie in the middle ground between AI and Cognitive Science; see Cognitive Science: The Science of Intelligent Systems (Luger 1994) and Being There (Clark 1997). Computation and Intelligence edited by George Luger (1995) is a collection of classic papers that emphasizes the development of many different knowledge representation schemes."

8. EXERCISES (Section 7.6)

The twenty-three exercises exactly as Luger lists them:

"1. Common sense reasoning employs such notions as causality, analogy, and equivalence but uses them in a different way than do formal languages. For example, if we say 'Inflation caused Jane to ask for a raise,' we are suggesting a more complicated causal relationship than that found in simple physical laws. If we say 'Use a knife or chisel to trim the wood,' we are suggesting an important notion of equivalence. Discuss the problems of translating these and other such concepts into a formal language. 2. In Section 7.2.1 we presented some of the arguments against the use of logic for representing common sense knowledge. Make an argument for the use of logic in representing this knowledge. McCarthy and Hayes (1969) have an interesting discussion on this. 3. Translate each of the following sentences into predicate calculus, conceptual dependencies, and conceptual graphs: 'Jane gave Tom an ice cream cone.' 'Basketball players are tall.' 'Paul cut down the tree with an axe.' 'Place all the ingredients in a bowl and mix thoroughly.' 4. Read 'What's in a Link' by Woods (1985). Section IV of this article lists a number of problems in knowledge representation. Suggest a solution to each of these problems using logic, conceptual graphs, and frame notations. 5. Translate the conceptual graphs of Figure 7.28 into English sentences 6. The operations join and restrict define a generalization ordering on conceptual graphs. Show that the generalization relation is transitive. 7. Specialization of conceptual graphs using join and restrict is not a truth-preserving operation. Give an example that demonstrates that the restriction of a true graph is not necessarily true. However, the generalization of a true graph is always true; prove this. 8. Define a specialized representation language to describe the activities of a public library. This language will be a set of concepts and relations using conceptual graphs. Do the same thing for a retail business. What concepts and relations would these two languages have in common? Which would exist in both languages but have a different meaning? 9. Translate the conceptual graphs of Figure 7.28 into predicate calculus. 10. Translate the financial advisor knowledge base, Section 2.4, into conceptual graph form. 11. Give evidence from your own experience that suggests a script-like or frame-like organization of human memory. 12. Using conceptual dependencies, define a script for: a. A fast-food restaurant. b. Interacting with a used-car salesperson. c. Going to the opera. 13. Construct a hierarchy of subtypes for the concept vehicle; for example, subtypes of vehicle might be land_vehicle or ocean-vehicle. These would have further subtypes. Is this best represented as a tree, lattice, or general graph? Do the same for the concept move; for the concept angry. 14. Construct a type hierarchy in which some types do not have a common supertype. Add types to make this a lattice. Could this hierarchy be expressed using tree inheritance? What problems would arise in doing so? 15. Each of the following sequences of characters is generated according to some general rule. Describe a representation that could be used to represent the rules or relationships required to continue each sequence: a. 2,4,6,8, . . . b. 1,2,4,8,16, . . . c. 1,1,2,3,5,8, . . . d. 1,a,2,c,3,f,4, . . . e. o,t,t,f,f,s,s, . . . 16. Examples of analogical reasoning were presented in Section 7.3.2. Describe an appropriate representation and search strategy that would allow for identification of the best answer for this type problem. Create two more example analogies that would work with your proposed representation. Could the two following examples be solved? a. hot is to cold as tall is to {wall, short, wet, hold} b. bear is to pig as chair is to {foot, table, coffee, strawberry} 17. Describe a representation that could be used in a program to solve analogy problems like that in Figure 7.29. This class of problems was addressed by T. G. Evans (1968). The representation must be capable of capturing the essential features of size, shape, and relative position. 18. Brooks' paper (1991a) offers an important discussion on the role of representation in traditional AI. Read this paper, and comment on the limitations of explicit, general-purpose representational schemes. 19. At the end of Section 7.3.1, there are five potential issues that Brooks' subsumption architecture (1991a) must address to offer a successful general-purpose approach to problem-solving. Pick one or more of these and comment on it (them). 20. Identify five properties that an agent language should have to provide an agent-oriented internet service. Comment on the role of Java as a general-purpose agent language for building internet services. Do you see a similar role for CLOS? Why or why not? There is plenty of information on this topic on the internet itself. 21. There were a number of important issues presented near the end of Section 7.4 related to the creation of an agent-oriented solutions to problems. Pick one of these and discuss the issue further. 22. Suppose you were designing an agent system to represent an American football or alternatively a soccer team. For agents to cooperate in a defensive or in a scoring maneuver, they must have some idea of each other's plans and possible responses to situations. How might you build a model of another cooperating agent's goals and plans? 23. Pick one of the application areas for agent architectures summarized in Section 7.4. Choose a research or application paper in that area. Design an organization of agents that could address the problem. Break the problem down to specify issues of responsibility for each agent. List appropriate cooperation procedures."

9. QUICK-REFERENCE TABLE OF KEY DEFINITIONS

Term
Definition

Weak method problem solving
General search strategies using syntax-based heuristics, applicable across domains (Logic Theorist, GPS)

Strong method problem solving
Domain-specific knowledge encoded as if… then… rules (expert systems)

Knowledge representation hypothesis
Intelligent processes comprise structural ingredients that (a) we take to represent propositional knowledge, (b) formally cause the exhibited behaviour

Semantic network
Graph with concept nodes and labelled arcs representing relations

Inheritance
Storing properties at the highest appropriate level; subclasses inherit them

Collins and Quillian model
Hierarchical semantic network; reaction times explain storage at abstract levels

Case frame
Verb node with case links (agent, object, instrument, location, time)

Conceptual dependency theory
Four primitive conceptualizations (ACTs, PPs, AAs, PAs); primitive ACTs; canonical form

Primitive ACTs
ATRANS, PTRANS, PROPEL, MOVE, GRASP, INGEST, EXPEL, MTRANS, MBUILD, CONC, SPEAK, ATTEND

Canonical form
All sentences with the same meaning have identical internal representations

Script
Structured representation of a stereotyped sequence of events (entry conditions, results, props, roles, scenes)

Script match problem
Which of several scripts applies? No algorithm guarantees correct choice

Between-the-lines problem
Unexpected events can break a script

MOP
Memory organization packet — smaller components than scripts, dynamically combined

Frame
Static data structure with named slots for stereotyped entities (Minsky 1975)

Slot
A named field in a frame containing identification, relations, requirements, procedures, defaults, or instance info

Procedural attachment
Linking procedures (demons) to slots

Demon
Procedure invoked as a side effect of another action

Default value
Slot value assumed true when no contrary evidence exists

Conceptual graph
Finite, connected, bipartite graph with concept nodes (boxes) and conceptual relation nodes (ellipses)

Concept node
Represents a concrete or abstract object; labeled with type and referent

Conceptual relation node
Represents an n-ary relation between concepts

Marker
Unique token #n identifying an individual

Generic marker
* indicating an unspecified individual

Named variable
*X indicating the same unspecified individual in two places

Type hierarchy
Partial ordering on types; forms a lattice with universal type T and absurd type ⊥

Canonical formation rules
copy, restrict, join, simplify — not truth-preserving but meaning-preserving

Copy
Produce an exact copy of a graph

Restrict
Replace a generic marker with an individual marker, or a type with its subtype

Join
Combine two graphs by sharing identical concept nodes

Simplify
Remove duplicate relations

Propositional node
A box containing another conceptual graph; used for beliefs and modal concepts

neg
Unary operation asserting a proposition concept as false

or
Relation taking two propositions and representing their disjunction

Ontology
The representation scheme of a software module and the structure supporting its components

Knowledge service
Integration of multiple representational schemes (ontologies) into a user-oriented platform

Subsumption architecture
Layered finite state machines; no central control; higher layers suppress lower layers (Brooks)

AVOID, WANDER, EXPLORE
The three layers of Brooks' early subsumption architecture

Emergence
Overall intelligent behaviour arising from interactions of simpler agents

Copycat
Architecture with workspace, slipnet, coderack, mediated by temperature (Mitchell/Hofstadter)

Workspace
Global structure for building perceptual structures (Copycat)

Slipnet
Dynamically deformable semantic network with activation levels (Copycat)

Coderack
Priority-based probabilistic queue of codelets (Copycat)

Codelet
Small piece of executable code exploring part of the solution (Copycat)

Temperature
Measure of perceptual organization; controls randomness (Copycat)

Agent
Element of a society that can perceive aspects of its environment and affect it directly or through cooperation

Multi-agent system
Computer program with problem solvers that are situated, autonomous, flexible, and social

Situated
Receives input from environment and can effect changes within it

Autonomous
Interacts with environment without direct intervention

Flexible
Responsive (reacts) and proactive (plans)

Social
Interacts with other software or human agents

Four characteristics of multi-agent systems
Incomplete information/limited viewpoint; no global controller; decentralised knowledge/data; asynchronous reasoning

Seven agent application areas
Manufacturing, automated control, telecommunications, transportation,

information management, e-commerce, interactive games/theatre

10. WHAT IS NOT IN CHAPTER 7

Topic	Belongs to
Expert system architecture (detail)	Chapter 8
Rule-based expert systems (detail)	Chapter 8
Model-based and case-based reasoning	Chapter 8
Planning (detail)	Chapter 8
Reasoning under uncertainty (certainty factors, fuzzy logic, Dempster-Shafer)	Chapter 9
Bayesian belief networks	Chapter 9
Machine learning algorithms	Chapters 10–13
Connectionist networks (detail)	Chapter 11
Genetic algorithms (detail)	Chapter 12
Resolution theorem proving	Chapter 14
Natural language parsing (detail)	Chapter 15
Constructivist epistemology	Chapter 16

End of Chapter 7 notes, based strictly on Luger’s textbook.

Chapter 8: Strong Method Problem Solving

Complete Study Notes — Based strictly on Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th ed., Chapter 8

1. WHY CHAPTER 8 EXISTS

Luger opens the chapter with two quotations:
"The first principle of knowledge engineering is that the problem-solving power exhibited by an intelligent agent's performance is primarily the consequence of its knowledge base, and only secondarily a consequence of the inference method employed. Expert systems must be knowledge-rich even if they are methods-poor. This is an important result and one that has only recently become well understood in AI. For a long time AI has focused its attentions almost exclusively on the development of clever inference methods; almost any inference method will do. The power resides in the knowledge. —EDWARD FEIGENBAUM, Stanford University"
"Nam et ipsa scientia potestas est (the knowledge is power). —FRANCIS BACON (1620)"
"We continue studying issues of representation and intelligence by considering an important component of AI: knowledge-intensive or strong method problem solving."
"Human experts are able to perform at a successful level because they know a lot about their areas of expertise. This simple observation is the underlying rationale for the design of strong method or knowledge-based problem solvers (Introduction, Part III). An expert system, for example, uses knowledge specific to a problem domain to provide 'expert quality' performance in that application area. Generally, expert system designers acquire this knowledge with the help of human domain experts, and the system emulates the human expert's methodology and performance. As with skilled humans, expert systems tend to be specialists, focusing on a narrow set of problems. Also, like humans, their knowledge is both theoretical and practical: the human experts that provide the system's knowledge have generally augmented their own theoretical understanding of the problem domain with tricks, shortcuts, and heuristics for using the knowledge they have gained through problem-solving experience."
The three general characteristics of expert systems:
"Because of their heuristic, knowledge-intensive nature, expert systems generally:
Support inspection of their reasoning processes, both in presenting intermediate steps and in answering questions about the solution process.
Allow easy modification in adding and deleting skills from the knowledge base.
Reason heuristically, using (often imperfect) knowledge to get useful solutions."
Why inspection is important:
"The reasoning of an expert system should be open to inspection, providing information about the state of its problem solving and explanations of the choices and decisions that the program is making. Explanations are important for a human expert, such as a doctor or an engineer, if he or she is to accept the recommendations from a computer. Indeed, few human experts will accept advice from another human, let alone a machine, without understanding the justifications for it."
Why modifiability is important:
"The exploratory nature of AI and expert system programming requires that programs be easily prototyped, tested, and changed. AI programming languages and environments are designed to support this iterative development methodology. In a pure production system, for example, the modification of a single rule has no global syntactic side effects. Rules may be added or removed without requiring further changes to the larger program. Expert system designers often comment that easy modification of the knowledge base is a major factor in producing a successful program."
Why heuristics are important:
"A further feature of expert systems is their use of heuristic problem-solving methods. As expert system designers have discovered, informal 'tricks of the trade' and 'rules of thumb' are an essential complement to the standard theory presented in textbooks and classes. Sometimes these rules augment theoretical knowledge in understandable ways; often they are simply shortcuts that have, empirically, been shown to work."
The eight general problem categories (from Waterman 1986):
"Expert systems are built to solve a wide range of problems in domains such as medicine, mathematics, engineering, chemistry, geology, computer science, business, law, defense, and education. These programs address a variety of problems; the following list, from Waterman (1986), is a useful summary of general expert system problem categories."
"Interpretation—forming high-level conclusions from collections of raw data. Prediction—projecting probable consequences of given situations. Diagnosis—determining the cause of malfunctions in complex situations based on observable symptoms. Design—finding a configuration of system components that meets performance goals while satisfying a set of design constraints. Planning—devising a sequence of actions that will achieve a set of goals given certain starting conditions and run-time constraints. Monitoring—comparing a system's observed behavior to its expected behavior. Instruction—assisting in the education process in technical domains. Control—governing the behavior of a complex environment."
The chapter's roadmap:
"In this chapter we first examine the technology that makes knowledge-based problem solving possible. Successful knowledge engineering must address a range of problems, from the choice of an appropriate application domain to the acquisition and formalization of problem-solving knowledge. In Section 8.2 we introduce rule-based systems and present the production system as a software architecture for solution and explanation processes. Section 8.3 examines techniques for model-based and case-based reasoning. Section 8.4 presents planning, a process of organizing pieces of knowledge into a consistent sequence of actions that will accomplish a goal. Chapter 9 presents techniques for reasoning in uncertain situations, an important component of strong method problem solvers."

2. OVERVIEW OF EXPERT SYSTEM TECHNOLOGY (Section 8.1)

2.1 The Design of Rule-Based Expert Systems (Section 8.1.1)

"Figure 8.1 shows the modules that make up a typical expert system. The user interacts with the system through a user interface that simplifies communication and hides much of the complexity, such as the internal structure of the rule base. Expert system interfaces employ a variety of user styles, including question-and-answer, menu-driven, or graphical interfaces. The final decision on the interface type is a compromise between user needs and the requirements of the knowledge base and inferencing system."
The components:
"The heart of the expert system is the knowledge base, which contains the knowledge of a particular application domain. In a rule-based expert system this knowledge is most often represented in the form of if…then… rules, as in our examples of Section 8.2. The knowledge base contains both general knowledge as well as case-specific information."
"The inference engine applies the knowledge to the solution of actual problems. It is essentially an interpreter for the knowledge base. In the production system, the inference engine performs the recognize-act control cycle. The procedures that implement the control cycle are separate from the production rules themselves. It is important to maintain this separation of the knowledge base and inference engine for several reasons:
This separation makes it possible to represent knowledge in a more natural fashion. If … then… rules, for example, are closer to the way in which humans describe their problem-solving skills than is lower-level computer code.
Because the knowledge base is separated from the program's lower-level control structures, expert system builders can focus on capturing and organizing problem-solving knowledge rather than on the details of its computer implementation.
Ideally, the separation of knowledge and control allows changes to be made in one part of the knowledge base without creating side effects in others.
The separation of the knowledge and control elements of the program allows the same control and interface software to be used in a variety of systems. The expert system shell has all the components of Figure 8.1 except that the knowledge base and case-specific data are empty and can be added for a new application. The broken lines of Figure 8.1 indicate the shell modules."
Case-specific data:
"The expert system must keep track of case-specific data: the facts, conclusions, and other information relevant to the case under consideration. This includes the data given in a problem instance, partial conclusions, confidence measures of conclusions, and dead ends in the search process. This information is separate from the general knowledge base."
The explanation subsystem:
"The explanation subsystem allows the program to explain its reasoning to the user. These explanations include justifications for the system's conclusions, in response to how queries (Section 8.2), explanations of why the system needs a particular piece of data, why queries (Section 8.2), and, where useful, tutorial explanations or deeper theoretical justifications of the program's actions."
The knowledge-base editor:
"Many systems also include a knowledge-base editor. Knowledge-base editors help the programmer locate and correct bugs in the program's performance, often accessing the information provided by the explanation subsystem. They also may assist in the addition of new knowledge, help maintain correct rule syntax, and perform consistency checks on any updated knowledge base."
Expert system shells:
"An important reason for the decrease in design and deployment times for current expert systems is the availability of expert system shells. NASA has created CLIPS, JESS is available from Sandia National Laboratories, and we offer shells in Lisp and Prolog in our supplementary materials. Unfortunately, shell programs do not solve all of the problems involved in building expert systems. Although the separation of knowledge and control, the modularity of the production system architecture, and the use of an appropriate knowledge representation language all help with the building of an expert system, the acquisition and formalization of domain knowledge still remain difficult tasks."

2.2 Selecting a Problem and the Knowledge Engineering Process (Section 8.1.2)

"Expert systems involve a considerable investment of money and human effort. Attempts to solve a problem that is too complex, too poorly understood, or otherwise unsuited to the available technology can lead to costly and embarrassing failures. Researchers have developed guidelines to determine whether a problem is appropriate for expert system solution:"
The seven guidelines:
"1. The need for the solution justifies the cost and effort of building an expert system. Many expert systems have been built in domains such as mineral exploration, business, defense, and medicine where a large potential exists for savings in terms of money, time, and human life.
Human expertise is not available in all situations where it is needed. In geology, for example, there is a need for expertise at remote mining and drilling sites. Often, geologists and other engineers find themselves traveling large distances to visit sites, with resulting expense and wasted time. By placing expert systems at remote sites, many problems may be solved without needing a visit.
The problem may be solved using symbolic reasoning. Problem solutions should not require physical dexterity or perceptual skill. Robots and vision systems currently lack the sophistication and flexibility of humans.
The problem domain is well structured and does not require common sense reasoning. Highly technical fields have the advantage of being well studied and formalized: terms are well defined and domains have clear and specific conceptual models. In contrast, common sense reasoning is difficult to automate.
The problem may not be solved using traditional computing methods. Expert system technology should not be used where unnecessary. If a problem can be solved satisfactorily using more traditional techniques, then it is not a candidate.

Cooperative and articulate experts exist. The knowledge used by expert systems comes from the experience and judgment of humans working in the domain. It is important that these experts be both willing and able to share knowledge.

The problem is of proper size and scope. For example, a program that attempted to capture all of the expertise of a medical doctor would not be feasible; a program that advised MDs on the use of a particular piece of diagnostic equipment or a particular set of diagnoses would be more appropriate."

The three key people:

"The primary people involved in building an expert system are the knowledge engineer, the domain expert, and the end user. The knowledge engineer is the AI language and representation expert. His or her main task is to select the software and hardware tools for the project, help the domain expert articulate the necessary knowledge, and implement that knowledge in a correct and efficient knowledge base. Often, the knowledge engineer is initially ignorant of the application domain."

"The domain expert provides the knowledge of the problem area. The domain expert is generally someone who has worked in the domain area and understands its problem-solving techniques, such as shortcuts, handling imprecise data, evaluating partial solutions, and all the other skills that mark a person as an expert problem solver. The domain expert is primarily responsible for spelling out these skills to the knowledge engineer."

"As in most applications, the end user determines the major design constraints. Unless the user is happy, the development effort is by and large wasted. The skills and needs of the user must be considered throughout the design cycle: Will the program make the user's work easier, quicker, more comfortable? What level of explanation does the user need? Can the user provide correct information to the system? Is the interface appropriate? Does the user's work environment place restrictions on the program's use? An interface that required typing, for example, would not be appropriate for use in the cockpit of a fighter."

The development cycle:

"Like most AI programming, building expert systems requires a nontraditional development cycle based on early prototyping and incremental revision of the code. Generally, work on the system begins with the knowledge engineer attempting to gain some familiarity with the problem domain. This helps in communicating with the domain expert. This is done in initial interviews with the expert and by observing experts during the performance of their job. Next, the knowledge engineer and expert begin the process of extracting the expert's problem-solving knowledge. This is often done by giving the domain expert a series of sample problems and having him or her explain the techniques used in their solution. Video and/or audio tapes are often essential for capturing this process."

"It is often useful for the knowledge engineer to be a novice in the problem domain. Human experts are notoriously unreliable in explaining exactly what goes on in solving a complex problem. Often they forget to mention steps that have become obvious or even automatic to them after years of work in their field. Knowledge engineers, by virtue of their relative naiveté in the domain, can spot these conceptual jumps and ask for help."

"Once the knowledge engineer has obtained a general overview of the problem domain and gone through several problem-solving sessions with the expert, he or she is ready to begin actual design of the system: selecting a way to represent the knowledge, such as rules or frames, determining the search strategy, forward, backward, depth-first, best-first etc., and designing the user interface. After making these design commitments, the knowledge engineer builds a prototype."

"This prototype should be able to solve problems in a small area of the domain and provide a test bed for preliminary design assumptions. Once the prototype has been implemented, the knowledge engineer and domain expert test and refine its knowledge by giving it problems to solve and correcting its shortcomings. Should the assumptions made in designing the prototype prove correct, the prototype can be incrementally extended until it becomes a final system."

"Expert systems are built by progressive approximations, with the program's mistakes leading to corrections or additions to the knowledge base. In a sense, the knowledge base is 'grown' rather than constructed. Figure 8.2 presents a flow chart describing the exploratory programming development cycle. This approach to programming was investigated by Seymour Papert with his LOGO language (Papert 1980) as well as Alan Kay's work with Smalltalk at Xerox PARC. The LOGO philosophy argues that watching the computer respond to the improperly formulated ideas represented by the code leads to their correction (being debugged) and clarification with more precise code. This process of trying and correcting candidate designs is common to expert systems development, and contrasts with such neatly hierarchical processes as top-down design."

"It is also understood that the prototype may be thrown away if it becomes too cumbersome or if the designers decide to change their basic approach to the problem. The prototype lets program builders explore the problem and its important relationships by actually constructing a program to solve it. After this progressive clarification is complete, they can then often write a cleaner version, usually with fewer rules."

"The second major feature of expert system programming is that the program need never be considered 'finished.' A large heuristic knowledge base will always have limitations. The modularity of the production system model make it natural to add new rules or make up for the shortcomings of the present rule base at any time."

2.3 Conceptual Models and Their Role in Knowledge Acquisition (Section 8.1.3)

"Figure 8.3 presents a simplified model of the knowledge acquisition process that will serve as a useful 'first approximation' for understanding the problems involved in acquiring and formalizing human expert performance. The human expert, working in an application area, operates in a domain of knowledge, skill, and practice. This knowledge is often vague, imprecise, and only partially verbalized. The knowledge engineer must translate this informal expertise into a formal language suited to a computational system. A number of important issues arises in the process of formalizing human skilled performance:"

The four issues:

"1. Human skill is often inaccessible to the conscious mind. As Aristotle points out in his Ethics, 'what we have to learn to do, we learn by doing.' Skills such as those possessed by medical doctors are learned as much in years of internship and residency, with their constant focus on patients, as they are in physiology lectures, where emphasis is on experiment and theory. Delivery of medical care is to a great extent practice-driven. After years of performance these skills become highly integrated and function at a largely unconscious level. It may be difficult for experts to describe exactly what they are doing in problem solving.

Human expertise often takes the form of knowing how to cope in a situation rather than knowing what a rational characterization of the situation might be, of developing skilled performance mechanisms rather than a fundamental understanding of what these mechanisms are. An obvious example of this is riding a unicycle: the successful rider is not, in real time, consciously solving multiple sets of simultaneous differential equations to preserve balance; rather she is using an intuitive combination of feelings of 'gravity,' 'momentum,' and 'inertia' to form a usable control procedure.

We often think of knowledge acquisition as gaining factual knowledge of an objective reality, the so-called 'real world'. As both theory and practice have shown, human expertise represents an individual's or a community's model of the world. Such models are as influenced by convention, social processes, and hidden agendas as they are by empirical methodologies.

Expertise changes. Not only do human experts gain new knowledge, but also existing knowledge may be subject to radical reformulation, as evidenced by ongoing controversies in both scientific and social fields."

The conceptual model:

"Consequently, knowledge engineering is difficult and should be viewed as spanning the life cycle of any expert system. To simplify this task, it is useful to consider, as in Figure 8.4, a conceptual model that lies between human expertise and the implemented program. By a conceptual model, we mean the knowledge engineer's evolving conception of the domain knowledge. Although this is undoubtedly different from the domain expert's, it is this model that actually determines the construction of the formal knowledge base."

"Because of the complexity of most interesting problems, we should not take this intermediate stage for granted. Knowledge engineers should document and make public their assumptions about the domain through common software engineering methodologies. An expert system should include a requirements document; however, because of the constraints of exploratory programming, expert system requirements should be treated as co-evolving with the prototype. Data dictionaries, graphic representations of state spaces, and comments in the code itself are all part of this model. By publicizing these design decisions, we reduce errors in both the implementation and maintenance of the program."

"Knowledge engineers should save recordings of interviews with domain experts. Often, as knowledge engineers' understanding of the domain grows, they form new interpretations or discover new information about the domain. The recordings, along with documentation of the interpretation given them, play a valuable role in reviewing design decisions and testing prototypes. Finally, this model serves an intermediate role in the formalization of knowledge. The choice of a representation language exerts a strong influence on a knowledge engineer's model of the domain."

"The conceptual model is not formal or directly executable on a computer. It is an intermediate design construct, a template to begin to constrain and codify human skill. It can, if the knowledge engineer uses a predicate calculus model, begin as a number of simple networks representing states of reasoning through typical problem-solving situations. Only after further refinement does this network become explicit if…then… rules."

The questions asked in the context of a conceptual model:

"Questions often asked in the context of a conceptual model include: Is the problem solution deterministic or search-based? Is the reasoning data-driven, perhaps with a generate and test flavor, or goal-driven, based on a small set of likely hypotheses about situations? Are there stages of reasoning? Is the domain well understood and capable of providing deep predictive models, or is all problem-solving knowledge essentially heuristic? Can we use examples of past problems and their solutions to solve future problems directly, or must we first convert these examples into general rules? Is the knowledge exact or is it 'fuzzy' and approximate, lending itself to numeric ratings of certainty (Chapter 9)? Will reasoning strategies allow us to infer stable facts about the domain, or do change and uncertainty within the system require nonmonotonic reasoning, that is, the ability to make assertions about the domain that may later be modified or retracted (Section 9.1)? Finally, does the structure of the domain knowledge require us to abandon rule-based inference for alternative schemes such as neural networks or genetic algorithms (Part IV)?"

"The eventual users' needs should also be addressed in the context of the conceptual model: What are their expectations of the eventual program? Where is their level of expertise: novice, intermediate, or expert? What levels of explanation are appropriate? What interface best serves their needs?"

"Based on the answers to these and related questions, the knowledge obtained from domain experts, and the resulting conceptual model, we begin development of the expert system. Because the production system, first presented in Chapter 6, offers a number of inherent strengths for organizing and applying knowledge, it is often used as the basis for knowledge representation in rule-based expert systems."

3. RULE-BASED EXPERT SYSTEMS (Section 8.2)

"Rule-based expert systems represent problem-solving knowledge as if…then… rules. This approach lends itself to the architecture of Figure 8.1, and is one of the oldest techniques for representing domain knowledge in an expert system. It is also one of the most natural, and remains widely used in practical and experimental expert systems."

3.1 The Production System and Goal-Driven Problem Solving (Section 8.2.1)

"The architecture of rule-based expert systems may be best understood in terms of the production system model for problem solving presented in Part II. The parallel between the two is more than an analogy: the production system was the intellectual precursor of modern expert system architectures, where application of production rules leads to refinements of understanding of a particular problem situation. When Newell and Simon developed the production system, their goal was to model human performance in problem solving."

"If we regard the expert system architecture in Figure 8.1 as a production system, the domain-specific knowledge base is the set of production rules. In a rule-based system, these condition action pairs are represented as if…then… rules, with the premises of the rules, the if portion, corresponding to the condition, and the conclusion, the then portion, corresponding to the action: when the condition is satisfied, the expert system takes the action of asserting the conclusion as true. Case-specific data can be kept in the working memory. The inference engine implements the recognize-act cycle of the production system; this control may be either data-driven or goal-driven."

Forward vs backward search:

"Many problem domains seem to lend themselves more naturally to forward search. In an interpretation problem, for example, most of the data for the problem are initially given and it is often difficult to formulate an hypotheses or goal. This suggests a forward reasoning process in which the facts are placed in working memory and the system searches for an interpretation, as first presented in Section 3.2."

"In a goal-driven expert system, the goal expression is initially placed in working memory. The system matches rule conclusions with the goal, selecting one rule and placing its premises in the working memory. This corresponds to a decomposition of the problem's goal into simpler subgoals. The process continues in the next iteration of the production system, with these premises becoming the new goals to match against rule conclusions. The system thus works back from the original goal until all the subgoals in working memory are known to be true, indicating that the hypothesis has been verified. Thus, backward search in an expert system corresponds roughly to the process of hypothesis testing in human problem solving, as also first presented in Section 3.2."

"In an expert system, subgoals can be solved by asking the user for information. Some expert systems allow the system designer to specify which subgoals may be solved by asking the user. Others simply ask the user about any subgoals that fail to match rules in the knowledge base; i.e., if the program cannot infer the truth of a subgoal, it asks the user."

The automotive diagnosis example:

"As an example of goal-driven problem solving with user queries, we next offer a small expert system for analysis of automotive problems. This is not a full diagnostic system, as it contains only four very simple rules. It is intended as an example to demonstrate goal-driven rule chaining, the integration of new data, and the use of explanation facilities:"

"Rule 1: if the engine is getting gas, and the engine will turn over, then the problem is spark plugs."

"Rule 2: if the engine does not turn over, and the lights do not come on then the problem is battery or cables."

"Rule 3: if the engine does not turn over, and the lights do come on then the problem is the starter motor."

"Rule 4: if there is gas in the fuel tank, and there is gas in the carburetor then the engine is getting gas."

"To run this knowledge base under a goal-directed control regime, place the top-level goal, the problem is X, in working memory as shown in Figure 8.5. X is a variable that can match with any phrase, as an example, the problem is battery or cables; it will become bound to the solution when the problem is solved."

"Three rules match with this expression in working memory: rule 1, rule 2, and rule 3. If we resolve conflicts in favor of the lowest-numbered rule, then rule 1 will fire. This causes X to be bound to the value spark plugs and the premises of rule 1 to be placed in the working memory as in Figure 8.6. The system has thus chosen to explore the possible hypothesis that the spark plugs are bad. Another way to look at this is that the system has selected an or branch in an and/or graph (Chapter 3)."

"Note that there are two premises to rule 1, both of which must be satisfied to prove the conclusion true. These are and branches of the search graph representing a decomposition of the problem (finding whether the problem is spark plugs) into two subproblems (finding whether the engine is getting gas and whether the engine will turn over). We may then fire rule 4, whose conclusion matches with the engine is getting gas, causing its premises to be placed in working memory as in Figure 8.7."

"At this point, there are three entries in working memory that do not match with any rule conclusions. Our expert system will, in this situation, query the user directly about these subgoals. If the user confirms all three of these as true, the expert system will have successfully determined that the car will not start because the spark plugs are bad. In finding this solution, the system has searched the leftmost branch of the and/or graph presented in Figure 8.8."

"This is, of course, a very simple example. Not only is its automotive knowledge limited at best, but it also ignores a number of important aspects of real implementations. The rules are phrased in English, rather than a formal language. On finding a solution, a real expert system will tell the user its diagnosis (our model simply stops). Also, we should maintain enough of a trace of the reasoning to allow backtracking if necessary. In our example, had we failed to determine that the spark plugs were bad, we would have needed to back up to the top level and try rule 2 instead. Notice that this information is implicit in the ordering of subgoals in working memory of Figure 8.7 and in the graph of Figure 8.8. In spite of its simplicity, however, this example underscores the importance of production system based search and its representation by the and/or graph as a foundation for rule-based expert systems."

"Earlier we emphasized that an expert system needed to be open to inspection, easily modified, and heuristic in nature. The production system architecture is an important factor in each of these requirements. Ease of modification, for example, is supported by the syntactic independence of production rules: each rule is a chunk of knowledge that can be independently modified. There are semantic constraints, however, in that the meanings of the individual rules are related. These must be reconciled in any editing/modification process. We next discuss the generation of explanations and the monitoring of inference."

3.2 Explanation and Transparency in Goal-Driven Reasoning (Section 8.2.2)

"The production system implements a graph search. The explanation routines of Figure 8.1 keep track of the graph search and use a trace of the search to answer user queries. The production rules themselves document each step of the reasoning process."

"In general, the two questions answered by rule-based expert systems are why? and how? A why query occurs when the program asks the user for information and the user responds with why, meaning 'why did you ask for that information?' The answer is the current rule that the production system is attempting to fire. The answer to 'how did you get that result?' is the sequence of rules that were used to conclude a goal."

The worked dialogue:

"Suppose our automotive expert system has an explanation facility, and assume that the consultation has reached the stage represented by the working memory configuration in Figure 8.7. The following dialogue begins with the computer asking the user about the goals present in working memory. User responses are in bold type."

"gas in fuel tank? yes gas in carburetor? yes engine will turn over? why"

"Here the user enters a why query, i.e., why do you want to know whether the engine will turn over? The system responds with a literal presentation of the rule under question:"

"It has been established that: 1. the engine is getting gas, therefore if 2. the engine will turn over, then the problem is spark plugs."

"Because the user answered yes to the first two questions, rule 4 was able to fire, verifying the first premise of rule 1. When the why query was made, the system was trying to solve the second premise of rule 1. Note that the explanation for the system's behavior is just a restatement of the current rule under consideration. However, this provides an adequate explanation for why the program wants to know whether the engine will turn over."

"In the response to the why query, the system states that it has concluded that the engine is getting gas. Suppose the user then asks:"

"how the engine is getting gas"

"The system will respond to how it concluded that the engine is getting gas. The response is a trace of the reasoning that led to this conclusion, working back from the goal along the rules that support it to the user responses:"

"This follows from rule 4: if gas in fuel tank, and gas in carburetor then engine is getting gas. gas in fuel tank was given by the user gas in carburetor was given by the user"

"Once again, the production system architecture provides an essential basis for these explanations. Each cycle of the control loop selects and fires another rule. The program may be stopped after each cycle and inspected. Because each rule represents a complete chunk of problem-solving knowledge, the current rule provides a context for the explanation. Contrast this production system approach with more traditional program architectures: if we stop a C, C++, or Java program in mid execution, it is doubtful that the current statement will have much meaning."

"In summary, the knowledge-based system answers why queries by showing the current rule that it is attempting to fire; it answers how queries by giving a trace of the reasoning that led to a goal or subgoal. Although the mechanisms are conceptually simple, they can exhibit remarkable explanatory power if the knowledge base is organized in a logical fashion. The supplementary materials in Java, Lisp and Prolog, demonstrate the use of rule stacks and proof trees to implement these explanations."

"If explanations are to behave logically, it is important not only that the knowledge base gets the correct answer but also that each rule corresponds to a single logical step in the problem solving process. If a knowledge base combines several steps into a single rule, or if it breaks up the rules in an arbitrary fashion, it may get correct answers but seem vague, arbitrary, or illogical in responding to how and why queries. This can not only undermine the user's faith in the system but can also make the program much more difficult for its builders to understand and modify."

3.3 Using the Production System for Data-Driven Reasoning (Section 8.2.3)

"The automobile diagnosis demonstration of Section 8.2.1 illustrated the use of a production system to implement goal-driven search. Search was also depth-first in that it searched each subgoal found in the rule base exhaustively before moving onto any sibling goals. As we saw in Section 6.3, however, the production system is also an ideal architecture for data-driven reasoning. Example 6.3.1 demonstrated this process with the 8-puzzle and Examples 6.3.2 and 6.3.3 with the Knight's Tour. In each of these problems we did conflict resolution by taking the first rule found in the knowledge base and then followed the results of that rule. This gave the search a depth-first flavor, although there was no mechanism, such as backtracking, to handle the problem of 'dead ends' in the search space."

Breadth-first data-driven reasoning:

"Breadth-first search is even more common in data-driven reasoning. The algorithm for this is simple: compare the contents of working memory with the conditions of each rule in the rule base according to the order of the rules in the rule base. If the data in working memory supports a rule's firing the result is placed in working memory and then control moves on to the next rule. Once all rules have been considered, search starts again at the beginning of the rule set."

"Consider, for example, the automobile diagnosis problem and the rules of Section 8.2.1. If a piece of information that makes up (part of) the premise of a rule is not the conclusion of some other rule then that fact will be deemed 'askable' when control comes to the situation (rule) where that information is needed. For example, the engine is getting gas is not askable in the premise of rule 1, because that fact is a conclusion of another rule, namely rule 4."

"The breadth-first, data-driven example begins as in Figure 8.5, with no information in working memory, as in Figure 8.9. We first examine premises of the four rules in order to see what

information is 'askable'. The premise, the engine is getting gas, is not askable, so rule 1 fails and control moves to rule 2. The engine does not turn over is askable. Suppose the answer to this query is false, so the engine will turn over is placed in working memory, as in Figure 8.10."

"But rule 2 fails, since the first of two and premises is false, and consideration moves to rule 3, where again, the first premise fails. At rule 4, both premises are askable. Suppose the answer to both questions is true. Then there is gas in the fuel tank and there is gas in the carburetor are placed in working memory, as is the conclusion of the rule, the engine is getting gas."

"At this point all the rules have been considered so search now returns, with the new contents of working memory, to consider the rules in order a second time. As is seen in Figure 8.11, when the working memory is matched to rule 1, its conclusion, the problem is spark plugs, is placed in working memory. In this example no more rules will match and fire, and the problem-solving session is completed. A graph of the search process, with the information content of working memory (WM) as the nodes of the graph, is presented as Figure 8.12."

Opportunistic search:

"An important refinement on the breadth-first search strategy used in the previous example is what is called opportunistic search. This search strategy is simple: whenever a rule fires to conclude new information, control moves to consider those rules which have that new information as a premise. This makes any new concluded information (search does not change as the result of 'askable' premises) the controlling force for finding the next rules to fire. This is termed opportunistic because each conclusion of new information drives the search. By the accident of rule ordering, the very simple example just presented was also opportunistic."

Explanation in forward chaining:

"We conclude this section on data-driven reasoning with several comments on explanation and transparency in forward chaining systems. First, in comparison with goal-driven systems, Sections 8.2.1–2, data-driven reasoning is much less 'focused' in its search. The reason for this is obvious: in a goal-driven system, reasoning is in pursuit of a particular goal; that goal is broken into subgoals that support the top-level goal and these subgoals may be even further broken down. As a result, search is always directed through this goal and subgoal hierarchy. In data-driven systems this goal orientation does not exist. Rather, the search moves about the tree depending only on rule order and the discovery of new information. As a result, the progress of search can often seem to be very diffuse and unfocused."

"Second, and as a direct result of the first point, the explanation available to the user at any time in the search is quite limited. There is a rule-level accountability in that, when the user asks why some information is required, the why query in Section 8.2.2, the rule under consideration is presented. The explanation cannot go much further, however, unless explicit rule tracking is added into the system, say with opportunistic search. The diffuse nature of the data-driven search makes this difficult to do. Finally, when a goal is achieved, getting a full how explanation for that goal is also difficult. About the only thing that can be used as a partial and very limited explanation is a presentation of the contents of the working memory or a list of rules fired. But again, these will not offer the consistent focused accountability we saw with goal-driven reasoning."

3.4 Heuristics and Control in Expert Systems (Section 8.2.4)

"Because of the separation of the knowledge base and the inference engine, and the fixed control regimes provided by the inference engine, an important method for the programmer to control search is through the structuring and ordering of the rules in the knowledge base. This micro-managing of the rule set offers an important opportunity, especially as the control strategies required for expert-level problem-solving tend to be domain specific and knowledge intensive. Although a rule of the form if p, q, and r then s resembles a logical expression, it may also be interpreted as a series of procedures or steps for solving a problem: to do s, first do p, then do q, then do r. The role of rule and premise ordering was implicit in the examples of Section 8.2 just presented."

"This procedural method of rule interpretation is an essential component of practical knowledge use and often reflects the human expert's solution strategy. For example, we can order the premises of a rule so that what is most likely to fail or is easiest to confirm is tried first. This gives the opportunity of eliminating a rule (and hence a portion of the search space) as early as possible. Rule 1 in the automotive example tries to determine whether the engine is getting gas before it asks if the engine turns over. This is inefficient, in that trying to determine whether the engine is getting gas invokes another rule and eventually asks the user two questions. By reversing the order of the premises, a negative response to the query 'engine will turn over?' eliminates this rule from consideration before the more involved condition is examined, thus making the system more efficient."

"It also makes more sense to determine whether the engine is turning over before checking to see whether it is getting gas; if the engine won't turn over it doesn't matter whether or not it is getting gas! In Rule 4, the user is asked to check the fuel tank before checking the carburetor for gas. In this situation, it is performing the easier check first."

"There is an important point here, if the overall system is to be more efficient all aspects must be considered: rule order, organization of premises, cost of tests, amount of search eliminated through the answers to tests, the way the majority of cases occur, and so on. Thus the planning of rule order, the organization of a rule's premises, and the costs of different tests are all fundamentally heuristic in nature. In most rule-based expert systems these heuristic choices reflect the approaches taken by the human expert, and indeed can have erroneous results. In our example, if the engine is getting gas and turning over, the problem may be a bad distributor rather than bad spark plugs."

Data-driven control:

"Data-driven reasoning provides additional problems and opportunities for control of reasoning. Some of these include the high-level heuristics, such as refraction, recency (opportunistic search), and specificity presented in Section 6.3.3. A more domain-specific approach groups sets of rules according to stages of the solution process. For example, in diagnosing car problems we might create the four distinct stages of 1) organize situation, 2) collect the data, 3) do the analysis (there may be more than one problem with the car), and finally, 4) report to the user the conclusions and recommended fixes."

"This staged problem solving can be accomplished by creating descriptors for each stage of the solution and placing that description as the first premise in all rules that belong to that stage. For example, we might begin by placing the assertion organize situation in working memory. If no other stage descriptions occur in working memory, then only rules that have organize situation in their set of premises will fire. Of course, this should be the first premise for each of these rules. We would move to the next stage by having the last rule to fire in the organizational stage remove (retract) the fact that stage is organize solution and assert the new fact that the stage is data collection. All the rules in the data collection stage would then have their first premise IF stage is data collection and . . .. When the data collection stage is finished, the last rule would retract this fact, and assert the data analysis fact into working memory to match only those rules whose premises begin with the fact IF stage is data analysis . . .."

The RETE algorithm:

"In our discussion so far, we have described the behavior of the production system in terms of exhaustive considerations of the rule base. Although this is expensive, it captures the intended semantics of the production system. There are, however, a number of algorithms such as RETE (Forgy 1982) that can be used to optimize search for all potentially usable rules. Essentially, the RETE algorithm compiles rules into a network structure that allows the system to match rules with data by directly following a pointer to the rule. This algorithm greatly speeds execution, especially for larger rule sets, while retaining the semantic behavior we have described in this section."

The summary:

"To summarize, rules are the oldest approach to knowledge representation in expert systems, and remain an important technique for building knowledge-intensive problem solvers. An expert system's rules capture human expert knowledge as it is used in practice; consequently, they are often a blend of theoretical knowledge, heuristics derived from experience, and special-purpose rules for handling odd cases and other exceptions to normal practice. In many situations, this approach has proven effective. Nonetheless, strongly heuristic systems may fail, either on encountering a problem that does not fit any available rules, or by misapplying a heuristic rule to an inappropriate situation. Human experts do not suffer from these problems, because they have a deeper, theoretical understanding of the problem domain that allows them to apply the heuristic rules intelligently, or resort to reasoning from 'first principles' in novel situations. Model-based approaches, described next in Section 8.3.1 attempt to give an expert system this power and flexibility."

"The ability to learn from examples is another human capability that knowledge-intensive problem solvers emulate. Case-based reasoners, Section 8.3.3, maintain a knowledge base of example problem solutions, or cases. When confronted with a new problem, the reasoner selects from this stored set a case that resembles the present problem, and then attempts to apply a form of its solution strategy to this problem. In legal reasoning, the argument through precedent is a common example of case-based reasoning."

4. MODEL-BASED, CASE-BASED, AND HYBRID SYSTEMS (Section 8.3)

4.1 Introduction to Model-Based Reasoning (Section 8.3.1)

"Human expertise is an extremely complex amalgamation of theoretical knowledge, experience-based problem-solving heuristics, examples of past problems and their solutions, perceptual and interpretive skills and other abilities that are so poorly understood that we can only describe them as intuitive. Through years of experience, human experts develop very powerful rules for dealing with commonly encountered situations. These rules are often highly 'compiled', taking the form of direct associations between observable symptoms and final diagnoses, and hiding their more deeply explanatory foundations."

The MYCIN example:

"For example, the MYCIN expert system would propose a diagnosis based on such observable symptoms as 'headaches', 'nausea', or 'high fever'. Although these parameters can be indicative of an illness, rules that link them directly to a diagnosis do not reflect any deeper, causal understanding of human physiology. MYCIN's rules indicate the results of an infection, but do not explain its causes. A more deeply explanatory approach would detect the presence of infecting agents, note the resulting inflammation of cell linings, the presence of inter-cranial pressures, and infer the causal connection to the observed symptoms of headache, elevated temperatures, and nausea."

The semiconductor example:

"In a rule-based expert system example for semiconductor failure analysis, a descriptive approach might base a diagnosis of circuit failure on such symptoms as the discoloration of components (possibly indicating a burned-out component), the history of faults in similar devices, or even observations of component interiors using an electron microscope. However, approaches that use rules to link observations and diagnoses do not offer the benefits of a deeper analysis of the device's structure and function. A more robust, deeply explanatory approach would begin with a detailed model of the physical structure of the circuit and equations describing the expected behavior of each component and their interactions. It would base its diagnosis on signal readings from various locations in the device, using this data and its model of the circuit to determine the exact points of failure."

The limitations of first-generation expert systems:

"Because first-generation expert systems relied upon heuristic rules gained from the human expert's description of problem-solving techniques, they exhibited a number of fundamental limitations (Clancy 1985). If a problem instance did not match their heuristics, they simply failed, even though a more theoretical analysis would have found a solution. Often, expert systems applied

heuristics in inappropriate situations, such as where a deeper understanding of the problem would have indicated a different course. These are the limitations that model-based approaches attempt to address. A knowledge-based reasoner whose analysis is founded directly on the specification and functionality of a physical system is called a model-based system. In its design and use, a model-based reasoner creates a software simulation, often referred to as 'qualitative', of the function of that which is to be understood or fixed. (There are other types of model-based systems, of course, in particular, the logic-based and stochastic, which we present in Chapter 9.)"

The origins:

"The earliest model-based reasoners appeared in the mid-1970s and continued to mature through the 1980s (Davis and Hamscher 1992). It is interesting to note that some of the earliest work was intended to create software models of various physical devices, such as electronic circuits, for instructional purposes (deKleer 1976, Brown et al. 1982). In these early intelligent tutoring situations, the specifications for a device or circuit were reflected in sets of rules, e.g., Kirchoff's and Ohm's laws. The tutoring system both tested the student's knowledge of the device or circuit as well as conveyed to the student knowledge he or she might be missing. Rules were both the representation of the functionality of the hardware as well as the medium for conveying this knowledge to the student."

"From these early tutoring systems, where the task was to both model and teach the functionality of a system, qualitative model-based reasoners moved to trouble-shooting systems. In trouble-shooting faults in a physical system the model leads to sets of predicted behaviors. A fault is reflected in the discrepancy between predicted and observed behavior. The model-based system tells its user what to expect, and when observations differ from these expectations, how these discrepancies lead to identification of faults."

The three components of qualitative model-based reasoning:

"Qualitative model-based reasoning includes:

A description of each component in the device. These descriptions can simulate the behavior of the component.

A description of the device's internal structure. This is typically a representation of its components and their interconnections, along with the ability to simulate component interactions. The extent of knowledge of internal structure required depends on the levels of abstraction applied and diagnosis desired.

Diagnosis of a particular problem requires observations of the device's actual performance, typically measurements of its inputs and outputs. I/O measurements are easiest to obtain, but in fact, any measure could be used.

The task is then to determine which of the components could have failed in a way that accounts for the observed behaviors. This requires additional rules that describe known failure modes for the different components and their interconnections. The reasoner must find the most probable failures that can explain the observed system behavior."

Data structures:

"A number of data structures can be used for representing the causal and structural information in models. Many model-based program designers use rules to reflect the causality and functionality of a device. Rules can also be used to capture the relationships between components. An object-oriented system also offers an excellent representational tool for reflecting device and component structure within a model, with the slots of an object representing a device or component's state, and the object or class methods defining its functionality."

The adder example:

"To be more concrete in the design and evaluation of a model, we now consider several examples of device and circuit analysis from Davis and Hamscher (1992). Device behavior is represented by a set of expressions that capture the relationships between values on the terminals of the device. For the adder of Figure 8.13, there will be three expressions:

If we know the values at A and B, the value of C is A + B (the solid line). If we know C and A the value at B is C − A (the dashed line). If we know C and B, the value at A is C − B (the dotted line)."

"We need not have used an algebraic form to represent these relationships. We could equally well have used relational tuples or represented the constraints with Lisp functions. The goal in model-based reasoning is to represent the knowledge that captures the functionality of the adder."

The circuit example:

"In a second example, consider the circuit of three multipliers and two adders linked as in Figure 8.14. In this example the input values are given A to E and the output values at F and G. The expected output values are given in ( ) and the actual outputs in [ ]. The task is to determine where the fault lies that will explain this discrepancy. At F we have a conflict, expecting a 12 and getting a 10. We check the dependencies at this point and determine that the value at F is a function of ADD-1 which in turn depends on the outputs of MULT-1 and MULT-2. One of these three devices must have a fault, and so we have three hypotheses to consider: either the adder behavior is bad or one of its two inputs was incorrect, and the problem lies further back in the circuit."

"Reasoning from the result (10) at F and assuming correct behavior of ADD-1 and one of its inputs X (6), input Y to ADD-1 must be a 4. But that conflicts with the expectation of 6, which is the correct behavior of MULT-2 and inputs B and D. We have observed these inputs and know they are correct, so MULT-2 must be faulty. In a parallel argument, our second hypothesis is that ADD-1 is correct and MULT-1 is faulty."

"Continuing this reasoning, if the first input X to ADD-1 is correct and ADD-1 itself is correct, then the second input Y must be a 4. If it were a 4, G would be 10 rather than 12, so the output of MULT-2 must be a 6 and correct. We are left with the hypotheses that the fault lies in either MULT-1 or ADD-1 and we can continue to constrain these devices with further testing."

The three tasks:

"In our reasoning about the situation of Figure 8.14 we had three tasks:

Hypothesis generation, in which, given a discrepancy, we hypothesized which components of the device could have caused it.

Hypothesis testing, in which, given a collection of potential faulty components, we determined which of them could have explained the observed behavior.

Hypothesis discrimination, in which when more than one hypothesis survives the testing phase, as happened in the case of Figure 8.14, we must determine what additional information can be gathered to continue the search for the fault.

Finally, we should note that in the example of Figure 8.14 there was assumed to be a single faulty device. The world is not always this simple, although a single fault assumption is a useful, and often correct, heuristic."

Advantages and disadvantages:

"Because they are based on a theoretical understanding of the devices in question, qualitative model-based techniques remedy many of the limitations of more heuristic approaches. Rather than reasoning directly from observed phenomena to causal explanations, model-based approaches attempt to represent devices and configurations of devices on a causal or functional level. The program code reflects both the function of devices and the dependencies within a system of devices. Such models are often more robust than heuristic approaches. However, the down side of this explicit modeling of function is that the knowledge acquisition stage can be quite demanding and the resulting program large, cumbersome, and slow. Because heuristic approaches 'compile' typical cases into a single rule, they are often more efficient, so long as other system constraints are appropriate."

Deeper problems:

"There are deeper problems, however, with this approach. As with rule-based reasoning, the model of a system is just that, a model. It will of necessity be an abstraction of the system, and at some level of detail, be incorrect. For example, consider the input wires of Figure 8.14. In our discussion we considered these values as given and correct. We did not examine the state of the wire itself, and in particular, the other end where it joined the multipliers. What if the wire were broken, or had a faulty connection to the multiplier? If the user failed to detect this faulty connection, the model would not match the actual device. Any model attempts to describe the ideal situation, what the system is supposed to do, and not necessarily what the system does do. A 'bridging' fault is a contact point in the system where two wires or devices are inadvertently linked, as when a bad solder joint bridges between two wires that should not be in contact. Most model-based reasoning has difficulty hypothesizing a bridging fault because of the a priori assumptions underlying the model and the search methods for determining anomalies. Bridging faults are simply 'new' wires that aren't part of the original design. There is an implicit 'closed world assumption' (Section 9.1) that the structure description of the model is assumed to be complete and anything not in the model simply doesn't exist."

"In spite of these shortcomings, model-based reasoning is an important addition to the knowledge engineer's tool kit. Researchers continue to expand our understanding of diagnosis, both how human experts do it so efficiently, as well as how better algorithms can be implemented on machines (Stern and Luger 1997, Pless et al. 2006)."

4.2 Model-Based Reasoning: A NASA Example (Section 8.3.2)

"NASA has supported its presence in space by developing a fleet of intelligent space probes that autonomously explore the solar system (Williams and Nayak 1996a, Bernard et al. 1998). This effort was begun with software for the first probe in 1997 and the launch of Deep Space 1 in 1998. To achieve success through years in the harsh conditions of space travel, a craft needs to be able to radically reconfigure its control regime in response to failures and then plan around these failures during its remaining flight. To achieve acceptable cost and fast reconfiguration, one-of-a-kind modules will have to be put together quickly to automatically generate flight software. Finally, NASA expects that the set of potential failure scenarios and possible responses will be much too large to use software that supports preflight enumeration of all contingencies. Instead, the spacecraft will have to reactively think through all the consequences of its reconfiguration options."

Livingstone:

"Livingstone (Williams and Nayak 1996b) is an implemented kernel for a model-based reactive self-configuring autonomous system. The model-based reasoning representation language for Livingstone is propositional calculus, a shift from the first-order predicate calculus (Chapter 2), the traditional representation language for model-based diagnosis. Williams and Nayak felt, based on their past research on building fast propositional logic conflict-based algorithms for diagnosis (de Kleer and Williams 1989), that a fast reactive system, performing significant deduction in the sense/response loop, was possible."

"A long-held vision of model-based reasoning has been to use a single centralized model to support a variety of engineering tasks. For model-based autonomous systems this means using a single model to support a diversity of execution tasks. These include keeping track of developing plans (Section 8.4), confirming hardware modes, reconfiguring hardware, detecting anomalies, diagnosis, and fault recovery. Livingstone automates all these tasks using a single model and a single core algorithm, thus making significant progress towards achieving the vision for model-based problem solvers."

The propulsion system:

"Figure 8.15 shows an idealized schematic of the main engine subassembly of Cassini, the most complex spacecraft built to that time. It consists of a helium tank, a fuel tank, an oxidizer tank, a pair of main engines, regulators, latch valves, pyro valves, and pipes. The helium tank pressurizes the two propellant tanks, with the regulators acting to reduce the high helium pressure to a lower working pressure. When propellant paths to a main engine are open (the valve icon is unfilled), the pressurized tank forces fuel and oxidizer into the main engine, where they combine, spontaneously ignite, and produce thrust. The pyro valves can be fired exactly once, that is, they can change state only once, either going from open to closed or vice versa. Their function is to isolate parts of the main engine subsystem until they are needed, or to permanently isolate failed components. The latch valves are controlled using valve drivers (not shown in Figure 8.15) and the Acc (accelerometer) senses the thrust generated by the main engines."

"Starting from the configuration shown in Figure 8.15, the high-level goal of producing thrust can be achieved using a variety of different configurations: thrust can be provided by either of the main engines and there are a number of ways of opening propellant paths to

either main engine. For example, thrust can be provided by opening the latch valves leading to the engine on the left, or by firing a pair of pyros and opening a set of latch valves leading to the engine on the right. Other configurations correspond to various combinations of pyro firings. The different configurations have different characteristics since pyro firings are irreversible actions and since firing pyro valves requires significantly more power than opening or closing latch valves."

"Suppose the main engine subsystem has been configured to provide thrust from the left main engine by opening the latch valves leading to it. Suppose that this engine fails, for example, by overheating, so that it fails to provide the required thrust. To ensure that the desired thrust is provided even in this situation, the spacecraft must be transitioned to a new configuration in which thrust is now provided by the main engine on the right side. Ideally, this is achieved by firing the two pyro valves leading to the right side and opening the remaining latch valves, rather than firing additional pyro valves."

"A configuration manager constantly attempts to move the spacecraft into lowest cost configurations that achieve a set of high-level dynamically changing goals. When the spacecraft strays from the chosen configuration due to failures, the manager analyzes sensor data to identify the current configuration of the spacecraft, and then moves the space craft to a new configuration which, once again, achieves the desired configuration goals. A configuration manager is a discrete control system that ensures that the spacecraft's configuration always achieves the set point defined by the configuration goals. The planning algorithms that support the configuration manager are presented in Section 8.4.4."

The configuration manager:

"Reasoning about the configurations (and autonomous reconfigurations) of a system requires the concepts of operating and failure modes, reparable failures, and configuration changes. NASA expresses these concepts in a state space diagram: reparable failures are transitions from a failure state to a nominal state; configuration changes are between nominal states; and failures are transitions from a nominal to a failure state."

"Williams and Nayak (1997) view an autonomous system as a combination of a reactive configuration manager and a high level planner. The planner, for details see Section 8.4, generates a sequence of hardware configuration goals. The configuration manager evolves the transition system of the application domain, here the propulsion system of the spacecraft, along the desired trajectory. Thus configuration management is achieved by sensing and controlling the state of a transition system."

"A model-based configuration manager is a configuration manager that uses a specification of the transition system to compute the desired sequence of control values. In the example of Figure 8.15, each hardware component is modeled by a component transition system. Component communication, denoted by wires (links) in the figure is modeled by shared propositional terms between the corresponding component transition systems."

"The configuration manager makes extensive use of the model to infer the system's current state and to select optimal control actions to meet configuration goals. This is essential in situations where mistakes can lead to disaster, ruling out simple trial and error methods. The model-based configuration manager uses a model to determine the desired control sequence in two stages: mode estimation and mode reconfiguration (ME and MR in Figure 8.16). ME incrementally generates the set of all system trajectories consistent with the plant transition model and the sequence of system control and sensed values. MR uses a plant transition model and the partial trajectories generated by ME up to the current state to determine a set of control values such that all predicted trajectories achieve the configuration goal of the next state. Both ME and MR are reactive. ME infers the current state from knowledge of the previous state and observations within the current state. MR only considers actions that achieve the configuration goal within the next state."

4.3 Introduction to Case-Based Reasoning (Section 8.3.3)

"Heuristic rules and theoretical models are two types of information human experts use to solve problems. Another powerful strategy experts use is reasoning from cases, examples of past problems and their solutions. Case-based reasoning (CBR) uses an explicit database of problem solutions to address new problem-solving situations. These solutions may be collected from human experts through the knowledge engineering process or may reflect the results of previous search-based successes or failures. For example, medical education does not rely solely on theoretical models of anatomy, physiology, and disease; it also depends heavily on case histories and the intern's experience with other patients and their treatment. CASEY (Koton 1988a, b) and PROTOS (Bareiss et al. 1988) are examples of case-based reasoning applied to medicine."

"Lawyers select past law cases that are similar to their client's and that suggest a favorable decision, and try to convince the court that these similarities merit similar findings. Although general laws are made by democratic processes, their interpretation is usually based on legal precedents. How a law was interpreted in some earlier situation is critical for its current interpretation. Thus, an important component of legal reasoning is identifying from case law precedents for decisions in a particular case. Rissland (1983) and Rissland and Ashley (1987) have designed case-based reasoners to support legal arguments."

"Computer programmers often reuse their code, adapting an old program to fit a new situation with similar structure. Architects draw on their knowledge of esthetically pleasing and useful buildings of the past to design new buildings that people find pleasing and comfortable. Historians use stories from the past to help statesmen, bureaucrats, and citizens understand past events and plan for the future. The ability to reason from cases is fundamental to human intelligence."

"Other obvious areas for reasoning from cases include design, where aspects of a successfully executed artifact may be appropriate for a new situation, and diagnosis, where the failures of the past often recur. Hardware diagnosis is a good example of this. An expert in this area, besides using extensive theoretical knowledge of electronic and mechanical systems, brings past successful and failed experiences in diagnosis to bear on the current problem. CBR has been an important component of many hardware diagnostic systems, including work on the maintenance of signal sources and batteries in earth orbiting satellites (Skinner and Luger 1992) and the failure analysis of discrete component semiconductors (Stern and Luger 1997)."

The advantages:

"CBR offers a number of advantages for the construction of expert systems. Knowledge acquisition can be simplified if we record a human expert's solutions to a number of problems and let a case-based reasoner select and reason from the appropriate case. This would save the knowledge engineer the trouble of building general rules from the expert's examples; instead, the reasoner would generalize the rules automatically, through the process of applying them to new situations."

"Case-based approaches can also enable an expert system to learn from its experience. After reaching a search-based solution to a problem, a system can save that solution, so that next time a similar situation occurs, search would not be necessary. It can also be important to retain in the case base information about the success or failure of previous solution attempts; thus, CBR offers a powerful model of learning. An early example of this is Samuel's (1959, Section 4.1.1) checker-playing program, where board positions that were found through search or experience to be important are retained in the chance that this position might occur again in a later game."

The four-step process:

"Case-based reasoners share a common structure. For each new problem they:

Retrieve appropriate cases from memory. A case is appropriate if its solution may be successfully applied to the new situation. Since reasoners cannot know this in advance, they typically use the heuristic of choosing cases that are similar to the problem instance. Both humans and artificial reasoners determine similarity on the basis of common features: for example, if two patients share a number of common features in their symptoms and medical histories, there is a good probability that they have the same disease and will respond to the same treatment. Retrieving cases efficiently also requires that the case memory be organized to aid such retrieval. Typically, cases are indexed by their significant features, enabling efficient retrieval of cases that have the most features in common with the current problem. The identification of salient features is highly situation dependent.

Modify a retrieved case so that it will apply to the current situation. Typically, a case recommends a sequence of operations that transform a starting state into a goal state. The reasoner must transform the stored solution into operations suitable for the current problem. Analytic methods, such as curve fitting the parameters common to stored cases and new situations can be useful; for example, to determine appropriate temperatures or materials for welding. When analytic relations between cases are not available more heuristic methods may be appropriate; for example, in help desks for hardware diagnosis.

Apply the transformed case. Step 2 modifies a stored case, which, when applied, may not guarantee a satisfactory problem solution. This may require modifications in the solution case, with further iterations of these first three steps.

Save the solution, with a record of success or failure, for future use. Storage of the new case requires updating of the index structure. There are methods that can be used to maintain indices, including clustering algorithms (Fisher 1987) and other techniques from machine learning (Stubblefield and Luger 1996)."

Data structures:

"The data structures for case-based reasoning can be quite varied. In the simplest situation, cases are recorded as relational tuples where a subset of the arguments record the features to be matched and other arguments point to solution steps. Cases can also be represented as more complex structures, such as proof trees. A fairly common mechanism for storing cases is to represent cases as a set of large situation–action rules. The facts that describe the situation of the rule are the salient features of the recorded case and the operators that make up the action of the rule are the transformations to be used in the new situation. When this type of rule representation is used algorithms such as RETE (Forgy 1982) can be used to organize and optimize the search for appropriate cases."

The salient features problem:

"The most difficult issue in case-based problem solving, regardless of the data structure chosen for case representation, is the selection of salient features for the indexing and retrieval of cases. Kolodner (1993) and others actively involved in case-based problem solving set as a cardinal rule that cases be organized by the goals and needs of the problem solver. That is, that a careful analysis be made of case descriptors in the context of how these cases will be used in the solution process."

"For example, suppose a weak communication signal problem occurs in a satellite at 10:24:35 GMT. Analysis is made, and it is also determined that the power system is low. The low power can occur because the solar panels are not properly oriented towards the sun. The ground controllers make adjustments in the satellite's orientation, the power improves, and the communication signal is again strong. There are a number of salient features that might be used to record this case, the most obvious being that there is a weak communication signal or that the power supply is low. Another feature to describe this case is that the time of the problem was 10:24:35 GMT. The goals and needs of the problem solver in this case suggest that the salient features are weak communication signal and/or low power supply; the time this all happens may well be irrelevant, unless, of course, the fault occurs just after the sun disappears over the horizon (Skinner and Luger 1995)."

"Another essential problem to be addressed in CBR is the representation of such notions as weak signal or low power. Since the precise situation will probably never again be matched, e.g., some exact real number describing signal strength, the reasoner will probably represent the values as a range of real numbers indicating, for example, good, borderline-good, weak, and danger-alert levels."

Kolodner's preference heuristics:

"Kolodner (1993) offers a set of possible preference heuristics to help organize the storage and retrieval of cases. These include:

Goal-directed preference. Organize cases, at least in part, by goal descriptions. Retrieve cases that have the same goal as the current situation.

Salient-feature preference. Prefer cases that match the most important features or those matching the largest number of important features.

Specify preference. Look for as exact as possible matches of features before considering more general matches.
Frequency preference. Check first the most frequently matched cases.
Recency preference. Prefer cases used most recently.
Ease of adaptation preference. Use first cases most easily adapted to the current situation."

Advantages:

"Case-based reasoning has a number of advantages for the design of expert systems. Once knowledge engineers have arrived at the proper case representation, continued knowledge acquisition is straightforward: simply gather and store more cases. Often, case acquisition can be done from historical records or by monitoring current operations, minimizing demands on the human expert's time."

Theoretical questions:

"In addition, CBR raises a number of important theoretical questions relating to human learning and reasoning. One of the most subtle and critical issues raised by CBR is the question of defining similarity. Although the notion that similarity is a function of the number of features that two cases have in common is quite reasonable, it masks a number of profound subtleties. For example, most objects and situations have a large number of potential descriptive properties; case-based reasoners typically select cases on the basis of a tiny retrieval vocabulary. Typically, case-based reasoners require that the knowledge engineer define an appropriate vocabulary of highly relevant features. Although there has been work on enabling a reasoner to determine relevant features from its own experience (Stubblefield 1995), determining relevance remains a difficult problem."

"Another important problem in case-based reasoning deals with store/compute trade-offs. As a case-based reasoner acquires more cases, it becomes more intelligent and better able to solve a variety of target problems. Indeed, as we add cases to a reasoner, its performance will improve – up to a point. The problem is that as the case base continues to grow, the time needed to retrieve and process an appropriate case also grows. A decline in efficiency for large case bases can also be due to overlapping concepts, noise, and the distribution of problem types. One solution to this problem is to only save the 'best' or 'prototype' cases, deleting those that are redundant or used infrequently; i.e., forgetting those cases that fail to prove useful. See Samuel's (1959) retention algorithm for saving checker board positions for an important early example of this approach. In general, however, it is not clear how we can automate such decisions; this remains an active research area (Kolodner 1993)."

Explanation:

"An automated explanation for why a solution is recommended is also difficult for a case-based reasoner. When asked why a solution was selected to remedy a current situation, the only explanation the system can provide is to say that this particular fix worked at a previous time period. There may also be weak explanations based on similarities of top-level goal descriptions between the current situation and a set of stored cases. In the example of the satellite communication problem, the relevant case was chosen on the basis of a weak communication signal. This mode of reasoning has no deeper explanation than that it worked before in a similar situation. But as noted previously, this may be sufficient explanation for many situations."

Explanation of failure:

"Many researchers, however, feel that the simple repetition of top-level goals and the case to be applied offers insufficient explanation (Leake 1992, Stern and Luger 1992), especially when we need an explanation of why some fix doesn't work. Take the satellite situation again. Suppose the solar panel reorientation works but three hours later the signal again becomes weak. Using both the frequency and recency heuristics, we again reorient the solar panel. In three hours the weak signal recurs. And again three hours later; we always apply the same fix. This example is based on an actual satellite situation in which it was found that a more complex problem existed, namely a gyroscope was overheating and giving a distorted reading that disoriented the satellite. The system that finally solved the problem used a model-based reasoner to simulate the behavior of the satellite to determine the root causes of the weak communication signal (Skinner and Luger 1995)."

Analogy:

"Case-based reasoning is related to the problem of learning through analogy. To reuse past experiences we must both recognize the salient features of the past as well as build a mapping of how that experience may be used in the present situation. Transformational analogy (Carbonell 1983) is an example of a case-based approach to problem solving. It solves new problems by modifying existing solutions until they may be applied to the new instance. Operators that modify complete problem solutions define a higher-level of abstraction or T-space in which states are problem solutions and operators transform these solutions, as in Figure 8.17. The goal is to transform a source solution into a possible solution for the target problem. Operators modify solutions in ways such as inserting or deleting steps in a solution path, reordering steps in a solution, splicing new solutions into a portion of an old solution, or changing the bindings of parameters in the current solution."

"Transformational analogy typifies the approach used by case-based problem solving. Later work has refined the approach, considering such issues as the representation of cases, strategies for organizing a memory of prior cases, retrieval of relevant prior cases, and the use of cases in solving new problems. For further information on case-based reasoning, see Hammond (1989) and Kolodner (1988a, b). Reasoning through analogies is discussed further in the context of symbol-based machine learning (Section 10.5.4)."

4.4 Hybrid Design: Strengths/Weaknesses of Strong Method Systems (Section 8.3.4)

"Successes in building expert systems that solve hard, practical problems have demonstrated the truth of the central idea behind knowledge-based systems: that the power of a reasoner is in its domain knowledge rather than the sophistication of its reasoning methods. This observation, however, raises one of the central issues in artificial intelligence: that of knowledge representation. At a practical level, every knowledge engineer must make choices about how to represent domain knowledge in a way that will be most appropriate to the given domain. Knowledge representation also raises a number of theoretically important, intellectually difficult issues, such as the handling of missing and uncertain information, the measurement of a representation's expressiveness, the relationship between a representation language and such issues as learning, knowledge acquisition, and the efficiency of a reasoner."

"In this chapter, we considered a number of basic approaches to knowledge representation: rule-based expert systems, model-based reasoners, and case-based reasoners. As an aid to practical knowledge engineering, we summarize the strengths and weaknesses of each knowledge-intensive approach to problem solving."

Rule-based reasoning — advantages:

"The advantages of a rule-based approach include:
The ability to use, in a very direct fashion, experiential knowledge acquired from human experts. This is particularly important in domains that rely heavily on heuristics to manage complexity and/or missing information.
Rules map into state space search. Explanation facilities support debugging.
The separation of knowledge from control simplifies development of expert systems by enabling an iterative development process where the knowledge engineer acquires, implements, and tests individual rules.
Good performance is possible in limited domains. Because of the large amounts of knowledge required for intelligent problem solving, expert systems are limited to narrow domains. However, there are many domains where design of an appropriate system has proven extremely useful.
Good explanation facilities. Although the basic rule-based framework supports flexible, problem-specific explanations, it must be mentioned that the ultimate quality of these explanations depends upon the structure and content of the rules. Explanation facilities differ widely between data- and goal-driven systems."

Rule-based reasoning — disadvantages:

"Disadvantages of rule-based reasoning include:
Often the rules obtained from human experts are highly heuristic in nature, and do not capture functional or model-based knowledge of the domain.
Heuristic rules tend to be 'brittle' and can have difficulty handling missing information or unexpected data values.
Another aspect of the brittleness of rules is a tendency to degrade rapidly near the 'edges' of the domain knowledge. Unlike humans, rule-based systems are usually unable to fall back on first principles of reasoning when confronted with novel problems.
Explanations function at the descriptive level only, omitting theoretical explanations. This follows from the fact that heuristic rules gain much of their power by directly associating problem symptoms with solutions, without requiring (or supporting) deeper reasoning.
The knowledge tends to be very task dependent. Formalized domain knowledge tends to be very specific in its applicability. Currently, knowledge representation languages do not approach the flexibility of human reasoning."

Case-based reasoning — advantages:

"The advantages of case-based reasoning include:
The ability to encode historical knowledge directly. In many domains, cases can be obtained from existing case histories, repair logs, or other sources, eliminating the need for intensive knowledge acquisition with a human expert.
Allows shortcuts in reasoning. If an appropriate case can be found, new problems can often be solved in much less time than it would take to generate a solution from rules or models and search.
It allows a system to avoid past errors and exploit past successes. CBR provides a model of learning that is both theoretically interesting and practical enough to apply to complex problems.
Extensive analysis of domain knowledge is not required. Unlike a rule-based system, where the knowledge engineer must anticipate rule interactions, CBR allows a simple additive model for knowledge acquisition. This requires an appropriate representation for cases, a useful retrieval index, and a case adaptation strategy.
Appropriate indexing strategies add insight and problem-solving power. The ability to distinguish differences in target problems and select an appropriate case is an important source of a case-based reasoner's power; often, indexing algorithms can provide this functionality automatically."

Case-based reasoning — disadvantages:

"The disadvantages of case-based reasoning include:
Cases do not often include deeper knowledge of the domain. This handicaps explanation facilities, and in many situations it allows the possibility that cases may be misapplied, leading to poor quality or wrong advice.
A large case base can suffer problems from store/compute trade-offs.
It is difficult to determine good criteria for indexing and matching cases. Currently, retrieval vocabularies and similarity matching algorithms must be carefully hand crafted; this can offset many of the advantages CBR offers for knowledge acquisition."

Model-based reasoning — advantages:

"The advantages of model-based reasoning include:
The ability to use functional/structural knowledge of the domain in problem-solving. This increases the reasoner's ability to handle a variety of problems, including those that may not have been anticipated by the system's designers.
Model-based reasoners tend to be very robust. For the same reasons that humans often retreat to first principles when confronted with a novel problem, model-based reasoners tend to be thorough and flexible problem solvers.
Some knowledge is transferable between tasks. Model-based reasoners are often built using scientific, theoretical knowledge. Because science strives for generally applicable theories, this generality often extends to model-based reasoners.
Often, model-based reasoners can provide causal explanations. These can convey a deeper understanding of the fault to human users, and can also play an important tutorial role."

Model-based reasoning — disadvantages:

"The disadvantages of model-based reasoning include:
A lack of experiential (descriptive) knowledge of the domain. The heuristic methods used by

rule-based approaches reflect a valuable class of expertise.
It requires an explicit domain model. Many domains, such as the diagnosis of failures in electronic circuits, have a strong scientific basis that supports model-based approaches. However, many domains, such as some medical specialties, most design problems, or many financial applications, lack a well-defined scientific theory. Model-based approaches cannot be used in such cases.
High complexity. Model-based reasoning generally operates at a level of detail that leads to significant complexity; this is, after all, one of the main reasons human experts have developed heuristics in the first place.
Exceptional situations. Unusual circumstances, for example, bridging faults or the interaction of multiple failures in electronic components, can alter the functionality of a system in ways difficult to predict using an a priori model."

Hybrid design:

"An important area of research and application is the combination of different reasoning models. With a hybrid architecture two or more paradigms are integrated to get a cooperative effect where the strengths of one system can compensate for the weakness of another. In combination, we can address the disadvantages noted in the previous discussion."
"For example, the combination of rule-based and case-based systems can:
Offer a natural first check against known cases before undertaking rule-based reasoning and the associated search costs.
Provide a record of examples and exceptions to solutions through retention in the case base.
Record search-based results as cases for future use. By saving appropriate cases, a reasoner can avoid duplicating costly search."
"The combination of rule-based and model-based systems can:
Enhance explanations with functional knowledge. This can be particularly useful in tutorial applications.
Improve robustness when rules fail. If there are no heuristic rules that apply to a given problem instance, the reasoner can resort to reasoning from first principles.
Add heuristic search to model-based search. This can help manage the complexity of model-based reasoning and allow the reasoner to choose intelligently between possible alternatives."
"The combination of model-based and case-based systems can:
Give more mature explanations to the situations recorded in cases.
Offer a natural first check against stored cases before beginning the more extensive search required by model-based reasoning.
Provide a record of examples and exceptions in a case base that can be used to guide model-based inference.
Record results of model-based inference for future use."
"Hybrid methods deserve the attention of researchers and application developers alike. However, building such systems is not a simple matter, requiring the resolution of such problems as determining which reasoning method to apply in a given situation, deciding when to change reasoning methods, resolving differences between reasoning methods, and designing representations that allow knowledge to be shared."
"We next consider planning, or the organization of procedures into potential solutions."

5. PLANNING (Section 8.4)

"The task of a planner is to find a sequence of actions that allow a problem solver, such as a control system, to accomplish some specific task. Traditional planning is very much knowledge-intensive, since plan creation requires the organization of pieces of knowledge and partial plans into a solution procedure. Planning plays a role in expert systems in reasoning about events occurring over time. Planning has many applications in manufacturing, such as process control. It is also important for designing a robot controller."

5.1 Introduction to Planning: Robotics (Section 8.4.1)

"To begin, we consider examples from traditional robotics. (Much of modern robotics uses reactive control rather than deliberative planning, Sections 7.3.1 and 8.4.3.) The steps of a traditional robot plan are composed of the robot's atomic actions. In our example, we do not describe atomic actions at the micro-level: 'turn the sixth stepper motor one revolution.' Instead, we specify actions at a higher level, such as by their effects on a world. For example, a blocks world robot might include such actions as 'pick up object a' or 'go to location x.' The micro-control is built into these higher-level actions."

The example plan:

"Thus, a sequence of actions to 'go get block a from room b' might be:
put down whatever is now held
go to room b
go over to block a
pick up block a
leave room b
return to original location"
"Plans are created by searching through a space of possible actions until the sequence necessary to accomplish the task is discovered. This space represents states of the world that are changed by applying each of the actions. The search terminates when the goal state (the appropriate description of the world) is produced. Thus, many of the issues of heuristic search, including finding A* algorithms, are also appropriate in planning."
"The act of planning does not depend on the existence of an actual robot to carry out the plans, however. In the early years of computer planning (1960s), entire plans were formulated before the robot performed its first act. Thus plans were devised without the presence of a robot at all! More recently, with the implementation of sophisticated sensing and reactive devices, research has focused on more integrated plan/action sequencing."

The issues:

"Traditional planning relies on search techniques, and raises a number of interesting issues. For one, the description of the states of the world may be considerably more complex than in previous examples of search. Consider the number of predicates that may be necessary to describe rooms, corridors, and objects in a robot's environment. Not only must we represent the robot's world; we must also represent the effect of atomic actions on that world. The full description of each state of the problem space can be large."
"Another difference in planning is the need to characterize what is not changed by a particular action. Picking up an object does change (a) the location of the object and (b) the fact that the robot hand is now grasping the object. It does not change (a) the locations of the doors and the rooms or (b) the locations of other objects. The specification of what is true in one state of the world and exactly what is changed by performing some action in the world has become known as the frame problem (McCarthy 1980, McCarthy and Hayes 1969). As the complexity of the problem space increases, the issue of keeping track of the changes that occur with each action and the features of a state description that remain unchanged becomes more important. We present two solutions for coping with the frame problem, but, as will be seen, neither of these is totally satisfactory."
"Other important issues include generating plans, saving and generalizing good plans, recovering from unexpected plan failures (part of the world might not be as expected, perhaps by being accidentally moved from its anticipated location), and maintaining consistency between the world and a program's internal model of the world."

The blocks world:

"In the examples of this section, we limit our robot's world to a set of blocks on a tabletop and the robot's actions to an arm that can stack, unstack, and otherwise move the blocks about the table. In Figure 8.18 we have five blocks, labeled a, b, c, d, e, sitting on the top of a table. The blocks are all cubes of the same size, and stacks of blocks, as in the figure, have blocks directly on top of each other. The robot arm has a gripper that can grasp any clear block (one with no block on top of it) and move it to any location on the tabletop or place it on top of any other block (with no block on top of it)."

The robot tasks:

"The robot arm can perform the following tasks (U, V, W, X, Y, and Z are variables):
goto(X,Y,Z) Goto location described by coordinates X, Y, and Z. This location might be implicit in the command pickup (W) where block W has location X, Y, Z. pickup(W) Pick up block W from its current location. It is assumed that block W is clear on top, the gripper is empty at the time, and the computer knows the location of block W. putdown(W) Place block W down at some location on the table and record the new location for W. W must be held by the gripper at the time. stack(U,V) Place block U on top of block V. The gripper must be holding U and the top of V must be clear of other blocks. unstack(U,V) Remove block U from the top of V. U must be clear of other blocks, V must have block U on top of it, and the hand must be empty before this command can be executed."

The state description predicates:

"The state of the world is described by a set of predicates and predicate relationships:
location(W,X,Y,Z) Block W is at coordinates X, Y, Z. on(X,Y) Block X is immediately on top of block Y. clear(X) Block X has nothing on top of it. gripping(X) The robot arm is holding block X. gripping( ) The robot gripper is empty. ontable(W) Block W is on the table."
"ontable(W) is a short form for the predicate location(W,X,Y,Z), where Z is the table level. Similarly, on(X,Y) indicates that block X is located with its bottom coincident with the top of block Y. We can greatly simplify the world descriptions by having the computer record the present location(X,Y,Z) of each block and keep track of its movements to new locations. With this location assumption, the goto command becomes unnecessary; a command such as pickup(X) or stack(X) implicitly contains the location of X."

STATE 1:

"The blocks world of Figure 8.18 may now be represented by the following set of predicates. We call this collection of predicates STATE 1 for our continuing example. Because the predicates describing the state of the world for Figure 8.18 are all true at the same time, the full state description is the conjunction (∧) of all these predicates.
STATE 1 ontable(a). on(b, a). clear(b). ontable(c). on(e, d). clear(c). ontable(d). gripping( ). clear(e)."

The truth relations:

"Next, a number of truth relations (in the declarative sense) or rules for performance (in the procedural sense) are created for clear(X), ontable(X), and gripping( ):
(∀ X) (clear(X) ← ¬ (∃Y) (on(Y,X)))
(∀Y) (∀ X) ¬ (on(Y,X) ← ontable(Y))
(∀Y) gripping( ) ↔¬ (gripping(Y))"
"The first statement says that if block X is clear, there does not exist any block Y such that Y is on top of X. Interpreted procedurally, this says 'to clear block X go and remove any block Y that might be on top of X.'"

The operators:

"We now design rules to operate on states and produce new states. In doing so, we again impute a procedural semantics to a predicate logic-like representation. The operators (pickup, putdown, stack, unstack) are:
(∀ X) (pickup(X) → (gripping(X) ← (gripping( ) ∧ clear(X) ∧ ontable(X)))).
(∀ X) (putdown(X) → ((gripping( ) ∧ ontable(X) ∧ clear(X)) ← gripping(X))).
(∀X) (∀Y) (stack(X,Y) → ((on(X,Y) ∧ gripping( ) ∧ clear(X)) ← (clear(Y) ∧ gripping(X)))).
(∀X)(∀Y) (unstack(X,Y) → ((clear(Y) ∧ gripping(X)) ← (on(X,Y) ∧ clear(X) ∧ gripping( )))."
"Consider the fourth rule: for all blocks X, pickup(X) means gripping(X) if the hand is empty and X is clear. Note the form of these rules: A → (B ← C). This says that operator A produces new predicate(s) B when condition(s) C is (are) true. We use these rules to generate new states in a space. That is, if predicates C are true in a state, then B is true in its child state. Thus, operator A can be used to create a new state described by predicates B when predicates C are true. Alternative approaches to creating these operators include STRIPS (Nilsson 1980 and Section 8.4.2) and Rich and Knight's (1991) do function."

The frame axioms:

"We must first address the frame problem before we can use these rule relationships to generate new states of the blocks world. Frame axioms are rules to tell what predicates describing a state are not changed by rule applications and are thus carried over intact to help describe the new state of the world. For example, if we apply the operator pickup block b in Figure 8.18, then all predicates related to the rest of the blocks remain true in the child state. For our world of blocks we can specify several such frame rules:
(∀ X) (∀Y) (∀ Z) (unstack(Y,Z) → (ontable(X) ← ontable(X))).
(∀ X) (∀Y) (∀ Z) (stack(Y,Z) → (ontable(X) ← ontable(X)))."

"These two rules say that ontable is not affected by the stack and unstack operators. This is true even when X and Z are identical; if Y = Z either 6 or 7 above won't be true."
"Other frame axioms say that on and clear are affected by stack and unstack operators only when that particular on relation is unstacked or when a clear relation is stacked. Thus, in our example, on(b,a) is not affected by unstack(c,d)."
"Similarly, frame axioms say that clear(X) relations are unaffected by gripping(Y) even when X = Y or gripping( ) is true. More axioms say that gripping does not affect on(X,Y) relations but affects only the ontable(X) relation where X is gripped. A number of further frame axioms need to be specified for our example."
STATE 2:
"Together, these operators and frame axioms define a state space, as illustrated by the operator unstack. unstack(X,Y) requires three conditions to be true simultaneously, namely on(X,Y), gripping( ), and clear(X). When these conditions are met the new predicates gripping(X) and clear(Y) are produced by applying the unstack operator. A number of other predicates also true for STATE 1 will remain true in STATE 2. These states are preserved for STATE 2 by the frame axioms. We now produce the nine predicates describing STATE 2 by applying the unstack operator and the frame axioms to the nine predicates of STATE 1, where the net result is to unstack block e:
STATE 2 ontable(a). on(b,a). clear(b). ontable(c). clear(c). clear(d). ontable(d). gripping(e). clear(e)."
The summary:
"To summarize:
Planning may be seen as a state space search. New states are produced by general operators such as stack and unstack plus frame rules. The techniques of graph search may be applied to find a path from the start state to the goal state. The operations on this path constitute a plan."
"Figure 8.19 shows an example of a state space searched by applying the operators as described above. If a goal description is added to this problem-solving process, then a plan may be seen as a set of operators that produces a path that leads from the present state of this graph to the goal. (See Section 3.1.2.)"
"This characterization of the planning problem defines its theoretical roots in state space search and predicate calculus representation and inferencing. However, it is important to note how complex this manner of solution can be. In particular, using the frame rules to calculate what remains unchanged between states can add exponentially to the search, as can be seen from the complexity of our very simple blocks example. In fact, when any new predicate descriptor is introduced, for color, shape, or size, new frame rules must be defined to relate it to all other appropriate actions!"
"This discussion also assumes that the subproblems that make up a task are independent and may thus be solved in an arbitrary order. This is very seldom the case in interesting and/or complex problem domains, where the preconditions and actions required to achieve one subgoal can often conflict with the preconditions and actions required to achieve another. Next we illustrate these problems and discuss an approach to planning that greatly assists in handling this complexity."

5.2 Using Planning Macros: STRIPS (Section 8.4.2)

"STRIPS, developed at what is now SRI International, stands for STanford Research Institute Planning System (Fikes and Nilsson 1971, Fikes et al. 1972). This controller was used to drive the SHAKEY robot of the early 1970s. STRIPS addressed the problem of efficiently representing and implementing the operations of a planner. It addressed the problem of conflicting subgoals and provided an early model of learning; successful plans were saved and generalized as macro operators, which could be used in similar future situations. In the remainder of this section, we present a version of STRIPS-style planning and triangle tables, the data structure used to organize and store macro operations."
"Using our blocks example, the four operators pickup, putdown, stack, and unstack are represented as triples of descriptions. The first element of the triple is the set of preconditions (P), or conditions the world must meet for an operator to be applied. The second element of the triple is the add list (A), or the additions to the state description that are a result of applying the operator. Finally, there is the delete list (D), or the items that are removed from a state description to create the new state when the operator is applied. These lists are intended to eliminate the need for separate frame axioms. We can represent the four operators in this fashion:"
Operator
Preconditions (P)
Add list (A)
Delete list (D)
pickup(X)
gripping( ) ∧ clear(X) ∧ ontable(X)
gripping(X)
ontable(X) ∧ gripping( )
putdown(X)
gripping(X)
ontable(X) ∧ gripping( ) ∧ clear(X)
gripping(X)
stack(X,Y)
clear(Y) ∧ gripping(X)
on(X,Y) ∧ gripping( ) ∧ clear(X)
clear(Y) ∧ gripping(X)
unstack(X,Y)
clear(X) ∧ gripping( ) ∧ on(X,Y)
gripping(X) ∧ clear(Y)
gripping( ) ∧ on(X,Y)
"The important thing about the add and delete lists is that they specify everything that is necessary to satisfy the frame axioms! Some redundancy exists in the add and delete list approach. For example, in unstack the add of gripping(X) could imply the delete of gripping( ). But the gain of this redundancy is that every descriptor of a state that is not mentioned by the add or delete remains the same in the new state description."
"A related weakness of the precondition-add-delete list approach is that we are no longer using a theorem-proving process to produce (by inference) the new states. This is not a serious problem, however, as proofs of the equivalence of the two approaches can guarantee the correctness of the precondition-add-delete method. The precondition-add-delete list approach may be used to produce the same results we produced with the inference rules and frame axioms in our earlier example. The state space search, as in Figure 8.19, would be identical for both approaches."
The conflicting subgoals problem:
"A number of other problems inherent in planning are not solved by either of the two approaches presented so far. In solving a goal we often divide it into subproblems, for instance, unstack(e,d) and unstack(b,a). Attempting to solve these subgoals independently can cause problems if the actions needed to achieve one goal actually undo the other. Incompatible subgoals may result from a false assumption of linearity (independence) of subgoals. The non-linearity of a plan/action space can make solution searches unnecessarily difficult or even impossible."
"We now show a very simple example of an incompatible subgoal using the start state STATE 1 of Figure 8.18. Suppose the goal of the plan is STATE G as in Figure 8.20, with on(b,a) ∧ on(a,c) and blocks d and e remaining as in STATE 1. It may be noted that one of the parts of the conjunctive goal on(b,a) ∧ on(a,c) is true in STATE 1, namely on(b,a). This already satisfied part of the goal must be undone before the second subgoal, on(a,c), can be accomplished."
Triangle tables:
"The triangle table representation (Fikes and Nilsson 1971, Nilsson 1980) is aimed at alleviating some of these anomalies. A triangle table is a data structure for organizing sequences of actions, including potentially incompatible subgoals, within a plan. It addresses the problem of conflicting subgoals within macro actions by representing the global interaction of sequences of operations. A triangle table relates the preconditions of one action to the postconditions, the combined add and delete lists, of actions preceding it."
"Triangle tables are used to determine when that macro operator could be used in building a plan. By saving these macro operators and reusing them, STRIPS increases the efficiency of its planning search. Indeed, we can generalize a macro operator, using variable names to replace the block names in a particular example. Then we can call the new generalized macro to prune search. In Chapter 10, with our presentation of learning in symbol based environments, we discuss techniques for generalizing macro operations."
"The reuse of macro operators also helps to solve the problem of conflicting subgoals. As the following example illustrates, once the planner has developed a plan for goals of the form stack(X,Y) ∧ stack(Y,Z), it may store and reuse that plan. This eliminates the need to break the goal into subgoals and avoids the complications that may follow."
"Figure 8.21 presents a triangle table for the macro action stack(X,Y) ∧ stack(Y,Z). This macro action can be applied to states where on(X,Y) ∧ clear(X) ∧ clear(Z) is true. This triangle table is appropriate for starting STATE 1 with X = b, Y = a, and Z = c."
"The atomic actions of the plan are recorded along the diagonal. These are the four actions, pickup, putdown, stack, and unstack, discussed earlier in this section. The set of preconditions of each of these actions are in the row preceding that action, and the postconditions of each action are in the column below it. For example, row 5 lists the preconditions for pickup(X) and column 6 lists the postconditions (the add and delete lists) of pickup(X). These postconditions are placed in the row of the action that uses them as preconditions, organizing them in a manner relevant to further actions. The triangle table's purpose is to properly interleave the preconditions and postconditions of each of the smaller actions that make up the larger goal. Thus, triangle tables address non-linearity issues in planning on the macro operator level; Partial-Order Planners (Russell and Norvig 1995) and other approaches have further addressed these issues."
The kernel:
"One advantage of triangle tables is the assistance they can offer in attempting to recover from unexpected happenings, such as a block being slightly out of place, or accidents, such as dropping a block. Often an accident can require backing up several steps before the plan can be resumed. When something goes wrong with a solution the planner can go back into the rows and columns of the triangle table to check what is true. Once the planner has figured out what is still true within the rows and columns, it then knows what the next step must be if the larger solution is to be restarted. This is formalized with the notion of a kernel."
"The nth kernel is the intersection of all rows below and including the nth row and all columns to the left of and including the nth column. In Figure 8.21 we have outlined the third kernel in bold. In carrying out a plan represented in a triangle table, the ith operation (that is, the operation in row i) may be performed only if all predicates contained in the ith kernel are true. This offers a straightforward way of verifying that a step can be taken and also supports systematic recovery from any disruption of the plan. Given a triangle table, we find and execute the highest-numbered action whose kernel is enabled. This not only lets us back up in a plan but also allows for the possibility that an unexpected event might let us jump forward in a plan."
"The conditions in the leftmost column are the preconditions for the macro action as a whole. The conditions in the bottom row are the conditions added to the world by the macro operator. Thus a triangle table may be saved as a macro operator with its own set of pre and post conditions or add and delete lists."
"Of course, the triangle table approach does lose some of the semantics of the previous planning models. Notice, for example, that only those postconditions of an act are retained that are also preconditions of later acts. Thus, if guaranteed correctness is a desired result, further verification of the triangle tables, perhaps with additional information that might allow sequences of triangle tables to be composed, might be desirable."
"Other problems arise with the use of macro operators in planning. As the number of macro operators increases, the planner has more powerful operations to use, decreasing the size of the state space that must be searched. Unfortunately, at each step of the search, all of

these operators must be examined. The pattern matching needed to determine whether an operator may be applied can add considerable overhead to the search process, often counteracting the gains made by saving macro operations. The problems of determining when a macro operation should be saved and how to determine the next operator to use remain the subject of much research. In the next section we describe an algorithm under which many subgoals may be simultaneously satisfied, teleo-reactive planning."

5.3 Teleo-Reactive Planning (Section 8.4.3)

"Since the early work in planning described in the previous section (Fikes and Nilsson 1971), there have been a number of significant advances. Much of this work has been done on domain independent planning, but more recently more domain specific planning, where distributed sense/react mechanisms are employed, has become important. In the next sections we describe a domain independent system, teleo-reactive planning, as well as a more domain dependent planner, Burton, from NASA. For a summary of the first two decades of planning research we recommend (Allen et al. 1990)."

"Teleo-reactive (TR) planning was first proposed by Nilsson (1994) and Benson (1995) at Stanford University. TR planning offers a general purpose architecture, with application to many domains where control of complex subsystems must be coordinated to accomplish a higher level goal. Thus it combines the top-down approach of hierarchical control architectures with an 'agent-based' (Section 7.4) or bottom-up assist. The result is a system that can accomplish complex problem solving through the coordination of simple task-specific agents. The justification for this approach is that simple agents have the advantage of working in smaller and more constrained problem spaces. The higher level controller, on the other hand, can make more global decisions about the entire system, for example, how the current result of a purely local decision can affect the result of the general problem-solving task."

"Teleo-reactive control combines aspects of feedback-based control and discrete action planning. Teleo-reactive programs sequence the execution of actions that have been assembled into a goal-oriented plan. Unlike more traditional AI planning environments (Weld 1994), no assumptions are made that actions are discrete and uninterruptible and that every action's effects are completely predictable. To the contrary, teleo-actions can be sustained over extended periods of time, that is, teleo-actions are executed as long as the actions' preconditions are met and the associated goal has not yet been achieved. Nilsson (1994) refers to this type of action as durative. Durative actions can be interrupted when some other action closer to the top-level goal is activated. A short sense-react cycle ensures that when the environment changes the control actions also quickly change to reflect the new state of the problem solution."

The TR tree:

"Teleo-reactive action sequences are represented by a data structure called a TR tree as in Figure 8.22. A TR tree is described by a set of condition →action pairs (or production rules, Section 6.3). For example:

C0 → A0 C1 → A1 C2 → A2 … Cn → An"

"where the Ci are the conditions and the Ai are the associated actions. We refer to C0 as the top-level goal of the tree and A0 as the null action, indicating that nothing further needs to be done once the top-level goal is achieved. At each execution cycle of the teleo-reactive system, each Ci is evaluated from the top of the rules to the bottom (C0, C1, … Cn) until the first true condition is found. The action associated with this true condition is then performed. The evaluation cycle is then repeated at a frequency that approximates the reactivity of circuit-based control."

"The Ci → Ai productions are organized in such a way that each action Ai, if continually executed under normal conditions, will eventually make some condition higher in the TR rule tree, Figure 8.22, true. TR tree execution may be seen as adaptive in that if some unanticipated event in the control environment reverses the effect of previous actions, TR execution will fall back to the lower level rule condition that reflects that condition. From that point it will restart its work towards satisfying all higher level goals. Similarly, if something good inadvertently happens, TR execution is opportunistic in that control will automatically shift to the action of that true condition."

TR tree construction:

"TR trees can be constructed with a planning algorithm that employs common AI goal reduction methods. Starting from the top-level goal, the planner searches over actions whose effects include achievement of that goal. The preconditions of those actions generate a new set of subgoals, and this procedure recurses. Termination is achieved when the preconditions of the leaf nodes are satisfied by the current state of the environment. Thus the planning algorithm regresses from the top-level goal through goal reduction to the current state. Actions, of course, often have side effects and the planner must be careful to verify that an action at any level does not alter conditions that are required as preconditions of actions at a higher level in the TR tree. Goal reduction is thus coupled with constraint satisfaction, where a variety of action reordering strategies are used to eliminate possible constraint violations."

"Thus, TR planning algorithms are used to build plans whose leaf nodes are satisfied by the current state of the problem environment. They usually do not build complete plans, that is plans that can start from any world state, because such plans are generally too large to store or to execute efficiently. This final point is important because sometimes an unexpected environmental event can shift the world to a state in which no action's preconditions in the TR tree are satisfied and some form of replanning is necessary. This is usually done by reactivating the TR planner."

The applications:

"Benson (1995) and Nilsson (1994) have used teleo-reactive planning in a number of application domains including the control of distributed robot agents and building a flight simulator. Klein et al. (1999, 2000) have used a teleo-reactive planner to build and test a portable control architecture for the acceleration of a charged particle beam. These latter researchers gave a number of reasons for the use of a teleo-reactive controller in the domain of particle beam controllers:

Accelerator beams and their associated diagnostics are typically dynamic and noisy.

The achievement of accelerator tuning goals is often affected by stochastic processes, RF breakdown, or oscillations in the beam source.

Many of the actions required for tuning are durative. This is especially true of tweaking and optimization operations that need to be continued until specific criteria are met.

TR trees offer an intuitive framework for encoding tuning plans acquired from accelerator physicists. In fact with very little help, the physicists themselves were able to develop their own TR trees."

5.4 Planning: A NASA Example (Section 8.4.4)

"In this section we describe how a planner can be implemented in the context of a model-based reasoner. We continue the NASA example of Williams and Nayak (1996b) introduced in Section 8.3.2. Livingstone is a reactive configuration manager that uses a compositional, component-based model of the space craft propulsion system to determine configuration actions, as is seen in Figure 8.23."

"Each propulsion component is modeled as a transition system that specifies the behaviors of the operating and failure modes of the component, the nominal and failure transitions between modes, and the costs and likelihoods of transitions, as in Figure 8.24. In Figure 8.24, open and closed are normal operation modes, but stuck open and stuck closed are failure modes. The open command has unit cost and causes a mode transition from closed to open, similarly for the close command. Failure transitions move the valve from normal operating modes to one of the failure modes with probability 0.01. Mode behaviors are specified using formulae in propositional logic, but transitions between modes are specified using formulae in a restricted temporal, propositional logic. The restricted temporal, propositional logic is adequate for modeling digital hardware, analog hardware using qualitative abstractions (de Kleer and Williams 1989, Weld and de Kleer 1990), and real time software using the models of concurrent reactive systems (Manna and Pnueli 1992). The spacecraft transition system model is a composition of its component transition systems in which the set of configurations of the spacecraft is the cross product of the sets of component modes. We assume that the component transition systems operate synchronously; that is, for each spacecraft transition every component performs a transition."

"A model-based configuration manager uses its transition-system model to both identify the current configuration of the spacecraft, called mode estimation ME, and move the spacecraft into a new configuration that achieves the desired configuration goals, called mode reconfiguration, MR. ME incrementally generates all spacecraft transitions from the previous configuration such that the models of the resulting configurations are consistent with the current observations. Thus, in Figure 8.25, a situation is shown where the left engine is firing normally in the previous state, but no thrust is observed in the current state. ME's must identify the configurations into which the spacecraft has transitioned that accounts for this observation. The figure shows two possible transitions, corresponding to one of the main engine valves failing. The failed or stuck closed valves are circled. Many other transitions, including unlikely double faults, can also account for the observations."

"MR determines the commands to be sent to the spacecraft such that the resulting transitions put the spacecraft into a configuration that achieves the configuration goal in the next state, as in Figure 8.26. This figure shows a situation in which mode identification has identified a failed main engine valve leading to the left engine. MR reasons that normal thrust can be restored in the next state if an appropriate set of valves leading to the right engine is opened. The figure shows two of the many configurations that can achieve the desired goal, when the circled valves are commanded to change state. Transitioning to the configuration at the top has lower cost because only necessary pyro valves (see the discussion in Section 8.3.2) are fired. The valves leading to the left engine are turned off to satisfy a constraint that, at most, one engine can fire at a time. The use of a spacecraft model in both ME and MR ensures that configuration goals are achieved correctly."

"Both ME and MR are reactive (see Section 6.4). ME infers the current configuration from knowledge of the previous configuration and current observations. MR only considers commands that achieve the configuration goal in the next state. Given these commitments, the decision to model component transitions as synchronous is key. An alternative is to model multiple component transitions through interleaving. However, interleaving can place an arbitrary distance between the current configuration and a goal configuration, defeating the desire to limit inference to a small fixed number of states. Hence model component transitions are synchronous. If component transitions in the underlying hardware-software are not synchronous, Livingstone's modeling assumption is that all interleavings of transitions are correct and support achieving the desired configuration. This assumption is removed in Burton, a follow-on of Livingstone, whose planner determines a sequence of control actions that produce all desired transitions (Williams and Nayak 1997). NASA is using the Burton architecture on a mission called Tech Sat 21."

"For Livingstone, ME and MR need not generate all transitions and control commands, respectively. Rather all that is required is just the most likely transitions and an optimal control command. These are efficiently regenerated by recasting ME and MR as combinatorial optimization problems. In this reformulation, ME incrementally tracks the likely spacecraft trajectories by always extending the trajectories leading to the current configurations by the most likely transitions. MR then identifies the command with the lowest expected cost that transitions from the likely current configurations to a configuration that achieves the desired goal. These combinatorial optimization problems are efficiently solved using a conflict-directed best-first search algorithm. See Williams and Nayak (1996a, 1996b, 1997) for a more formal characterization of ME and MR as well as a more detailed description of the search and planning algorithms."

6. EPILOGUE AND REFERENCES (Section 8.5)

"The architecture for the rule-based expert system is the production system. Whether the final product is data-driven or goal-driven, the model for the software is production system-generated graph search, as presented in

Chapter 6. We implement production system-based expert system shells in Java, Prolog, and Lisp in the auxiliary materials supplied with this book. The rules of these systems can include certainty measures in a limited form for design of a heuristic-based search."

Luger's recommended references:

"A number of references complement the material presented in this chapter; especially recommended is a collection of the original MYCIN publications from Stanford entitled Rule-Based Expert Systems by Buchanan and Shortliffe (1984). Other early expert systems work is described in Waterman (1968), Michie (1979), Patil et al. (1981), Clancy (1983), and Clancy and Shortliffe (1984a, 1984b). For a robust implementation of data-driven rule-based search, we recommend the CLIPS software, distributed by NASA. An excellent reference for CLIPS is Giarratano and Riley (1989, 2005)."

"Other important books on general knowledge engineering include Building Expert Systems by Hayes-Roth et al. (1984), A Guide to Expert Systems by Waterman (1986), Expert Systems: Concepts and Examples by Alty and Coombs (1984), Expert Systems Technology: A Guide by Johnson and Keravnou (1985), and Expert Systems and Fuzzy Systems by Negoita (1985), Introduction to Expert Systems by Jackson (1986), and Expert Systems: Tools and Applications by Harmon et al. (1988). See also Introduction to Expert Systems by Ignizio (1991), An Introduction to Expert Systems by Mockler and Dologite (1992), Expert Systems: Design and Development by John Durkin (1994), and conference proceedings in AI applications and expert systems, IEA/AIE (2004 - 2007)."

"Because of the domain specificity of expert system solutions, case studies are an important source of knowledge in the area. Books in this category include Expert Systems: Techniques, Tools and Applications by Klahr and Waterman (1986), Competent Expert Systems: A Case Study in Fault Diagnosis by Keravnou and Johnson (1986), The CRI Directory of Expert Systems by Smart and Langeland-Knudsen (1986), Developments in Expert Systems by Coombs (1984), and Developing and Managing Expert Systems by Prerau (1990). We especially recommend Expert Systems: Design and Development by Durkin (1994) for its vast number of practical suggestions on building systems."

"A number of techniques for knowledge acquisition have been developed. For more information on specific methodologies see Knowledge Acquisition: Principles and Guidelines by McGraw and Harbison-Briggs (1989) and Knowledge Engineering by Chorafas (1990), as well as An Introduction to Expert Systems by Mockler and Dologite (1992), and books and conference proceedings on expert systems IEA/AIE (2004 - 2007)."

"Case-based reasoning is an offshoot of earlier research by Schank's group at Yale and their work on scripts (see Sections 7.1.3 - 4). Case-Based Reasoning by Kolodner (1993) is an excellent introduction to the CBR technology. Other CBR-related work from Kolodner and her research group include (Kolodner 1987, 1991). Leake (1992, 1996) offers important comment on explanations in case-based systems. CBR software is now available in commercial software products that support the case-based technology."

"Model-based reasoning had its origins in the explicit representation of medicine, logic circuits, and other mathematical domains, often used for teaching purposes (deKleer 1975, Weiss et al. 1977, Brown and Burton 1978, Brown and VanLehn 1980, Genesereth 1982, Brown et al. 1982). More modern research issues in MBR are presented in Readings in Model-Based Diagnosis (edited by Hamscher et al. 1992), Diagnosis Based on Description of Structure and Function (Davis et al. 1982), and diagnosis in the context of multiple faults (deKleer and Williams 1987, 1989). Skinner and Luger (1995) and other writers on agent architectures (Russell and Norvig 1995, 2003) describe hybrid expert systems, where the interactions of multiple approaches to problem solving can create a synergistic effect with the strengths of one system compensating for the weaknesses of others."

"The planning section demonstrates some of the data structures and search techniques used for general-purpose planners. Further references include ABSTRIPS or ABstract specification for STRIPS generator relationships (Sacerdotti 1974), Warren's work (1976), and NOAH for nonlinear or hierarchical planning (Sacerdotti 1975, 1977). For a more modern planner see teleo-reactive planning, Section 15.3 (Benson and Nilsson 1995)."

"Meta-planning is a technique for reasoning not just about the plan but also about the process of planning. This can be important in expert systems solutions. References include Meta-DENDRAL for DENDRAL solutions (Lindsay et al. 1980) and Teiresias for MYCIN solutions (Davis 1982). For plans that interact continuously with the world, that is, model a changing environment, see McDermott (1978)."

"Further research includes opportunistic planning using blackboards and planning based on an object-oriented specification (Smoliar 1985). There are several surveys of planning research in the Handbook of Artificial Intelligence (Barr and Feigenbaum 1989, Cohen and Feigenbaum 1982) and the Encyclopedia of Artificial Intelligence (Shapiro 1992). Non-linearity issues and partial-order planning are presented in Russell and Norvig (1995). Readings in Planning (Allen et al. 1990) is relevant to this chapter."

"Our description and analysis of the NASA model-based reasoning and planning algorithms was taken from Williams and Nayak (1996a, 1996b, 1997). We thank Williams and Nayak, as well as AAAI Press, for allowing us to cite their research."

7. EXERCISES (Section 8.6)

The twenty-two exercises exactly as Luger lists them:

"1. In Section 8.2 we introduced a set of rules for diagnosing automobile problems. Identify possible knowledge engineers, domain experts, and potential end users for such an application. Discuss the expectations, abilities, and needs of each of these groups. 2. Take Exercise 1 above. Create in English or pseudocode 15 if-then rules (other than those prescribed in Section 8.2) to describe relations within this domain. Create a graph to represent the relationships among these 15 rules. 3. Consider the graph of Exercise 2. Do you recommend data-driven or goal-driven search? breadth-first or depth-first search? In what ways could heuristics assist the search? Justify your answers to these questions. 4. Pick another area of interest for designing an expert system. Answer Exercises 1–3 for this application. 5. Implement an expert system using a commercial shell program. These are widely available for personal computers as well as larger machines. We especially recommend CLIPS from NASA (Giarratano and Riley 1989) or JESS from Sandia National Laboratories. 6. Critique the shell you used for Exercise 5. What are its strengths and weaknesses? What would you do to improve it? Was it appropriate to your problem? What problems are best suited to that tool? 7. Create a model-based reasoning system for a simple electronic device. Combine several small devices to make a larger system. You can use if... then... rules to characterize the system functionality. 8. Read and comment on the paper Diagnosis based on description of structure and function (Davis et al., 1982). 9. Read one of the early papers using model-based reasoning to teach children arithmetic (Brown and Burton 1978) or electronic skills (Brown and VanLehn 1980). Comment on this approach. 10. Build a case-based reasoner for an application of your choice. One area might be for selecting computer science and engineering courses to complete an undergraduate major or a MS degree. 11. Use commercial software (check the WWW) for building the case-based reasoning system of Exercise 10. If no software is available, consider building such a system in Prolog, Lisp, or Java. 12. Read and comment on the survey paper Improving Human Decision Making through Case-Based Decision Aiding by Janet Kolodner (1991). 13. Create the remaining frame axioms necessary for the four operators pickup, putdown, stack, and unstack described in rules 4 through 7 of Section 8.4. 14. Use the operators and frame axioms of the previous question to generate the search space of Figure 8.19. 15. Show how the add and delete lists can be used to replace the frame axioms in the generation of STATE 2 from STATE 1 in Section 8.4. 16. Use add and delete lists to generate the search space of Figure 8.19. 17. Design an automated controller that could use add and delete lists to generate a graph search similar to that of Figure 8.19. 18. Show two more incompatible (precondition) subgoals in the blocks world operators of Figure 8.19. 19. Read the ABSTRIPS research (Sacerdotti 1974) and show how it handles the linearity (or incompatible subgoal) problem in planning. 20. In Section 8.4.3 we presented a planner created by Nilsson and his students at Stanford (Benson and Nilsson 1995). Teleo-reactive planning allows actions described as durative, i.e., that must continue to be true across time periods. Why might teleo-reactive planning be preferred over a STRIPS-like planner? Build a teleo-reactive planner in Prolog, Lisp or Java. 21. Read Williams and Nayak (1996, 1996a) for a more complete discussion of their model-based planning system. 22. Expand the propositional calculus representation scheme introduced by Williams and Nayak (1996a, p. 973) for describing state transitions for their propulsion system."

8. QUICK-REFERENCE TABLE OF KEY DEFINITIONS

Term
Definition

Strong method problem solving
Knowledge-intensive problem solving using domain-specific knowledge

Expert system
Program using domain-specific knowledge to provide expert-quality performance

Knowledge base
Contains domain knowledge (rules) and case-specific information

Inference engine
Applies the knowledge; performs the recognize-act cycle

User interface
Simplifies communication; hides complexity

Explanation subsystem
Answers why and how queries

Knowledge-base editor
Helps locate and correct bugs; maintains rule syntax; consistency checks

Expert system shell
Architecture minus the knowledge base and case-specific data

Domain expert
Provides knowledge of the problem area

Knowledge engineer
AI specialist who implements knowledge in a program

End user
Determines major design constraints

Exploratory programming
Iterative prototype-test-revise development cycle

Conceptual model
Knowledge engineer's evolving conception of the domain knowledge; lies between expertise and implementation

Rule-based expert system
Knowledge represented as if…then… rules

Goal-driven (backward) chaining
Goal matched against rule conclusions; premises become subgoals

Data-driven (forward) chaining
Facts matched against rule conditions; conclusions added to working memory

Opportunistic search
When a rule fires, consider those rules with the new information as premise

Why query
User asks why the system needs a piece of data; answer is the current rule

How query
User asks how a conclusion was reached; answer is the trace of rules

RETE algorithm
Compiles rules into a network for efficient pattern matching

Model-based reasoning
Analysis founded on specification and functionality of a physical system

Qualitative model
Software simulation of the function of a device

Hypothesis generation
Given a discrepancy, hypothesize which components could have caused it

Hypothesis testing
Given potential faulty components, determine which explain the observed behaviour

Hypothesis discrimination
Gather additional information to distinguish between surviving hypotheses

Bridging fault
Fault where two wires/devices are inadvertently linked; difficult for model-based reasoning

Case-based reasoning (CBR)
Uses an explicit database of past problem solutions to address new situations

Case retrieval
Finding appropriate cases from memory based on similarity

Case adaptation
Modifying a retrieved case to apply to the current situation

Salient features
The most relevant features for indexing and retrieval

Goal-directed preference
Retrieve cases with the same goal as the current situation

Salient-feature preference
Prefer cases matching the most important features

Specify preference
Prefer exact matches before more general matches

Frequency preference
Prefer most frequently matched cases

Recency preference
Prefer most recently used cases

Ease of adaptation preference
Prefer cases most easily adapted

Transformational analogy
Case-based approach modifying existing solutions to new problems (T-space)

Hybrid architecture
Integration of two or more reasoning paradigms

Planning
Finding a sequence of actions to achieve a goal

Frame problem
Representing what does **not** change when an action occurs

Frame axioms
Rules specifying what is unchanged by an operator

STRIPS
STanford Research Institute Planning System (Fikes & Nilsson 1971)

Preconditions (P)
Conditions the world must meet for an operator

Add list (A)
Additions to the state description from applying the operator

Delete list (D)
Items removed from the state description by the operator

Incompatible subgoals
Subgoals whose actions undo each other (non-linearity)

Triangle table
Data structure organising sequences of actions, including potentially incompatible subgoals

Kernel
Intersection of rows below and columns left of the nth row/column; determines when step n can be executed

Teleo-reactive (TR) planning
General-purpose architecture coordinating simple task-specific agents to accomplish a top-level goal

Durative action
Action sustained over time, interrupted when another closer to the top-level goal activates

TR tree
Condition → action pairs, evaluated top to bottom until first true condition

Livingstone
NASA's model-based reactive self-configuring autonomous system kernel

Mode estimation (ME)
Identifies the current configuration of the spacecraft

Mode reconfiguration (MR)
Moves the spacecraft to a new configuration achieving the desired goals

Burton
Follow-on to Livingstone; planner determines a sequence of control actions producing desired transitions

9. WHAT IS NOT IN CHAPTER 8

Topic	Belongs to
Reasoning under uncertainty (certainty factors, fuzzy logic, Dempster-Shafer)	Chapter 9
Truth maintenance systems	Chapter 9
Bayesian belief networks	Chapter 9
Machine learning algorithms	Chapters 10–13
Neural networks (detail)	Chapter 11
Genetic algorithms (detail)	Chapter 12
Resolution theorem proving	Chapter 14
Natural language parsing (detail)	Chapter 15
Constructivist epistemology	Chapter 16

'
End of Chapter 8 notes, based strictly on Luger’s textbook.Create print ready notes for my exam prepation. If you can, take screenshots of diagrams that were text only and include them. Make sure you get the formatting and unicode and symbols right, no blocks or unkown stuff.

Selected Textbook Diagram Extracts

These figure extracts are supplementary visual references. Captions and nearby explanations in the main notes remain the primary study text.

Chapter 1 — The Turing Test

problem. As it is used an
leading to new questions a

1.1.4 The Turing Tes

One of the earliest papers
relation to the modern digi
Alan Turing. *Computing M*
its assessment of the argun
machine and its answers t
to the theory of computabi
actually be made to think.
(what is thinking? what is
question of intelligence be

The *Turing test* measu
that of a human being, ar
test, which Turing calle
counterpart in rooms apar
(Figure 1.1). The interroga
know which entity is actua
of a textual device such

Chapter 1 — The Turing Test
Source: Luger, 6th edition, PDF page 38.

Chapter 1 — Biological neuron

the human brain or the evo

A simple schematic o
of branched protrusions, c
receive signals from othe

Chapter 1 — Biological neuron
Source: Luger, 6th edition, PDF page 54.

Chapter 3 — Directed graph and paths

nodes (as in Figure 3.2), th

A graph is *directed* i
graph are usually drawn a
that can be crossed in eith
no direction indicators at a
crossed from node a to no

A *path* through a gra

Chapter 3 — Directed graph and paths
Source: Luger, 6th edition, PDF page 108.

Chapter 4 — Heuristic search illustration



Figure

3. evaluate B4; oper
4. evaluate C4; oper
5. evaluate H3; oper
6. evaluate O2; oper
7. evaluate P3; the s

Figure 4.11 shows the
The states contained in op
of the search and closed

Chapter 4 — Heuristic search illustration
Source: Luger, 6th edition, PDF page 161.

Chapter 5 — Probability / stochastic methods illustration



Figure 5.1

Chapter 5 — Probability / stochastic methods illustration
Source: Luger, 6th edition, PDF page 201.

Chapter 6 — Search control / recursion illustration

are not changed by the execution

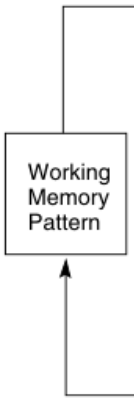


Figure 6.1 A pro
work
the c

Chapter 6 — Search control / recursion illustration
Source: Luger, 6th edition, PDF page 227.

Chapter 7 — Knowledge representation illustration

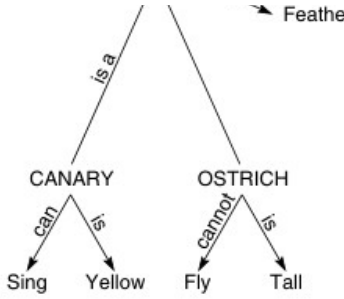
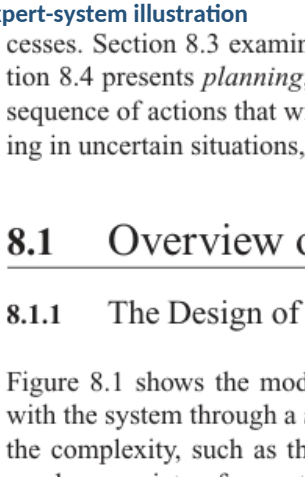


Figure 7.1 Semantic
in their re
response

There is psychological evide
humans also organize their knowl
appropriate levels of the taxonom
tion storage and management usi
hierarchy was derived from labora
questions about different propert

Chapter 7 — Knowledge representation illustration
Source: Luger, 6th edition, PDF page 255.

Chapter 8 — Expert-system illustration


cesses. Section 8.3 examir
tion 8.4 presents *planning*
sequence of actions that w
ing in uncertain situations,

8.1 Overview

8.1.1 The Design of

Figure 8.1 shows the mod
with the system through a
the complexity, such as th

Chapter 8 — Expert-system illustration
Source: Luger, 6th edition, PDF page 304.